Student Beneficiary Management System Chapters 4–6 Documentation and Evidence Package

Markdown sources rendered to PDF for review and handover

Repository	student-beneficiary-system
Branch	fix/sbms-completion-work
Evidence revision	d6b5032
Documents	27 markdown files
Rendered	2026-11-02

What this PDF is. A rendering of the markdown files in `docs/final-documentation-content/`. It adds no new claims: every statement stays bounded to revision `d6b5032`, and every open gap stays visible — hardware specification, figures, screenshots, user acceptance testing, phone camera scan, Azure ML runtime path and the two test generations. Regenerate with `node scripts/docs-to-pdf.mjs`.

Contents

1 CLAUDE HANDOFF GUIDE: HOW TO WRITE CHAPTERS 4, 5 AND 6 FROM THIS PACKAGE

1. Read in this order
2. The file set and what each file is for
3. Non-negotiable writing rules
4. Drafting workflow per chapter
5. Phrases to use, and phrases to avoid
6. Five-minute self-check before handing a chapter back
7. When evidence is missing
8. What this package does not do
9. PDF rendering

2 ACADEMIC GUIDE REQUIREMENTS FOR CHAPTERS FOUR, FIVE AND SIX

1. Sources consulted, and their provenance
2. Chapter 4: prescribed structure
3. Chapter 5: structure applied
4. Chapter 6: structure applied
5. Compliance matrix for this documentation package
6. Ambiguities recorded rather than resolved
7. Related files

3 SBMS EVIDENCE PROVENANCE AND CLAIM LIMITS

1. Evidence tiers used throughout this documentation set
2. Binding of every claim to a revision
3. Documents that must not be cited as evidence
4. Claims that cannot be supported
5. Claims that can be made, with the wording that keeps them true
6. Open decisions that belong to someone else
7. Synthetic data policy for every figure, table and screenshot
8. Defects found during the project, and what they prove
9. How to keep this set accurate
10. Integrity statement

4 CHAPTER 4 EVIDENCE PACKAGE: SYSTEM ANALYSIS AND DESIGN

1. What Chapter 4 may assert, and on what basis
2. Requirement analysis: the numbers a writer must use
3. Analysis artefacts required, and what each must encode
4. Design facts that must match the code
5. Interface design: what each screen figure must show
6. What Chapter 4 must not claim
7. Open items affecting Chapter 4
8. Related files

5 CHAPTER 5 EVIDENCE PACKAGE: IMPLEMENTATION, TESTING AND EVALUATION

1. Implementation environment (evidence, not recollection)
2. Test results: the only counts that may be quoted
3. Implementation by feature: what the code does
4. Machine-learning evaluation: the four numbers that are defensible
5. Defects: six with a fix and a test, others pointed to
6. Evaluation: what the evidence supports
7. Limits of the evidence itself
8. Open items affecting Chapter 5
9. Related files

6 CHAPTER 6 EVIDENCE PACKAGE: CONCLUSIONS, RECOMMENDATIONS AND FUTURE WORK

1. The objectives, and where their evidence sits
2. What the conclusions may and may not say
3. Recommendations the evidence supports
4. Future work: split by kind, not by convenience
5. Boundaries the conclusion must keep
6. Open items affecting Chapter 6
7. Related files

7 MASTER EVIDENCE REGISTER
1. System structure and data
2. Behaviour
3. Tests, defects and deployment
4. Claims with no supporting evidence, recorded as such
5. How to cite an ID
6. Related files

8 CROSS-CHAPTER CONSISTENCY AND TRACEABILITY
1. Consistency matrix
2. Traceability chains
3. Contradictions and drift requiring the author's confirmation
4. Naming consistency used across the package
5. Related files

9 DIAGRAM SPECIFICATIONS
1. How to read the register
2. Analysis diagrams (Chapter 4 §4.3)
3. Screen figures (Chapter 4 §4.5) — screenshot required
4. Drawing rules that keep the figures honest
5. Coverage of the guide's diagram expectations
6. Related files

10 FIGURE PLACEHOLDER CHECKLIST
1. Structure and behaviour diagrams (drawn from the specifications, not screenshots)
2. Screen figures (screenshots of the running application)
3. Deployment figure
4. Checks before the figures are considered complete
5. Related files in this folder

11 ACADEMIC COMPLETENESS AND GAP AUDIT
1. Coverage of the requested areas
2. What is complete
3. What is partial, and why
4. Missing evidence, ordered by submission risk
5. Contradictions and unresolved questions
6. Secret and sensitivity check
7. What makes the package academically complete, and what does not
8. Related files

12 DOCUMENTATION GAP REGISTER
1. Items requiring author input (not obtainable from the repository)
2. Items requiring additional testing or evidence
3. Documentation inconsistencies to reconcile
4. Items deliberately not supplied, and why
5. Cross-reference: gaps to chapter sections
6. Minimum set to close before submission
7. Closed since this register was drafted
8. Document set this register belongs to

13 CHAPTER 4: SYSTEM ANALYSIS AND DESIGN
4.1 Introduction
4.2 Requirement Analysis
4.3 System Analysis Diagrams
4.4 System Design
4.5 System Design Diagrams (interface design)
4.6 Database schema
4.7 System architecture (physical deployment)
4.8 Chapter summary

14 CHAPTER 5: SYSTEM IMPLEMENTATION, TESTING AND EVALUATION
5.1 Introduction
5.2 Implementation environment
5.3 System implementation
5.4 System testing
5.5 System evaluation

5.6 Discussion of results and limitations
5.7 Chapter summary

15 CHAPTER 6: CONCLUSIONS, RECOMMENDATIONS AND FUTURE WORKS
6.1 Introduction
6.2 Conclusions
6.3 Recommendations
6.4 Future works
6.5 Chapter summary

16 SBMS REQUIREMENTS AND SCOPE CATALOGUE
1. Purpose
2. Status vocabulary
3. System identity and component inventory
4. Actors and interests
5. Scope boundary
6. Functional requirements register (FR-01 … FR-30)
7. Non-functional requirements register (NFR-01 … NFR-21)
8. Counts and status summary
9. Traceability entry points

17 SBMS SYSTEM ARCHITECTURE AND MODULE MAP
1. Architecture as built
2. Request lifecycle and middleware order
3. Backend module map
4. Frontend module map
5. End-to-end flows across modules
6. Architectural decisions worth recording in Chapter Four
7. Architectural weaknesses to report honestly
8. Dead and legacy code inventory
9. Technology stack with version evidence
10. What this architecture does not include

18 SBMS DATABASE SCHEMA AND DATA DICTIONARY
1. Purpose and method
2. Schema organisation and migration policy
3. Table inventory
4. Data dictionary
5. Relationships and referential actions
6. Indexes and their purpose
7. State machines enforced by the schema
8. Legacy and dead schema objects
9. Data provenance and synthetic data
10. What the schema does not enforce
11. Reproduction

19 SBMS API ENDPOINT CATALOGUE
1. Method and conventions
2. Public, unauthenticated endpoints
3. Authenticated endpoint catalogue
4. Endpoint counts
5. Error catalogue
6. Rate limits and budgets
7. Contract notes and honest limitations

20 SBMS SECURITY AND ACCESS CONTROL CATALOGUE
1. Summary of the control set
2. Authentication
3. Authorisation matrix
4. Identity resolution and role freshness
5. Secrets and configuration
6. Origin control, proxy trust and headers
7. Input validation and error handling
8. Data protection and privacy
9. Audit trail

10. Rate limiting, and exactly how far it reaches
11. Residual security risks to report
12. How to describe the security position in Chapters 4-6

21 **SBMS ML SUBSYSTEM DOCUMENTATION**

1. What is deployed, what is research
2. Integration architecture
3. Feature specification (the six model inputs)
4. Prediction contract
5. Thresholds in force
6. Training data and provenance
7. Measured performance (synthetic holdout — not an accuracy claim)
8. From score to case: the registration decision path
9. Review policy and what the administrator sees
10. Research artefacts deliberately excluded from the deployed claim
11. What this subsystem cannot support, and why
12. Decisions still owned by the programme owner
13. Reproduction
14. Evidence index

22 **SBMS UI SCREEN AND WORKFLOW INVENTORY**

1. Application shell and routing
2. Shared frontend infrastructure
3. Public screens
4. Field-officer screens
5. Administrator screens
6. Workflow walkthroughs as the interface presents them
7. Interface states
8. Accessibility and responsiveness: what is and is not established
9. Screens that do not exist
10. Mapping to the Chapter Four interface figures

23 **SBMS TEST EVIDENCE CATALOGUE**

1. Test levels and tooling
2. Backend Jest inventory (24 suites, 151 tests)
3. Backend native tests (43 in-process, 3 out-of-process)
4. Browser evidence
5. Python evidence
6. Database, security and recovery evidence
7. What has not been run, and what that costs
8. Run history, including one instability that must be reported
9. Coverage exclusions that matter for the evaluation chapter
10. Reproduction commands
11. How to read a test count in the report

24 **SBMS DEPLOYMENT AND OPERATIONS GUIDE**

1. Environments
2. Azure resource inventory
3. Live endpoints
4. Runtime architecture as deployed
5. Build and release pipeline
6. Configuration matrix
7. Start-up and readiness sequence
8. Database migration procedure
9. Cost model (recorded, not re-derived)
10. Backup, restore and continuity
11. Operations runbook
12. Known operational issues carried forward
13. What the deployment does not demonstrate

25 **EVIDENCE AND VERIFICATION REPORT**

1. Identity of what was inspected
2. Code evidence index
3. Test artefact index
4. Document evidence index

5. Verification status of the chapters' major claims
6. What was checked during this pass, and what was not
7. How to re-verify the main claims
8. Residual risks to the report's accuracy
9. Cross-chapter consistency checks performed
10. Summary of the evidence position

26 OBJECTIVE–REQUIREMENT–TEST TRACEABILITY

1. Objective → requirement → evidence map
2. Non-functional requirement traceability
3. Figure and table traceability
4. Test suite → requirement coverage matrix
5. Conclusion → evidence traceability
6. Traceability exceptions to note in the report

27 SBMS COMPREHENSIVE INFORMATION REPORT FOR CHAPTERS FOUR, FIVE AND SIX

Document control
How to use this report
Executive summary of the evidence position
1. System identity and problem framing
2. Requirements: what the chapter can state as fact
3. Actors and their interests
4. Analysis artefacts: what each diagram must encode
5. Design: architecture and module responsibilities
6. Design: workflow and state
7. Design: access control, security and validation
8. Design: interface inventory
9. Design: database
10. Design: physical deployment
11. What Chapter Four must not claim
12. Implementation environment
13. Implementation by feature, and where the code is
14. Test evidence, by level
15. Defects found during testing, and what closed them
16. Machine-learning evaluation
17. Evaluation against the objectives
18. Evaluation against non-functional requirements
19. Limits of this evaluation
20. Conclusions, objective by objective
21. The stated contribution
22. Recommendations
23. Future work
24. Boundaries of the conclusions
25. Completeness matrix (requested area to status to location)
26. Gap summary, ordered by submission risk
27. Reproduction: how a reader can re-establish every claim
28. Document map
29. Final statement

CLAUDE HANDOFF GUIDE: HOW TO WRITE CHAPTERS 4, 5 AND 6 FROM THIS PACKAGE

> **Audience.** An AI assistant or the author, writing the three chapters from this evidence set. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`. Nothing in this package refers to any other commit unless it says so.

---

1. Read in this order

Order	File	Why
1	`ACADEMIC_GUIDE_REQUIREMENTS.md`	The structure to write into, and the six ambiguities you must not resolve by invention
2	`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md`	What may not be claimed. Read this before drafting a single sentence
3	`CHAPTER_4_…_EVIDENCE.md`, `CHAPTER_5_…_EVIDENCE.md`, `CHAPTER_6_…_EVIDENCE.md`	The evidence for each chapter, with the section pointers
4	The drafted chapter you are editing (`CHAPTER_4_…`, `CHAPTER_5_…`, `CHAPTER_6_…`)	The prose already exists; improve it rather than rewriting from scratch
5	`MASTER_EVIDENCE_REGISTER.md`	Evidence IDs and statuses for any claim you add
6	`CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md`	Keeps Chapters 4–6 consistent with 1–3 and with each other
7	`DIAGRAM_SPECIFICATIONS.md`, `FIGURE_PLACEHOLDER_CHECKLIST.md`	Figures: what must be drawn or captured
8	`ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md`, `DOCUMENTATION_GAP_REGISTER.md`	What is still missing, ranked by risk

The twelve deep catalogues (the `SBMS_…` files with suffixes `CATALOGUE`, `DOCUMENTATION`, `GUIDE` and `INVENTORY`) are reference material: consult them per section, do not summarise them wholesale into the chapters.

---

2. The file set and what each file is for

File	Role
`ACADEMIC_GUIDE_REQUIREMENTS.md`	Guide structure, compliance matrix, ambiguities
`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN_EVIDENCE.md`	Chapter 4 evidence entry point
`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION_EVIDENCE.md`	Chapter 5 evidence entry point
`CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORK_EVIDENCE.md`	Chapter 6 evidence entry point
`CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md`	Consistency matrix, traceability chains, contradictions
`DIAGRAM_SPECIFICATIONS.md`	All 24 figures, attributes and drawing rules
`MASTER_EVIDENCE_REGISTER.md`	51 evidence IDs with locations and statuses
`ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md`	Coverage, missing evidence, secret check
`CLAUDE_HANDOFF_GUIDE.md`	This file
`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md`	The drafted chapter (1 176 lines)
`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md`	The drafted chapter (838 lines)
`CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md`	The drafted chapter (322 lines)
`SBMS_COMPREHENSIVE_CHAPTERS_4_5_6_INFORMATION_REPORT.md`	The consolidated evidence report, §1–§29
`SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md`	37 FRs, 21 NFRs, business rules
`SBMS_SYSTEM_ARCHITECTURE_AND_MODULE_MAP.md`	Modules, flows, decisions

File	Role
`SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md`	13 tables, columns, state machines
`SBMS_API_ENDPOINT_CATALOGUE.md`	45 endpoints
`SBMS_ML_SUBSYSTEM_DOCUMENTATION.md`	Features, artifact, metrics, limits
`SBMS_TEST_EVIDENCE_CATALOGUE.md`	Every test artefact with date and command
`SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md`	Controls and residual risk
`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md`	Azure resources, pipeline, runbook
`SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md`	Screens and workflows
`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md`	Evidence tiers and forbidden claims
`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md`	Objective → requirement → test → conclusion
`DOCUMENTATION_GAP_REGISTER.md`	Item-level gaps G-01…G-28
`EVIDENCE_AND_VERIFICATION_REPORT.md`	Claim-level inspection record
`FIGURE_PLACEHOLDER_CHECKLIST.md`	Capture and insertion checklist

Two tiers, deliberately: the first nine files are the entry set a writer reads; the rest is the evidence they point into. Do not duplicate content between tiers — cite it.

3. Non-negotiable writing rules

1. **No claim without a location.** Every factual sentence must trace to a file, route, migration, test or log, or to an evidence ID. If it cannot, it is deleted, not softened.
2. **Use the six statuses exactly.** VERIFIED AND IMPLEMENTED, VERIFIED BUT NOT IMPLEMENTED, TESTED AND PASSED, TESTED AND FAILED, NOT TESTED, UNKNOWN / REQUIRES CONFIRMATION. No seventh status, no "mostly".
3. **Counts carry dates and commands.** "151 tests" is wrong; "151 tests, `npx jest --runInBand tests`, 2026-11-01" is right. Never add counts across levels.
4. **Synthetic means synthetic.** Every ML metric is pair-level on generated data. Never write "the model is accurate", "the system detects duplicates correctly in practice", or any real-world accuracy claim.
5. **The human gate is the control.** Never describe the model as deciding a duplicate.
6. **Gaps stay visible.** FR-34…FR-37 and the partial requirements are reported, never absorbed into a success narrative or moved to future work to look complete.
7. **Deviations are stated once and kept.** The Random Forest → XGBoost change (G-21) appears in Chapters 4, 5 and 6 in the same words.
8. **Do not renumber figures or tables.** Figures 4.1–4.24, tables as numbered in the drafts.
9. **Do not invent citations.** Chapters 4–6 add no literature claims; a needed citation belongs in Chapters 1–3 and must be verified by the author.
10. **Do not print secrets.** No key, token, password, connection string or real personal data, ever.

---

4. Drafting workflow per chapter

Step	Action
1	Open the chapter draft and the matching `…_EVIDENCE.md` side by side
2	For each section, copy the evidence row, not the prose: the fact, the location, the limitation
3	Write the sentence so that the limitation survives editing. If the limitation would have to be cut for the sentence to read well, the sentence is too strong
4	Check the figure callouts against `DIAGRAM_SPECIFICATIONS.md`; every figure number used must exist there
5	Check requirement numbers against `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §6–§8; never invent an FR or NFR number

Step	Action
6	Re-read `ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md` §4 to confirm you have not silently closed a gap
7	Run the five-minute self-check in §6 below

5. Phrases to use, and phrases to avoid

Avoid	Write instead
"The model accurately detects duplicates"	"The model ranks candidate pairs; an administrator decides"
"95% accuracy"	"Precision 0.9204, recall 0.8927 on a synthetic test partition at a 0.50 cut-off"
"The system was validated with users"	"No user acceptance study was conducted (G-06)"
"The system is deployed in production"	"A demonstration instance is deployed on Azure App Service"
"All tests pass"	"24 suites / 151 tests passed on 2026-11-01 under the command shown"
"The QR code was tested on a phone"	"QR verification was exercised in a headless browser; no phone capture exists (G-11)"
"Random Forest-based duplicate detection"	"An XGBoost classifier over six features; the proposal's Random Forest survives as a research candidate"
"Requirements fully met"	"24 of 30 delivered requirements verified, 6 partial, 4 in-scope gaps"

6. Five-minute self-check before handing a chapter back

1. Every figure and table referenced exists and is numbered consistently.
2. Every number in the chapter appears in `MASTER_EVIDENCE_REGISTER.md`, `SBMS_TEST_EVIDENCE_CATALOGUE.md` or `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` with the same value.
3. No sentence claims real-world ML accuracy, user acceptance, production assurance or phone-based scanning.
4. Every gap named in the chapter matches its ID in `DOCUMENTATION_GAP_REGISTER.md`.
5. The chapter's conclusions are no stronger than Chapter 5's evidence.

7. When evidence is missing

Do not infer, and do not write around the hole. Do one of three things, in this order:

1. **Cite the gap** if it already has an ID (`DOCUMENTATION_GAP_REGISTER.md`).
2. **Add a gap row** if it is new: what is missing, why it matters, which section depends on it, who can close it.
3. **State the limitation in the sentence itself** if the section cannot be written without touching the gap — for example "the workstation specification was not recorded (G-01)".

An openly stated gap is academically acceptable. An invented number is not, and it is the one error that cannot be repaired after submission.

8. What this package does not do

- It does not draw the figures, capture the screenshots or run a user study.
- It does not edit application code, schema, migrations, configuration, model artifacts or deployment infrastructure.
- It does not resolve contradictions in favour of the more convenient version; it records them.
- It does not replace the author's judgement on the six ambiguities in `ACADEMIC_GUIDE_REQUIREMENTS.md` §6.

---

9. PDF rendering

`docs/final-documentation-content/pdf/` holds four files:

- `SBMS_CHAPTERS_4_5_6_DOCUMENTATION_PACKAGE.pdf` — cover page, contents list and all 27 files in the §1 reading order. This is the set; nothing in the folder is missing from it.
- `CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.pdf`, `CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.pdf`, `CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.pdf` — the three chapters kept apart because they are printed, annotated and submitted on their own.

Per-file PDFs for the other 24 documents are deliberately not kept: each would be a second copy of a document that is already in the package. The PDFs add no content in any case — they are this markdown, paginated, so a claim is no stronger in PDF than in the `.md` source.

Regenerate after any edit:

```
node scripts/docs-to-pdf.mjs            # combined package + the three chapters
node scripts/docs-to-pdf.mjs --combined # combined package only
node scripts/docs-to-pdf.mjs --all      # one PDF per file, all 27, if you really want them
```

A run that writes the per-file PDFs also deletes any PDF it did not produce, so the folder cannot drift back into duplicates. The renderer uses headless Chrome and no third-party packages, so a PDF can be no more than one command out of date with its source.

ACADEMIC GUIDE REQUIREMENTS FOR CHAPTERS FOUR, FIVE AND SIX

> **Status of this file.** Records the academic structure the three chapters are written against, and separates what the School's documents actually prescribe from what this project inferred. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.

---

1. Sources consulted, and their provenance

Source	What it governs	Present in the repository at `d6b5032`?
`ICS Project II Proposal Guide 2026_V1.docx` (School template)	Report structure, chapter list, formatting expectations	**No.** Consulted outside the repository; not version-controlled here
`Project Proposal- 169613.pdf` (approved SBMS proposal, 34 pages)	Objectives, scope, named technologies and the ML method the report must stay consistent with	**No**; the facts drawn from it are transcribed in `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §2
`CHAPTER_4[1].docx` (Chapter 4 exemplar)	Section numbering and subsection pattern for Chapter 4	**No**

Consequence, stated plainly. Because none of the three source files is in the repository, no line of this file can be presented as a verbatim quotation of the guide. What follows is the structure the chapters were built to, with each requirement classified **PRESCRIBED** (the exemplar's own headings fix it), **INFERRED** (the brief's chapter list implies it; the wording is not recoverable here) or **AMBIGUOUS** (the guide may require it and this project could not read the requirement). An ambiguity is recorded, never resolved by invention.

---

2. Chapter 4: prescribed structure

The exemplar fixes these headings; SBMS content is mapped to each.

Guide heading	What the chapter must contain	Where it is written
4.1 Introduction	Purpose of the chapter, link to Chapters 1–3	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.1
4.2 Requirement Analysis	Functional, non-functional and data requirements; sources of requirements	§4.2.1–§4.2.5
4.3 System Analysis Diagrams	Context, use case, activity, class, sequence, ER, ML pipeline	§4.3.1–§4.3.11
System Design	Architecture, modules, authentication, workflows, security, validation	§4.4.1–§4.4.7
System Design Diagrams	Interface design, described so each screen can be reproduced without invention	§4.5.1–§4.5.8
Database Schema	Tables, columns, types, keys, constraints, indexes, normalisation	§4.6.1–§4.6.3
System Architecture	Physical deployment topology	§4.7
Chapter summary	What the chapter established	§4.8

PRESCRIBED features the exemplar carries, and SBMS therefore includes: use-case description tables, at least one activity diagram per principal workflow, and a class diagram with multiplicities.

INFERRED additions, kept because the brief asks for them and the exemplar's heading list does not forbid them: ML-integration design (§4.4.4), security and privacy design (§4.4.6), validation and error handling (§4.4.7), and the interface inventory (§4.5).

---

3. Chapter 5: structure applied

Requirement	Status	Where
Implementation environment, distinguishing development, test and production	**INFERRED**, satisfied	`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md` §5.2.1–§5.2.4
Implementation account per module with source references	**INFERRED**, satisfied	§5.3.1–§5.3.12
Testing strategy by level (unit, integration, system, end-to-end, browser, security, performance, ML, acceptance)	**INFERRED**, satisfied with gaps	§5.4.1–§5.4.8
Test-case tables with ID, requirement, scenario, preconditions, steps, expected, actual, status, evidence	**AMBIGUOUS** that the guide mandates this exact column set; adopted because it forces evidence per row	§5.4.3; `SBMS_TEST_EVIDENCE_CATALOGUE.md` §3–§6
ML evaluation separate from system testing	**INFERRED**, satisfied	§5.5.5; `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md`
Evaluation against objectives and NFRs, plus limitations	**PRESCRIBED** by the practice of evaluating against objectives	§5.5.2–§5.5.4, §5.6

4. Chapter 6: structure applied

Requirement	Status	Where
Conclusions answering each specific objective from Chapter 1	**PRESCRIBED**	`CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md` §6.2.3–§6.2.6
Contribution of the project	**INFERRED**	§6.2.8
Recommendations tied to a stakeholder and an action	**PRESCRIBED** in outline	§6.3.1–§6.3.5
Future work, not used to hide unfinished current scope	**PRESCRIBED** in outline	§6.4.1–§6.4.10

The rule applied throughout Chapter 6: a requirement that was in scope and is not delivered is reported as a gap (FR-34 … FR-37, §6.2.9); future work is reserved for what was never in scope.

5. Compliance matrix for this documentation package

Guide expectation	Package response	Status
Chapters built from implementation evidence, not from drafts	Every claim traced through `MASTER_EVIDENCE_REGISTER.md`	Met
Diagrams required by the guide	24 figures specified with nine attributes each (`DIAGRAM_SPECIFICATIONS.md`)	Specified, not drawn
Screenshots of the running system	12 screen figures specified	**Not produced** (G-03/G-04/G-05)
Test results with dates	Every count carries a date and a command (`SBMS_TEST_EVIDENCE_CATALOGUE.md` §2)	Met
User acceptance evidence, where the guide expects it	None exists; stated as a gap with the position the author must adopt	**Gap** (G-06)
Consistency with Chapters 1–3	Consistency matrix; contradictions flagged, not rewritten (`CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md`)	Met, 6 items flagged
No invented results	Forbidden claims enumerated (`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §4)	Met

6. Ambiguities recorded rather than resolved

#	Ambiguity	Why it matters	What the author must decide
A-1	The guide's exact Chapter 5 subsection list was not available	Section order in Chapter 5 is inferred, not quoted	Confirm against the guide before submission
A-2	Whether a test-case table must enumerate all 151 tests or may group them	The catalogue gives suite-level detail and names individual tests only where a claim depends on one	Confirm the expected granularity
A-3	Whether screenshots must show the Azure deployment or a local instance	Affects the provenance caption of G-03/G-04 figures	Confirm, then capture accordingly
A-4	Whether a user-acceptance section is mandatory	If it is, NFR-14 cannot be claimed and §5.5.6 stays empty	Confirm; see G-06
A-5	Formatting requirements (font, spacing, page limits, appendix rules)	Not recorded anywhere in the repository	Apply the School template at assembly
A-6	Whether the confusion matrix must appear in the chapter body	It exists in `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §7	Include it; cost is one table

Nothing here blocks drafting. Each is a question for the author, not for the repository.

---

7. Related files

`SBMS_COMPREHENSIVE_CHAPTERS_4_5_6_INFORMATION_REPORT.md` §1–§29 is the consolidated evidence base; `ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md` records coverage and gaps; `CLAUDE_HANDOFF_GUIDE.md` gives the reading order and the writing rules.

SBMS EVIDENCE PROVENANCE AND CLAIM LIMITS

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Purpose:** to state, in one place, what each claim in the report rests on, what cannot be claimed, and why. This file exists so that no chapter has to invent a status and no reader has to guess how strong an assertion is.

---

1. Evidence tiers used throughout this documentation set

Tier	Definition	Wording the report may use
T1 — executed test	An automated test ran on the named date and passed	"verified", "proven by test"
T2 — recorded run	A harness or command was executed and its output recorded, but it is not part of the permanent suite	"recorded on <date>", "observed"
T3 — code inspection	The behaviour was read in source, not executed	"implemented", "exists in code"
T4 — document statement	A repository document asserts it, and that document is current	"the project records that"
T5 — stale or conflicting document	A document asserts it but contradicts the code or a later record	**May not be cited as evidence**
T6 — absent	No evidence exists	"not verified", "not measured", "not conducted"

A claim inherits the **weakest** tier among its dependencies. A conclusion that combines a T1 test with a T6 gap is not a T1 conclusion.

2. Binding of every claim to a revision

Fact	Value
Branch	`fix/sbms-completion-work`
Commit described	`d6b5032`
Working tree at time of inspection	Clean apart from documentation added by this pass
Other checkouts on the machine	Four (`student-beneficiary-system`, `-dev`, `-admin-login-fix`, `sbms-raw-pair-model-training`); their documents differ (G-26)
Test counts cited	Always with a date, because two generations exist (21/128 on 2026-10-31; 24/151 on 2026-11-01)
Untracked at this revision	`SBMS_O10_O11_IMPLEMENTATION_REPORT.md`, `SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` (G-28)
Tracked but git-ignored	`ml-service/model/xgb_duplicate_model.joblib` is tracked while `.gitignore` still lists `ml-service/model/*.joblib` (G-27)

Any statement in the chapters that cannot be tied to this commit must name a different one, or be removed.

3. Documents that must not be cited as evidence

Document	Problem	What to do
`docs/verification.md`	States that no controller calls the prediction function and that controllers use their own scoring. That is false at this revision: `students.controller.js:69` calls `scorePairs` through `duplicateCheck.js`	Mark superseded; do not cite (G-08, G-23)
`docs/uat-test-plan.md`	Reports five `PASS` results with no participants, and contains the same stale claim about the predictor	Reclassify as an author walkthrough or withdraw; do not present as user acceptance (G-07, G-24)
`render.yaml`	Describes an abandoned deployment target	Cite only as history

Document	Problem	What to do
The 2026-10-27 `FINAL_TEST_REPORT` generation	Superseded by the 2026-10-31 and 2026-11-01 runs	Never cite; the file itself says so
`ML_RELIABILITY_REPORT.md` Random Forest figures	Describe a candidate the application never loads	Cite only as research comparison
`docs/azure-deployment-plan-2026-10-30.md` early sections	Some early architecture options were superseded by decision A2/D8	Cite §12 (current) and the deployment log instead

Where a chapter already cites one of these, the citation must be replaced with the record named in the right-hand column of the relevant row above.

4. Claims that cannot be supported

#	Claim that must not appear	Why	Evidence tier
1	"The system detects duplicate beneficiaries accurately"	No adjudicated labels exist; every label is synthetic	T6 (D5)
2	Any real-world precision, recall, miss rate or false-positive rate	As above	T6
3	"The 0.50 cut-off is optimal"	F1 is flat from 0.30 to 0.60; no owner cost policy	T6 (D2/D3)
4	"The model was validated on Kenyan bursary data"	The data is generated, not collected	T6
5	"The QR code was scanned by a phone camera"	No phone session was recorded; the harness is headless Chrome	T6 (G-11)
6	"The system was tested by real users" / any usability result	No user acceptance study was conducted	T6 (G-06)
7	"The system was tested under production load"	No load test; the only timing is 22.67 ms for 50 pairs	T6
8	"The deployment is production-ready"	No monitoring, no backup schedule, no penetration test, demonstration data	T6
9	"Payments are made by the system"	SBMS records payments; it never transfers money	T3 (FR-37)
10	"QR verification confirms identity, eligibility or payment"	It confirms existence only, by design	T1 (10 tests)
11	"Certificates prove that funds were disbursed"	A certificate confirms an approval, and the PDF says so	T1 (17 tests)
12	"User acceptance criteria were met"	No acceptance criteria were recorded or measured	T6
13	"The supervisor's findings SBMS-01…18 were addressed"	Their wording could not be recovered; no ID can be closed	T6 (`SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` §4, §11)
14	"The Docker image was built and run here"	The engine does not run in this environment; the image is evidenced from CI	T6
15	"The audit log is immutable"	It is append-only by convention only	T3
16	"All 151 tests are branch-complete"	Several requirements are covered only at route level	T1 partial

5. Claims that can be made, with the wording that keeps them true

Claim	Wording to use	Basis
The register enforces a five-school constraint	"The API and the database both refuse a school outside the five approved names"	T1 (`students.test.js`; `students_school_supported_check`)

Claim	Wording to use	Basis
Screening cannot be bypassed by a failure	"Registration aborts with 503 rather than creating an unscreened record"	T1 plus T3
A human decides every duplicate and every payment	"Scores create review cases only; approval, payment and certification are role-gated human actions, and the database refuses an approval without verification"	T1 (`workflow.test.js` #2-#6) plus schema constraints
The queue cannot explode with registry size	"At most one case is created per registration under the `top-1` policy, with suppressed candidates counted and audited"	T1 (`duplicateAggregation.test.js`) plus T2
Public verification discloses nothing	"The student endpoint returns a boolean; the certificate endpoint returns authenticity facts without name, school, UPI or amount"	T1 (15 tests across two suites)
Session authorisation cannot be forged from the token	"The role is re-read from the database on every request and an inactive account is refused"	T1 (`roleAuthorization.test.js`) plus T3
Model provenance is auditable per case	"Each case stores the model version (artifact SHA-256), the threshold in force and the six feature values"	T1 plus T3
The synthetic model separates the designed classes	"On a synthetic holdout of 12 000 pairs the deployed artifact scores precision 0.9204, recall 0.8927 and F1 0.9063 at the 0.50 cut-off"	T2 (`docs/ml-decision-sheet-2026-10-30.md`) — always with the word *synthetic*
The model does not memorise	"The in-sample to holdout F1 gap is 1.33 points and grouped cross-validation reproduces the holdout within 0.0004"	T2 plus T1 (integrity tests)
A restore was demonstrated	"A dump was restored into an independent database and verified by table count, migration ledger and row digests, on a local disposable database"	T2
The deployment works end to end	"On 2026-10-31 the deployed instance answered health, readiness, authentication, role, QR, ML, CORS and persistence checks; on 2026-11-01 the registration form and identifier capture were verified live"	T2, dated
The business chain is gated correctly	"An 11-test suite drives the real application against a real database and proves the verification gate, the duplicate gate, the amount ceiling, the clearance evidence rule, the acknowledgement discrepancy rule and the audit sequence"	T1

Every one of these remains true only while the date and the revision stay attached to it.

6. Open decisions that belong to someone else

ID	Decision	Owner	Effect on the report
D1	Adjudication protocol and reviewer roles for real pairs	Programme owner / supervisor	Until answered, no accuracy claim is possible
D2	Acceptable miss rate	Programme owner	Determines whether 0.50 is defensible
D3	Reviewer capacity per week	Programme owner	Determines whether the queue is affordable
D4	Whether to introduce candidate blocking	Project owner (needs code change and tests)	FR-34 remains a stated gap
D5	Adjudication ownership and data-protection basis	Institution / owner	Blocks every real-data metric
SBMS-01… SBMS-18	The supervisor's eighteen findings	Supervisor	Their wording is unavailable, so none can be marked resolved (`SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` §4)
G-01	Workstation and device specification	Author	Chapter Five §5.2.1 stays incomplete without it
G-03/G-04/G-05	Figures and screenshots	Author	Chapters Four and Five cannot be illustrated without them
G-06	A position on user acceptance	Author	NFR-14 stays unverified

The report must present these as open, and must not fill any of them with a plausible-looking invention. That rule is the single most important one in this documentation set.

7. Synthetic data policy for every figure, table and screenshot

Rule	Reason
Every number in a results table must name the file that produced it	A reader must be able to re-run or re-read the source
Synthetic figures must carry the word *synthetic* in the caption	Otherwise a reader will read them as field results
No screenshot may contain a live demonstration record created during testing	The deployment holds `E2E Probe` rows; those are not synthetic seeds and must not appear (G-05)
`students.is_synthetic` distinguishes seeded from registered rows	The only reliable marker in the data
No beneficiary name, UPI, national identifier or phone may appear in any figure	Privacy, even for synthetic rows, sets the habit correctly
Where a figure cannot be produced, the placeholder checklist governs	`FIGURE_PLACEHOLDER_CHECKLIST.md` §4

8. Defects found during the project, and what they prove

Defect	Symptom	Root cause	Status	Why it belongs in the report
Payment clearance never worked	`PATCH /api/disbursements/:id/payment-status` returned 500 for every change	PostgreSQL `42P08`: one parameter reused for `VARCHAR(20)`, `VARCHAR(50)` and a text comparison	Fixed with explicit casts	A defect found by testing, not by inspection; it proves the value of the workflow suite
Stale QR result on a second scan	A valid code followed by an invalid one still read "Record found"	Verification ran once on mount; a hash-only change does not remount	Fixed with a `hashchange` listener	Shows why a browser harness was necessary
StrictMode regression	The first fix reported "invalid" in development	The effect re-read an already-stripped URL	Fixed by capturing the token during render and keying the effect on it	Shows that a fix must be re-tested in the development configuration
`risk_level: critical` branch unreachable	The UI could never style a case red	The API only emitted `review`	Resolved by publishing `priority` and `exact_identifier_matches`	Shows a contract mismatch between tiers
High-priority cut-off dead configuration	Every model case was `high`; the screen rendered "high 0.00"	Queue prioritised on the review cut-off and published a null threshold	Fixed	Shows a configuration value that was never exercised
Registration form omitted UPI and birth certificate	The critical exact-identifier rule could only fire for seeded rows	The form posted five fields; the API accepted eight	Fixed 2026-11-01 and verified live with 11 checks	Shows a real functional gap closed by evidence

Reporting these is not a weakness of the report; concealing them would be. Each one is dated, attributed to its evidence document and closed with the change that closed it.

9. How to keep this set accurate

Trigger	What must be updated
A test suite is re-run	`SBMS_TEST_EVIDENCE_CATALOGUE.md` §8 and any count quoted in the chapters
A migration is added or applied	`SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` §2-§4 and the readiness contract
A route is added or re-guarded	`SBMS_API_ENDPOINT_CATALOGUE.md` §3 and the security matrix §3
The model is retrained	`SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §4-§7 and every figure quoted from it
A screen is added	`SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md` §1 and the figure map §10

Trigger	What must be updated
The deployment changes	`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md` §2-§13
A gap closes	`DOCUMENTATION_GAP_REGISTER.md` and the requirement status in `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §6-§7
A new claim is wanted	This file §4 first: if it appears there, the claim cannot be made

10. Integrity statement

This documentation set was produced by reading the repository at `d6b5032`, the recorded run documents, and the test suites; no test suite was re-executed during this pass, and no code, migration, configuration or existing record was modified. Where the repository and a document disagree, the disagreement is stated rather than resolved in favour of the more convenient version. Where evidence is absent, the absence is reported. No supervisor finding, user result, production metric or accuracy figure has been invented, and none will be found in the chapters that cite this file.

CHAPTER 4 EVIDENCE PACKAGE: SYSTEM ANALYSIS AND DESIGN

> **What this file is.** The evidence a writer needs for Chapter 4, condensed to "what may be stated, on what basis, and where the full detail lives". It is the Chapter 4 entry point of the nine-file handover set. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.
>
> **Where the full detail lives.** Prose: `CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` (§4.1–§4.8). Architecture: `SBMS_SYSTEM_ARCHITECTURE_AND_MODULE_MAP.md`. Schema: `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md`. Interfaces: `SBMS_API_ENDPOINT_CATALOGUE.md`, `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md`. Controls: `SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md`. Figures: `DIAGRAM_SPECIFICATIONS.md`.

---

1. What Chapter 4 may assert, and on what basis

Assertion the chapter needs	Basis (code-level)	Status
Three-tier system: React SPA, Express API, PostgreSQL, Python ML service	`frontend/`, `backend/`, `ml-service/`, `docker-compose.yml`	VERIFIED AND IMPLEMENTED
Three roles, enforced server-side, role re-read per request	`backend/src/middleware/auth.js` and route guards	VERIFIED AND IMPLEMENTED
45 HTTP endpoints in ten families	`SBMS_API_ENDPOINT_CATALOGUE.md` §2–§4	VERIFIED AND IMPLEMENTED
13 tables, 14 forward migrations with a ledger and checksums	`backend/src/db/migrations/`, `backend/src/db/schema.sql`; schema doc §2–§3	VERIFIED AND IMPLEMENTED
Screening compares each new record with the whole registry, then scores pairs in the ML service	`backend/src/services/duplicateCheck.js`, `duplicateAggregation.js`, `mlDuplicateReview.js` (which spawns `ml-service/predict.py --server`)	VERIFIED AND IMPLEMENTED
The ML score is advisory: an administrator decides, with a recorded reason	`backend/src/controllers/duplicateFlags.controller.js`; `duplicate_flags` state machine	VERIFIED AND IMPLEMENTED
Payments are gated in the database as well as the API	schema doc §9 (constraints and triggers)	VERIFIED AND IMPLEMENTED
QR codes identify a student record only — never identity, eligibility or payment	`backend/src/controllers/verification.controller.js`; 10 route tests	VERIFIED AND IMPLEMENTED
Design is reproducible from specifications alone	24 figure specifications, each with implementation references	Specifications complete; drawings not produced

2. Requirement analysis: the numbers a writer must use

Group	Count	Verified	Partial	Deviated	Gaps
Functional requirements delivered (FR-01…FR-30)	30	24	6	0	0
Proposal requirements carried as deviations (FR-31…FR-33)	3	0	0	3	0
Functional gaps reported as gaps (FR-34…FR-37)	4	0	0	0	4
Non-functional requirements (NFR-01…NFR-21)	21	14	7	0	0

The six partial FRs are FR-12, FR-19, FR-25, FR-26, FR-28 and FR-29; the seven partial NFRs are NFR-01, NFR-13, NFR-14, NFR-16, NFR-18, NFR-19 and NFR-21. These are counted from the status cells in `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §6–§8, not maintained as a separate figure.

The deviation the chapter must state, not hide (G-21): the proposal named a Random Forest over four similarity features; the delivered classifier is XGBoost over six features (`ml-service/model/`). Three of the six inputs are the proposal's features; three are added signals. Chapter Four states this in the FR-31 row of §4.2.2 and again in §4.4.4, and Chapter Six repeats it in §6.2.4.

3. Analysis artefacts required, and what each must encode

Artefact	Must encode	Specification
System context (Figure 4.1)	SBMS as one process; four external actors; three external systems	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.3.1
Use case (Figure 4.2)	Three actors outside the boundary; 14 use cases; `<<include>>` screening	§4.3.2
Activity: registration and screening (Figure 4.3)	Four swimlanes; validation failure path; advisory-score branch	§4.3.3
Activity: review, approval, payment gate (Figure 4.4)	Three swimlanes; terminal states; reminder and escalation	§4.3.4
Class diagram (Figure 4.5)	13 classes with multiplicities and state enums	§4.3.5
Sequence: registration (Figure 4.6)	Synchronous API, subprocess ML call, write-then-flag ordering	§4.3.6
Sequence: duplicate decision (Figure 4.7)	Reveal-then-decide, audit rows, terminal outcome	§4.3.7
Sequence: approval to certificate (Figure 4.8)	Snapshot, token, PDF, revocation path	§4.3.8
Sequence: public verification (Figure 4.9)	Fragment token, rate limiter, existence-only response	§4.3.9
ER diagram (Figure 4.10)	Chen notation, all 13 tables, cardinalities, FK actions	§4.3.10
ML pipeline (Figure 4.11)	Offline lane (data → features → train → artifact) and online lane (request → score → advisory flag)	§4.3.11
Architecture (Figure 4.12)	Tiers, trust boundaries, the human decision gate drawn explicitly	§4.4.1

4. Design facts that must match the code

Design area	Fact	Evidence
Deployment	Frontend and API on Azure App Service; PostgreSQL Flexible Server; ML service containerised or run as a subprocess	`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md` §3–§7
Authentication	Short-lived JWT, bcrypt hashing, role claims re-validated per request	`backend/src/middleware/auth.js`
Validation	Zod schema per route, defined in each controller; five-school constraint from `backend/src/config/schools.js`; phone and identifier normalisation in `backend/src/services/studentRecordFormat.js`	`backend/src/controllers/`
Screening	Advisory lock per sweep; whole-registry comparison; `top-1` aggregation in `duplicateAggregation.js`; model version pinned per scan	`backend/src/services/`
State machines	Duplicate flag, award, disbursement and certificate lifecycles reject illegal transitions	schema doc §9
Privacy	Masked identifiers in the queue; explicit audited reveal	`backend/src/controllers/duplicateFlags.controller.js`
ML integration	Six features; 0.50 cut-off; score stored; ML failure degrades to manual review, never a silent pass	`ml-service/predict.py`; `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §5–§6

5. Interface design: what each screen figure must show

Twelve screen figures (4.13–4.24) are specified in §4.5.1–§4.5.8 and itemised in `FIGURE_PLACEHOLDER_CHECKLIST.md` §2. Each specification names the route, the audience, the data requested, the controls that must be visible and the states the screen must render, so a screenshot can be checked against the specification rather than the specification being reverse-engineered from a screenshot.

6. What Chapter 4 must not claim

- A number of users, records or deployments: no production usage exists, so no usage figure is supportable.
- That the ML model decides duplicates: the human gate is a design control and must be drawn as a boundary.
- That figures exist: they are specified only, and every placeholder says so.
- That a phone camera scanned a printed code: no such capture exists (G-11).

7. Open items affecting Chapter 4

Item	Effect	Owner
G-03/G-04/G-05 figures and screenshots	The interface section cannot be illustrated	Author
A-2 use-case table granularity	§4.3.2 may need more or fewer use-case tables	Author
A-3 screenshot provenance	Captions must say whether the instance is Azure or local	Author

`ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md` §3 carries the full gap list; `MASTER_EVIDENCE_REGISTER.md` carries the evidence IDs behind every row above.

8. Related files

`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` (the chapter), `DIAGRAM_SPECIFICATIONS.md` (figure register), `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` (schema), `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md` (screens).

CHAPTER 5 EVIDENCE PACKAGE: IMPLEMENTATION, TESTING AND EVALUATION

> **What this file is.** The Chapter 5 entry point of the nine-file handover set: the implementation and test facts a writer may assert, their evidence, and the exact numbers with dates. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.
>
> **Full detail:** `CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md` (§5.1–§5.7), `SBMS_TEST_EVIDENCE_CATALOGUE.md` (every artefact, command and date), `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` (features, artifact, metrics), `SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md` (environments, pipeline).

---

1. Implementation environment (evidence, not recollection)

Item	Value	Status
Operating system	Windows 11 (win32)	Recorded
Node.js / npm	v22.11.0 / 10.9.0	Recorded
Python	3.11.9 in a service virtual environment	Recorded
PostgreSQL	17.11 client and server	Recorded
Docker	CLI 29.9.2; engine **not running** here, image built by CI	Recorded
Browser automation	Headless Chrome over the DevTools Protocol from Node 22's `WebSocket`	Recorded
Hardware specification	**Not recorded**; logs show 7.4 GB RAM with about 0.5 GB free	**Blocking gap G-01**
Test database	Disposable `sbms_test` on loopback, role `sbms_test_runner`, guarded by `testDatabaseGuard.js`	Recorded

The hardware gap must be written as a gap. It is honest to say the workstation was not captured, that memory was observed at 7.4 GB with about 0.5 GB free during sweeps, and that one sweep completed 149 or 150 of 151 tests under memory pressure (G-02). It is not honest to invent a processor, memory size or storage type.

2. Test results: the only counts that may be quoted

Level	Command	Result	Date
Backend Jest, whole tree	`npx jest --runInBand tests`	**24 suites / 151 tests pass**	2026-11-01
Backend Jest, earlier generation	`npm test`	21 suites / 128 tests pass	2026-10-31
Backend native, in-process	`node --test` over seven files	**43/43 pass**	2026-10-31
Backend native, out-of-process	`node --test test/api.smoke.test.js`	**3/3 pass**	2026-10-31
Frontend production build	`npm.cmd --prefix frontend run build`	**exit 0**, 2 006 modules, 10.11 s	2026-11-01
ML service	`unittest discover -s ml-service/tests`	**33/33 pass**	2026-10-31
ML research helper	`unittest discover -s experiments/sbms_dataset`	**7/7 pass**	2026-10-31
Browser: registration and review	`node scripts/browser-smoke.mjs`	**23/23 pass**	2026-10-30
Browser: QR page	`node scripts/qr-smoke.mjs`	**9/9 pass**	2026-10-30
Browser: registration fields, live deployment	`node scripts/registration-fields-smoke.mjs`	**11/11 pass**	2026-11-01
Database restore	`pg_dump` then `pg_restore` into an independent database	Pass, digests compared	2026-10-31
Phone-camera QR scan	manual	**Not run**	—

Level	Command	Result	Date
Browser check of `/verify-certificate`	no harness exists	**Not run**	—
User acceptance	—	**Not conducted**	—
Load or scale test	—	**Not run**	—
Docker build in this environment	—	**Not run** (engine unavailable)	—

Two rules, both mandatory. A count is never quoted without its date, because the suite grew from 21/128 to 24/151 between 2026-10-31 and 2026-11-01. And counts are never summed across levels: 151 Jest tests, 46 native tests, 43 browser checks, 40 Python tests are four separate positions, not a total of 280.

`scripts/admin-reveal-smoke.mjs` exists but was **not** re-run (the seeded administrator password had drifted). It is listed as not run, not as passed and not as skipped.

3. Implementation by feature: what the code does

Feature	Implementation	Tests
Authentication and RBAC	Two sign-in endpoints, JWT, role re-read, user administration	4 + 6 + 1
Registration and validation	Zod schema, five-school constraint, phone and identifier normalisation	9
Duplicate screening	Advisory lock, whole-registry comparison, Python subprocess, `top-1` aggregation, provenance	15 + 13 + 8 + 6 + 1 + 4
Duplicate review	Masked queue, audited reveal, decision with reason, terminal states	9
Eligibility scoring	Five weighted factors, four categories, persisted result	Route and validation only
Award lifecycle	Submission, verification with evidence, approval and rejection with reasons	5
Payments and reconciliation	Issue, clearance with evidence, officer-assisted acknowledgement, discrepancy detection, exceptions	4 + 4 + 4
Student QR identification	32 random bytes, fragment token, existence-only endpoint, dedicated limiter	10 + 4
Award certificates	Snapshot row, reference, token, A4 landscape PDF, revocation, `not_current`	12 + 5
Dashboard, reports, audit, settings	Totals and trend, date-range PDF, audit trail, three cut-offs	6 (authorisation); content by inspection
Operations	Health, readiness, start-up guards, migration ledger	1 + 5 + migration unit tests

4. Machine-learning evaluation: the four numbers that are defensible

Item	Value	Basis
Model	XGBoost over six similarity features	`ml-service/model/`, `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §5
Live cut-off	0.50: precision 0.9204, recall 0.8927, F1 0.9063, miss 10.73%	Synthetic test partition; operating-point script output
Hard-negative false-positive rate at 0.50	7.72%	Same partition; the number to quote for "how often a wrong pair is flagged"
Independent per-pair baseline	0.35% of 2 000 synthetic unrelated pairs	`docs/ml-evaluation-baseline-2026-10.md`

The 7.72% and 0.35% answer **different questions** (one is the operating point on a constructed hard-negative set, the other a base rate on unrelated pairs). Quote both or neither; quoting one alone misleads. Ranking quality: ROC-AUC 0.9685, PR-AUC 0.9727. Every figure is synthetic, so no real-world accuracy claim is available.

The distinction the chapter must preserve: these are pair-level classification metrics on generated data. They are not beneficiary-level duplicate-detection performance and they do not survive contact with real records until

adjudicated labels exist.

Research-only artifact: `ml-service/model/research/rf_attachment_style_candidate.joblib` is a ten-feature Random Forest candidate. It is not the production model, and its metrics must not be mixed with the six-feature model's (§5.3.4, `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §11).

5. Defects: six with a fix and a test, others pointed to

#	Defect	Resolution	Validation
1	Duplicate sweep could double-flag under concurrency	Advisory lock around the sweep	Concurrency tests
2	Reveal endpoint leaked full identifiers before adjudication	Masked by default; reveal audited	Route tests
3	QR token in the path was logged	Token moved to the URL fragment	Route and browser tests
4	Certificate PDF ignored `not_current` awards	Snapshot row plus explicit state check	12 route tests
5	Readiness endpoint returned healthy with a stale queue	Queue freshness check added	Readiness tests
6	Payment recorded without verification state	Database-level gate in addition to the API check	Integration tests

The wider defect history — date-of-birth feature skew, registration 500, the deployed `/api/verify` 404, the queue-closes-on-open bug and the readiness false pass — is recorded in `docs/SBMS_PROJECT_HANDOVER_2026-10.md` §13.2–§13.4 and `docs/azure-deployment-log-2026-10-31.md` §7–§8. Chapter 5 §5.4.7 scopes its table to the six and points to these sources rather than implying completeness.

6. Evaluation: what the evidence supports

Objective (Chapter 1)	Extent achieved	Limitation that must be stated
(i) Investigate existing registration and tracking challenges	Achieved as literature and workflow analysis	No field study, no interviews recorded
(ii) Design architecture, schema and ML pipeline	Achieved and implemented	Classifier deviates from the proposal (G-21)
(iii) Implement registration, unique identifiers, QR and RBAC	Achieved	Physical QR scan not exercised (G-11)
(iv) Validate, test and evaluate	Partly achieved	No real data, no user acceptance, no load test

What the tests establish: the delivered behaviour matches the specified behaviour at route and workflow level, across 151 backend tests, 43 browser checks and 40 Python tests, with a reproducible command for each. What they do not establish: fitness for a real bursary office, accuracy on real records, usability, scale, or production assurance.

7. Limits of the evidence itself

Limit	Consequence for the chapter
Results are recorded from runs on one workstation, not from CI for every suite	The chapter states the command, environment and date for each count
Two test generations exist (21/128 and 24/151)	Only the dated, current generation is quoted; the older one is footnoted if mentioned at all
One sweep was incomplete under memory pressure	The stability caveat appears wherever the 151 figure appears
Browser checks depend on a seeded account; one harness was not re-run	The chapter says so rather than implying a full browser sweep
No user acceptance, no load test, no phone capture, no production assurance	Each is named as a gap, with the gap ID

8. Open items affecting Chapter 5

Item	Effect	Owner
G-01 hardware specification	§5.2.1 cannot be completed	Author
G-06 user-acceptance position	§5.5.6 stays empty; NFR-14 stays partial	Author
G-11 phone-camera scan	The identification mechanism rests on a headless-browser check	Author
G-02 149/150-of-151 instability	Every 151 figure needs the caveat	Author
G-25 two test-count generations	A count without a date is misleading	Author

9. Related files

`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md` (the chapter), `SBMS_TEST_EVIDENCE_CATALOGUE.md` (test inventory), `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` (ML evidence), `MASTER_EVIDENCE_REGISTER.md` (evidence IDs).

CHAPTER 6 EVIDENCE PACKAGE: CONCLUSIONS, RECOMMENDATIONS AND FUTURE WORK

> **What this file is.** The Chapter 6 entry point of the nine-file handover set: what each conclusion may rest on, the recommendations the evidence supports, and the future work that is genuinely future rather than unfinished. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.
>
> **Full detail:** `CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md` (§6.1–§6.5), `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §1 and §5, `SBMS_COMPREHENSIVE_CHAPTERS_4_5_6_INFORMATION_REPORT.md` §20–§24.

---

1. The objectives, and where their evidence sits

The four specific objectives are reproduced as working statements below. **The exact Chapter 1 sentences are not in this repository** — Chapters 1–3 exist only outside it — so the author must paste the originals from the report and confirm they match these statements before submission. Chapter 6 must answer the originals, not this file.

#	Working statement	Implemented functionality	Evidence	Extent	Limitation
(i)	Investigate the challenges of manual beneficiary registration and tracking	Requirement and workflow analysis, problem framing	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.1–§4.2; proposal §1	Achieved as analysis	No field study or interview record exists
(ii)	Design the architecture, database schema and ML pipeline, incorporating a Random Forest classifier	Three-tier architecture, 13 tables, 14 migrations, six-feature XGBoost pipeline	§4.4, §4.6, §4.7; `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md`	Achieved with a stated deviation	Classifier is XGBoost, not Random Forest (G-21)
(iii)	Implement registration, unique identifiers, QR identification and role-based access control	Registration with validation, generated identifiers, existence-only QR verification, three-role RBAC	`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md` §5.3.2–§5.3.8	Achieved	Physical QR scan never exercised (G-11)
(iv)	Validate, test and evaluate the system's effectiveness	151 backend tests, 46 native tests, 43 browser checks, 40 Python tests, ML evaluation, restore rehearsal	§5.4; `SBMS_TEST_EVIDENCE_CATALOGUE.md`	Partly achieved	No real data, no user acceptance, no load test

2. What the conclusions may and may not say

Conclusion	Supportable wording	Basis
The system is built and behaves as specified	"Route-level and workflow-level behaviour matches the specification across 151 backend tests, 43 browser checks and 40 Python tests, each with its date in §5.4."	Test artefacts with dates
Duplicate detection is functional	"The pipeline scores each candidate pair against the whole registry and surfaces the result to an administrator, who decides."	Code and tests
Duplicate detection is accurate	**Not supportable.** No adjudicated labels; synthetic data only	`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §4
The ML component adds value	"It reduces the reviewer's candidate set to a ranked top-1 per record and stores provenance per flag." Value against a manual baseline is unmeasured	Design evidence only
The system improves on manual processes	**Only as a design argument.** No before-and-after measurement and no users	Gap G-06
The payment chain is controlled	"Payment state is gated in the database as well as the API, and every state change is audited."	Schema constraints, audit tests
The system is production-ready	**Not supportable.** Demonstration instance; no monitoring, backup schedule or penetration test	§5.6.6

3. Recommendations the evidence supports

#	Problem or opportunity	Evidence	Action	Benefit	Implementation consideration
R-1	Labels are synthetic, so accuracy is unknown	ML doc §4, §10	Build an adjudicated ground-truth set from the decisions already stored in `duplicate_review_events`	Real precision, recall and a defensible cut-off	Needs data-sharing permission and a labelling protocol
R-2	Whole-registry comparison is quadratic	Screening service; 22.67 ms at 50 pairs	Add blocking or candidate generation before scoring	Keeps review volume viable as records grow	Blocks on the ML service, not the database
R-3	The score is uncalibrated and unexplained	ML doc §11	Calibrate probabilities and expose the top contributing features per flag	A reviewer can judge a flag instead of trusting it	Feature names must stay stable across retraining
R-4	Flag volume may exceed review capacity	Screening design	Set an explicit flag budget per cycle and measure reviewer throughput	Prevents an unusable queue	Needs an owner decision on the cost of a missed duplicate
R-5	Operation and assurance are thin	`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md` §9	Add monitoring, a backup schedule and a scheduled restore drill	Reduces operational risk	Costs money; Azure monitoring already available
R-6	Identity assurance rests on a printed token	QR design; G-11	Define a re-issue and revocation policy and test one scan on a phone	Closes a known weak point	Policy decision belongs to the programme owner

Each recommendation is tied to a stakeholder in Chapter 6 §6.3 and to the limitation that motivates it.

4. Future work: split by kind, not by convenience

Category	Items	Why it is future work, not a gap
Existing functionality	Blocking, calibration, score explanation, flag budget	The current scope delivers screening and adjudication; these improve it
Additional capability	Multi-registry operation, shared identity across programmes, confirmation of receipt	Never in the approved scope
ML	Real-data evaluation, retraining pipeline, drift monitoring, cost-sensitive threshold selection	Blocked by the absence of labelled data
Security	Penetration test, secret rotation, key management, rate-limit tuning	Assurance work, outside the project's testing scope
Scale and operations	Load testing, monitoring, backup schedule, high availability	Requires a production budget
Research	Comparing blocking strategies; reviewer efficiency against a manual baseline	Needs a field setting

Not to be listed here: the functional gaps FR-34…FR-37 and the deviations FR-31…FR-33. Those were in scope and are reported as missed or altered in §6.2.9.

5. Boundaries the conclusion must keep

Boundary	What it excludes
Data	Every metric is synthetic; nothing is claimed about Kenyan bursary records
Users	No student, officer, guardian or school participated
Scale	Nothing above 50 candidate pairs was timed
Device	Chrome only; no phone camera; no physical printer
Operation	Demonstration instance; no monitoring, backup schedule or high availability
Security	Control inventory, not an assurance statement

Boundary	What it excludes
Governance	Five owner decisions unanswered; no supervisor finding can be closed without its wording

6. Open items affecting Chapter 6

Item	Effect	Owner
G-21 classifier deviation	Must be stated in §6.2.4, not omitted	Author
G-06 user acceptance	§6.4.9 argues future work from an absence, which must be admitted	Author
Exact Chapter 1 objective wording	§6.2.3–§6.2.6 must answer the originals	Author

7. Related files

`CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md` (the chapter),
`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` (objective → requirement → test → conclusion),
`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` (claim limits).

MASTER EVIDENCE REGISTER

> **What this file is.** The central register: one ID per substantive claim, with the exact repository location that supports it, the verification status, and the limitation that keeps the claim honest. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.
>
> **Status vocabulary** (the six categories the brief requires): **IMPL** = VERIFIED AND IMPLEMENTED; **NOT IMPL** = VERIFIED BUT NOT IMPLEMENTED; **PASS** = TESTED AND PASSED; **FAIL** = TESTED AND FAILED; **NOT TESTED**; **UNKNOWN** = UNKNOWN / REQUIRES CONFIRMATION.
>
> **Evidence hierarchy applied:** code and schema first, then executed tests, then reproducible runs, then deployment logs, then version-controlled documentation, then drafts, and never a draft as proof of implementation.

---

1. System structure and data

ID	Claim	Evidence	Status	Limitation
EV-01	The API is an Express application with 14 controllers and 12 route modules	`backend/src/index.js`; `backend/src/routes/*.js`; `backend/src/controllers/*.js`	IMPL	—
EV-02	The frontend is a React SPA with 14 page components, 3 shared components, an auth context and a hash-token hook	`frontend/src/pages/`, `frontend/src/components/`, `frontend/src/context/AuthContext.jsx`, `frontend/src/hooks/useHashToken.js`	IMPL	—
EV-03	The ML service is a separate Python package invoked as a subprocess from Node	`ml-service/predict.py --server`; `backend/src/services/duplicateCheck.js:262`	IMPL	Spawned per process; not containerised in the local path
EV-04	PostgreSQL holds 13 application tables	`backend/src/db/schema.sql`; `backend/src/db/migrations/`; schema doc §2	IMPL	Table count excludes the migration ledger
EV-05	Schema changes are forward-only, ledger-tracked and checksum-verified	14 forward migrations plus one `.down.sql` in `backend/src/db/migrations/`; `backend/src/services/databaseStartup.js`	IMPL	Not replayed against a fresh cluster in this pass
EV-06	Database readiness is checked before traffic is accepted	`backend/src/services/databaseReadiness.js`	IMPL	Readiness false-pass was a defect, now fixed (see §3)
EV-07	Payment state is gated by database constraints as well as API checks	schema doc §9; `disbursements` constraints	IMPL	Triggers inspected, not fuzzed

2. Behaviour

ID	Claim	Evidence	Status	Limitation
EV-08	Two sign-in endpoints issue a JWT; bcrypt hashes passwords	`backend/src/controllers/auth.controller.js`; `backend/src/config/jwtSecret.js`	IMPL	bcrypt cost not benchmarked on the host (NFR-02)
EV-09	Role is re-read from the database on each request, not trusted from the token	`backend/src/middleware/auth.js`; `requireRole.js`	IMPL	—
EV-10	Login and verification endpoints are rate limited	`loginRateLimit.js`, `verificationRateLimit.js`, `certificateRateLimit.js`	IMPL	Limits are per-process, so horizontal scale weakens them
EV-11	Registration validates fields with Zod inside the controller	`backend/src/controllers/students.controller.js`	IMPL	—
EV-12	Only five configured schools are accepted	`backend/src/config/schools.js`	IMPL	The five are a project assumption, not a ministry list

ID	Claim	Evidence	Status	Limitation
EV-13	Screening compares the submitted record with every registry record	`students.controller.js:75`; `mlDuplicateReview.js:84` (`scorePairs`)	IMPL	Quadratic; 22.67 ms measured at 50 pairs only
EV-14	Pair scores are aggregated to one candidate per record (`top-1`)	`backend/src/services/duplicateAggregation.js`	IMPL	Aggregation choice is a design decision, not validated against alternatives
EV-15	Model version is pinned for the duration of a scan	`duplicateFlags.controller.js:188–196`	IMPL	Guard raises on change; no retry logic
EV-16	ML failure degrades to manual review rather than a silent pass	`backend/src/services/duplicateCheck.js`	IMPL	Degradation path tested at route level only
EV-17	The duplicate queue masks identifiers; reveal is explicit and audited	`duplicateFlags.controller.js`; `duplicate_review_events`	IMPL	—
EV-18	Decisions require a reason and move the case to a terminal state	`duplicateFlags.controller.js`; state machine in schema doc §9	IMPL	—
EV-19	Eligibility is computed from five weighted factors into four categories	`backend/src/services/eligibility.js`, `bursaryRules.js`	IMPL	Arithmetic partially tested (FR-12 partial)
EV-20	Awards require verification evidence before approval	`bursaryAwards.controller.js`; `bursary_award_verification_events`	IMPL	—
EV-21	Disbursements support issue, clearance, officer-assisted acknowledgement and discrepancy detection	`disbursements.controller.js`; `disbursement_reconciliation_events`	IMPL	—
EV-22	QR tokens are 32 random bytes carried in the URL fragment	`verification.controller.js`; `frontend/src/hooks/useHashToken.js`	IMPL	Fragment handling stops token leakage into logs; it does not make the token secret to the holder
EV-23	Public verification asserts existence only	`verification.controller.js`; `verificationRoute.test.js`	PASS	Confirms existence, never identity, eligibility or payment
EV-24	Certificates store a snapshot, carry a reference and a token, and produce an A4 landscape PDF	`certificates.controller.js`; `certificateDocument.js`	IMPL	PDF content asserted at route level, not visually verified (FR-26 partial)
EV-25	Certificate state `not_current` is honoured when an award changes	`certificates.controller.js`; `bursary_certificates`	IMPL	—
EV-26	Audit rows are written for administrative and reveal actions	`backend/src/services/auditLog.js`	IMPL	Append-only by convention, not by constraint (NFR-21 partial)

3. Tests, defects and deployment

ID	Claim	Evidence	Status	Limitation
EV-27	The Jest suite passes 24 suites / 151 tests	`npx jest --runInBand tests`, 2026-11-01	PASS	One earlier sweep returned 149 or 150 of 151 under memory pressure (G-02)
EV-28	The native suite passes 43/43 in-process and 3/3 out-of-process	`node --test`, 2026-10-31	PASS	Separate layer; not additive with EV-27
EV-29	33 ML-service tests and 7 research-helper tests pass	`unittest discover`, 2026-10-31	PASS	Unit-level only
EV-30	Browser checks pass 23/23, 9/9 and 11/11	`scripts/browser-smoke.mjs`, `qr-smoke.mjs`, `registration-fields-smoke.mjs`, 2026-10-30 and 2026-11-01	PASS	A fourth harness, `admin-reveal-smoke.mjs`, was not re-run (admin password drifted) and is recorded as not run
EV-31	The frontend builds for production	`npm --prefix frontend run build`, exit 0, 2026-11-01	PASS	Build only; no bundle-size budget asserted

ID	Claim	Evidence	Status	Limitation
EV-32	A backup can be restored into an independent database with matching digests	`pg_dump` / `pg_restore`, 2026-10-31	PASS	One rehearsal on synthetic data (NFR-16 partial)
EV-33	Six defects were found in testing and fixed, each with a validating test	`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md` §5.4.7	PASS	Table is scoped to six; the wider history is in the handover and Azure logs
EV-34	A deployed instance answers health and registration-field checks	`docs/azure-deployment-log-2026-10-31.md`; `registration-fields-smoke.mjs`	PASS	Demonstration instance, not production
EV-35	The ML operating point is precision 0.9204, recall 0.8927, F1 0.9063 at a 0.50 cut-off, with a 7.72% hard-negative false-positive rate	`ml-service/evaluate_operating_point.py` output, transcribed in `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §7	PASS	**Synthetic data only**; pair-level, not beneficiary-level
EV-36	The independent per-pair baseline is 0.35% on 2 000 synthetic unrelated pairs	`docs/ml-evaluation-baseline-2026-10.md`	PASS	Answers a different question from EV-35; both must be quoted together
EV-37	The production model is XGBoost over six features, not the proposal's Random Forest over four	`ml-service/model/`; `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §5	PASS	Deviation must be stated in Chapters 4, 5 and 6 (G-21)
EV-38	A ten-feature Random Forest exists only as a research candidate	`ml-service/model/research/rf_attachment_style_candidate.joblib`	IMPL	Must never be presented as the production model
EV-39	Screening latency is 22.67 ms at 50 candidate pairs	`performance.test.js`	PASS	Nothing measured above 50 pairs; no load test exists
EV-40	The Docker image is built by CI, not locally	`.github/workflows/`; `Dockerfile`	IMPL	Engine unavailable on the development host, so no local build was observed

4. Claims with no supporting evidence, recorded as such

ID	Claim	Why it has no evidence	Status
EV-41	Real-world duplicate-detection accuracy	No adjudicated labels exist	NOT TESTED
EV-42	User acceptance and usability	No study, no participants, no consent record	NOT TESTED
EV-43	Behaviour under load or at scale	No load test was run	NOT TESTED
EV-44	A phone camera scanning a printed QR code	No device capture exists	NOT TESTED
EV-45	Penetration-test resistance	No assessment was commissioned	NOT TESTED
EV-46	Production uptime, monitoring or backup schedule	Demonstration instance only	NOT TESTED
EV-47	The exact Chapter 1 objective wording	Chapters 1–3 are not in this repository	UNKNOWN
EV-48	The wording of supervisor findings SBMS-01…SBMS-18	Unrecoverable; `SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` §4 records the loss	UNKNOWN
EV-49	Whether the deployment's ML path is containerised or subprocess-spawned at runtime	Azure configuration shows the app service; the local path spawns Python	UNKNOWN (G-22)
EV-50	Whether `docs/verification.md` and `docs/uat-test-plan.md` remain part of the evidence base	Both conflict with the implementation; the decision is the author's	UNKNOWN (G-07, G-08)
EV-51	Whether the hardware specification can be reconstructed	Never recorded	UNKNOWN (G-01)

5. How to cite an ID

In a chapter: "(EV-27, `MASTER_EVIDENCE_REGISTER.md`)" or, where a section is more useful, the section reference the row already carries. Two rules:

1. **An ID never travels alone.** Any claim quoted from this register must keep the limitation column with it; the limitation is the part that makes the claim defensible.
2. **No new ID without a location.** If a sentence in a draft cannot be tied to a row here, either the sentence is wrong or a row is missing; it must not be written and left unsupported.

---

6. Related files

`EVIDENCE_AND_VERIFICATION_REPORT.md` §2–§6 (claim-level index in prose), `SBMS_TEST_EVIDENCE_CATALOGUE.md` §2 (every test artefact with date and command), `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §4 (the forbidden claims), `ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md` §3 (the same gaps in academic-coverage form).

CROSS-CHAPTER CONSISTENCY AND TRACEABILITY

> **What this file is.** The required consistency matrix between the earlier chapters (One to Three) and the evidence, the traceability chains that connect them to Chapters Four to Six, and the contradictions that need the author's decision. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.
>
> **Limitation on the left-hand column.** Chapters 1–3 are **not in this repository**. Their content is represented here by the approved proposal (`Project Proposal- 169613.pdf`) as transcribed in `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §2 and by the cross-chapter checks already recorded in `EVIDENCE_AND_VERIFICATION_REPORT.md` §9. Any row marked **NOT INDEPENDENTLY VERIFIABLE** means the original wording could not be re-read here, not that it is wrong.

---

1. Consistency matrix

Element	Original documentation	Verified implementation / evidence	Consistency status
Problem statement	Manual registration and tracking of beneficiaries is slow, error-prone and permits duplicate beneficiaries	Registration, screening, adjudication and payment-tracking modules all exist and are tested; the duplicate problem is addressed within a single registry	**CONSISTENT** — with the scope limit that cross-registry duplicates are out of scope (§1 of the catalogue)
General objective	Digitise registration, identification, verification and bursary tracking, with ML-assisted duplicate detection under human review	Three-tier system, 45 endpoints, 13 tables, advisory ML score, mandatory administrator decision	**CONSISTENT**
Specific objectives (i)–(iv)	Investigate challenges; design architecture, schema and ML pipeline with a Random Forest; implement registration, identifiers, QR and RBAC; validate and evaluate	Objective (i) as analysis; (ii) delivered with **XGBoost over six features**; (iii) delivered; (iv) partly delivered	**DRIFT on (ii)** — classifier altered; **PARTIAL on (iv)** — no real data, no UAT, no load test
Project scope	One registry, three roles, QR identification, awards, payments, certificates, reports	All present; six in-scope items remain gaps (FR-34…FR-37) and three proposal items are carried as deviations (FR-31…FR-33)	**CONSISTENT with declared exceptions** — every exception is numbered, none is silent
Functional requirements	Proposal §1.3 feature list, expanded into 37 numbered requirements in this package	30 delivered (24 verified, 6 partial), 3 deviations, 4 gaps	**CONSISTENT** — the numbering is this package's, and it maps back in `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §6
Non-functional requirements	Performance, security, usability, maintainability, availability expectations	21 numbered NFRs; 14 verified, 7 partial; no load test, no usability study	**CONSISTENT with 7 partials** — the partials are named, not averaged away
Methodology	Design Science Research with Agile delivery (proposal §3.2–§3.3)	Artefact-centred build with iterative test-driven increments, evidenced by commit history and test growth (21/128 → 24/151)	**CONSISTENT** — the methods chapter's claim is supported by the build record, not contradicted by it
Technology choices	React frontend, Node/Express API, PostgreSQL, Python ML, Azure deployment	Exactly these (`frontend/`, `backend/`, `ml-service/`, `docs/deployment.md`, Azure logs)	**CONSISTENT**
ML methodology	Random Forest over four features (`jaro_winkler`, `dob_match`, `school_match`, `guardian_match`)	XGBoost over six features: the four above **minus** guardian-only framing plus `upi_match` and `birth_certificate_match`; the Random Forest survives only as a research candidate	**DRIFT** — must be stated in Chapters 4, 5 and 6 (G-21)

1.1 Where the ML row needs care

The delivered feature set is `jaro_winkler`, `dob_match`, `school_match`, `guardian_match`, `upi_match`, `birth_certificate_match` — six features, of which three are the proposal's, one is a reformulation (`guardian_match`

now covers two guardian fields), and two are new identity signals. The research artifact `ml-service/model/research/rf_attachment_style_candidate.joblib` is a ten-feature Random Forest and is **not** the production model. A writer who mixes the two produces a contradiction that a supervisor can find in one page.

---

2. Traceability chains

Chain	Link 1 (Chapter 1–3)	Link 2 (Chapter 4)	Link 3 (Chapter 5)	Link 4 (Chapter 6)
Objective (i)	Proposal §1.1–§1.2, problem and challenges	§4.1–§4.2 requirement analysis	§5.5.2 evaluation of the objective	§6.2.3
Objective (ii)	Proposal §1.3.1 ii, §3.2.3	§4.4, §4.6, §4.7, §4.3.11	§5.3.1–§5.3.4	§6.2.4
Objective (iii)	Proposal §1.3.1 iii–iv	§4.2.2 (FR-01…FR-30), §4.4.2–§4.4.3	§5.3.2–§5.3.8, §5.4.3	§6.2.5
Objective (iv)	Proposal §1.3.1 v	§4.2.3 (NFR-01…NFR-21)	§5.4, §5.5, §5.6	§6.2.6–§6.2.7
Requirements	Proposal feature list	37 FRs and 21 NFRs, numbered in this package	Test coverage per requirement (`SBMS_TEST_EVIDENCE_CATALOGUE.md` §3–§6)	§6.2.7 and §6.2.9
Contribution	Proposal §1.4 (expected contribution)	§4.4.4 human-in-the-loop design	§5.5.5 ML evaluation and its limits	§6.2.8

`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` carries the same chains at requirement granularity, including the NFR table and the per-suite coverage matrix; this section is the chapter-level view.

---

3. Contradictions and drift requiring the author's confirmation

#	Item	What contradicts what	What the author must do
C-1	Classifier	Proposal: Random Forest, four features. Delivered: XGBoost, six features	Confirm the wording of the deviation in Chapters 4, 5 and 6 (already drafted) and in Chapter 3 if that chapter names the classifier
C-2	Deployment architecture	Proposal/Chapter 3 describe a containerised ML service; the delivered path also runs the model as a subprocess inside the API host	State both paths, and which one the Azure deployment uses (G-22)
C-3	`docs/verification.md`	Claims no controller calls the predictor; `students.controller.js:75` calls `scorePairs`	Mark the file superseded, or exclude it from the evidence base (G-08)
C-4	`docs/uat-test-plan.md`	Reports five `PASS` results with no participants, and one sentence that contradicts the implementation	Reclassify as an author walkthrough, or run a study (G-06, G-07)
C-5	Test totals	`docs/FINAL_TEST_REPORT.md` contains both 21/128 (2026-10-31) and 24/151 (2026-11-01)	Quote counts with their dates only (G-25)
C-6	Objectives wording	The exact Chapter 1 sentences could not be re-read here	Paste the originals into Chapter 6 and check §6.2.3–§6.2.6 answer them verbatim, not by paraphrase

No contradiction was resolved silently: each of the six is either stated in the drafts as a deviation, excluded from the evidence base, or listed here for decision.

---

4. Naming consistency used across the package

Name used	Meaning	Do not substitute
`admin`, `field_officer`	The two role values in the code and database	Administrator/Field Officer only in prose, never as identifiers
`students`, `duplicate_flags`, `duplicate_review_events`	Tables	Never `beneficiaries` for the primary table

Name used	Meaning	Do not substitute
`bursary_awards`, `bursary_award_verification_events`	Award lifecycle	Never `awards` alone in a schema reference
`disbursements`, `disbursement_reconciliation_events`	Payment lifecycle	Never `payments` for the table
`bursary_certificates`	Certificate snapshot	Never `certificates` for the table
`audit_logs`, `reports`	Audit and reporting	—
`beneficiaries` (legacy)	Five legacy endpoints, kept for compatibility	Not part of the current design narrative

---

5. Related files

`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` (requirement-level traceability), `EVIDENCE_AND_VERIFICATION_REPORT.md` §9 (the checks behind the matrix), `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §2 and §6 (proposal transcription and the FR register), `ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md` §4 (the same contradictions in gap form).

DIAGRAM SPECIFICATIONS

> **What this file is.** The consolidated register of every diagram the three chapters require. **No diagram is drawn here, deliberately:** a diagram invented for a report rather than derived from the implementation misrepresents the system. Each row fixes the nine attributes the academic guide asks for — number and title, purpose, actors/entities, relationships, flow direction, boundaries, labels and decision points, implementation reference, and how it is to be drawn — and points to the prose specification that carries the detail.
>
> Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.

---

1. How to read the register

Column	Meaning
No. / Title	Figure number as used in the chapters; do not renumber
Type	UML or structured notation to draw in
Elements	Actors, classes, tables or components that must appear
Flow / relationships	The connections a reader must be able to follow
Boundary	What the figure must show as inside or outside the system
Decision points / labels	Branches and edge labels that must be visible
Implementation reference	Where the drawn content is proved by code
Detail	Section holding the full prose specification

Status of every figure: SPECIFIED, NOT DRAWN. Twelve of them (4.13–4.24) are screenshots and cannot be produced from specifications at all; they are marked SCREENSHOT REQUIRED and tracked as gaps G-03/G-04.

---

2. Analysis diagrams (Chapter 4 §4.3)

No. / Title	Type	Elements	Flow / relationships	Boundary	Decision points	Implementation reference	Detail
Fig 4.1 System context	Context (Yourdon)	SBMS process; Field Officer, Administrator, Student/guardian, Public verifier; e-mail, Azure hosting, PostgreSQL	Data and control crossing the boundary both ways	SBMS as a single rounded process	—	`backend/src/routes/`, `docs/deployment.md`	§4.3.1
Fig 4.2 Use case	UML use case	3 actors; 14 use cases; `<<include>>` screening	Actor to use case; include arrows	Actors outside a named rectangle	Screening include on registration	route inventory	§4.3.2
Fig 4.3 Activity: registration and screening	UML activity, 4 swimlanes	Officer, API, ML service, Database	Validate → persist → screen → flag → confirm	Swimlanes per tier	Validation failure; ML unavailable; duplicate above cut-off	`students.controller.js`, `mlDuplicateReview.js`	§4.3.3
Fig 4.4 Activity: review, approval, payment	UML activity, 3 swimlanes	Administrator, Field Officer, Database	Queue → reveal → decide → approve → verify → pay → reconcile	Per-tier swimlanes	Reject; duplicate-confirmed; verification rejected; discrepancy	`duplicateFlags.controller.js`, `bursaryAwards.controller.js`, `disbursements.controller.js`	§4.3.4

No. / Title	Type	Elements	Flow / relationships	Boundary	Decision points	Implementation reference	Detail
Fig 4.5 Class	UML class	13 classes incl. `Student`, `DuplicateFlag`, `BursaryAward`, `Disbursement`, `BursaryCertificate`, `AuditLog`	Associations with multiplicities; state enums as attributes	—	—	`backend/src/db/schema.sql`, `backend/src/db/migrations/`	§4.3.5
Fig 4.6 Sequence: registration	UML sequence	Officer browser, API, validator, ML client, Python model, DB	Synchronous calls; ML call inside the request	API as mediator	ML timeout path	`students.controller.js`, `mlDuplicateReview.js`	§4.3.6
Fig 4.7 Sequence: duplicate decision	UML sequence	Administrator, API, DB, audit	Fetch queue → reveal (audited) → decide → write terminal state	—	Confirm / dismiss	`duplicateFlags.controller.js`	§4.3.7
Fig 4.8 Sequence: approval to certificate	UML sequence	Field Officer, Administrator, API, PDF generator, storage	Approve → verify → pay → issue certificate → revoke path	—	`not_current` outcome	`certificates.controller.js`	§4.3.8
Fig 4.9 Sequence: public verification	UML sequence	Phone browser, API, limiter, DB	Token in fragment → API lookup → existence-only response	No authenticated session	Token invalid; rate limited; revoked	`verification.controller.js`	§4.3.9
Fig 4.10 Entity-relationship	Chen	13 entities; key attributes; relationships	1:N, 1:1, M:N resolutions; FK actions	—	—	migrations; schema doc §2–§9	§4.3.10
Fig 4.11 ML pipeline	Two-lane flow	Offline: data → features → split → train → artifact. Online: request → features → score → flag → human	Left-to-right per lane; artifact bridging the lanes	Model boundary vs application	Cut-off 0.50; ML unavailable	`ml-service/`, `train_*.py`	§4.3.11
Fig 4.12 System architecture	Tiered block diagram	Browser SPA; App Service API; PostgreSQL; ML service; file storage	HTTPS between tiers; subprocess or HTTP to model	Trust boundaries drawn	—	`docker-compose.yml`, deployment guide §3	§4.4.1

3. Screen figures (Chapter 4 §4.5) — screenshot required

Each of these is a capture of the running application against **synthetic data only**. The specification states the route, audience, data requested, controls and states, so the capture can be checked rather than posed.

No. / Title	Route	Must show	Implementation reference	Detail
Fig 4.13 Role-specific sign-in	`/login`, `/admin/login`, `/field-officer/login`	SBMS identity block, e-mail and password fields, role switch, error state	`frontend/src/pages/Login.jsx`	§4.5.1
Fig 4.14 Registry list (field officer)	`/students`	Header, role badge, search, table of synthetic students, pagination	`Students.jsx`	§4.5.2
Fig 4.15 Eligibility assessment	`/students/:id/eligibility`	Five inputs, computed category, persisted result	`Eligibility.jsx`	§4.5.2
Fig 4.16 Registration form	`/students/new`	Two-column name fields, identifier fields, five-school selector, validation messages	`StudentRegistration.jsx`	§4.5.3

No. / Title	Route	Must show	Implementation reference	Detail
Fig 4.17 Registration confirmation with screening outcome	post-register view	QR code, generated identifier, screening outcome banner	`StudentRegistration.jsx`, `students.controller.js`	§4.5.3
Fig 4.18 Administrator dashboard	`/dashboard`	The counters the endpoint returns, trend, recent activity	`AdminDashboard.jsx`, `dashboard` route	§4.5.4
Fig 4.19 Duplicate review queue	`/duplicate-review`	Two records per case, masked identifiers, score, reveal control, decision buttons	`DuplicateReview.jsx`	§4.5.5
Fig 4.20 Awards and disbursements	`/disbursements`	Awards table, verification state, payment state, exception panel, certificate action	`Disbursements.jsx`	§4.5.6
Fig 4.21 Administrative support screens	`/reports`, `/audit`, `/users`, `/settings`	Four panels: date-range export, audit rows, user administration, three cut-offs	`Reports.jsx`, `AuditLogs.jsx`, `UserManagement.jsx`, `Settings.jsx`	§4.5.7
Fig 4.22 Student-record verification	`/verify`	Minimal layout, single input, existence-only result	`Verification.jsx`	§4.5.8
Fig 4.23 Certificate verification	`/verify-certificate`	Four outcome states: verified, revoked, not current, unknown	`CertificateVerification.jsx`	§4.5.8
Fig 4.24 Physical deployment architecture	—	Azure resources, network path, TLS, database, storage, ML runtime	`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md` §3–§7	§4.7

4. Drawing rules that keep the figures honest

1. **Derive, do not decorate.** Every element must be traceable to a file, route or migration listed in the *Implementation reference* column. If an element cannot be traced, it does not belong in the figure.
2. **Draw the human gate.** Figures 4.3, 4.4, 4.7, 4.11 and 4.12 must show the administrator's decision as a step and as a boundary. A diagram that runs model output straight into a state change misrepresents the design.
3. **Keep identifiers as they are.** `field_officer`, `duplicate_flags`, `bursary_certificates` — the diagram must use the code's names, or the figure will not match Chapters Four and Five.
4. **Show failure paths.** ML unavailable, validation failure, rate limit and revoked token are part of the design; omitting them makes the system look more certain than it is.
5. **Captions state provenance.** Screenshots carry the instance (Azure or local), the date and the fact that the data is synthetic. A caption that omits this invites the reader to assume production data.
6. **No invented actors.** There are three roles and one public verifier. A fourth actor in a figure would need a fourth role in the code, and there is none.
7. **One numbering scheme.** Figures are 4.1…4.24 as fixed above; do not renumber when assembling the document, and do not reuse a number for a different figure.

5. Coverage of the guide's diagram expectations

Guide expectation	Figures	Status
Context diagram	4.1	Specified
Use case diagram and use case descriptions	4.2 plus §4.3.2 tables	Specified
Activity diagrams for principal workflows	4.3, 4.4	Specified
Sequence diagrams	4.6, 4.7, 4.8, 4.9	Specified
Class diagram	4.5	Specified
Entity-relationship diagram	4.10	Specified
Architecture diagram	4.12	Specified

Guide expectation	Figures	Status
Deployment diagram	4.24	Specified
System interface / screen designs	4.13–4.23	**Screenshots required** (G-04)
ML pipeline (project-specific)	4.11	Specified

---

6. Related files

`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.3, §4.5, §4.7 (full prose specifications), `FIGURE_PLACEHOLDER_CHECKLIST.md` (capture checklist and pre-insertion checks), `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md` (screens and workflows behind the figures).

FIGURE PLACEHOLDER CHECKLIST

Purpose. Chapter 4 specifies twenty-four figures but contains no drawings: each position is marked `[Insert Figure n: … here]` and followed by a written specification of what the figure must show. This checklist is the working list for the drawing and screenshot pass, so that no figure is produced from memory and none contradicts the implementation. It adds no new requirements to Chapter 4.

Rules that apply to every figure

1. Capture or draw against the revision documented in `EVIDENCE_AND_VERIFICATION_REPORT.md` §1 (`fix/sbms-completion-work` @ `d6b5032`).
2. Screenshots use **synthetic data only** and must show no real beneficiary record (gap G-05). The production database currently holds two `E2E Probe` demonstration rows; remove them first (gap G-13).
3. Never show a password, JWT, `.env` value, connection string, database host or account administrator e-mail.
4. Use one viewport size for all screen captures and one export setting for all diagrams, so the figures look like one document (suggest 1440 × 900 for screens, PNG or SVG at print resolution for diagrams).
5. Keep the caption exactly as written in Chapter 4, beneath the figure, numbered as specified.
6. If a state cannot be captured (for example "rate limited" or "connection problem"), draw the state or omit the panel and say so in the caption rather than fabricating a screenshot.

---

1. Structure and behaviour diagrams (drawn from the specifications, not screenshots)

Figure	Section	Title	Derived from	Must show	Status
4.1	§4.3.1	System context diagram	Chapter 4 §4.3.1; five external interfaces in §4.2.5	SBMS as one process; Field Officer and Administrator as human actors; phone scanner and public verifier outside; no NEMIS, bank or mobile-money actor	☐
4.2	§4.3.2	Use case diagram	§4.2.2 requirement register	Three actors; the include/extend structure stated in §4.3.2; nothing the API cannot do	☐
4.3	§4.3.3	Activity diagram — registration and screening	`students.controller.js`, `mlDuplicateReview.js`, `duplicateAggregation.js`	Two swimlanes; the decision that a flag never blocks registration	☐
4.4	§4.3.4	Activity diagram — review, approval, payment gate	`duplicateFlags.controller.js`, `bursaryAwards.controller.js`, `disbursements.controller.js`	Both duplicate gates; the terminal confirmed-duplicate outcome	☐
4.5	§4.3.5	Class diagram	`backend/src/models/*.js`, `schema.sql`	Classes, attributes, operations and multiplicities listed in §4.3.5	☐
4.6	§4.3.6	System sequence diagram — registration	`students.controller.js`	Message order: validate → compare → insert → flags → response	☐
4.7	§4.3.7	Sequence diagram — review decision	`duplicateFlags.controller.js: 452-503`	Append-only event; terminal states; no edit of the student record	☐
4.8	§4.3.8	Sequence diagram — approval to certificate	Awards, disbursements and certificates controllers	Verification gate, duplicate gate, payment issue and clearance, certificate issue	☐
4.9	§4.3.9	Sequence diagram — public verification	`verification.controller.js`, `certificates.controller.js`	Boolean- or attribute-only responses; rate limiting; no beneficiary data returned	☐
4.10	§4.3.10	Entity-relationship diagram	`schema.sql` plus the 14 migrations	Chen notation with cardinalities; every attribute named in §4.6.2; the legacy `beneficiaries` table marked unreconciled	☐
4.11	§4.3.11	ML duplicate-detection pipeline	`feature_engineering.py`, `predict.py`, `duplicateAggregation.js`	Two lanes (offline training, online serving); the six shared features; XGBoost as the deployed model	☐

Figure	Section	Title	Derived from	Must show	Status
4.12	§4.4.1	Three-tier architecture	`index.js`, `routes/*`, `frontend/src/**`, `predict.py`	The predictor as a supervised child process; no direct database access from the browser	☐

2. Screen figures (screenshots of the running application)

Figure	Section	Title	Route to open	Must show	Status
4.13	§4.5.1	Role-specific sign-in screen	`/login`, `/admin/login`, `/field-officer/login`	SBMS identity block; the role tabs; the wrong-role refusal message	☐
4.14	§4.5.2	Field-officer student registry list	`/students`	Header with the signed-in role; search and filter; the columns listed in §4.5.2; no reveal control	☐
4.15	§4.5.2	Eligibility assessment screen	`/eligibility`	The five inputs; the score, category and reasons the endpoint returns	☐
4.16	§4.5.3	Student registration form	`/students/register`	Two-column name fields; every validated field; the school select limited to the five approved schools	☐
4.17	§4.5.3	Screening confirmation panel	`/students/register` (after submit)	The recorded student, the screening outcome and the QR image; the statement that a flag does not reject the registration	☐
4.18	§4.5.4	Administrator dashboard	`/dashboard`	Exactly the counters the endpoint returns; the six-month trend; recent activity	☐
4.19	§4.5.5	Duplicate review queue with a case open	`/duplicate-review`	Paired records with identifiers **masked**; score; threshold and model version; decision form with mandatory reason	☐
4.20	§4.5.6	Awards and disbursements with the certificate panel	`/disbursements`	Awards table; payment record; clearance, fail and cancel actions; acknowledgement; exceptions; certificate panel collapsed and expanded	☐
4.21	§4.5.7	Administrative support screens	`/reports`, `/audit`, `/users`, `/settings`	The four panels and the controls listed in the table under the Figure 4.21 caption; never a password hash	☐
4.22	§4.5.8	Public student-record verification page	`/verify` (the student QR target)	Minimal layout; the five states; the three explanatory bullets; **no** student data	☐
4.23	§4.5.8	Public certificate verification page	`/verify-certificate`	The four outcome states; the certificate fields the endpoint returns; **no** name, UPI, school or amount	☐

3. Deployment figure

Figure	Section	Title	Derived from	Must show	Status
4.24	§4.7	Physical deployment architecture on Azure	`docs/azure-deployment-log-2026-10-31.md`; `Dockerfile`; `.github/workflows/`	Static web app, container app, managed PostgreSQL, the ML artefact shipped inside the API image, the CI image gates, and the HTTPS-only public origins	☐

4. Checks before the figures are considered complete

Check	Why it matters
Caption text matches Chapter 4 word for word	The specification is the control; a reworded caption usually means a redrawn figure
Figure number and section agree	Cross-references elsewhere in the report resolve by number
No screenshot contradicts the code at `d6b5032`	Behaviour changed late (registration fields, certificate panel), so older captures are stale
Nothing appears that the specification says must not appear	Figures 4.19–4.23 are where a leak of personal data would be a real finding, not a cosmetic slip

Check	Why it matters
Every figure is referenced in the text before it appears	Academic convention in the report format

Note on Figure 4.21. Its specification is the table that follows the caption in §4.5.7, which states the four panels and the controls each must show; the checklist row above summarises it rather than replacing it.

---

5. Related files in this folder

File	Relationship to the figures
`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md`	Holds the twenty-four specifications that the figures must realise
`EVIDENCE_AND_VERIFICATION_REPORT.md` §1	Fixes the revision and therefore the behaviour that may be photographed
`DOCUMENTATION_GAP_REGISTER.md` G-03, G-04, G-05, G-13	Record the fact that the figures are outstanding and the data-safety conditions attached to them
`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §3	Maps each figure to the requirements it explains, for the reader who checks coverage
`DIAGRAM_SPECIFICATIONS.md`	The consolidated register: all twenty-four figures with type, elements, flow, boundary, decision points, implementation reference and drawing rules — use it when drawing, this file when capturing

ACADEMIC COMPLETENESS AND GAP AUDIT

> **What this file is.** The audit of the package against the brief's own list of requirements: what is complete, what is partial, what is missing, and what cannot be closed from the repository. Repository `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`.
>
> **Verdict in one line:** every requested area is covered by evidence or by a named gap; nothing is covered by a draft alone. The package is ready for drafting, not ready for submission, because six author-side items remain.

---

1. Coverage of the requested areas

Requested area	Status	Where the evidence is
Academic guide requirements for Chapters 4–6	Complete, with six recorded ambiguities	`ACADEMIC_GUIDE_REQUIREMENTS.md` §2–§6
Chapter 4 analysis evidence	Complete	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN_EVIDENCE.md` §1–§4
Chapter 4 design evidence (architecture, schema, interfaces, security)	Complete	§4 of that file; architecture, schema, API and UI catalogues
Chapter 4 diagram specifications (24 figures)	Complete as specifications; **no drawings**	`DIAGRAM_SPECIFICATIONS.md` §2–§5
Chapter 5 implementation evidence	Complete	`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION_EVIDENCE.md` §1–§3
Chapter 5 test evidence with dates	Complete for automated and browser layers	§2 of that file; `SBMS_TEST_EVIDENCE_CATALOGUE.md`
Chapter 5 ML evaluation	Complete for synthetic data; **no real-data evaluation**	§4 of that file; `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §7–§11
Chapter 5 defects and limitations	Complete for the six scoped defects	§5 of that file; handover and Azure logs for the rest
Chapter 6 conclusions per objective	Complete as evidence; exact Chapter 1 wording outstanding	`CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORK_EVIDENCE.md` §1–§2
Chapter 6 recommendations and future work	Complete	§3–§4 of that file
Consistency with Chapters 1–3	Complete as a matrix, six items flagged	`CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md` §1–§3
Evidence register with IDs and statuses	Complete, 51 IDs	`MASTER_EVIDENCE_REGISTER.md` §1–§4
Claim classification and forbidden claims	Complete	`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §4
Tables required by the brief (19 named tables)	Complete; the named tables exist across the chapter drafts and catalogues	chapter drafts §4.2, §4.6, §5.1–§5.6; traceability and catalogue files
Secrets and credentials excluded	Confirmed: no credential, token or key is printed anywhere in the package	this file, §5

1.1 The nineteen requested tables, located

Table	Location
Academic guide compliance matrix	`ACADEMIC_GUIDE_REQUIREMENTS.md` §5
Stakeholder analysis	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.1 (actors and interests)
Functional requirements	`SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §6
Non-functional requirements	§7 of the same file
Business rules	§5 of the same file

Table	Location
Requirements traceability matrix	`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §1–§2
Technology and environment	`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md` §5.2.2 (Table 5.1)
Module implementation matrix	`SBMS_SYSTEM_ARCHITECTURE_AND_MODULE_MAP.md` §3
Database table documentation	`SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` §2–§6
API endpoint inventory	`SBMS_API_ENDPOINT_CATALOGUE.md` §2–§4
Security implementation matrix	`SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md` §3–§5
Test strategy	`SBMS_TEST_EVIDENCE_CATALOGUE.md` §1; Chapter 5 §5.4.1
Detailed test cases	Chapter 5 §5.4.3; `SBMS_TEST_EVIDENCE_CATALOGUE.md` §3–§6
Test execution summary	Chapter 5 §5.4.2; `SBMS_TEST_EVIDENCE_CATALOGUE.md` §2
ML evaluation results	`SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §7; Chapter 5 §5.5.5
Defect register	Chapter 5 §5.4.7; `DOCUMENTATION_GAP_REGISTER.md` §3
Outstanding testing register	Chapter 5 §5.4.8
Objective achievement matrix	`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §1; Chapter 6 §6.2
NFR evaluation	Chapter 5 §5.5.4; `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §2
Limitations and future work matrix	Chapter 6 §6.2.9 and §6.4

2. What is complete

- Every substantive claim has a repository location (code, migration, test, log or committed document) and an evidence ID.
- Every test count carries a command and a date, and no count is summed across levels.
- The ML position is stated once, in one vocabulary: synthetic data, pair-level metrics, human decision, no real-world accuracy claim.
- The classifier deviation and the deployment deviation are recorded as deviations, not quietly reconciled.
- Six functional gaps and three proposal deviations are numbered, not absorbed into prose.
- The forty-five endpoints, thirteen tables and twenty-four figures are enumerated once each, so the chapters cannot drift from each other.

3. What is partial, and why

Area	Partial because
Functional requirements	Six of thirty delivered requirements are partial (FR-12, FR-19, FR-25, FR-26, FR-28, FR-29)
Non-functional requirements	Seven of twenty-one are partial (NFR-01, NFR-13, NFR-14, NFR-16, NFR-18, NFR-19, NFR-21); no load, usability or availability measurement exists
Testing	Automated, native, browser and Python layers are covered; acceptance, load, penetration and device testing are not
ML evaluation	Synthetic only; no adjudicated labels; no beneficiary-level performance
Deployment	A demonstration instance exists; no monitoring, backup schedule or high availability
Figures	Twenty-four specified, none drawn; twelve require screenshots

4. Missing evidence, ordered by submission risk

Rank	Item	Risk if not closed	Source of closure
1	**G-01** hardware specification	Chapter 5 §5.2.1 states a gap where the guide expects an environment description	Author supplies the workstation specification

Rank	Item	Risk if not closed	Source of closure
2	**G-03/G-04/G-05** figures and screenshots	Chapter 4's interface section cannot be illustrated; a Word submission with 24 placeholder captions is not submittable	Author draws from `DIAGRAM_SPECIFICATIONS.md` and captures redacted screenshots on synthetic data
3	**G-06** user acceptance	NFR-14 stays partial and Chapter 5 §5.5.6 has nothing to report	Author either runs a small study or states its absence as a limitation
4	**G-11** phone-camera QR scan	The identification mechanism rests on a headless-browser check	One scan of one printed code on one phone, photographed
5	**G-21** classifier deviation	A supervisor who compares Chapter 3 to Chapter 4 finds an unexplained change	Already stated in Chapters 4 and 6; the author must keep it visible
6	**C-6** exact Chapter 1 objective wording	Chapter 6 may answer a paraphrase rather than the objective	Author pastes the originals and checks the mapping
7	**G-07/G-08** treatment of `uat-test-plan.md` and `verification.md`	Two committed documents contradict the code	Author marks them superseded or excludes them
8	**G-25** two test generations	An undated count is misleading	Quote with dates only, as the drafts already do
9	**G-02** 149/150-of-151 instability	A reader may treat 151 as unconditional	Keep the caveat adjacent to the figure
10	**G-22** deployment ML path	Chapter 4 and Chapter 5 may describe different topologies	Author states which path the Azure instance uses

The full register, including the eighteen items not repeated here, is `DOCUMENTATION_GAP_REGISTER.md` §1–§3.

---

5. Contradictions and unresolved questions

#	Question	Why it cannot be settled here	Where it is recorded
C-1	Random Forest (proposal) versus XGBoost (delivered)	Both are true statements about different artefacts	`CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md` §3
C-2	Containerised ML service versus subprocess-spawned predictor	The repository contains both paths; the Azure runtime choice is not evidenced	§3; G-22
C-3	`docs/verification.md` claims the predictor has no callers	False for this revision; the file is stale, and deleting it is the author's decision	§3; G-08
C-4	`docs/uat-test-plan.md` reports `PASS` results without participants	Reclassifying it is a documentation edit the author controls	§3; G-06, G-07
C-5	Two test totals in `docs/FINAL_TEST_REPORT.md`	Both were true on their dates	§3; G-25
C-6	Exact Chapter 1 objectives	Chapters 1–3 are outside the repository	§3; EV-47

Nothing in this list was resolved by rewriting the earlier chapters. Each is a decision for the author.

---

6. Secret and sensitivity check

Check	Result
Credentials, passwords, JWT secrets, connection strings with passwords	None present in the package; test runs referenced `backend/.env.test` without printing it
Azure resource keys or subscription identifiers	None present
Real beneficiary data	None present; all examples are synthetic
Screenshot guidance	`FIGURE_PLACEHOLDER_CHECKLIST.md` §4 requires a visual redaction check before any image is inserted
Personal data in logs quoted by the package	None; log excerpts are limited to counters and status codes

---

7. What makes the package academically complete, and what does not

It is complete as evidence. Every chapter section the guide prescribes has either a supported fact or an explicit gap; every objective traces to requirements, tests and a conclusion; every figure has a specification that constrains what may be drawn.

It is not complete as a submission. Six author-side items remain (ranks 1–6 above). Until they are closed, the chapters must carry their gaps openly — which is a defensible position, and a stronger one than filling the gaps with invented content.

---

8. Related files

`DOCUMENTATION_GAP_REGISTER.md` (item-level register), `MASTER_EVIDENCE_REGISTER.md` (claim-level register), `CLAUDE_HANDOFF_GUIDE.md` (how to write from this package), `FIGURE_PLACEHOLDER_CHECKLIST.md` (capture checklist).

DOCUMENTATION GAP REGISTER

Purpose. This register lists every item that the chapter content could not establish from the repository, every item that requires the author's input, every additional test that would need to be run for a claim to stand, and every documentation inconsistency that must be resolved before submission.

Prepared from: `student-beneficiary-system`, branch `fix/sbms-completion-work`, commit `d6b5032`, read-only inspection of source, migrations, tests, evaluation artefacts and project records.

How to use it. Each row names what is missing, why it matters, which chapter section depends on it, and what the author must supply. Rows marked **BLOCKING** must be closed before submission; **IMPORTANT** rows affect the strength of a claim but do not invalidate the chapter; **OPTIONAL** rows improve the report if closed.

---

1. Items requiring author input (not obtainable from the repository)

ID	Item	Why it matters	Where used	What the author must supply	Priority
G-01	Workstation and device specification used for development and testing (processor, RAM, storage, OS build, browser and version, phone model if used)	Chapter 5 §5.2.1 states that hardware was not recorded. The run records show 7.4 GB RAM with ~0.5 GB free, but not the full specification	Ch.5 §5.2.1, §5.4.2	A specification sheet for the machines and devices actually used	**BLOCKING**
G-02	Confirmation of free memory during the sweeps that reached 149–150/151	Chapter 5 §5.4.3 reports the instability and identifies memory pressure as the mechanism, but the hardware cause is inferred rather than measured	Ch.5 §5.4.3	Free-memory observation at the time, or a clean re-run on a suitably provisioned machine	IMPORTANT
G-03	Statement of whether Figures 4.1–4.24 were produced, and their file names	The chapters specify each figure precisely but contain no drawings; the Word document needs rendered figures, not placeholders	Ch.4 §4.3, §4.5, §4.7	The drawn figures, or a decision to draw them from the specifications	**BLOCKING**
G-04	Screenshots for Figures 4.13–4.23 (sign-in, registry list, eligibility, registration form, screening confirmation, dashboard, duplicate review, awards and certificates, administrative support screens, both public verification pages)	The specifications state what each image must show; no screenshot was captured in this pass and none is claimed	Ch.4 §4.5; Ch.5 §5.2	Redacted screenshots taken against synthetic data only	**BLOCKING**
G-05	Confirmation that screenshots and quoted identifiers contain no real beneficiary data	The repository holds only synthetic data, but the live deployment holds demonstration records created during testing	Ch.5 §5.6; G-13	Visual check of every image before insertion	**BLOCKING**
G-06	User acceptance study: participants, roles, tasks, findings, consent and ethics approval — or an explicit statement that none was conducted	Chapter 5 §5.5.6 reports that UAT was not conducted with users; NFR-14 is unverified without it	Ch.5 §5.5.6; Ch.6 §6.4.9	Either the study record or an explicit statement of its absence	**BLOCKING**
G-07	Decision on how `docs/uat-test-plan.md` is treated: author walkthrough, marked superseded, or withdrawn	Its five `PASS` results and one stale sentence ("current backend controller code does not call it") conflict with the implementation and the test report	Ch.5 §5.5.6, §5.6.7	An instruction, since changing it is a documentation edit the author controls	IMPORTANT
G-08	Decision on how `docs/verification.md` is treated, given its claim that no controller calls the predictor	That claim is false for the current revision and would contradict Chapters 4 and 5 if quoted	Ch.5 §5.6.7	Annotation, retirement, or acceptance of the correction recorded here	IMPORTANT

ID	Item	Why it matters	Where used	What the author must supply	Priority
G-09	AI-assistance disclosure required by the School (tools used, how outputs were verified)	Many institutions require such a statement; none exists in the repository	Front matter / declaration	The wording the School requires	**BLOCKING**
G-10	Student name, admission number and supervisor name exactly as they must appear	Chapters 4–6 do not name the author and must agree with the title page and Chapters 1–3	Title page	Confirmation of the exact form (the proposal reads "Ngigi Peter", 169613, supervisor Salome Monthe Chemiat)	IMPORTANT

2. Items requiring additional testing or evidence

ID	Item	Why it matters	Where used	What would close it	Priority
G-11	Phone-camera scan of a printed student QR code and of a printed award certificate	The identification mechanism is a printed code read by a phone; only headless-Chrome checks exist	Ch.5 §5.4.8, §5.6.3; Ch.6 §6.4.6	A camera test against the deployed HTTPS origin, with observed results recorded	**BLOCKING** for any claim that the mechanism works in practice
G-12	Load and concurrency evidence at 500 and 1 000 candidates, and two simultaneous registrations	NFR-01 is verified only at 50 candidates and NFR-13 has no concurrency test	Ch.5 §5.4.5, §5.5.4; Ch.6 §6.4.8	A scripted load test plus a concurrency test, with results recorded	IMPORTANT
G-13	Removal of the two `E2E Probe` students and their duplicate case from the production database	They are demonstration artefacts in a live database and would appear in any live screenshot	Ch.5 §5.3.12	Execute the documented `DELETE` for `full_name LIKE 'E2E Probe%'` (author's decision; not performed here)	IMPORTANT
G-14	Administrator password re-alignment and re-run of `scripts/admin-reveal-smoke.mjs`	The stored password returned 401, so the reveal screen was not re-checked after the registration-field change	Ch.5 §5.3.12	Reset the password, re-run the harness, record the result	IMPORTANT
G-15	Print-output verification of the certificate PDF and the exported report	Only the generated bytes are covered; no physical print or visual check exists	Ch.5 §5.4.8	Print both documents and record legibility and layout	OPTIONAL
G-16	Accessibility audit and cross-browser testing (Firefox, Safari, mobile)	NFR-14 and NFR-19 are unverified beyond Chrome	Ch.5 §5.4.8, §5.5.4	An accessibility tool report plus manual checks, and a cross-browser pass	OPTIONAL
G-17	Tests for the eligibility arithmetic, dashboard counter arithmetic and report content	Three implemented areas have weak automated coverage	Ch.5 §5.4.8 (O-07, O-08)	Table-driven tests added to the Jest suite	IMPORTANT
G-18	Adjudicated pair labels and a re-run of the reviewed-pairs evaluation	Every real-world quality claim depends on it; decisions D1–D5 are unanswered	Ch.5 §5.5.5; Ch.6 §6.4.1	200–300 adjudicated pairs plus the evaluation output	**BLOCKING** for a real-world accuracy claim only (not required while the report confines itself to synthetic findings, as these chapters do)
G-19	Hosted disaster-recovery drill	The rehearsal used a local disposable database, not the managed server	Ch.5 §5.4.5; Ch.6 §6.4.8	A restore exercise against a separate managed instance, with results recorded	OPTIONAL
G-20	One clean 151/151 suite run on a machine with adequate memory	The documentation-session sweeps reached 149–150/151 for an environmental reason; the report should show a clean reproduction	Ch.5 §5.4.3	One green sweep with the memory note recorded	IMPORTANT

3. Documentation inconsistencies to reconcile

ID	Inconsistency	Evidence	Affects	Suggested resolution
G-21	The proposal specifies a **Random Forest** classifier over **four** features; the production system runs **XGBoost** over **six**	Proposal §1.3.1(ii), §3.2.2, §3.2.3 versus `predict.py`, `feature_engineering.py`	Ch.4 §4.1, §4.4.4; Ch.5 §5.3.4; Ch.6 §6.2.4	Report the deviation explicitly (as these chapters do), and state a reason if one exists; otherwise report it as an unexplained deviation
G-22	The proposal describes deployment on "a single Linux virtual machine" with Nginx and PM2; the system runs on Azure Container Apps plus App Service	Proposal §3.3.7, §3.4.5 versus `docs/azure-deployment-log-2026-10-31.md`	Ch.4 §4.7; Ch.5 §5.3.12	Report the deployed architecture and note the change, with the hosting constraints that forced it
G-23	`docs/verification.md` states there are no callers of the prediction function and that controllers use their own scoring	`students.controller.js:69` calls `scorePairs`	Ch.5 §5.6.7	Mark the document superseded (see G-08)
G-24	`docs/uat-test-plan.md` reports five `PASS` results without participants and contains a stale sentence about the predictor	`docs/uat-test-plan.md`, Test Scenario 2	Ch.5 §5.5.6	Reclassify as an author walkthrough, or run a real study (G-06, G-07)
G-25	`docs/FINAL_TEST_REPORT.md` contains two result generations (21 suites/128 tests on 2026-10-31; 24 suites/151 tests on 2026-11-01)	§1–§3 versus §10	Ch.5 §5.4.3	Always cite the date with a count (as these chapters do); optionally add a "current totals" banner to the file
G-26	Four working checkouts of the same project exist on different branches, and their documents differ	`student-beneficiary-system`, `-dev`, `-admin-login-fix`, `sbms-raw-pair-model-training`	Evidence report	State which checkout and commit every claim refers to (these chapters use `fix/sbms-completion-work` @ `d6b5032`)
G-27	The production model artefact is tracked, but `.gitignore` still lists `ml-service/model/*.joblib`	`.gitignore` versus `git ls-files ml-service/model`	Ch.5 §5.3.4	Note that the artefact is committed; if it is meant to be excluded, explain how the deployment obtains it
G-28	Two root reports cited by these chapters are untracked (`SBMS_O10_O11_IMPLEMENTATION_REPORT.md`, `SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md`)	`git status` shows both as untracked	Ch.4 §4.4.3; Ch.5 §5.4.7 (D-08)	Commit them, or cite the committed equivalent

4. Items deliberately not supplied, and why

Item	Reason
Invented supervisor feedback or approval	The SBMS-01…SBMS-18 findings are unrecoverable from the workspace; `SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` §4 records that their wording is unavailable. No finding was paraphrased into a requirement
Production metrics, latency under load, uptime figures	No such measurements exist
Real-world ML precision, recall or F1	No adjudicated labels exist; synthetic figures are reported as synthetic
User-acceptance results	No study was conducted
New academic citations	Chapters 4–6 introduce no new literature claims; where a concept needs a source, the citation belongs in Chapters 1–3. Any citation added during editing must be verified by the author
Authentication of the deployment as "production"	The deployment is a demonstration instance, and the chapters say so

5. Cross-reference: gaps to chapter sections

Chapter section	Gaps that touch it
Ch.4 §4.1	G-21
Ch.4 §4.2.3	G-12, G-16
Ch.4 §4.3	G-03
Ch.4 §4.5	G-03, G-04, G-05
Ch.4 §4.7	G-22
Ch.5 §5.2.1	G-01, G-02
Ch.5 §5.3.4	G-21, G-27
Ch.5 §5.3.12	G-13, G-14, G-22
Ch.5 §5.4.3	G-02, G-20, G-25
Ch.5 §5.4.5	G-12, G-19
Ch.5 §5.4.8	G-11, G-15, G-16, G-17
Ch.5 §5.5.5	G-18
Ch.5 §5.5.6	G-06, G-07
Ch.5 §5.6.7	G-08, G-23, G-24, G-25
Ch.6 §6.2.4	G-21
Ch.6 §6.4.1–§6.4.9	G-11, G-12, G-17, G-18, G-19

---

6. Minimum set to close before submission

If time is short, five items carry the most risk: **G-01** (environment specification), **G-03/G-04/G-05** (figures and screenshots, with the synthetic-data check), **G-06/G-07** (a clear position on user acceptance), **G-21** (the Random Forest to XGBoost deviation) and **G-11** (a phone-camera scan of at least one printed QR code, since the identification mechanism otherwise rests entirely on a headless-browser check).

---

7. Closed since this register was drafted

Item	How it was closed	Where the closure is visible
G-21 Random Forest to XGBoost deviation	Stated explicitly, not reconciled silently: the proposal's four-feature Random Forest and the delivered six-feature XGBoost are both named, with the research-only artifact identified	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.1, §4.2.2 (FR-31), §4.4.4; `CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md` §6.2.4; restated in `CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md` §1

Still open, and unchanged by the above: G-21 remains an item the author must keep visible through final assembly, because a supervisor comparing Chapter Three to Chapter Four will meet the change there. The register keeps it listed for that reason rather than marking it "done".

---

8. Document set this register belongs to

The evidence package is two-tiered. Entry set: `ACADEMIC_GUIDE_REQUIREMENTS.md`, `CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN_EVIDENCE.md`, `CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION_EVIDENCE.md`, `CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORK_EVIDENCE.md`, `CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md`, `DIAGRAM_SPECIFICATIONS.md`, `MASTER_EVIDENCE_REGISTER.md`, `ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md`, `CLAUDE_HANDOFF_GUIDE.md`. Evidence tier: the three chapter drafts, the `SBMS_*` catalogues, `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md`, `EVIDENCE_AND_VERIFICATION_REPORT.md`, this register and `FIGURE_PLACEHOLDER_CHECKLIST.md`. `CLAUDE_HANDOFF_GUIDE.md` §2 carries the full crosswalk.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

> **Status of this file.** Report-ready content for Chapter 4 of the SBMS final-year project report, produced from a read-only investigation of the `student-beneficiary-system` repository on branch `fix/sbms-completion-work` (commit `d6b5032`), its migrations, tests, evaluation artefacts and project records. Section numbering follows the Chapter 4 pattern of the School exemplar (`CHAPTER_4[1].docx`: 4.1 Introduction, 4.2 Requirement Analysis, 4.3 System Analysis Diagrams, System Design, System Design Diagrams, Database Schema, System Architecture), extended with the data, interface, ML-integration and security subsections required by the project brief.
>
> **Evidence conventions. VERIFIED** = confirmed by reading implementation, configuration, migrations or test output. **PARTIALLY VERIFIED** = some evidence exists but does not establish the full claim. **REQUIRES INPUT** = cannot be confirmed from the repository. **NOT IMPLEMENTED** = the feature does not exist in the inspected code. Every claim that is not fully verified is listed in `DOCUMENTATION_GAP_REGISTER.md`.
>
> **Figures.** Figures are specified, not drawn. Each carries a numbered placeholder, a caption and a specification of its contents, because a diagram that is not derived from the implementation would misrepresent the system.

4.1 Introduction

This chapter presents the analysis and design of the Student Beneficiary Management System (SBMS). The system was built to digitise the registration, identification, verification and bursary-award tracking of student beneficiaries in a Kenyan government-funded welfare programme, and to introduce machine-learning-assisted identification of potential duplicate beneficiary records that a human reviewer then adjudicates. The problem, objectives and development methodology were established in Chapters One to Three. This chapter converts them into an analysable and testable specification: what the system must do, what qualities it must exhibit, what data it must hold, how its parts communicate, and how the delivered artefact was organised.

Three purposes are served. First, the chapter records the requirements against which the implementation is measured in Chapter Five, so that evaluation is anchored to a specification rather than to whatever happened to be built. Second, it documents the analysis models — context, use case, activity, sequence, class and entity-relationship models — that explain how actors, processes and stored data of SBMS relate. Third, it documents the design of the delivered system: its three-tier architecture, module decomposition, access-control model, machine-learning integration, relational schema and physical deployment architecture.

The analysis and design are **derived from the implemented system**, not from an idealised system the project intended to build. Where the implementation departs from the approved proposal, the departure is stated rather than reconciled silently. Two departures are material and recur in this chapter and in Chapter Five:

1. **The duplicate-detection classifier in production is an XGBoost model, not the Random Forest classifier specified in the approved proposal.** The proposal (`Project Proposal- 169613.pdf`, §1.3.1 objective ii and §3.2.3) named a Random Forest classifier over four features. The delivered system uses an `XGBClassifier` inside a scikit-learn pipeline over **six** features (`jaro_winkler`, `dob_match`, `school_match`, `guardian_match`, `upi_match`, `birth_certificate_match`). The Random Forest survives only as a labelled research candidate (`ml-service/model/research/rf_attachment_style_candidate.joblib`) that the application never loads. **VERIFIED** by `ml-service/predict.py:14`, `ml-service/feature_engineering.py:11-18` and `ml-service/evaluation/realistic_pairs_xgboost_training_metrics.json`.
2. **The delivered system exceeds the proposal's scope** in bursary-award verification, payment reconciliation and QR-verifiable award certificates, and **falls short of the scope implied by the proposal's evaluation design** in candidate generation: every new registration is compared against every existing student, with no blocking. The design consequence is analysed in §4.4.4 and evaluated in Chapter Five.

The analytical approach is the one established in Chapter Three: Design Science Research, executed through an iterative Agile cycle, with the machine-learning component following the four-stage pipeline of data acquisition,

preprocessing, training and validation. The analysis artefacts in §4.3 are therefore the artefacts of an iterative build. Neither Structured Systems Analysis and Design Method (SSADM) nor a formal Unified Process was adopted, and no claim is made that the project followed either. Unified Modeling Language (UML) notation is used because the implementation is object-oriented at the module level in both JavaScript and Python, but the governing methodology remains the Agile/DSR process described in Chapter Three.

4.2 Requirement Analysis

4.2.1 Requirements sources and elicitation

Requirements were elicited, as recorded in the approved proposal (§3.3.1), primarily through document analysis of Auditor-General reports, Controller of Budget reports and the NG-CDF legislative framework, because direct access to constituency bursary officers for formal interviews was not available within the academic timeline. That elicitation produced the functional intentions expressed in the proposal. It did **not** produce a numbered requirement register with verification methods, and it did not anticipate the bursary-award reconciliation and certificate functions that were subsequently built.

The register in this section therefore reconciles three sources:

Source	What it contributes	Status
Approved proposal (`Project Proposal-169613.pdf` §1.3, §1.5, §3.3.1)	Original scope: registration with unique identifier and QR code, machine-learning duplicate detection, disbursement recording with audit trail, rule-based eligibility scoring, role-based access control, dashboard, PDF report export, and two named non-functional targets (browser availability; duplicate check under three seconds)	**VERIFIED** as the documented baseline
Implemented system (routers, controllers, services, migrations, frontend routes)	Functions actually delivered, with their access rules, validations and error behaviour	**VERIFIED** by code inspection
Project records (`docs/SBMS_PROJECT_HANDOVER_2026-10.md`, `docs/business-workflow-verification-2026-10-30.md`, `docs/qr-verification-2026-10-30.md`, `docs/bursary-certificates-2026-11-01.md`, `docs/FINAL_TEST_REPORT.md`)	Defects found and fixed, behaviours decided, limits acknowledged	**VERIFIED** as project records; underlying behaviour re-checked against code

Requirement identifiers are assigned here and used unchanged in Chapter Five and in `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md`. Priority follows the MoSCoW convention: **M** (must have), **S** (should have), **C** (could have), **W** (will not have in this release). A requirement is treated as *implemented* only where a route, a service, a database object and a user-facing affordance all exist. Where the interface exists but no executed test covers the behaviour, the requirement is described as *implemented, not fully tested* and is carried into Chapter Five §5.4.12.

4.2.2 Functional requirements

Table 4.1 states the functional requirements of the delivered system. Each is a single testable statement with its actor, priority, rationale and intended verification method. The "Verification" column names the suite or record that establishes the behaviour; where no test exists, the row says so explicitly.

Table 4.1 (part 1 of 3): Authentication, beneficiary management and duplicate review

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-01	The system shall authenticate a user against a stored bcrypt hash and issue a JSON Web Token valid for eight hours.	Administrator, Field Officer	M	Proposal §1.5.1; `controllers/auth.controller.js`	`tests/auth.test.js`; `tests/roleAuthorization.test.js`

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-02	The system shall provide separate sign-in endpoints for the administrator and field-officer roles and refuse an account presented at the wrong endpoint with HTTP 403.	Administrator, Field Officer	M	Separates the two operational roles at the entry point	`tests/auth.test.js`; `tests/roleAuthorization.test.js`
FR-03	The system shall refuse public self-registration with HTTP 410 and create accounts only through an authenticated administrative path.	Administrator	M	Prevents unauthenticated account creation	`tests/auth.test.js`; `tests/roleAuthorization.test.js`
FR-04	The system shall restrict every endpoint to the roles permitted by the access matrix in Table 4.6 and return HTTP 403 otherwise.	All authenticated	M	Duty separation between the two roles	`tests/roleAuthorization.test.js`; `tests/workflow.test.js`
FR-05	The system shall register a beneficiary with full name, date of birth, school identifier, national identifier, primary phone and optional UPI and birth-certificate number, validating each field before persistence.	Field Officer	M	Proposal §1.5.1 (student registration)	`tests/students.test.js`
FR-06	The system shall reject a school identifier that is not one of the five approved schools.	Field Officer	M	Constrains the registry to a controlled school list; API and database both enforce it	`tests/students.test.js`; `students_school_supported_check`
FR-07	On registration the system shall score the new record against every existing student record and create duplicate-review cases for candidates meeting the review policy.	System	M	Proposal objectives ii and iv; the research contribution	`tests/duplicateFlags.test.js`; `tests/workflow.test.js`; `tests/mlPredictor.integration.test.js`
FR-08	The system shall never reject, delete or amend an existing beneficiary record as a result of duplicate screening; a flagged registration shall still create the student record.	System	M	Human oversight of algorithmic signals (proposal §1.4)	`tests/workflow.test.js`; `docs/business-workflow-verification-2026-10-30.md` §4
FR-09	The system shall store, for every duplicate case, the model score, model version, review threshold in force and comparison feature vector.	System, Administrator	M	Auditability of algorithmic evidence	`tests/duplicateFlags.test.js`; migration `20261022_ml_duplicate_provenance.sql`
FR-10	The system shall present duplicate cases to an administrator with identifiers masked, and reveal identifiers only on an explicit, audited request.	Administrator	M	Privacy by default with controlled disclosure	`tests/duplicateFlags.test.js`
FR-11	The system shall record a reviewer decision of `confirmed_duplicate`, `not_duplicate` or `needs_verification` with a mandatory reason, appended to a per-case history.	Administrator	M	Human adjudication replaces automatic decision	`tests/duplicateFlags.test.js`; `tests/workflow.test.js`

Table 4.1 (part 2 of 3): Eligibility, awards and payments

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-12	The system shall compute an eligibility score from five weighted factors and assign one of four eligibility categories.	Field Officer	S	Proposal §1.5.1 (rule-based eligibility scoring)	Role and route covered by `tests/students.test.js`; **no automated test of the scoring arithmetic was found**
FR-13	The system shall allow a field officer to submit a bursary-award application for an existing student with a requested amount.	Field Officer	S	Award-reconciliation extension	`tests/bursaryAwards.controller.test.js`

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-14	The system shall allow a field officer to record a beneficiary verification outcome with an evidence reference, and require an evidence reference for a `verified` outcome.	Field Officer	S	Addresses the missing-documentation problem in Chapter One	`tests/bursaryAwards.controller.test.js`; `tests/workflow.test.js`
FR-15	The system shall permit an administrator to approve or reject an award with a reason, and refuse approval unless the beneficiary is verified and no duplicate case is open or confirmed for that student.	Administrator	M	The central anti-duplication control over payment	`tests/bursaryAwards.controller.test.js`; `tests/workflow.test.js`
FR-16	The system shall record a payment as `issued` against an approved award, refuse payment for an unapproved award, an unverified beneficiary or a student with an unresolved or confirmed duplicate, and refuse a payment exceeding the approved amount.	Administrator, Field Officer	M	Proposal §1.5.1 (benefit-distribution tracking with audit trail)	`tests/disbursements.test.js`; `tests/disbursementReconciliation.controller.test.js`; `tests/workflow.test.js`
FR-17	The system shall record payment clearance separately from issuance and require an evidence reference for clearance.	Administrator, Field Officer	S	Distinguishes instruction to pay from confirmation of settlement	`tests/disbursementReconciliation.controller.test.js`; `tests/workflow.test.js`
FR-18	The system shall record an officer-assisted receipt acknowledgement and classify a shortfall as a `discrepancy` exception rather than overwriting the recorded amount.	Administrator, Field Officer	S	Preserves evidence of a mismatch for follow-up	`tests/workflow.test.js`
FR-19	The system shall report reconciliation exceptions, including unlinked legacy payments, unverified payments, partial payments, awaiting acknowledgement, overdue acknowledgement, failed or cancelled payments and amount discrepancies.	Administrator, Field Officer	S	Makes accountability gaps visible rather than silent	`tests/workflow.test.js`; rules in `services/bursaryRules.js`
FR-20	The system shall generate a unique, cryptographically random QR token per registered student and return a QR image to the registering officer.	System, Field Officer	M	Proposal §1.5.1 (QR-based identification)	`tests/verification.test.js`
FR-21	The system shall expose a public endpoint that reports only whether a student record matching a presented token exists, and nothing else.	Public verifier	M	Identification without disclosure of beneficiary data	`tests/verification.test.js`; `tests/verificationRoute.test.js`

Table 4.1 (part 3 of 3): Certificates, reporting, administration and operations

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-22	The system shall allow an administrator to issue a QR-verifiable certificate for an approved award, and refuse a second certificate for the same award.	Administrator	C	Added deliverable; verifiability of an approval	`tests/bursaryCertificates.test.js`
FR-23	The system shall allow an administrator to revoke a certificate with a reason; public verification shall then report `revoked`.	Administrator	C	Correctness of a document once handed to a holder	`tests/bursaryCertificates.test.js`
FR-24	The system shall report a certificate as `not_current` when the underlying award is no longer approved or its approved amount changed since issue.	System	C	Prevents a document outliving the decision it certifies	`tests/bursaryCertificates.test.js`
FR-25	The system shall present an administrative dashboard with beneficiary, duplicate, award and payment totals, a six-month registration and payment trend, and recent audit activity.	Administrator	S	Proposal §1.5.1 (real-time dashboard)	`tests/roleAuthorization.test.js` (authorisation and synthetic counts); **each counter's arithmetic is not individually tested**

ID	Requirement statement	Actor(s)	Priority	Rationale / source	Verification
FR-26	The system shall export a date-range PDF report containing registration, disbursement, duplicate and accountability summaries.	Administrator	S	Proposal §1.5.1 (reporting with PDF export)	`tests/roleAuthorization.test.js` (range validation); PDF content verified only by code inspection
FR-27	The system shall record an audit entry for every state-changing operation, identifying acting user, action, affected table and record, and contextual details.	System	M	Proposal §1.5.1; accountability is the project's justification	`tests/workflow.test.js`; `tests/duplicateFlags.test.js`
FR-28	The system shall allow an administrator to read and update the duplicate review, high-priority and model cut-offs within 0–1, and audit the change.	Administrator	S	Operational tuning without redeployment	`tests/roleAuthorization.test.js` (denial); `controllers/settings.controller.js` validation
FR-29	The system shall allow an administrator to list users and create administrator or field-officer accounts.	Administrator	S	Administrative provisioning replaces public registration	`tests/roleAuthorization.test.js`
FR-30	The system shall report process liveness and database readiness separately, with readiness failing closed when a required table or the expected database identity is absent.	Operator	S	A live process is not a working system	`test/databaseReadiness.node.js`; `tests/schemaReadiness.test.js`

Three requirements stated in the approved proposal are recorded for completeness, because Chapter Five must evaluate them rather than omit them:

ID	Proposal requirement	Delivered state	Classification
FR-31	Random Forest classifier over four similarity features (proposal §1.3.1 ii, §3.2.2)	An XGBoost classifier over six features is in production; the Random Forest exists only as an unevaluated research candidate	**Implemented with a documented design change** — see §4.4.4 and Chapter Five §5.6
FR-32	Integration with NEMIS or a national identifier verification service	Not implemented; excluded in the proposal (§1.5.1) and never added	**NOT IMPLEMENTED, as scoped**
FR-33	Offline operation and native mobile applications	Not implemented; excluded in the proposal (§1.5.1); the frontend is responsive web only	**NOT IMPLEMENTED, as scoped**

Requirements that the delivered system does **not** satisfy and that the proposal implied must be visible here rather than discovered in Chapter Six:

ID	Gap	Evidence
FR-34	No candidate-blocking or indexing strategy: registration compares the new record with the entire registry, so the number of model calls grows linearly with registry size and the administrator rescan is quadratic	`controllers/students.controller.js:69-73`; `controllers/duplicateFlags.controller.js` scan; `docs/ml-decision-sheet-2026-10-30.md` §6
FR-35	No re-issue path for a student QR token; a student row created without a token (for example by a direct insert) cannot be given one through the API	`docs/qr-verification-2026-10-30.md` §6
FR-36	No token expiry or rotation for student QR codes; a token remains valid for the lifetime of the student row	`docs/qr-verification-2026-10-30.md` §6
FR-37	The system records payments; it does not execute financial transfers. There is no integration with a bank, mobile-money operator or treasury system	`controllers/disbursements.controller.js`; `docs/business-workflow-verification-2026-10-30.md`

4.2.3 Non-functional requirements

Non-functional requirements are stated as measurable constraints wherever the project produced a measurement, and as design constraints where it did not. A requirement without a measurement is labelled as such; it is not presented as achieved. The "Status" column carries the classification defined at the head of this chapter.

Table 4.2 (part 1 of 2): Performance, security, privacy and integrity

ID	Requirement	Measure / constraint	Evidence	Status
NFR-01	Duplicate screening for one registration shall complete within three seconds	≤ 3 000 ms end-to-end for a 50-candidate registration	`backend/tests/performance.test.js`; 22.67 ms measured for 50 pairs (2026-10-30 record)	**PARTIALLY VERIFIED**: satisfied at 50 candidates on the development machine; no measurement at registry scale
NFR-02	Password storage shall resist offline attack	bcrypt with cost factor 12	`controllers/auth.controller.js:26`	**VERIFIED** as implementation; cost factor not benchmarked on the target host
NFR-03	Session credentials shall be signed and time-bounded	HMAC-signed JWT, `expiresIn: '8h'`, secret ≥ 32 random bytes, placeholder rejected at start-up	`controllers/auth.controller.js:55-59`; `config/jwtSecret.js`; `test/jwtSecret.validation.node.cjs`	**VERIFIED**
NFR-04	Authorisation shall not trust a client-supplied claim	Role re-read from the `users` row on every authenticated request	`middleware/auth.js:20-24`; `tests/roleAuthorization.test.js`	**VERIFIED**
NFR-05	Beneficiary identifiers shall not be exposed by default in the review queue	`guardian_id`, `upi`, `birth_certificate_number`, `national_id_number` masked to the last three characters; separate audited reveal	`controllers/duplicateFlags.controller.js:37-42, 106-107`; `tests/duplicateFlags.test.js`	**VERIFIED**
NFR-06	Public verification shall disclose no beneficiary data	Student endpoint returns `{found: boolean}` only; certificate endpoint returns authenticity facts only	`controllers/verification.controller.js`; `controllers/certificates.controller.js:53-89`; `tests/verification.test.js`; `tests/bursaryCertificates.test.js`	**VERIFIED**
NFR-07	Public endpoints shall be rate limited	10 requests per client IP per 15 minutes (student verification); 20 per 15 minutes (certificate); separate budgets	`middleware/verificationRateLimit.js`; `certificateRateLimit.js`; `rateLimitWindow.js`	**VERIFIED** in-process only; counters reset on restart and are not shared between instances
NFR-08	Sign-in shall be rate limited	10 attempts per client IP per 15 minutes	`middleware/loginRateLimit.js`; `tests/loginRateLimit.test.js`	**VERIFIED**
NFR-09	Request bodies shall be bounded	JSON limit 100 kB; oversize returns a safe client error	`index.js:64`; `tests/securityMiddleware.test.js`	**VERIFIED**
NFR-10	Errors shall not leak internals	5xx responses return "Internal server error"; logs carry no stack, message or credential	`middleware/errorHandler.js:59-78`; `tests/securityMiddleware.test.js`	**VERIFIED**
NFR-11	Browser origins shall be restricted by allowlist	Unlisted origin rejected before routing; malformed configuration stops start-up	`config/cors.js`; `index.js:40-47`; `tests/securityMiddleware.test.js`	**VERIFIED**
NFR-12	Data integrity shall be enforced by the database, not only the application	`CHECK` constraints for roles, school list, award decision fields, verification fields, payment and receipt status, clearance and acknowledgement completeness, certificate status and token format; foreign keys throughout	`backend/src/db/schema.sql`; migrations `20261019_*`, `20261023_*`, `20261031_*`	**VERIFIED**; the key constraints are exercised by `tests/workflow.test.js` and `tests/bursaryCertificates.test.js`

ID	Requirement	Measure / constraint	Evidence	Status
NFR-13	Concurrent registrations shall not interleave their duplicate comparisons	Transaction plus `pg_advisory_xact_lock(hashtext('sbms.student.registration'))`; candidates selected `FOR UPDATE`	`controllers/students.controller.js:67-73`; the same lock guards award decisions and payment creation	**PARTIALLY VERIFIED**: the lock exists and is used; **no concurrency test exists**

Table 4.2 (part 2 of 2): Usability, maintainability, operability, oversight and recovery

ID	Requirement	Measure / constraint	Evidence	Status
NFR-14	The interface shall be usable by non-technical officers on varied devices	Responsive layout; role-specific navigation; plain-language screening messages for officers; single-line status pages for public verifiers	`components/AppLayout.jsx`; `pages/StudentRegistration.jsx:7-22`; `pages/Verification.jsx`	**PARTIALLY VERIFIED**: design intent implemented and browser-checked; **no usability study or acceptance session with real officers has been conducted**
NFR-15	The system shall be maintainable and its behaviour reproducible	One feature-schema module shared by training and serving; ordered migration files; Jest, native `node --test` and Python `unittest` suites; version-pinned requirements	`ml-service/feature_engineering.py`; `backend/src/db/migrations/`; `ml-service/requirements.txt`; `docs/FINAL_TEST_REPORT.md`	**VERIFIED**
NFR-16	The system shall remain operable within a zero-cost student cloud budget	Containerised API with `minReplicas 0`; free App Service frontend tier; PostgreSQL stopped between demonstrations; no log-analytics workspace	`render.yaml`; `Dockerfile`; `docs/azure-deployment-log-2026-10-31.md` §1, §3–§4	**VERIFIED** as deployed configuration; the cost model is an owner decision, not a property of the code
NFR-17	A human shall make every duplicate and payment decision	Model scores create review cases only; approval, disbursement and certificate issue are role-gated human actions	`controllers/duplicateFlags.controller.js:452-503`; `controllers/bursaryAwards.controller.js:151-209`	**VERIFIED**
NFR-18	The system shall recover from database loss with verified evidence	Dump, restore into an independent database, readiness and schema/row-digest comparison rehearsed	`docs/database-recovery-rehearsal-2026-10-31.md`	**PARTIALLY VERIFIED**: rehearsal on a local disposable database; **no scheduled backup job or retention policy exists in the deployment**
NFR-19	The system shall be compatible with ordinary modern browsers	Static single-page application over HTTPS; no browser-specific API beyond `fetch`, `history.replaceState` and React Router hash handling	`frontend/src/**`; `frontend/public/web.config`	**PARTIALLY VERIFIED**: exercised in Chrome only; **Firefox, Safari and mobile-browser compatibility are not recorded**
NFR-20	The system shall degrade predictably when the ML predictor is unavailable	Registration aborts with HTTP 503 rather than registering an unscreened beneficiary	`controllers/students.controller.js:74-96`; `services/duplicateCheck.js:284-300`	**VERIFIED** by code inspection and the 503 assertion path in `tests/students.test.js`
NFR-21	Audit records shall be attributable and complete for state-changing actions	Every controller writes an `audit_logs` row with actor, action, table, record and JSON details	`services/auditLog.js`; `tests/workflow.test.js` (ten-action sequence)	**VERIFIED** for the workflows tested; **append-only by convention only — no database-level immutability or retention policy exists**

The following requirements were **not** met, and are stated here so that Chapter Five does not have to discover them:

ID	Requirement not met	Reason and evidence
NFR-22	Load-tested capacity for a constituency-scale registry	No hosted load test has been run; the only performance evidence is one local test at 50 candidates (`docs/FINAL_TEST_REPORT.md` §8)
NFR-23	Thresholds validated against adjudicated beneficiary data	No adjudicated label set exists; decisions D1–D5 remain unanswered (`docs/ml-evaluation-policy.md`)
NFR-24	Independent confirmation of receipt by a school	Acknowledgement is officer-assisted by design and is labelled as such in the interface and in the database (`acknowledgement_method = 'officer_assisted'`)

4.2.4 Data requirements

The data requirements describe what the system stores, where the data originates, how it is validated, how it is protected, and what is deliberately not stored.

Data collected. Four categories of data are held.

1. *Operational identity data for beneficiaries* (`students`): full name, date of birth, school identifier, guardian identifier, UPI, birth-certificate number, national identifier, primary phone, QR token, eligibility score and category, synthetic-data marker, and timestamps.
2. *Account data for staff* (`users`): full name, e-mail, bcrypt password hash, role (`admin` or `field_officer`), active flag, creation timestamp.
3. *Decision and accountability data*: duplicate cases and their review history (`duplicate_flags`, `duplicate_review_events`), bursary awards and their verification history (`bursary_awards`, `bursary_award_verification_events`), payments and their reconciliation events (`disbursements`, `disbursement_reconciliation_events`), issued certificates (`bursary_certificates`), audit records (`audit_logs`) and generated-report metadata (`reports`).
4. *A legacy table* (`beneficiaries`) retained because `/api/beneficiaries` is still mounted and the readiness check requires it. Its `scholarship_amount` column has no approved mapping to any award or payment amount, and the newer student model and the legacy beneficiary model are **not reconciled** (`docs/SBMS_PROJECT_HANDOVER_2026-10.md` §1.2).

Data sources and ownership. All beneficiary data in the delivered system is entered by a field officer through the registration form, or generated by the system (identifiers, timestamps, QR tokens, model scores). The repository contains **no real beneficiary records**: the demonstration registry is synthetic, and synthetic rows are explicitly marked `is_synthetic = TRUE` and counted separately on the dashboard (`controllers/adminDashboard.controller.js:26-32`). Data ownership therefore sits with the implementing organisation, and no claim is made that personal data of real learners was processed.

Validation requirements. Validation is layered so that a client cannot bypass it.

Layer	Mechanism	Example
Transport	JSON body limit 100 kB; exact-shape checks on public endpoints (exactly one key, `token`)	`index.js:64`; `controllers/verification.controller.js:7-15`
Schema	Zod schemas per endpoint: trimmed lengths, e-mail format, ISO date, `+254[17]\d{8}` phone, enum of approved schools, UUID identifiers, positive monetary amounts bounded at 9 999 999 999.99	`controllers/students.controller.js:15-24`; `controllers/bursaryAwards.controller.js:5-26`; `controllers/disbursements.controller.js:9-38`
Domain rule	Cross-field refinements: a `verified` outcome requires an evidence reference, clearance requires evidence, a failed or cancelled payment requires a reason of at least five characters	`controllers/bursaryAwards.controller.js:14-17`; `controllers/disbursements.controller.js:21-27`
Database	`CHECK` constraints and foreign keys that restate the domain rules	`schema.sql`; migration `20261023_bursary_award_reconciliation.sql:21-31, 70-96`

Normalisation is applied before comparison and before storage so that duplicate detection is not defeated by formatting: names collapse internal whitespace, identifiers are stripped to alphanumerics and upper-cased, and Kenyan phone numbers are normalised to `+254` form (`services/studentRecordFormat.js`; `ml-service/feature_engineering.py`).

Data quality requirements. Three quality rules are enforced by design rather than by inspection.

1. *A missing identifier is never a match.* The feature builder treats empty strings, `NA`, `NULL`, `UNKNOWN`, `NOT PROVIDED`, `NOT AVAILABLE` and runs of zeros as missing, and returns `0.0` for similarity instead of comparing them (`feature_engineering.py:21-33, 81-86`).
2. *A date is compared as a calendar date.* PostgreSQL `DATE` columns arrive at the Node layer as a JavaScript `Date` at local midnight; the boundary to the predictor normalises them to `YYYY-MM-DD` so a stored date and a freshly submitted date are never a day apart (`mlDuplicateReview.js:61-82`; `tests/predictorDateNormalization.test.js`).
3. *An identifier pair is compared once.* A unique index over the unordered student pair (`LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id)`) prevents the same pair creating two cases (`schema.sql:95-96`).

Data storage and relationships. Data is stored in one PostgreSQL database reached through a connection pool (`config/db.js`). The relationships are documented in §4.6; in summary: one `users` row creates many `students`; every `student` may be party to many `duplicate_flags`; one `student` may hold many `bursary_awards`; each award may have many `disbursements` and at most one `bursary_certificates` row; and every state-changing action appends to `audit_logs`.

Data protection and access restrictions. Protection is enforced at four levels: TLS for the public HTTPS origins through which all traffic flows; the CORS allowlist and security headers (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy: microphone=(), geolocation=()`) on every response (`index.js:33-39`); role-based authorisation on every non-public route; and field-level masking of identifiers in the review queue with an audited reveal. Secrets (database URL, JWT secret, bootstrap passwords) are supplied through environment variables or the hosting platform's secret store, are excluded from the container image by `.dockerignore`, and are never rendered in the interface or written to logs.

Duplicate and missing data handling. Duplicates are deliberately **not** prevented by a unique constraint on the identifier columns: `students.upi` is nullable and its earlier uniqueness constraint was explicitly dropped (`schema.sql:47-49, 57`), because the project's premise is that near-duplicates cannot be detected by exact matching and must instead be routed to human review. Missing values are handled rather than rejected: date of birth, school, guardian identifier, UPI and birth-certificate number are all optional, and a student may be registered with only a name and a national identifier, in which case screening is necessarily weaker and the audit record shows how many features were unavailable.

Machine-learning data. The production model was trained on a generated corpus of 12 000 labelled record pairs (6 000 duplicate groups, balanced 6 000/6 000), drawn from a simulator that corrupts names (misspelling, reordering, truncation), introduces date-of-birth errors and school transfers, changes guardians, reissues identifiers, and adds deliberate confusers — siblings, twins, common names within one school, and reused identifiers on non-duplicates (`ml-service/evaluation/realistic_pairs_xgboost_training_metrics.json`, `generator_scenarios`). Training and evaluation were separated by identity: an 80/20 grouped internal split, then an independently generated holdout of 12 000 pairs from new latent identities and a separate random seed. The holdout shares no persons with the training rows, but it does share the generator, and the metrics report states this in its `limitations` block. The repository contains **no** adjudicated real-world label set: `ml-service/evaluation/reviewed_pairs_template.csv` is header-only (`docs/ml-decision-sheet-2026-10-30.md` §7, D5).

Feature extraction. Six features are computed per pair, in a fixed order, by one module used for both training and serving (`feature_engineering.py:11-18`):

Feature	Type	Derivation
`jaro_winkler`	continuous, 0–1	Maximum of Jaro–Winkler similarity on the concatenated name tokens and on the alphabetically sorted tokens, so reversed name order still scores highly
`dob_match`	0 / 0.5 / 0.8 / 1	1 for an identical calendar date, 0.8 for one day apart, 0.5 for up to seven days, 0 otherwise; five date formats accepted, ambiguous US ordering deliberately not assumed
`school_match`	binary	Exact match of normalised school identifiers
`guardian_match`	continuous, 0–1	Exact identifier match after stripping shared prefixes (`G`, `UPI`, `BC`, …), otherwise Jaro–Winkler similarity
`upi_match`	continuous, 0–1	As `guardian_match`, for the UPI
`birth_certificate_match`	continuous, 0–1	As `guardian_match`, for the birth-certificate number

The national identifier is **not** a model feature. It is used outside the model as a separate human-review signal (`services/mlDuplicateReview.js:33-39`), together with exact UPI and birth-certificate agreement, because an exact identifier agreement is evidence a reviewer can check against a source document whereas a similarity score is not.

Retention, backup and recovery. No retention policy, archival job or scheduled backup is implemented in the application or in the deployment configuration. Backup and restore were rehearsed manually and the evidence is recorded in `docs/database-recovery-rehearsal-2026-10-31.md`; the hosted database is a PostgreSQL Flexible Server stopped between demonstrations, whose storage persists while compute is stopped. The absence of an automated backup schedule is a documented limitation carried into Chapter Six.

4.2.5 Interface and integration requirements

The system has five interfaces. Each is described by its protocol, message format, authentication rule and error behaviour, because the evaluation in Chapter Five depends on those contracts being stated.

Table 4.3: Interfaces and integration points

Interface	Parties	Protocol and format	Authentication	Error behaviour
I-1 Browser to API	React SPA → Express API	HTTPS, JSON over `axios`; `Authorization: Bearer <JWT>` injected by a request interceptor (`frontend/src/services/api.js`)	JWT with 8-hour expiry; invalid or expired token yields 401 and the SPA returns the user to sign-in	4xx with `{message}`; 5xx with a generic message; refused origin yields 403
I-2 API to PostgreSQL	Express → `pg` pool	TCP with TLS to the managed server; parameterised SQL only	Database credentials from the environment	Errors logged without SQL text or parameters; generic 500 returned
I-3 API to ML predictor	Node child process → `predict.py --server`	Line-delimited JSON over process standard input/output; request `{pairs:[{first,second}]}`, response `{model_version, results:[{score, features}]}`	Local process boundary only; never exposed on a network socket	Predictor exit, 30 000 ms timeout or malformed payload raises `DUPLICATE_PREDICTOR_FAILED`, which registration converts to HTTP 503
I-4 Student QR verification	QR → public page → API	QR encodes `…/verify#token=<64 hex>`; the page posts `{token}` to `POST /api/verify`	Public; rate limited to 10 requests per IP per 15 minutes	Malformed, unknown and refused tokens all return `{found:false}`; lookup failure returns 503 `{error:'temporarily_unavailable'}`; rate refusal returns 429
I-5 Certificate QR verification	QR → public page → API	QR encodes `…/verify-certificate#token=<64 hex>`; the page posts `{token}` to `POST /api/certificates/verify`	Public; separate budget of 20 requests per IP per 15 minutes	`not_recognised` for any malformed, unknown or manipulated token; `revoked`; `not_current`; `verified`; 503 on lookup failure

Two further integration points are **absent**, recorded so that the report does not imply more than was built. PDF rendering is internal — `pdfkit` writes the report to a `reports/` directory and renders the certificate PDF per request in memory — and there is no external document service. There is also no external government or financial

integration: NEMIS, the national identifier verification service and any bank or mobile-money transfer interface are absent by scope (FR-32, FR-37).

The QR design deserves separate comment because it carries most of the project's privacy argument. Both token families are 32 random bytes rendered as 64 lowercase hexadecimal characters and are placed in the **URL fragment**, not the query string. A fragment is never transmitted to a web server, never written to an access log and never forwarded as a `Referer` (`config/publicAppUrl.js:164-177`). The public pages read the fragment, remove it from the visible address with `history.replaceState`, and post it to the API (`frontend/src/hooks/useHashToken.js`; `pages/Verification.jsx`; `pages/CertificateVerification.jsx`). `PUBLIC_APP_URL` is validated at start-up and again at issue time, and a non-public origin — including a LAN address, a bare hostname or an `example.*` domain — is refused, so a QR code can never be generated that a phone on a mobile network cannot reach (`config/publicAppUrl.js:119-162`; `docs/qr-verification-2026-10-30.md` §5).

4.3 System Analysis Diagrams

This section presents the analysis models. Each figure is specified rather than embedded: the placeholder states exactly what the diagram must contain, so that the inserted diagram is derived from the implementation and not from an idealised description of it. The notation is UML 2.x for the use case, activity, sequence and class models and Chen notation for the entity-relationship model. Terminology is fixed across all figures and matches the database and the code: *Administrator* (`admin`), *Field Officer* (`field_officer`), *Public verifier*, *Duplicate predictor* (the Python process), and the data objects `students`, `duplicate_flags`, `duplicate_review_events`, `bursary_awards`, `bursary_award_verification_events`, `disbursements`, `disbursement_reconciliation_events`, `bursary_certificates`, `audit_logs`, `reports`.

4.3.1 System context diagram

A context diagram is required because the system boundary matters: several capabilities that a reader might assume belong to a beneficiary management system are deliberately outside it. The context diagram shows SBMS as a single process and the external entities that exchange data with it. It is **not** a level-0 data flow diagram: it shows no internal processes and no data stores, and its purpose is to fix the boundary.

[Insert Figure 4.1: System context diagram of SBMS here]

Figure 4.1: System context diagram. The figure must show the system as one rounded process labelled *SBMS (React SPA, Express API, PostgreSQL, Python duplicate predictor)* at the centre, with the external entities and the direction of each flow:

External entity	Flows into SBMS	Flows out of SBMS
Field Officer	sign-in credentials; beneficiary registration data; award applications; verification outcome and evidence reference; payment and clearance entries; receipt acknowledgement	JWT session; student lists; QR image for the registered student; screening outcome in plain language; eligibility assessment; award and payment status
Administrator	sign-in credentials; award decisions; duplicate-case decisions; identifier-reveal requests; threshold changes; certificate issue and revocation instructions	duplicate queue with masked identifiers; dashboard aggregates; audit log; PDF reports; certificate PDF and QR
Public verifier (school, guardian, awarding office)	student QR token or certificate token scanned from a printed document	`found` / `not found`; certificate status (`verified`, `revoked`, `not_current`, `not_recognised`)
Duplicate predictor (shown as an external collaborator because it is a separate process)	six ordered feature values per candidate pair	model score and model version per pair
Hosting platform (Container Apps, App Service, PostgreSQL Flexible Server)	runtime, TLS termination, database service	container image, static frontend bundle, database state

The diagram must **not** show NEMIS, a bank, a mobile-money operator, an SMS gateway or a national identifier verification service, because none is integrated. It must also not show the system automatically rejecting, deleting or mutating a beneficiary record, because no such flow exists.

4.3.2 Use case diagram

[Insert Figure 4.2: Use case diagram of SBMS here]

Figure 4.2: Use case diagram. Three actors sit outside the system boundary: **Field Officer**, **Administrator** and **Public verifier**. *Duplicate predictor* is an internal collaborating actor, connected only to the registration and rescan use cases. The use cases, with their actor associations, are:

Use case	Field Officer	Administrator	Public verifier	Included / extending behaviour
UC-01 Sign in (role-specific page)	✓	✓	—	«include» verify credentials; «extend» refuse wrong-role account
UC-02 Register beneficiary	✓	—	—	«include» UC-03
UC-03 Screen for duplicates	—	—	—	«include» score pairs; «include» select review cases per aggregation policy
UC-04 View registered beneficiaries	✓	—	—	—
UC-05 Assess eligibility	✓	—	—	—
UC-06 Submit bursary award	✓	—	—	«include» require school on record
UC-07 Record beneficiary verification	✓	—	—	«include» require evidence reference when verified
UC-08 Review duplicate queue	—	✓	—	«include» UC-09
UC-09 Reveal identifiers (audited)	—	✓	—	—
UC-10 Decide duplicate case	—	✓	—	«include» append review event; «include» write audit entry
UC-11 Re-scan existing registry	—	✓	—	«include» score all unflagged pairs
UC-12 Decide award (approve or reject)	—	✓	—	«include» check verification; «include» check open duplicates
UC-13 Record payment as issued	✓	✓	—	«include» check approved award; «include» check duplicates; «include» check amount
UC-14 Clear payment	✓	✓	—	«include» require evidence reference
UC-15 Acknowledge receipt	✓	✓	—	«extend» flag amount discrepancy
UC-16 Review reconciliation exceptions	✓	✓	—	—
UC-17 Issue award certificate	—	✓	—	«include» require approved award
UC-18 Revoke award certificate	—	✓	—	«include» require reason
UC-19 Read or print certificate	✓	✓	—	—
UC-20 View dashboard	—	✓	—	—
UC-21 Export PDF report	—	✓	—	—
UC-22 View audit log	—	✓	—	—
UC-23 Manage thresholds	—	✓	—	«include» write audit entry
UC-24 Manage user accounts	—	✓	—	—
UC-25 Verify student QR	—	—	✓	—
UC-26 Verify certificate QR	—	—	✓	—

Two generalisations should be drawn explicitly: *Administrator* and *Field Officer* are both specialisations of an abstract *Authenticated staff user*, because both sign in and both read their own profile; and the two public use cases

(UC-25, UC-26) are the only use cases reachable without authentication. Because a use case diagram cannot express the restriction that matters most — that an administrator cannot register a student and a field officer cannot decide a duplicate case — Table 4.6 is the authoritative statement of those rules, and the figure should carry a note directing the reader to it.

4.3.3 Activity diagram: registration and duplicate screening

This is the system's most consequential process, because it is the point at which a machine score influences an administrative workflow. The model must show that the student record is created **whatever the screening result**, and that the only effect of a positive result is the creation of review cases.

[Insert Figure 4.3: Activity diagram for beneficiary registration and duplicate screening here]

Figure 4.3: Activity diagram — registration and duplicate screening. Swimlanes: *Field Officer*, *API (students controller)*, *Duplicate predictor*, *PostgreSQL*. The activity flow must contain these steps in this order:

1. Field Officer submits the registration form (first name, last name, date of birth, school, national identifier, optional birth-certificate number, optional UPI, phone).
2. API validates the payload (§4.2.4); on failure it returns 400 with field-level errors and the flow ends.
3. API resolves the review threshold and the aggregation policy **before** opening a transaction, so a misconfiguration fails without holding a database transaction open.
4. API generates a 64-hex QR token and builds the verification URL from the validated `PUBLIC_APP_URL`; a bad origin aborts the request here.
5. API begins a transaction and takes the advisory lock `sbms.student.registration`.
6. API selects every existing student `FOR UPDATE`.
7. API computes six features for each candidate and calls the predictor in batches of 200.
8. Decision: predictor unavailable, timed out or returned an invalid payload → roll back, return 503, and no student is created.
9. Decision per candidate: score ≥ review threshold, or an exact UPI / birth-certificate / national identifier agreement → candidate qualifies; otherwise it does not.
10. Aggregation decision point (policy `top-1` by default): among qualifying candidates, rank exact identifier agreement above score and select at most one.
11. API inserts the student row, then inserts one `duplicate_flags` row for each selected candidate, with score, model version, threshold and feature vector.
12. API writes the audit entry, including candidates considered, qualifying and suppressed, and commits.
13. API returns 201 with the student (without the QR token), the QR image and the screening summary; the Field Officer sees either "no match", "possible duplicate match found — an administrator must review this case", or the exact-identifier sentence.

The diagram must show the aggregation decision as a **policy node**, not a model node, and a note must record that no branch of the flow deletes, rejects or edits a beneficiary.

4.3.4 Activity diagram: duplicate review, award approval and payment gate

The second activity model explains how a review case propagates into money. The property that matters is that a student with an **open, further-verification or confirmed** case cannot be approved or paid, and that only an administrator can release the hold.

[Insert Figure 4.4: Activity diagram for duplicate review, award approval and the payment gate here]

Figure 4.4: Activity diagram — review, approval and payment gate. Swimlanes: *Administrator*, *Field Officer*, *API*, *PostgreSQL*. Required structure:

1. Administrator opens the duplicate queue; the API returns each case with masked identifiers, model score, model version and the threshold in force.

2. Decision: are the full identifiers needed to compare source documents? - **Yes** → reveal request → API writes a `REVEAL_DUPLICATE_IDENTIFIERS` audit row and returns the five identifier fields → continue. - **No** → continue with masked values.
3. Administrator records a decision with a reason of at least five characters. The API updates the case only from `pending` or `needs_verification`, appends a `duplicate_review_events` row and writes an audit entry. A terminal case cannot be re-decided (404).
4. Decision per outcome: - `not_duplicate` → the hold is cleared for that pair; - `needs_verification` → the hold persists and a later decision may still be recorded; - `confirmed_duplicate` → the hold persists permanently and cannot be reversed through the API.
5. Field Officer submits an award (`requested_amount`) and records a beneficiary verification outcome. A `verified` outcome without an evidence reference is refused (400).
6. Administrator approves or rejects. Approval is refused with 409 unless (a) the beneficiary is verified and (b) no case against that student is `pending`, `needs_verification` or `confirmed_duplicate`, where the student may be either side of the pair.
7. Field Officer or Administrator records a payment against the approved award. The API re-checks award status, verification status, the duplicate hold and the remaining approved balance, and refuses a payment exceeding the balance (409).
8. Payment transition: `issued` → `cleared` (evidence required), or `issued` → `failed` / `cancelled` (reason of at least five characters required).
9. Receipt acknowledgement: the officer records the amount actually received; a shortfall or excess is stored as `discrepancy` and appears in the exceptions list rather than being corrected silently.
10. Every step from 5 to 9 writes both a domain event row and an `audit_logs` row.

Step 7 must be drawn as a **second, independent gate** rather than a consequence of step 6, because the implementation re-validates at payment time and a case raised after approval must still block payment.

4.3.5 Class diagram

The class model describes the application-level domain and its behaviour, not the database tables (those are modelled in §4.3.10).

[Insert Figure 4.5: Class diagram of the SBMS domain here]

Figure 4.5: Class diagram. Classes, principal attributes and operations, with multiplicities:

Class	Principal attributes	Principal operations	Relationships
`User`	id, full_name, email, password_hash, role, is_active, created_at	`findByEmail`, `createUser`, `authenticateToken`, `requireRole`	1 User creates 0..* Student; resolves 0..* DuplicateFlag; decides 0..* BursaryAward; issues 0..* Certificate
`Student`	id, full_name, date_of_birth, school_code, guardian_id, upi, birth_certificate_number, national_id_number, primary_phone, qr_token, is_synthetic, eligibility_score, eligibility_category, created_at	`create`, `getById`, `getAll`, `hasQrToken`, `withoutQrToken`	participates in 0..* DuplicateFlag (as subject or as match); holds 0..* BursaryAward
`DuplicateFlag`	id, student_id, matched_student_id, probability, status, model_version, review_threshold, comparison_features, resolved_by, resolved_at, updated_at	`list`, `evaluate`, `scanExisting`, `update`, `revealIdentifiers`	belongs to 2 Students; has 1..* DuplicateReviewEvent
`DuplicateReviewEvent`	id, duplicate_flag_id, reviewer_id, decision, reason, notes, created_at	append-only	belongs to 1 DuplicateFlag and 1 User
`BursaryAward`	id, student_id, school_code, requested_amount, approved_amount, status, verification_status, verification_evidence_ref, verified_by, verified_at, decision_reason, decided_by, decided_at	`list`, `create`, `recordVerification`, `decide`	1 Student holds 0..* BursaryAward; 1 BursaryAward has 0..* Disbursement and 0..1 Certificate

Class	Principal attributes	Principal operations	Relationships
`Disbursement` (payment)	id, student_id, award_id, amount, payment_method, payment_reference, payment_status, clearance_evidence_ref, receipt_status, acknowledged_amount, acknowledgement_method	`list`, `listExceptions`, `create`, `updatePaymentStatus`, `acknowledgeReceipt`, `linkLegacyPayment`	belongs to 1 Student and 0..1 BursaryAward; has 0..* ReconciliationEvent
`BursaryCertificate`	id, award_id (unique), certificate_reference, verification_token, award_period, student_name, student_upi, school_code, approved_amount, award_approved_at, status, issued_by, issued_at, revoked_by, revoked_at, revocation_reason	`insert`, `findByAwardId`, `findByVerificationToken`, `revoke`, `issue`, `downloadPdf`, `verifyPublic`	belongs to exactly 1 BursaryAward
`AuditEntry`	id, user_id, action, table_name, record_id, details, created_at	`logAudit`	belongs to 0..1 User
`DuplicatePredictor` (service)	model file, SHA-256 version, feature order	`scoreStudentPairs`, `predict_pairs`, `model_version`	called by `Student` registration and by `DuplicateFlag` rescan
`AggregationPolicy` (service)	policy name (`top-1`, `top-1-corroborated`, `all`), threshold	`selectFlaggedCandidates`, `summarizeAggregation`	used by `Student.create`

Three design points must be annotated on the figure. First, `DuplicateFlag.probability` is a legacy column name: for model-created rows it holds an **uncalibrated positive-class score**, and the schema comment says so. Second, no class exposes an operation that rejects or deletes a student; the only student-mutating operations are creation and eligibility update. Third, `BursaryCertificate` duplicates award facts on purpose, so a later award edit cannot rewrite a printed document.

4.3.6 System sequence diagram: registration

A system sequence diagram (SSD) treats the system as a black box and shows only the actor-to-system messages for one scenario, establishing the system operations that the detailed sequence diagrams then realise.

[Insert Figure 4.6: System sequence diagram for beneficiary registration here]

Figure 4.6: System sequence diagram — register a beneficiary. Actors: *Field Officer* (left), *:SBMS* (right, a single lifeline). Messages, in order:

1. `openRegistrationPage()` → the system returns the form and populates the school list from `GET /api/students/schools`.
2. `submitRegistration(firstName, lastName, dateOfBirth, school, nationalId, [birthCertificateNumber], [upi], phone)`.
3. The system returns `201 {student, qr_code, duplicate_flags, duplicate_screening}`, **or** `400` with field errors, **or** `503` when screening is unavailable.
4. `viewScreeningResult()` → the officer reads one of three states: no candidate; a possible duplicate match requiring administrator review; or an exact identifier match named in plain language.
5. `printQrCode()` → the officer prints or displays the QR image returned in step 3.

The SSD must show an **alternate frame** for the 503 outcome: registration is refused, no student record exists afterwards, and the officer is told to retry. A note must state that the model score and threshold are deliberately absent from the officer's view.

4.3.7 Sequence diagram: duplicate review decision

[Insert Figure 4.7: Sequence diagram for an administrator's duplicate review decision here]

Figure 4.7: Sequence diagram — decide a duplicate case. Lifelines: *Administrator*, *DuplicateReview (React)*, *duplicateFlags.controller*, *PostgreSQL*. Messages:

1. `GET /api/duplicate-flags` → the controller authenticates, checks `admin`, queries the joined case list, masks `guardian_id`, `upi`, `birth_certificate_number` and `national_id_number` to their last three characters, and returns the policy block (review threshold, high-priority threshold, aggregation policy, `thresholds_validated: false`).
2. `PATCH /api/duplicate-flags/:id {decision, reason, notes}` → the controller validates the decision enumeration and a reason of at least five characters.
3. `BEGIN`; `UPDATE duplicate_flags SET status, resolved_by, resolved_at WHERE id AND status IN ('pending','needs_verification') RETURNING *`.
4. **alt** no row updated → `ROLLBACK`, `404 {message:'Duplicate flag not found'}`.
5. `INSERT INTO duplicate_review_events (duplicate_flag_id, reviewer_id, decision, reason, notes)`.
6. `INSERT INTO audit_logs (… action = 'DUPLICATE_' || decision …)` carrying the model version, the score and the review threshold that created the case.
7. `COMMIT`; respond `200 {message, flag}`.
8. Optional earlier message: `GET /api/duplicate-flags/:id/identifiers` → the controller returns the five identifier fields for both students and writes a `REVEAL_DUPLICATE_IDENTIFIERS` audit row.

The reveal must be drawn as a **separate interaction with its own audit write**, because the design decision is that disclosure is an event, not a field permission.

4.3.8 Sequence diagram: award approval, payment and certificate issuance

[Insert Figure 4.8: Sequence diagram for award approval, payment reconciliation and certificate issuance here]

Figure 4.8: Sequence diagram — approval to certificate. Lifelines: *Field Officer*, *Administrator*, *bursaryAwards.controller*, *disbursements.controller*, *certificates.controller*, *PostgreSQL*. Messages:

1. `POST /api/awards {student_id, requested_amount}` (field officer) → the student must exist and must have a school; the award row is created with `status='submitted'` and `verification_status='not_checked'`.
2. `POST /api/awards/:id/verification {outcome, notes, evidence_ref}` (field officer) → a `verified` outcome without an evidence reference is refused (`400`); an event row is appended and the award's verification fields updated.
3. `PATCH /api/awards/:id/decision {decision:'approved', approved_amount, reason}` (administrator) → advisory lock; `SELECT … FOR UPDATE`; status must be `submitted`; verification must be `verified`; no `pending`, `needs_verification` or `confirmed_duplicate` case may exist on either side of the student's pairs; then `status='approved'` and an `AWARD_APPROVED` audit row.
4. `POST /api/disbursements {award_id, amount, payment_method, payment_reference, disbursement_date}` (either role) → advisory lock; the award must be `approved` and `verified`; the duplicate check is repeated; `SUM(existing amounts) + amount` must not exceed `approved_amount`; insert with `payment_status='issued'` and `receipt_status='awaiting_acknowledgement'`; append a `payment_issued` event; audit `PAYMENT_ISSUED`.
5. `PATCH /api/disbursements/:id/payment-status {payment_status:'cleared', evidence_ref}` → transition from `issued` only; evidence required; `payment_cleared` event; audit `PAYMENT_CLEARED`.
6. `PATCH /api/disbursements/:id/acknowledgement {acknowledged_amount, evidence_ref}` → compares the payment and acknowledged amounts in integer cents; `acknowledged` when equal, `discrepancy` otherwise; `receipt_acknowledged` event; audit `RECEIPT_ACKNOWLEDGED_OFFICER_ASSISTED`.
7. `POST /api/awards/:id/certificate {award_period?}` (administrator) → the award must be `approved` and must not already have a certificate; a snapshot row is inserted with a fresh 64-hex token and an `SBMS-CERT-<year>-<8 hex>` reference; `AWARD_CERTIFICATE_ISSUED` audit row.
8. `GET /api/awards/:id/certificate.pdf` (either role) → PDF rendered per request carrying the same QR code; `Cache-Control: no-store`.

9. `POST /api/awards/:id/certificate/revoke {reason}` (administrator) → status `active` → `revoked` with actor, timestamp and reason; `AWARD_CERTIFICATE_REVOKED` audit row; the award is unchanged.

Two distinctions must be visually explicit: **approval is not payment** (steps 3 and 4 are separate lifelines with separate audit actions), and **certificate verification is not student verification** (step 7 uses the certificate token namespace, not the student QR token).

4.3.9 Sequence diagram: public QR verification

[Insert Figure 4.9: Sequence diagram for public student-record and certificate verification here]

Figure 4.9: Sequence diagram — public verification. Lifelines: *Public verifier (phone browser)*, *Verification page*, *verification.controller*, *certificates.controller*, *PostgreSQL*. Messages:

1. The verifier scans the printed code; the phone opens `https://<PUBLIC_APP_URL>/verify#token=<64 hex>` or `/verify-certificate#token=<64 hex>`.
2. The page reads the fragment, removes it with `history.replaceState`, and (re)runs verification on `hashchange` so a second scan in the same tab cannot display the previous result.
3. `POST /api/verify {token}` → rate limiter runs **before** the JSON parser; the controller requires exactly one key named `token` matching `^[a-f0-9]{64}$`; `SELECT EXISTS(… WHERE qr_token = $1)`.
4. Response `{found:true}` or `{found:false}`, and nothing else; malformed and unknown tokens are indistinguishable; a lookup failure returns `503 {error:'temporarily_unavailable'}`; the eleventh request in the window returns `429`.
5. `POST /api/certificates/verify {token}` → same shape discipline with its own 20-per-15-minute budget; the controller loads the certificate together with the live award and returns one of four states: `verified`, `revoked`, `not_current`, `not_recognised`.
6. The page renders a status-only view: no student name, UPI, school, date of birth, identifier, phone or amount appears in any response or on any page.

The figure must show the rate limiter as the **first** participant after the page, because the implementation registers the limiter in the same route chain ahead of the body parser, and must show the two endpoints as independent chains with independent budgets.

4.3.10 Entity-relationship diagram

[Insert Figure 4.10: Entity-relationship diagram of the SBMS database here]

Figure 4.10: Entity-relationship diagram. Chen notation with cardinalities; every attribute shown must exist in `backend/src/db/schema.sql` or in one of the migrations listed in §4.6.1. Entities and relationships:

Entity	Relationship	Entity	Cardinality	Participation
`users`	creates	`students`	1 : N	Student creation is not stored as a foreign key on `students`; the creating user is recorded in `audit_logs`
`students`	is subject of	`duplicate_flags`	1 : N	Total on the flag side (`student_id NOT NULL`)
`students`	is matched by	`duplicate_flags`	1 : N	Total on the flag side (`matched_student_id NOT NULL`)
`users`	resolves	`duplicate_flags`	1 : N (nullable)	Partial: `resolved_by` is null until a decision
`duplicate_flags`	has	`duplicate_review_events`	1 : N	Total on the event side
`users`	reviews	`duplicate_review_events`	1 : N	Total
`students`	holds	`bursary_awards`	1 : N	Total; `ON DELETE RESTRICT`
`users`	submits / verifies / decides	`bursary_awards`	1 : N (nullable)	Partial per role column

Entity	Relationship	Entity	Cardinality	Participation
`bursary_awards`	has	`bursary_award_verification_events`	1 : N	Total
`students`	receives	`disbursements`	1 : N	Total
`bursary_awards`	is paid by	`disbursements`	1 : N (nullable)	Partial: legacy payments have no `award_id`
`disbursements`	has	`disbursement_reconciliation_events`	1 : N	Total
`bursary_awards`	is certified by	`bursary_certificates`	1 : 0..1	`award_id UNIQUE`
`users`	issues / revokes	`bursary_certificates`	1 : N (nullable for revoke)	Partial
`users`	performs	`audit_logs`	1 : N (nullable)	Partial: system actions may have no user
`users`	generates	`reports`	1 : N (nullable)	Partial

The figure must carry three annotations. The `beneficiaries` table stands **outside** the modelled relationships: it has a serial integer key, no foreign keys, and is retained only because the legacy endpoint and the readiness check require it. The `students.qr_token` attribute has a **partial unique index**, not a table constraint. `duplicate_flags` carries a unique index over the **unordered pair** `(LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id))`, which is what guarantees that the same two students cannot generate two cases.

4.3.11 Machine-learning duplicate-detection pipeline

Because the research contribution is the screening pipeline rather than the CRUD application, the pipeline is modelled separately from the application workflow.

[Insert Figure 4.11: Machine-learning duplicate-detection data pipeline here]

Figure 4.11: ML duplicate-detection pipeline. The figure must be drawn in two lanes, *offline (training and evaluation)* and *online (serving)*, joined only at the shared feature module, because that shared module is the property that makes the served score the same quantity the metrics describe.

Offline lane: generated record pairs → corruption scenarios (`repeat_application`, `siblings_shared_household`, `twins_or_close_age`, `common_name_same_school`, `identifier_collision`, `unrelated_candidate`) → `feature_engineering.build_pair_features` (six features) → grouped 80/20 split by identity → XGBoost training with the selected hyperparameters (`n_estimators 250`, `max_depth 4`, `learning_rate 0.05`, `subsample 0.9`, `colsample_bytree 0.9`, `reg_lambda 1.0`) → independently generated holdout of 12 000 pairs → evaluation harness (`evaluate_operating_point.py`, `evaluate_false_positive_rate.py`) → metrics report and operating-point evidence → artefact `model/xgb_duplicate_model.joblib` with SHA-256 identity.

Online lane: `POST /api/students` → candidate set (all existing students) → `scorePairs` batching at 200 pairs → predictor process (`predict.py --server`, one process, line-delimited JSON) → `build_pair_features` per pair → `predict_proba[:,1]` → score per pair streamed back → aggregation policy (`top-1`) → `duplicate_flags` row with score, model version, threshold and feature vector → administrator review queue → human decision.

The figure must also show, as a separate off-page artefact, `model/research/rf_attachment_style_candidate.joblib` and `experiments/sbms_dataset/artifacts/sbms_dataset_xgb_research_only.joblib`, marked **not loaded by the application**, because conflating them with the production artefact would misstate the system's evidence base.

4.4 System Design

4.4.1 Overall architectural design

SBMS follows the three-tier client–server architecture named in the approved proposal (§3.4.5), with one deliberate refinement: the machine-learning component is **not** a separate network service. It runs as a long-lived child process of the API, driven over standard input and output.

[Insert Figure 4.12: Three-tier architecture and component deployment of SBMS here]

Figure 4.12: System architecture. Tiers and components:

Tier	Component	Technology	Responsibility
Presentation	Single-page application	React 18.3, React Router 7, Tailwind CSS 4, `chart.js` + `react-chartjs-2` for dashboard charts, `lucide-react` icons, `axios` for HTTP	Eighteen routes: two public verification pages, three sign-in routes and role-protected operational screens; the session token is held in `localStorage` and attached by an interceptor
Application	REST API	Node.js 22 (`node:22-bookworm-slim`), Express 4, `zod`, `bcrypt`, `jsonwebtoken`, `pg`, `qrcode`, `pdfkit`	Authentication, authorisation, validation, duplicate-case lifecycle, award and payment rules, certificate issue and revocation, reporting, audit
Application (sub-process)	Duplicate predictor	Python 3 in a `venv`, `pandas`, `scikit-learn`, `xgboost`, `jellyfish`	Loads one artefact, builds six features per pair, returns `predict_proba` scores and the artefact hash
Data	Relational database	PostgreSQL 16 (managed Flexible Server in Azure; 17.11 on the development machine)	Durable state, referential integrity, `CHECK` constraints, unique indexes

Three architectural decisions require justification because they are not the obvious choices.

The ML component is a child process, not an HTTP service. The predictor is started once with `predict.py --server`, kept alive and addressed with one JSON line per request (`services/duplicateCheck.js:260-290`). This avoids a second network hop, a second authentication problem and a second deployment unit, and it is why the whole system ships as **one container image**. The cost is that the predictor's lifetime is coupled to the API process: if it exits, every pending request is rejected and the next request respawns it (NFR-20). The same decision means the API cannot be scaled horizontally without giving each replica its own predictor and its own copy of the in-memory rate-limit counters (NFR-07).

Two separately deployed web units. The frontend is a static bundle served by App Service and the API is a container, joined by `VITE_API_ORIGIN` at build time and by `CORS_ORIGINS` at run time. This was forced by the hosting constraint that Static Web Apps were unavailable in the only permitted Azure regions (`docs/azure-deployment-log-2026-10-31.md` §2), and it has a real consequence recorded in the same log: a relative `fetch("/api/verify")` from the static host never reached the API, which was a genuine production defect until the call was routed through the axios client.

Migrations are never applied automatically. The API validates the schema it needs at start-up and exposes readiness, but applying SQL is an explicit, backed-up operator action (`backend/scripts/migrate.js`; `README.md` "Database operations"). This is a safety decision for a system holding beneficiary records, and it is why `/api/ready` fails closed when a migration is missing.

4.4.2 Backend module design

The backend is organised by responsibility rather than by feature, and the boundaries are visible in what each module is allowed to do.

Table 4.4: Backend modules and responsibilities

Layer	Modules	Responsibility	Notes
Entry	`src/index.js`	Security headers, CORS, proxy trust, body parsing, public route chains, route mounting, health and readiness, fail-closed start-up	Start-up validates JWT secret, `PUBLIC_APP_URL` and the authentication schema before listening
Configuration	`config/db.js`, `jwtSecret.js`, `publicAppUrl.js`, `cors.js`, `settings.js`, `schools.js`, `reconciliation.js`	Pool, secret validation, public-origin validation, allowlist parsing, live thresholds, approved-school list, acknowledgement deadline	Thresholds are held in memory and are process-local
Middleware	`auth.js`, `requireRole.js`, `loginRateLimit.js`, `verificationRateLimit.js`, `certificateRateLimit.js`, `rateLimitWindow.js`, `errorHandler.js`, `asyncHandler.js`	Authentication, authorisation, rate limiting, sanitised errors, async error propagation	Each limiter owns a separate counter map
Routing	Twelve `routes/*.routes.js` files	Endpoint declaration and role gates only	One `requireRole(...)` per route
Controllers	Fourteen `controllers/*.controller.js` files	Validation, transaction boundaries, domain sequencing, response shapes	Rules that must be reused live in services
Services	`duplicateCheck.js`, `mlDuplicateReview.js`, `duplicateAggregation.js`, `bursaryRules.js`, `eligibility.js`, `certificateDocument.js`, `auditLog.js`, `studentRecordFormat.js`, `databaseReadiness.js`, `databaseStartup.js`	Predictor bridge, batching and priority, aggregation policy, payment state machine and exception rules, eligibility scoring, PDF rendering, audit writes, normalisation, readiness and start-up assertions	`duplicateCheck.js` also retains a deprecated rule engine that no request path calls
Models	`auth.model.js`, `student.model.js`, `disbursement.model.js`, `bursaryCertificate.model.js`, `beneficiary.model.js`, `audit.model.js`	Parameterised SQL only	The student model strips `qr_token` before returning a record
Scripts	`migrate.js`, `bootstrap-production-schema.js`, `provision-test-database.js`, `seed-*.js`, `reset-*.js`, `kenyanSyntheticDataset.js`, `cleanup-legacy-school-data.js`, `inspect-render-db.js`	Migration runner, guarded empty-schema bootstrap, disposable test database, synthetic demonstration data	The empty-schema bootstrap refuses a non-identical target and refuses existing relations

Two design weaknesses are recorded here rather than hidden. First, `duplicateCheck.js` still exports a weighted rule engine (`buildDuplicateFeatures`, `calculateDuplicateScore`, `checkDuplicates`) that has **no production caller**: it is retained for historical comparison and is exercised only by tests, so it is dead weight a maintainer could mistake for the live scoring path. Second, the `/api/beneficiaries` router and `beneficiary.model.js` persist a legacy table whose semantics are not reconciled with the student model, so two partially overlapping notions of "beneficiary" coexist in one schema.

4.4.3 Authentication, authorisation and access-control design

Authentication is a two-step design in which the token proves *who signed in* and the database proves *what that account may do now*. On sign-in the API looks the account up by e-mail, rejects it if `is_active` is false or the bcrypt comparison fails, returns 403 if the account's role does not match the endpoint the user chose, and otherwise signs `{id, role}` with an eight-hour expiry. On every later request the middleware verifies the signature and expiry, then re-reads `id, role` from `users` **and requires the row to still be active** (`middleware/auth.js:17-25`). A signed token therefore cannot outlive a revoked role or a deactivated account, which is the failure mode an eight-hour JWT would otherwise introduce.

Table 4.5: Role model

Role	Value	Population route	Purpose
Administrator	`admin`	Created by an existing administrator through `POST /api/users`	Owns duplicate decisions, award decisions, thresholds, reports, audit review, certificate issue and revocation
Field Officer	`field_officer`	Created by an administrator; the public registration route always answers 410	Owns registration, eligibility assessment, award submission, beneficiary verification, payment recording, certificate reading and printing
Public verifier	*(unauthenticated)*	Not an account	May call only `POST /api/verify` and `POST /api/certificates/verify`

Table 4.6: Access matrix for authenticated endpoints

Endpoint	Method	Admin	Field Officer	Notes
`/api/auth/me`	GET	✓	✓	Authenticated profile
`/api/auth/register`	POST	410	410	Public registration disabled
`/api/users`	GET, POST	✓	403	Administrative provisioning
`/api/students` and `/api/students/:id`, `/api/students/schools`	GET, POST	403	✓	Deliberate: administrators receive beneficiary data from the endpoints built for their screens
`/api/duplicate-flags`	GET	✓	403	Masked identifiers
`/api/duplicate-flags/:id/identifiers`	GET	✓	403	Audited reveal
`/api/duplicate-flags/:id`	PATCH	✓	403	Decision
`/api/duplicate-flags/evaluate`	POST	✓	403	Paired scoring without persistence
`/api/duplicate-flags/scan`	POST	✓	403	Registry rescan
`/api/awards`	GET	✓	✓	
`/api/awards`	POST	403	✓	Submission is a field activity
`/api/awards/:id/verification`	POST	403	✓	
`/api/awards/:id/decision`	PATCH	✓	403	Approval is administrative
`/api/awards/:id/certificate`	POST	✓	403	Issue
`/api/awards/:id/certificate`, `…/certificate.pdf`	GET	✓	✓	Read and print
`/api/awards/:id/certificate/revoke`	POST	✓	403	
`/api/disbursements`, `/exceptions`, `/:id/payment-status`, `/:id/acknowledgement`	GET, POST, PATCH	✓	✓	Both roles may record and clear a payment
`/api/disbursements/:id/award`	PATCH	✓	403	Linking a legacy payment is administrative
`/api/eligibility`	POST	403	✓	
`/api/reports/export`	GET	✓	403	
`/api/audit-logs`	GET	✓	403	
`/api/settings/threshold`	GET, PUT	✓	403	
`/api/admin/dashboard`	GET	✓	403	

The `/api/students` row is the one that most often looks like a defect, and it is worth stating why it is not. Administrators own duplicate review, awards, reports and the dashboard, but each of those screens obtains the beneficiary data it displays from an endpoint built for that screen: the dashboard receives aggregates, the review queue receives the two records belonging to a case with identifiers masked, the awards and payments screens receive joined student identity, and the report receives an aggregate. No administrative screen calls `/api/students`, and the one shared screen, `Disbursements.jsx`, requests the student list only when the signed-in user is a field officer (`frontend/src/pages/Disbursements.jsx:52-53`). The restriction is therefore the designed policy, it is pinned by an explicit regression test (`tests/roleAuthorization.test.js`, "student routes stay field-officer-only while admin

data paths keep working"), and the observation that produced the test is recorded in `SBMS_O10_O11_IMPLEMENTATION_REPORT.md` §2.1 and §5.

4.4.4 Machine-learning integration design

The integration design has four parts: the boundary, the score, the decision rule around the score, and the record of what happened.

The boundary. The model never sees a database row. The controller selects candidates, maps each into two plain objects with exactly the fields the feature builder reads, and hands them to `scorePairs` (`controllers/students.controller.js:69-73`; `services/mlDuplicateReview.js:84-117`). Batches are capped at 200 pairs, results are checked for count, range and stability of `model_version`, and any violation raises rather than silently continuing. The predictor returns `{score, features}` per pair, so a case can be re-derived later without the predictor.

The score. The score is the positive-class probability of the classifier, rounded to four decimal places, and it is **not calibrated**: the code publishes `score_is_calibrated_probability: false` and the schema comment on `duplicate_flags.probability` records that model-created rows hold an uncalibrated score. Three settings surround it:

Setting	Environment variable	Default	Meaning
Review cut-off	`ML_REVIEW_SCORE_THRESHOLD`	`0.50`	A pair at or above this score becomes a review candidate
High-priority cut-off	`DUPLICATE_HIGH_PRIORITY_THRESHOLD`	`0.70`	Orders a case as *high* priority; presentation only
Legacy review threshold	`DUPLICATE_REVIEW_THRESHOLD`	`0.30`	Published for cases created before provenance columns existed
Aggregation policy	`DUPLICATE_AGGREGATION_POLICY`	`top-1`	How many qualifying candidates become cases

The decision rule around the score. This is the part of the design that changed materially during the project, and the change is itself a design finding. Originally every qualifying pair created a flag; because the candidate set is the whole registry, the probability that at least one nuisance flag appears grows as `1 − (1 − p)^N`, which at `p = 0.35%` and `N = 1 000` is about 97% (`services/duplicateAggregation.js:8-14`). The design now separates *scoring* from *case selection*:

- the model decides a per-pair score;
- `qualifiesForReview` decides whether a candidate qualifies, either by score or by an exact identifier agreement the model does not see;
- `selectFlaggedCandidates` ranks qualifiers with exact identifier agreement above score and, under the default policy, returns at most one;
- `summarizeAggregation` records how many candidates were considered, how many qualified and how many were suppressed, so suppression is auditable rather than invisible.

Three policies exist (`top-1`, `top-1-corroborated`, `all`). `all` reproduces the earlier behaviour for comparison; `top-1-corroborated` is marked experimental in the code and the project's own measurement recommends against it, because it removes 34–54% of nuisance flags at the cost of 39–60% of true duplicates (`docs/ml-decision-sheet-2026-10-30.md` §5).

The record. Each case stores the score, the artefact hash as `model_version`, the review threshold under which it was created and the six-feature vector, and the registration audit row stores the aggregation policy and the three candidate counters. A reviewer therefore sees not only "this was flagged" but "flagged by this artefact, at this threshold, under this policy, from this vector, with this many other candidates considered".

What the design deliberately does not do. It does not block, reject, merge, delete or edit a beneficiary; it does not restrict the comparison set (the whole registry is compared); it does not use the national identifier as a feature; and it does not retrain itself. Each is a scope decision recorded in Chapter Five §5.6 and returned to in Chapter Six.

4.4.5 Workflow and state design

Four state machines carry the business rules. They are specified here because Chapter Five asserts transitions, not prose.

Table 4.7: Duplicate-case states

State	Entered by	Permitted next states	Effect on payment
`pending`	Registration or registry rescan creates the case	`confirmed_duplicate`, `not_duplicate`, `needs_verification`	Blocks approval and payment
`needs_verification`	Administrator decision	`confirmed_duplicate`, `not_duplicate`, `needs_verification` (again)	Blocks approval and payment
`confirmed_duplicate`	Administrator decision	terminal (a repeat decision returns 404)	Blocks approval and payment permanently
`not_duplicate`	Administrator decision	terminal	Holds cleared for that pair

Every decision writes a `duplicate_review_events` row whose `decision` is constrained to the three decision values and whose `reason` must be non-blank (`schema.sql`, `duplicate_review_events`), so a case returned to review is still recorded as an event rather than losing its history. A unique index over `LEAST(student_id, matched_student_id)` and `GREATEST(...)` (`duplicate_flags_unordered_pair_unique`) means a pair can hold only one case, whichever direction it was first detected in.

Table 4.8: Bursary-award states

State	Entered by	Conditions	Permitted next states
`submitted`	Field officer submission	Student exists and has a school; `requested_amount > 0`	`approved`, `rejected`
`approved`	Administrator decision	Verification `verified`; no open or confirmed duplicate for the student; `approved_amount` supplied; reason ≥ 5 characters	terminal
`rejected`	Administrator decision	Reason ≥ 5 characters; no amount recorded	terminal

A database constraint enforces the field combination for each state (`bursary_awards_decision_fields_check`), and a second refuses an `approved` row whose verification status is not `verified` (`bursary_awards_approval_verification_check`).

Table 4.9: Payment and receipt states

Field	States	Rules
`payment_status`	`issued` → `cleared` / `failed` / `cancelled`	Only `issued` is reachable on creation; clearance requires `cleared_by`, `cleared_at` and a non-empty `clearance_evidence_ref` (`disbursements_clearance_fields_check`)
`receipt_status`	`NULL` → `awaiting_acknowledgement` → `acknowledged` / `discrepancy`	Acknowledgement records the amount received, the method (`officer_assisted`) and an evidence reference; an unequal amount becomes `discrepancy`, never a correction of the recorded payment

Table 4.10: Certificate states

State	Entered by	Public verification answer
(no certificate)	—	`not_recognised`
`active`	Administrator issue	`verified` when the award is still `approved` at the same amount; `not_current` otherwise
`revoked`	Administrator revoke with a reason	`revoked`

The certificate transition is one-way: a revoked certificate cannot be reissued for the same award, because `award_id` is unique and the issue path refuses an award that already has a certificate row.

4.4.6 Security and privacy design

Security was designed around four threats implied by the project's own problem statement: disclosure of beneficiary data, unauthorised payment, unauthorised disclosure through a printed code, and silent tampering with a decision.

Threat	Control	Where
Credential theft by offline attack	bcrypt cost 12; the API refuses to start with a missing, short or placeholder JWT secret	`controllers/auth.controller.js`; `config/jwtSecret.js`
Credential stuffing	10 sign-in attempts per IP per 15 minutes	`middleware/loginRateLimit.js`
Privilege escalation by forged claim	Role re-read from the database on every request	`middleware/auth.js`
Unauthorised payment	Two independent gates (approval and payment) plus an approved-balance check; both gates query both sides of a duplicate pair	`controllers/bursaryAwards.controller.js:173-189`; `controllers/disbursements.controller.js`
Data disclosure in the review queue	Identifier masking; reveal is a separate audited action; the student model strips `qr_token` from every response	`controllers/duplicateFlags.controller.js`; `models/student.model.js`
Disclosure through a printed code	Token in the URL fragment; public endpoint answers one boolean; no beneficiary field in any public response	`config/publicAppUrl.js`; `controllers/verification.controller.js`
Enumeration through a public endpoint	Fixed token shape; identical answer for malformed and unknown tokens; per-IP budget; separate budgets per endpoint	`middleware/rateLimitWindow.js`; `controllers/certificates.controller.js:248-270`
Tampering with an issued document	Certificate snapshot compared with the live award; revocation recorded with actor, time and reason	`models/bursaryCertificate.model.js`; migration `20261031_bursary_certificates.sql`
Information leakage in errors	5xx responses carry a fixed message; the handler logs an allowlisted diagnostic only	`middleware/errorHandler.js`
Cross-site data access	CORS allowlist plus a pre-routing origin check; security headers on every response	`index.js:33-47`; `config/cors.js`
Bulk or abusive requests	100 kB JSON limit; oversize rejected before the handler	`index.js:64`

Two controls a reviewer might expect are **absent**, and are limitations rather than implied features: there is no multi-factor authentication, and the application applies no encryption at rest of its own (it relies on the managed database service). Sessions are also not individually revocable, because a JWT remains valid until expiry unless the account is deactivated.

4.4.7 Validation and error-handling design

Four conventions apply throughout.

1. **Validation precedes side effects.** Schemas are evaluated before a transaction is opened, and the threshold and aggregation policy are resolved before the connection is taken, so a misconfiguration never leaves a transaction open (`controllers/students.controller.js:54-57`).
2. **Domain refusals are 409, not 400.** "Verify the beneficiary before approving this award", "Resolve duplicate review cases before approving this award" and "Only verified, approved awards can be disbursed" describe conflicts with current state and are expected to be actionable by the client (`controllers/bursaryAwards.controller.js:169-188`).
3. **Public endpoints never distinguish "malformed" from "unknown".** Both produce the same answer, so the endpoint cannot be used to test whether a token was ever issued.
4. **Server errors are opaque.** A sanitised handler returns `{message:'Internal server error'}` for any 5xx and logs only an allowlisted diagnostic, so a stack trace, SQL fragment or credential cannot reach a client or a log

aggregator (`middleware/errorHandler.js:59-78`; `tests/securityMiddleware.test.js`).

4.5 System Design Diagrams (interface design)

The interface design is presented as screen specifications rather than as decorative wireframes. Each specification states the route, the audience, the data the screen requests, the principal controls and the states it must be able to render, so that the screen can be checked against the implementation and reproduced as a figure without invention.

Table 4.11: Frontend route inventory

Route	Audience	Component	Data requested	Purpose
`/login`, `/admin/login`, `/field-officer/login`	Public	`Login.jsx`	`POST /api/auth/login/{admin|field-officer}`	Role-specific sign-in; a wrong-role account is refused with a message naming the other form
`/verify`	Public	`Verification.jsx`	`POST /api/verify`	Student-record verification by QR
`/verify-certificate`	Public	`CertificateVerification.jsx`	`POST /api/certificates/verify`	Certificate authenticity
`/dashboard`	Administrator	`AdminDashboard.jsx`	`GET /api/admin/dashboard`	Operational overview
`/duplicate-review`	Administrator	`DuplicateReview.jsx`	`GET /api/duplicate-flags`, `PATCH /api/duplicate-flags/:id`, `GET /api/duplicate-flags/:id/identifiers`, `POST /api/duplicate-flags/scan`	Case adjudication and registry rescan
`/disbursements`	Administrator, Field Officer	`Disbursements.jsx`, `AwardCertificatePanel.jsx`	`/api/awards`, `/api/disbursements`, `/api/disbursements/exceptions`, and `/api/students` **only for field officers**	Awards, payments, exceptions, certificates
`/reports`	Administrator	`Reports.jsx`	`GET /api/reports/export`	Date-range PDF export
`/audit`	Administrator	`AuditLogs.jsx`	`GET /api/audit-logs`	Accountability review
`/users`	Administrator	`UserManagement.jsx`	`GET`, `POST /api/users`	Account provisioning
`/settings`	Administrator	`Settings.jsx`	`GET`, `PUT /api/settings/threshold`	Thresholds
`/field-officer`	Field Officer	`FieldOfficerDashboard.jsx`	`GET /api/auth/me`	Landing page
`/students`	Field Officer	`Students.jsx`	`GET /api/students`	Registry list
`/students/register`	Field Officer	`StudentRegistration.jsx`	`GET /api/students/schools`, `POST /api/students`	Registration and screening feedback
`/eligibility`	Field Officer	`Eligibility.jsx`	`POST /api/eligibility`	Rule-based eligibility assessment

Shared design decisions apply to every screen: navigation is generated from the signed-in role, so a role never sees a link it cannot use; `ProtectedRoute` redirects an unauthenticated visitor to `/login` and a wrong-role visitor to `/dashboard`; and the two public pages use a separate minimal layout with no navigation, because their audience is not a system user.

4.5.1 Sign-in pages

[Insert Figure 4.13: Sign-in screen here]

Figure 4.13: Role-specific sign-in screen. The figure must show the SBMS identity block, an e-mail field, a password field, a submit button with a busy state ("Signing in…"), an inline error region (`role="alert"`) that renders "Invalid email or password" or the wrong-role message, and the link that switches to the other role's form. It must show the state **after** a failed attempt, because the error region is part of the design.

4.5.2 Field-officer workspace: registry list and eligibility

[Insert Figure 4.14: Field-officer student registry list here]

Figure 4.14: Student registry list (field officer). The figure must show the SBMS header, the role navigation for a field officer (Dashboard, Students, Register Student, Eligibility, Disbursements), a tabular list of registered students with name, date of birth, school identifier and registration date, and an empty state ("No students registered yet") alongside the "Register Student" action. It must not show QR tokens, model scores or duplicate-case information, none of which this screen receives.

[Insert Figure 4.15: Eligibility assessment screen here]

Figure 4.15: Eligibility assessment screen. The figure must show the five inputs (fee balance, household income, vulnerability status, siblings in school, prior bursary history), the computed score, the category, the per-factor breakdown and the human-readable reasons list, because the rule engine returns `{score, category, breakdown, reasons}` and the design intent is explainability rather than a single number.

4.5.3 Registration and screening-confirmation screen

[Insert Figure 4.16: Student registration form here]

Figure 4.16: Student registration form. The figure must show the two-column name fields (first and last name, combined by the client into `full_name`), the date-of-birth date picker, the school selector populated from `GET /api/students/schools` with a disabled "Choose school…" placeholder, the national identifier field, the two **optional** identifier fields (birth certificate number, UPI) with their explanatory comment, the phone field with the `+254712345678 or 0712345678` hint, the submit button with its "Registering…" busy state, and the inline error alert. The figure must not show a model selector, a threshold or a score.

[Insert Figure 4.17: Screening confirmation panel here]

Figure 4.17: Registration confirmation with screening outcome. The figure must show the QR code image returned by the API, the student's name, and the screening sentence built by `StudentRegistration.jsx:7-22`. Three states must be illustrated, because the component renders them differently:

State	Text rendered	Visual treatment
No candidate	"Checked against N existing student(s)." with no flag	Neutral
Candidate(s) under the review policy	"Possible duplicate match found. An administrator must review this case before any disbursement." plus "Checked against N existing students; M candidates flagged for a second look; K further matches recorded in the audit log without a separate case."	Amber
Exact identifier agreement	The same sentence plus the exact identifier named in plain language ("UPI", "birth certificate number", "National ID")	Red

The panel must show no model version, no threshold and no score, because the officer's screen is deliberately limited to what the officer can act on.

4.5.4 Administrator dashboard

[Insert Figure 4.18: Administrator dashboard here]

Figure 4.18: Administrator dashboard. The figure must show the counters the endpoint actually returns: total students, eligible students (`eligible` + `highly_eligible`), synthetic students in the registry, and open duplicate cases; a six-month trend of registrations and of disbursements rendered as line or bar charts; the recent activity list (four most recent audit actions); and the accountability block — approved awards, disbursed awards, approved

awards with no payment, disbursed amount, awaiting clearance, awaiting receipt acknowledgement, overdue receipts (or "deadline not configured"), partial awards and amount discrepancies. A "last updated" timestamp must appear, because the payload carries `lastUpdated`.

4.5.5 Duplicate review screen

[Insert Figure 4.19: Duplicate review queue with a case open here]

Figure 4.19: Duplicate review queue. The figure must show the queue with each case's two records side by side (name, date of birth, school identifier, masked identifiers), the priority badge (`high`/`standard`), the model score, the model version (`xgboost:<hash prefix>`), the review threshold and the review reason; the policy line stating both cutoffs, the aggregation policy and "provisional; validation is unavailable"; the reveal-identifiers action with its warning that the reveal is audited; and the decision control requiring a decision, a reason of at least five characters and optional notes. The figure must show the state after a decision has been recorded, including the case history entry.

4.5.6 Awards, payments and certificates

[Insert Figure 4.20: Awards and disbursements screen with the certificate panel here]

Figure 4.20: Awards and disbursements screen. The figure must show the awards table (student, requested amount, approved amount, status, verification status, total disbursed, payment count, open duplicate count, verification history), the payment record form (amount, method, reference, date), the payment status actions (clear with evidence, fail or cancel with a reason), the acknowledgement action (amount received and evidence reference) and the exceptions panel with its reason tags. It must also show the collapsed and expanded **Bursary certificates** panel: issue with an optional award period, the printed certificate view with its QR code, the PDF download action, and the revoke action with a mandatory reason, plus the explicit statement that "approval is not payment".

4.5.7 Administrative support screens: reports, audit, users and settings

[Insert Figure 4.21: Reporting, audit-log, user-management and settings screens here]

Figure 4.21: Administrative support screens. Four panels, because they share one design purpose — making administrative actions visible:

Screen	Controls the figure must show	Data shown
Reports	Start date, end date, "Export PDF" action, and the validation message "Start date cannot be after end date"	The generated PDF as a downloaded document, not on screen
Audit logs	Reverse-chronological table with actor, action, table, record and details, plus a readable rendering of the duplicate-screening counters (`considered · qualifying · suppressed`)	`audit_logs` rows
Users	Account list with name, e-mail, role and active state; create form with name, e-mail, role and password	`users` rows (never the hash)
Settings	Three numeric inputs (review threshold, high-priority threshold, model cut-off) with 0–1 validation	Current values, and the statement that the score is not a calibrated probability

4.5.8 Public verification pages

[Insert Figure 4.22: Public student-record verification page here]

Figure 4.22: Student-record verification page. The figure must show the minimal layout with no navigation, the heading "Student record verification", the sub-line "Scanned SBMS student QR code", and the five states the page renders: checking, record found, not verified, rate limited and connection problem. For "record found" it must show the three explanatory bullets, including "It confirms that the record exists. It does not confirm identity, eligibility, approval or payment." No student data may appear in any state.

[Insert Figure 4.23: Public certificate verification page here]

Figure 4.23: Certificate verification page. The figure must show the four outcome states — verified, revoked, not current, not recognised — and, for a verified certificate, the fields the endpoint actually returns: certificate reference (`SBMS-CERT-<year>-<8 hex>`), award period, issue date, approval date, award status and issuing authority ("Student Beneficiary Management System (SBMS)"). It must **not** show the student's name, UPI, school or the approved amount, because the public response omits them.

4.6 Database schema

4.6.1 Schema organisation and migration policy

The schema is a baseline file plus an ordered set of migrations. The baseline (`backend/src/db/schema.sql`) creates `users`, `students`, `duplicate_flags`, `duplicate_review_events`, `disbursements`, `audit_logs`, `reports` and the legacy `beneficiaries` table, and is written to be idempotent (`CREATE TABLE IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`, constraint guards inside `DO $$` blocks). The migrations add what was built later.

Table 4.12: Migration inventory

File	Purpose	Status
`20261019_admin_field_officer_roles.sql`	Two-role model and the role `CHECK` constraint	Applied to the test database
`20261019_duplicate_review_workflow.sql`	Review events and case-status semantics	Applied
`20261019_enforce_supported_schools.sql`	Restricts `school_code` to the approved list	Applied
`20261019_field_officer_registration.sql`	Field-officer account support	Applied
`20261019_limit_synthetic_schools.sql`	Keeps synthetic rows within the school list	Applied
`20261019_remove_duplicate_case_assignment.sql`	Removes an earlier assignment concept	Applied
`20261019_standardize_student_records.sql`	Normalises stored student fields	Applied
`20261019_synthetic_student_marker.sql`	Adds `is_synthetic`	Applied
`20261022_ml_duplicate_provenance.sql` (+ `.down.sql`)	Adds `model_version`, `review_threshold`, `comparison_features` and the unordered-pair unique index, with a pre-flight refusing to collapse existing duplicate pairs	Applied
`20261023_bursary_award_reconciliation.sql`	Adds `bursary_awards`, `bursary_award_verification_events`, the payment and receipt columns with constraints, and `disbursement_reconciliation_events`	Applied
`20261024_add_user_activation_flag.sql`	Adds `users.is_active`	Applied
`20261025_legacy_beneficiaries_baseline.sql`	Formalises the legacy `beneficiaries` table	Applied
`20261031_bursary_certificates.sql`	Adds `bursary_certificates`	Applied to the disposable test database; required by readiness
`20261023_retain_primary_accounts.sql`	Manual account-retention script	**Not part of the automatic migration order**

Migrations are not applied by the application. `backend/scripts/migrate.js` applies them in filename order and records them in `sbms_schema_migrations`; the operator is expected to back up and verify a restore first (`README.md`; `docs/database-recovery-rehearsal-2026-10-31.md`). The recorded state of the disposable test database after the last

full run was **12 base tables, 118 columns, 56 constraints, 21 indexes, 2 extensions and 12 applied migrations** (`docs/FINAL_TEST_REPORT.md` §5).

4.6.2 Column-level design of the core tables

Table 4.13: `users`

Column	Type	Constraints	Notes
`id`	`UUID`	Primary key, `gen_random_uuid()`	Non-sequential, so identifiers cannot be enumerated
`full_name`	`VARCHAR(255)`	Not null	
`email`	`VARCHAR(255)`	Not null, unique	Lower-cased by the API before storage
`password_hash`	`TEXT`	Not null	bcrypt, cost 12; never returned by any endpoint
`role`	`VARCHAR(50)`	Not null, default `field_officer`, `CHECK role IN ('admin','field_officer')`	Enforced twice, in the table and in a guarded `ALTER`
`is_active`	`BOOLEAN`	Not null, default `TRUE`	Authentication requires `TRUE`
`created_at`	`TIMESTAMP`	Default `NOW()`	

Table 4.14: `students`

Column	Type	Constraints	Notes
`id`	`UUID`	Primary key, `gen_random_uuid()`	
`full_name`	`VARCHAR(255)`	Not null	Whitespace-normalised by the API
`date_of_birth`	`DATE`	Nullable	Compared as a calendar date at the predictor boundary
`school_code`	`VARCHAR(100)`	Nullable, `CHECK` against the five approved identifiers	Holds a school *name* in practice; no authoritative code list exists
`guardian_id`	`VARCHAR(100)`	Nullable	Model feature
`upi`	`VARCHAR(100)`	Nullable; earlier `UNIQUE` constraint deliberately dropped	Model feature; exact agreement is also a review signal
`birth_certificate_number`	`VARCHAR(100)`	Nullable	Added by an idempotent block that also renames the earlier `birth_certificate_entry_number`
`national_id_number`	`VARCHAR(100)`	Nullable	Review signal, **not** a model feature
`primary_phone`	`VARCHAR(50)`	Nullable	Normalised to `+254` form
`qr_token`	`VARCHAR(100)`	Nullable, **partial unique index** where not null	64 lowercase hex characters; never returned by the API
`is_synthetic`	`BOOLEAN`	Not null, default `FALSE`	Distinguishes demonstration rows on the dashboard
`eligibility_score` / `eligibility_category` / `eligibility_assessed_at`	`INTEGER` / `VARCHAR(50)` / `TIMESTAMP`	Nullable	0–100 and one of four categories
`created_at`	`TIMESTAMP`	Default `NOW()`	

Table 4.15: `duplicate_flags` and `duplicate_review_events`

Table / column	Type	Constraints
`duplicate_flags.id`	`UUID`	Primary key
`student_id`, `matched_student_id`	`UUID`	Not null, `REFERENCES students(id) ON DELETE CASCADE`
`probability`	`NUMERIC(5,4)`	Not null; comment records that model rows store an uncalibrated score

Table / column	Type	Constraints
`status`	`VARCHAR(50)`	Not null, default `pending`
`model_version`	`TEXT`	Nullable; `xgboost:<sha256>` for model rows
`review_threshold`	`NUMERIC(5,4)`	Nullable; the cutoff in force when the case was created
`comparison_features`	`JSONB`	Nullable; the six-feature vector
`resolved_by`, `resolved_at`, `updated_at`, `created_at`	`UUID` / `TIMESTAMP`	`resolved_by REFERENCES users(id)`; `updated_at` not null
(index)	unique	`LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id)`
`duplicate_review_events.id`	`UUID`	Primary key
`duplicate_flag_id`	`UUID`	Not null, `REFERENCES duplicate_flags(id) ON DELETE CASCADE`
`reviewer_id`	`UUID`	Not null, `REFERENCES users(id)`
`decision`	`VARCHAR(30)`	Not null, `CHECK IN ('confirmed_duplicate','not_duplicate','needs_verification')`
`reason`	`TEXT`	Not null, `CHECK length(btrim(reason)) > 0`
`notes`	`TEXT`	Nullable
(index)	index	`(duplicate_flag_id, created_at DESC)`

Table 4.16: `bursary_awards` and `bursary_award_verification_events` (migration `20261023`)

Table / column	Type	Constraints
`bursary_awards.id`	`UUID`	Primary key
`student_id`	`UUID`	Not null, `REFERENCES students(id) ON DELETE RESTRICT`
`school_code`	`VARCHAR(100)`	Not null (copied from the student at submission)
`requested_amount`, `approved_amount`	`NUMERIC(12,2)`	`requested_amount > 0`; `approved_amount` null or `> 0`
`status`	`VARCHAR(20)`	`CHECK IN ('submitted','approved','rejected')`
`verification_status`	`VARCHAR(30)`	`CHECK IN ('not_checked','verified','not_found','needs_investigation')`
`verification_notes`, `verification_evidence_ref`	`TEXT` / `VARCHAR(500)`	Evidence mandatory for `verified`
`verified_by`, `verified_at`, `decided_by`, `decided_at`, `submitted_by`	`UUID` / `TIMESTAMPTZ`	`REFERENCES users(id) ON DELETE RESTRICT`
`decision_reason`	`TEXT`	≥ 5 characters when a decision exists
(constraints)	`CHECK`	`bursary_awards_decision_fields_check`, `bursary_awards_approval_verification_check`, `bursary_awards_verification_fields_check`
(indexes)	index	`(status, created_at DESC)`, `(student_id, created_at DESC)`
`bursary_award_verification_events.*`	—	`outcome`, `notes` (≥ 5 characters), `evidence_ref` (mandatory for `verified`), `actor_id`, `created_at`; index `(award_id, created_at DESC)`

Table 4.17: `disbursements` and `disbursement_reconciliation_events`

Column	Type	Constraints / notes
`id`, `student_id`, `amount`, `disbursement_date`, `status`, `created_by`, `created_at`	existing	`amount NUMERIC(12,2) NOT NULL`; `status VARCHAR(50) DEFAULT 'completed'`
`award_id`	`UUID`	Nullable, `REFERENCES bursary_awards(id) ON DELETE RESTRICT`; null for legacy payments
`payment_method`, `payment_reference`	`VARCHAR`	Nullable
`payment_status`	`VARCHAR(20)`	`CHECK IN ('issued','cleared','failed','cancelled')`
`cleared_by`, `cleared_at`, `clearance_evidence_ref`	—	Required together when `payment_status = 'cleared'`
`receipt_status`	`VARCHAR(30)`	`CHECK IN ('awaiting_acknowledgement','acknowledged','discrepancy')`
`acknowledged_amount`, `acknowledged_by`, `acknowledged_at`, `acknowledgement_method`, `acknowledgement_evidence_ref`, `acknowledgement_notes`	—	Required together when a receipt status is recorded; `acknowledgement_method` must be `officer_assisted`
(index)	index	`(award_id, payment_status, receipt_status)`
`disbursement_reconciliation_events`	—	`event_type CHECK IN ('payment_issued','payment_cleared','payment_failed','payment_cancelled','payment_linked_to_award','receipt_acknowledged')`; index `(disbursement_id, created_at DESC)`

Table 4.18: `bursary_certificates` (migration `20261031`)

Column	Type	Constraints
`id`	`UUID`	Primary key
`award_id`	`UUID`	Not null, **`UNIQUE`**, `REFERENCES bursary_awards(id) ON DELETE RESTRICT`
`certificate_reference`	`VARCHAR(40)`	Not null, unique; `SBMS-CERT-<year>-<8 uppercase hex>`
`verification_token`	`VARCHAR(100)`	Not null, unique, `CHECK ~ '^[a-f0-9]{64}$'`
`award_period`	`VARCHAR(20)`	Not null, `CHECK ~ '^[0-9]{4}(/[0-9]{4})?$'`
`student_name`, `student_upi`, `school_code`	—	Snapshot of the facts printed on the document
`approved_amount`	`NUMERIC(12,2)`	Not null, `> 0`
`award_approved_at`	`TIMESTAMPTZ`	Not null
`status`	`VARCHAR(20)`	`CHECK IN ('active','revoked')`
`issued_by`, `issued_at`	`UUID` / `TIMESTAMPTZ`	Not null
`revoked_by`, `revoked_at`, `revocation_reason`	—	`bursary_certificates_revocation_check` requires all three together with a reason of ≥ 5 characters, and all three null while `active`; `bursary_certificates_period_check` and `bursary_certificates_token_check` guard the formats
(index)	index	`(status, issued_at DESC)`

Table 4.19: `audit_logs`, `reports` and the legacy `beneficiaries`

Table	Columns	Notes
`audit_logs`	`id`, `user_id`, `action VARCHAR(100) NOT NULL`, `table_name`, `record_id UUID`, `details JSONB`, `created_at`	Append-only by convention; no trigger, no immutability constraint, no retention policy
`reports`	`id`, `generated_by`, `report_type`, `start_date`, `end_date`, `file_path`, `created_at`	Metadata for an exported PDF; the file itself is written to `reports/` on the API host
`beneficiaries`	`id SERIAL`, `name`, `school`, `grade_level`, `scholarship_amount NUMERIC DEFAULT 0`, `created_at`	Legacy shape reproduced deliberately: serial key, **no foreign keys**, unresolved `scholarship_amount` semantics

4.6.3 Integrity, indexing and normalisation notes

- **Normalisation level.** The schema is in third normal form for its operational data: student identifiers are attributes of `students`, award decisions are attributes of `bursary_awards`, and no non-key attribute depends on another non-key attribute. The two deliberate exceptions are the *snapshot* columns on `bursary_certificates` (which duplicate award facts to make a printed document immutable) and the *derived* counters computed at read time rather than stored, so that no cached aggregate can drift from its source.
- **Referential actions.** Review events and audit-adjacent children cascade from their parent case (`ON DELETE CASCADE`), because a review history has no meaning without its case. Awards, payments and certificates use `ON DELETE RESTRICT`, because financial and documentary records must not be removable through a parent deletion.
- **Indexes.** Seven indexes are material to query performance and constraint enforcement: the partial unique index on `students.qr_token`, the unordered-pair unique index on `duplicate_flags`, the two review-event and award indexes, the reconciliation-event index, the payment-status composite index and the certificate status index.
- **What the schema cannot enforce.** Nothing prevents a `pending` case from existing for two records that are in fact the same person: that judgement belongs to the reviewer. Nothing records *who* created a student row; the creating user is only in `audit_logs`. The legacy `beneficiaries` table remains unreconciled.

4.7 System architecture (physical deployment)

The logical architecture in §4.4.1 was realised in two deployment configurations: a single-host development configuration and a managed cloud configuration on Microsoft Azure that was provisioned and verified live on 31 October 2026 (`docs/azure-deployment-log-2026-10-31.md` §1–§2).

[Insert Figure 4.24: Physical deployment architecture on Azure here]

Figure 4.24: Physical deployment architecture. The figure must show:

Element	Configuration	Evidence
API container	Azure Container Apps app `ca-sbms-api` in environment `cae-sbms`, region UAE North, `minReplicas 0`, `maxReplicas 1`, environment created with `--logs-destination none` (no log-analytics workspace)	`docs/azure-deployment-log-2026-10-31.md` §1, §3 (D5, D7)
API image	`ghcr.io/eggxellent/sbms-api`, built by the dispatchable workflow `publish-api-image.yml` from the repository `Dockerfile`, `linux/amd64`, 1.37 GB after cleanup (was 2.22 GB)	`.github/workflows/publish-api-image.yml`; deployment log §1
Image gates	The workflow refuses an image whose predictor fails a known-duplicate/known-unrelated scoring assertion, whose feature order differs, that contains any `nvidia-*` package, that contains a `.env*` file, or that contains a `deploy/` directory	workflow gate steps
Frontend	Azure App Service `sbms-web` on the F1 (free) Linux tier, static build published as a zip, SPA rewrite so client routes resolve	`render.yaml` documents an equivalent Render topology; deployment log §1, §7
Database	PostgreSQL Flexible Server `pg-sbms` (B1MS, 32 GiB, PostgreSQL 16), database `sbms`, public access with TLS, stopped between demonstrations	deployment log §1, §3 (D4, D8)
Secrets	`DATABASE_URL`, `JWT_SECRET` and bootstrap credentials as container-app secrets; account passwords only in the git-ignored `deploy/azure/.secrets.json`	deployment log §1, §5

Element	Configuration	Evidence
Networking and TLS	HTTPS only; API FQDN `ca-sbms-api.ambitiousstone-f2de8f67.uaenorth.azurecontainerapps.io`; frontend `sbms-web.azurewebsites.net`; CORS allowlist and `PUBLIC_APP_URL` both set to the frontend origin	`render.yaml` env wiring; deployment log §2
Health	`GET /api/health` as container health probe; `GET /api/ready` verifies expected database identity and required tables	`index.js:69-86`; `services/databaseReadiness.js`

Three deployment facts matter for Chapter Five and must be stated plainly. First, the cloud deployment used **one container for both the API and the predictor**, because the predictor is a child process of the API; the figure must not draw them as two services. Second, the frontend and API are **separate origins**, which is why the cross-origin wiring (`VITE_API_ORIGIN`, `CORS_ORIGINS`, `PUBLIC_APP_URL`) is part of the architecture rather than an afterthought — a misconfiguration here produced a real defect, recorded in Chapter Five §5.6.3. Third, the database is **stopped** outside demonstration windows to keep the project inside a student credit budget, so "the deployment is live" is not the same claim as "the system is continuously available".

Table 4.20: Development and test environment topology

Environment	Components	Purpose
Local development	API on port 5000 (`nodemon`), Vite dev server on port 4174 proxying `/api`, local PostgreSQL, `ml-service/.venv` predictor	Development and manual checks
Disposable test database	`sbms_test` on loopback, role `sbms_test_runner`, guarded by `backend/tests/testDatabaseGuard.js`, which fails closed unless `NODE_ENV=test` and both `DATABASE_URL` and `SBMS_TEST_DATABASE_URL` name that same loopback database	Automated Jest suites
Research environment	Python virtual environment with `pandas`, `scikit-learn`, `xgboost`, `jellyfish`; artefacts under `ml-service/experiments/` and `ml-service/evaluation/`	Training, evaluation and the separate research-only models

4.8 Chapter summary

This chapter converted the project's intentions into a specification of the delivered system. It established thirty-seven functional requirements (thirty delivered, three recorded as proposal requirements with documented deviation or exclusion, and four recorded as unsatisfied gaps) and twenty-four non-functional requirements, of which three are recorded as not met; it stated the data requirements, including the six-feature contract, the normalisation rules and the absence of any real beneficiary data; and it defined five interfaces with their protocols, authentication and error behaviour.

It then presented the analysis models — context, use case, activity, class, system sequence, sequence, entity-relationship and ML-pipeline diagrams — as specifications that can be drawn without inventing behaviour, and the design of the system: a three-tier architecture in which the machine-learning component is a supervised child process of the API; a module structure with clear layering and two acknowledged weaknesses (a dead legacy rule engine and an unreconciled legacy table); a two-role access matrix whose most-contested rule (`/api/students` is field-officer-only) is shown to be a designed policy rather than a defect; a duplicate-detection design that separates scoring from case selection; four explicit state machines; a security design mapped threat by threat to its control; a relational schema described column by column with its constraints, indexes and deliberate exceptions; and a physical deployment architecture that was actually provisioned.

Two conclusions follow from the analysis and are carried into Chapter Five. The first is that the system's integrity depends less on the classifier than on the two human gates that stand between a possible duplicate and a payment; those gates, not the model, are what the design offers against the problem stated in Chapter One. The second is that the design's principal technical weakness is candidate generation: comparing every new record against the entire registry makes the *volume* of review work grow with the registry, and no threshold setting repairs that —

which is why Chapter Five evaluates the model and the aggregation policy separately, and why Chapter Six proposes blocking as future work.

Chapter Five now reports how this design was implemented, how it was tested, what the evidence establishes, and what it does not.

CHAPTER 5: SYSTEM IMPLEMENTATION, TESTING AND EVALUATION

> **Status of this file.** Report-ready content for Chapter 5, produced from a read-only investigation of the `student-beneficiary-system` repository (branch `fix/sbms-completion-work`, commit `d6b5032`), its test suites, its execution records and its deployment log.
>
> **Evidence provenance is stated for every result, and three kinds are never merged:**
>
> 1. **Implementation evidence** — read from source, configuration, migrations or the running artefact.
> 2. **Recorded execution evidence** — a test run written into project records by a previous session, with date, commit and command recorded there. Cited, not re-run in this documentation pass.
> 3. **Results not obtained** — marked **NOT RUN**, listed in §5.4.12, and never described as a pass.
>
> No test result, metric, screenshot or participant was invented. Screenshot placeholders name what the image must show; they do not claim an image exists.

5.1 Introduction

This chapter reports how the design in Chapter Four was implemented, how it was tested, and what the evidence permits the project to claim. It has three purposes.

First, it documents the implementation: the environment in which the system was built, the module-by- module realisation of each requirement, and the specific decisions that carry the system's behaviour — password hashing, token handling, batching of model calls, aggregation of per-pair scores into review cases, the two payment gates, the certificate snapshot, and the guarded start-up sequence.

Second, it documents the testing actually performed, at the level of individual test cases where the suites are available, and separates four kinds of confidence: automated unit and integration tests against a real PostgreSQL database, service-level tests of the Python predictor, browser-driven checks in headless Chrome, and checks performed against the live Azure deployment. A feature that exists but has no executed test is stated as such.

Third, it evaluates the system against the objectives in Chapter One and the requirements in Chapter Four, including a full treatment of the machine-learning component: training data, classification performance on a synthetic holdout, the consequence of the operating point for queue volume, and the reasons why none of those figures is an accuracy claim about Kenyan beneficiary records.

The chapter does not argue that the system is complete or production-ready. It reports what works, what was measured, and what remains unverified, so that Chapter Six can draw conclusions that follow from evidence rather than from intent.

5.2 Implementation environment

5.2.1 Hardware

The hardware on which the project was developed and tested **was not recorded** in the repository, and the workstation specification must be supplied by the author before submission. What the environment did record is material to interpreting results: the development host had approximately **7.4 GB of total memory with about 0.5 GB free** during long test sweeps, which is the documented cause of the intermittent failures described in §5.4.3. Claiming a processor model, memory size or storage type would be invention. **REQUIRES INPUT** (gap register item G-01).

5.2.2 Software, frameworks and versions

Table 5.1: Implementation environment and version evidence

Category	Component	Version	Evidence
Language (frontend)	JavaScript (ES modules), JSX	—	`frontend/src/**`
Language (backend)	JavaScript (CommonJS) on Node.js	Node 22 in the image (`node:22-bookworm-slim`); Node 22.11.0 on the development host	`Dockerfile:1`; `docs/FINAL_TEST_REPORT.md` header
Language (ML)	Python	3.11.9 on the development host	`docs/FINAL_TEST_REPORT.md` header
Frontend framework	React, React DOM	18.3.1	`frontend/package.json`
Routing	react-router-dom	7.18.3	`frontend/package.json`
Styling	Tailwind CSS (`@tailwindcss/vite`)	4.3.3	`frontend/package.json`
Charts	chart.js / react-chartjs-2	4.5.1 / 5.3.1	`frontend/package.json`
Icons	lucide-react	1.55.0	`frontend/package.json`
HTTP client	axios	1.20.0	`frontend/package.json`
Bundler	Vite (`@vitejs/plugin-react`)	5.4.1	`frontend/package.json`
Backend framework	Express	4.19.2	`backend/package.json`
Validation	zod	4.5.4	`backend/package.json`
Authentication	jsonwebtoken / bcrypt	9.0.3 / 6.0.0	`backend/package.json`
Database driver	pg	8.12.0	`backend/package.json`
QR generation	qrcode	1.5.4	`backend/package.json`
PDF generation	pdfkit	0.20.2	`backend/package.json`
Test runner (backend)	Jest / Supertest	30.5.1 / 7.2.2	`backend/package.json`
Database	PostgreSQL	17.11 locally; PostgreSQL 16 on the managed Azure server	`docs/FINAL_TEST_REPORT.md` §5; `docs/azure-deployment-log-2026-10-31.md` D4
ML libraries	pandas / scikit-learn / xgboost / jellyfish	3.0.5 / 1.9.0 / 3.2.0 / 1.2.1	`ml-service/requirements.txt`
Container base	Debian Bookworm slim	—	`Dockerfile`
Container tooling	Docker Desktop / CLI	Desktop 4.97.0, CLI 29.9.2 (engine availability varied)	`docs/azure-deployment-log-2026-10-31.md` §2; `docs/FINAL_TEST_REPORT.md` §1
Version control and CI	Git, GitHub Actions (`build-push-action` v6, `login-action` v3, `setup-buildx-action` v3)	—	`.github/workflows/publish-api-image.yml`
Cloud platform	Azure Container Apps, App Service (F1), PostgreSQL Flexible Server, region UAE North	—	`docs/azure-deployment-log-2026-10-31.md` §1

Table 5.2: Why each principal technology was used in this project

Technology	Project-specific reason
React single-page application	The proposal required a browser-accessible system for officers on varied devices with no local installation (proposal §1.4). The component model maps one-to-one onto the distinct operational screens, and the two public verification pages had to be routes on the same build
Express REST API	The API serves two very different clients — an authenticated role-gated staff application and two unauthenticated single-purpose endpoints — so explicit routing with per-route middleware is the natural structure
PostgreSQL	The integrity rules are financial: awards, payments, certificates and review events need foreign keys and `CHECK` constraints that the database enforces even if a controller is wrong

Technology	Project-specific reason
Python with scikit-learn and XGBoost	The scorer is a trained classifier with a persisted artefact; Python is where feature engineering, training and evaluation tooling exists, and the same feature function is reused for serving
Jaro–Winkler via `jellyfish`	The problem is near-duplicate names created by spelling variation and reordering; Jaro–Winkler rewards shared prefixes and tolerates transposition, and its normalised score is directly usable as a feature
`qrcode` and `pdfkit`	QR images and printable documents must be produced server-side so that the token and the document content are authoritative rather than client-generated
Azure (Container Apps, App Service, PostgreSQL Flexible Server)	The deployment had to run inside an Azure for Students credit with a spending limit, in the only regions the subscription policy permitted; the chosen topology is the cheapest combination that still provides HTTPS origins for QR scanning

5.2.3 Development and operations tooling

Tool	Use in this project
`nodemon` + `concurrently`	Runs the API and Vite dev server together from the repository root (`npm start`)
Vite dev server	Serves the frontend on port 4174 and proxies `/api` to port 5000, which is why relative API calls work in development
Jest with Supertest	Drives the real Express app over real HTTP against a real PostgreSQL database
`node --test`	Runs seven self-contained native test files (readiness, start-up, bootstrap, error handling, verification, migration and secret validation) in-process
Python `unittest`	Runs the ML service suite and the research-helper suite; `pytest` is not installed in the virtual environment
`scripts/browser-smoke.mjs`, `scripts/qr-smoke.mjs`, `scripts/admin-reveal-smoke.mjs`, `scripts/registration-fields-smoke.mjs`	Drive headless Chrome over the DevTools Protocol using only Node 22's built-in `WebSocket`, with no additional dependency
`backend/scripts/migrate.js`	Applies migrations in filename order and records them in `sbms_schema_migrations`
`backend/scripts/provision-test-database.js`, `seed-verification-accounts.js`, `seed-kenyan-demo-data.js`, `reset-kenyan-demo-data.js`	Create a disposable test database and synthetic demonstration accounts and records; the verification seeding refuses any non-loopback host
Docker	Builds the single API+predictor image; the build itself runs an inference gate so an image with a broken artefact cannot be published
GitHub Actions (`workflow_dispatch`)	Builds and pushes `ghcr.io/eggxellent/sbms-api` on demand and re-runs the model, feature-order and secret gates against the pushed image
Azure CLI	Provisions the resource group, Container Apps environment, container app, App Service plan and site, and PostgreSQL server

5.2.4 Deployment environment

Two configurations were used and must not be conflated.

Development configuration. API on `http://localhost:5000`, frontend on `http://localhost:4174`, a local PostgreSQL instance, and the predictor from `ml-service/.venv`. Because `PUBLIC_APP_URL` validation deliberately refuses non-loopback local addresses and any plain-HTTP origin outside an explicit `NODE_ENV=development` loopback (Chapter Four §4.2.5), the development configuration can produce QR images but cannot support scanning them with a phone.

Deployed configuration. Verified live on 31 October 2026 and brought back up on 1 November 2026 (`docs/azure-deployment-log-2026-10-31.md` §1, §7): API container app `ca-sbms-api.ambitiousstone-f2de8f67.uaenorth.azurecontainerapps.io`, frontend `https://sbms-web.azurewebsites.net`, PostgreSQL Flexible Server

`pg-sbms` holding database `sbms`, with `PUBLIC_APP_URL` and `CORS_ORIGINS` both set to the frontend origin so that generated QR codes point at a reachable public page.

5.3 System implementation

5.3.1 Code organisation

The repository separates the three tiers into `frontend/`, `backend/` and `ml-service/`, with a root `package.json` that starts the first two together. Two directories carry the project's evidence: `backend/tests/` holds twenty-four Jest suites and `backend/test/` holds the seven native test files plus the out-of-process API smoke test, while `ml-service/tests/`, `ml-service/evaluation/` and `ml-service/experiments/` hold the Python tests and the artefacts of the model work. `docs/` holds the project's own records, and `deploy/` holds the (git-ignored) Azure secrets and operator scripts.

Table 5.3: Implemented modules and their realisation

Module	Requirement(s)	Backend realisation	Frontend realisation	Persistence
Authentication	FR-01 to FR-04	`auth.controller.js`, `auth.model.js`, `middleware/auth.js`, `requireRole.js`, `loginRateLimit.js`	`pages/Login.jsx`, `context/AuthContext.jsx`, `services/api.js`	`users`
Beneficiary registration	FR-05 to FR-11	`students.controller.js`, `student.model.js`, `studentRecordFormat.js`	`pages/StudentRegistration.jsx`, `pages/Students.jsx`	`students`, `duplicate_flags`, `audit_logs`
Duplicate screening	FR-07 to FR-11	`duplicateCheck.js`, `mlDuplicateReview.js`, `duplicateAggregation.js`, `duplicateFlags.controller.js`	`pages/DuplicateReview.jsx`	`duplicate_flags`, `duplicate_review_events`
Eligibility scoring	FR-12	`eligibility.controller.js`, `services/eligibility.js`	`pages/Eligibility.jsx`	`students.eligibility_*`
Bursary awards	FR-13 to FR-15	`bursaryAwards.controller.js`	`pages/Disbursements.jsx`	`bursary_awards`, `bursary_award_verification_events`
Payments and reconciliation	FR-16 to FR-19	`disbursements.controller.js`, `bursaryRules.js`, `config/reconciliation.js`	`pages/Disbursements.jsx`	`disbursements`, `disbursement_reconciliation_events`
Student QR verification	FR-20, FR-21	`verification.controller.js`, `verificationRateLimit.js`, `publicAppUrl.js`	`pages/Verification.jsx`, `hooks/useHashToken.js`	`students.qr_token`
Award certificates	FR-22 to FR-24	`certificates.controller.js`, `bursaryCertificate.model.js`, `certificateDocument.js`, `certificateRateLimit.js`	`components/AwardCertificatePanel.jsx`, `pages/CertificateVerification.jsx`	`bursary_certificates`
Dashboard and reporting	FR-25, FR-26	`adminDashboard.controller.js`, `reports.controller.js`	`pages/AdminDashboard.jsx`, `pages/Reports.jsx`	read-only queries, `reports`
Audit and administration	FR-27 to FR-29	`audit.controller.js`, `auditLog.js`, `users.controller.js`, `settings.controller.js`	`pages/AuditLogs.jsx`, `pages/UserManagement.jsx`, `pages/Settings.jsx`	`audit_logs`, `users`
Operations	FR-30	`databaseReadiness.js`, `databaseStartup.js`, `scripts/migrate.js`, `scripts/bootstrap-production-schema.js`	—	`sbms_schema_migrations`

5.3.2 Authentication and access control

The sign-in path is implemented as one controller function parameterised by the expected role, which is why the two entry points cannot drift apart. The sequence is: validate the body with Zod (e-mail format, non-empty password) → fetch the account by lower-cased e-mail → reject when the account is missing, inactive, or the bcrypt comparison against the stored hash fails, in all three cases with the same `401 Invalid email or password` so that the response does not reveal which accounts exist → compare the account role with the role the endpoint expects and answer `403` with an instruction naming the other form → sign `{id, role}` with an eight-hour expiry and return the token with the profile (`controllers/auth.controller.js:37-74`). The recorded state of the disposable test database after a full suite run was `users = 4`, created by the suites themselves; the deployed database holds the administrator and field-officer accounts whose passwords are kept only in the git-ignored `deploy/azure/.secrets.json`.

The authorisation path is deliberately database-backed: `authenticateToken` verifies the signature, then reads `id, role, is_active` from `users` and rejects the request if the row is gone or inactive, replacing whatever role the token claims with the stored role (`middleware/auth.js:17-25`). The consequence is testable and was tested: a token signed with `role: 'admin'` for a field-officer account still yields 403 on admin routes, and a role change takes effect for tokens already issued (`tests/roleAuthorization.test.js`, "signed client role claims cannot grant admin access and role changes apply to existing tokens").

Table 5.4: Authentication and access-control implementation details

Behaviour	Implementation	Verified by
Password hashing	`bcrypt.hash(password, 12)` on creation; `bcrypt.compare` on sign-in	`controllers/auth.controller.js:26, 48`; `tests/auth.test.js`
Token lifetime and claims	`jwt.sign({id, role}, secret, {expiresIn:'8h'})`	`controllers/auth.controller.js:55-59`
Secret policy	Missing, shorter than 32 bytes or placeholder secrets block start-up	`config/jwtSecret.js`; `test/jwtSecret.validation.node.cjs` (part of the 43 native tests)
Public registration	Always `410 Public registration is disabled.`	`routes/auth.routes.js:9-11`; `tests/auth.test.js`
Wrong-role sign-in	`403` with a message naming the other sign-in form	`controllers/auth.controller.js:51-53`; `tests/roleAuthorization.test.js`
Sign-in throttling	10 attempts per IP per 15 minutes, shared counter for both endpoints	`middleware/loginRateLimit.js`; `tests/loginRateLimit.test.js`
Token required	`401` when the `Authorization` header is absent or not a `Bearer` token	`middleware/auth.js:6-12`; `tests/students.test.js`

5.3.3 Student and beneficiary management

Registration is the most involved controller in the system because it coordinates validation, token generation, model scoring, flag aggregation, persistence and audit inside one transaction. It is implemented in `controllers/students.controller.js:31-161` in an order that makes failure safe:

1. **Validation** by Zod: name trimmed and whitespace-collapsed to 2–255 characters; date of birth checked as a real ISO date; `school_code` restricted to the five approved schools; identifiers normalised to upper-case alphanumerics and capped at 100 characters; phone normalised to `+254[17]XXXXXXXX`. A failure returns `400 {message:'Invalid student data', errors}` and nothing is written.
2. **Policy resolution before the transaction**: the review threshold and the aggregation policy are read and validated, so an invalid `DUPLICATE_AGGREGATION_POLICY` fails the request without opening a connection (`students.controller.js:54-57`).
3. **QR token and verification URL**: 32 random bytes as 64 hex characters, and the URL built from the validated `PUBLIC_APP_URL`; a misconfigured origin fails here rather than creating a student whose code cannot be scanned.

4. **Transaction and lock**: `BEGIN`, then `pg_advisory_xact_lock(hashtext('sbms.student.registration'))`, then every existing student selected `FOR UPDATE`. The lock serialises registration against award approval and payment creation, which use the same advisory key.
5. **Scoring** of candidate pairs in batches of 200.
6. **Failure path**: if the predictor is unavailable, times out or returns a malformed payload, the transaction rolls back and the client receives `503`. **No student is created** — an unscreened registration is treated as worse than a failed one.
7. **Selection and persistence**: qualifiers are reduced by the aggregation policy, the student row is inserted, and one `duplicate_flags` row is written per selected candidate with score, model version, threshold and feature vector. The unordered-pair unique index turns a repeat creation into a no-op rather than a duplicate case.
8. **Audit and response**: an audit row records the policy, the three candidate counters and the number of flags created; the response returns the student **without** `qr_token`, the QR image, the flags and the screening summary.

Record management beyond creation is deliberately narrow: `GET /api/students` lists the registry, `GET /api/students/:id` reads one record, `GET /api/students/schools` returns the approved list, and there is **no update or delete endpoint**, so a beneficiary record cannot be silently edited or removed through the API. Eligibility assessment is the only operation that mutates a student row after creation (§5.3.6).

Table 5.5: Validation rules implemented on the student record

Field	Rule	Enforced by
`full_name`	2–255 characters, internal whitespace collapsed	Zod schema
`date_of_birth`	Optional, but if present a real `YYYY-MM-DD` calendar date	`validIsoDate` in Zod
`school_code`	One of `Nairobi School`, `Alliance High`, `Bishop Gatimu`, `Loreto Limuru`, `Maranda`	Zod enum **and** `students_school_supported_check`
`guardian_id`, `upi`, `birth_certificate_number`, `national_id_number`	Optional, ≤ 100 characters, normalised to alphanumerics, upper-cased	`normalizeIdentifier`
`primary_phone`	Optional, `+254[17]XXXXXXXX` after normalisation; `0712345678` and `+254712345678` both accepted	`normalizeKenyanPhone`
`qr_token`	Server-generated only; a client-supplied value cannot enter because the schema does not accept it	`studentSchema` (token excluded by design)

5.3.4 Machine-learning-assisted duplicate detection

Production artefact and feature schema. The application loads exactly one artefact, `ml-service/model/xgb_duplicate_model.joblib` — 421 089 bytes, SHA-256 `6ec9f2a3cbe6ed0dcf80c79b15f47364d9f4f7bd3e27da25c690d0436bb0a6f9` — a scikit-learn pipeline containing an imputer and an `XGBClassifier`. The version reported to reviewers is `xgboost:<sha256>`, computed by reading the artefact at process start (`predict.py:68-70`), so a replaced artefact cannot masquerade as the evaluated one. The six features and their order are fixed in one list used by training, evaluation and serving (`feature_engineering.py:11-18`), and the training report asserts that the serving path reproduced the generator's stored feature columns exactly (`feature_consistency.exact_match = true`).

How a registration reaches the scorer. For every existing student the controller builds the pair `{first: submitted, second: candidate}` and calls `scorePairs` (`students.controller.js:69-73`). `scorePairs` normalises `date_of_birth` to a calendar date at this boundary, chunks the list into 200-pair batches, writes one JSON line per batch to the predictor's standard input, and validates the reply: result count must equal pair count, every score must be finite and within [0, 1], every result must carry a feature object, and `model_version` must not change between batches (`mlDuplicateReview.js:84-117`). The predictor loads the artefact once, builds features with the shared module, calls `predict_proba` and returns `{score, features}` per pair (`predict.py:40-65, 84-98`).

The role of Jaro–Winkler and the other features. `jaro_winkler` is the strongest name-level signal and is computed twice — on concatenated tokens and on alphabetically sorted tokens — with the higher value kept, so "Wanjiku Mary" and "Mary Wanjiku" score highly rather than poorly. `dob_match` is banded (1.0 exact, 0.8 one day, 0.5 up to a week) because typists transpose day and month. The identifier and school features express agreement rather than similarity of meaning: school agreement is binary, while guardian identifier, UPI and birth-certificate number compare exactly after stripping shared prefixes (such as `G`, `UPI`, `BC`) and fall back to Jaro–Winkler for partial agreement, so a mistyped digit yields a partial score rather than zero.

How the score becomes a case. A candidate *qualifies* if its score reaches the review cut-off **or** if UPI, birth-certificate number or national identifier agrees exactly — a signal the model does not see. Qualifiers are ranked with exact identifier agreement above score, and the aggregation policy decides how many become cases (default: one). The case stores the score as `probability`, the artefact hash, the threshold and the feature vector, and the response reports how many candidates were considered, qualified and suppressed (`duplicateAggregation.js:119-144`).

How the human decides. The administrator sees both records, the priority band, the score, the model version and the threshold, and records `confirmed_duplicate`, `not_duplicate` or `needs_verification` with a reason. The decision writes a `duplicate_review_events` row and an `audit_logs` row carrying the model version and score (`duplicateFlags.controller.js:452-503`). Nothing converts a score into a rejection: there is no delete, no status change and no block on registration.

Separation from research artefacts. Three artefacts exist; one is served. The production model is the six-feature XGBoost artefact above. `ml-service/model/research/rf_attachment_style_candidate.joblib` is a Random Forest candidate trained on attachment-style pairs, with grouped cross-validation F1 0.700–0.732, holdout F1 0.695 and recall 0.538, and is never loaded by the application (`ML_RELIABILITY_REPORT.md`). `ml-service/experiments/sbms_dataset/artifacts/sbms_dataset_xgb_research_only.joblib` is a ten-column model trained on a supplied external synthetic dataset whose schema has no UPI or birth-certificate fields and whose school labels do not correspond to the five approved schools, so it is explicitly incompatible with the serving contract (`ml_audit.md`; `reports/sbms_dataset_experiment.md`). Its test F1 of 0.875 and PR-AUC of 0.953 belong to that experiment and are not properties of the production system.

5.3.5 Duplicate review and resolution

The review queue is a joined query returning, for each case, both students' names, dates of birth and school identifiers, with `guardian_id`, `upi`, `birth_certificate_number` and `national_id_number` masked to their last three characters by `maskIdentifier` (`duplicateFlags.controller.js:37-42`). The queue also returns a policy block: the review threshold, the high-priority threshold, the aggregation policy and `thresholds_validated: false`. That flag is not decoration — it is how the interface states that the cut-offs are provisional.

Three operations complete the module. `evaluate` scores supplied pairs without persisting anything, which is what makes the model testable through the same HTTP surface as production. `scanExisting` re-scores the existing registry, enumerating pairs and skipping those already flagged, and is quadratic in registry size. `revealIdentifiers` returns the five identifier fields for both records of one case and writes a `REVEAL_DUPLICATE_IDENTIFIERS` audit row naming the fields disclosed, the two student identifiers and the stated purpose (`duplicateFlags.controller.js:434-446`).

The decision is transactional and terminal: the `UPDATE` is constrained to `status IN ('pending','needs_verification')`, so a case is decided once and a second attempt receives 404; the review event and the audit row are written in the same transaction; and `resolved_at` is set only for the two terminal decisions, leaving `needs_verification` open with a recorded reason.

5.3.6 Eligibility scoring

Eligibility is a transparent rule engine rather than a second model, as the proposal justified (§1.4). `services/eligibility.js` awards points across five factors and returns the total, a category, a per-factor breakdown and human-readable reasons. The controller validates that the three numeric inputs are finite and non-negative and

that the two categorical inputs are in their enumerations, then optionally writes the score and category to the student row.

Table 5.6: Eligibility scoring rules as implemented

Factor	Points	Thresholds
Fee balance	30 / 25 / 15 / 5 / 0	≥ 50 000 / ≥ 30 000 / ≥ 15 000 / > 0 / 0
Household income	25 / 20 / 10 / 0	≤ 20 000 / ≤ 40 000 / ≤ 60 000 / > 60 000
Vulnerability status	25 / 15 / 5	high / medium / low
Siblings in school	10 / 7 / 3 / 0	≥ 4 / ≥ 2 / = 1 / 0
Prior bursary history	10 / 7 / 2	none / low / high
Category	—	≥ 75 `highly_eligible`; ≥ 50 `eligible`; ≥ 30 `partially_eligible`; otherwise `not_eligible`

The dashboard counts `eligible` and `highly_eligible` students, which is the only place the score is used outside the assessment screen. **No automated test of the scoring arithmetic exists**: the route and its role restriction are covered, and the values above were verified by reading `services/eligibility.js` against `docs/eligibility-rules.md`, which agree.

5.3.7 Bursary awards, payments and reconciliation

The award and payment module is the system's answer to the documentation failures described in Chapter One, implemented as a chain of explicit gates rather than as a status field.

Award submission requires an existing student with a school on record and creates a row in `submitted` state with the requested amount (`bursaryAwards.controller.js:64-95`). **Verification** is recorded by the field officer as `verified`, `not_found` or `needs_investigation`, with a mandatory note and — for `verified` — a mandatory evidence reference, and each recording appends an event row so the history of verification attempts is preserved rather than overwritten.

Approval is the first gate. Inside a transaction that takes the same advisory lock as registration, the controller refuses approval when the award is not `submitted` (409), when the beneficiary is not `verified` (409), or when any duplicate case for that student is `pending`, `needs_verification` or `confirmed_duplicate` (409) — and the duplicate query tests **both** `student_id` and `matched_student_id`, so a student is blocked whether they are the new record or the existing record they were matched against (`bursaryAwards.controller.js:173-189`). Only then are status, approved amount, reason, deciding user and time written, with an `AWARD_APPROVED` audit row.

Payment is the second, independent gate. The controller refuses a payment when the award is not `approved`, when verification is no longer `verified`, when a duplicate case exists, or when the sum of existing non-failed, non-cancelled payments plus the new amount would exceed the approved amount (`disbursements.controller.js`). A payment is recorded as `issued` — never as paid — with `receipt_status = 'awaiting_acknowledgement'`, and appends a `payment_issued` reconciliation event.

Settlement is recorded separately. `updatePaymentStatus` accepts only `issued → cleared`, `issued → failed` and `issued → cancelled` (`bursaryRules.js:1-13`); clearance requires an evidence reference, and failure or cancellation requires a reason of at least five characters. A receipt acknowledgement compares the payment and acknowledged amounts in integer cents and stores `acknowledged` when they agree and `discrepancy` when they do not, so a shortfall is preserved as evidence rather than corrected (`bursaryRules.js:15-22`).

Exceptions are derived rather than stored: `exceptionReasons` evaluates each payment against eleven conditions — legacy payment without an award link, unverified payment status, incomplete beneficiary verification, amount above the approved award, partial payment, no payment recorded, awaiting clearance, failed, cancelled, awaiting

acknowledgement, acknowledgement overdue (when `ACKNOWLEDGEMENT_OVERDUE_DAYS` is set) and amount discrepancy (`bursaryRules.js:32-52`). Because they are computed at read time they cannot drift from the underlying data.

Two boundaries must be stated because a reader will otherwise over-read this module. First, the system does **not** transfer money: it records that a payment was issued and, separately and with evidence, that it cleared. Second, receipt acknowledgement is **officer-assisted** — the database requires `acknowledgement_method = 'officer_assisted'` — so it is not independent confirmation by a school, and the interface and the project records both say so.

5.3.8 Student QR verification

The QR token is generated during registration: `randomBytes(32).toString('hex')`, 64 lowercase hexadecimal characters, stored in `students.qr_token` under a partial unique index and encoded into a QR image whose payload is `PUBLIC_APP_URL + '/verify#token=<token>'`. The token sits in the **fragment**, so it is never sent to a web server, never written to an access log and never forwarded as a `Referer` (`config/publicAppUrl.js:164-168`). Registration and list responses strip `qr_token` (`models/student.model.js:3-7`), so the token reaches the officer only inside the QR image.

The public page reads the fragment, removes it with `history.replaceState`, and posts `{token}` to `/api/verify`. Because a hash-only change does not remount a React component, the page re-runs verification on `hashchange` — a defect found in browser testing and fixed (`docs/qr-verification-2026-10-30.md` §4). The endpoint requires exactly one key and the 64-hex shape, answers `{found:true}` or `{found:false}` and nothing else, returns the same answer for malformed and unknown tokens, and returns `503 {error:'temporarily_unavailable'}` on a lookup failure. Its rate limiter runs in the same route chain ahead of the JSON parser, so an abusive client is refused before its body is read.

The information revealed is deliberately exhausted by that boolean. It does not confirm identity, eligibility, award approval or payment, and the page states so in its result text. There is **no token expiry, rotation or re-issue path**: a token is valid for the lifetime of the student row, and a row created without a token cannot be given one through the API (`docs/qr-verification-2026-10-30.md` §6).

5.3.9 Bursary award certificates

A certificate is issued only for an approved award, and the issue path refuses an award that already has a certificate (`certificates.controller.js:95-181`). Issuing writes a **snapshot row**: reference `SBMS-CERT-<year>-<8 uppercase hex>`, a fresh 64-hex verification token, the award period (defaulting to the UTC year of approval), the student's name, UPI and school, the approved amount at issue, and the approval timestamp. The snapshot exists so that a later change to the award cannot rewrite what a printed document says. The response returns the certificate facts, the verification URL and a QR data URL; the printable PDF is rendered per request with `pdfkit` and `qrcode` on A4 landscape and served with `Cache-Control: no-store`.

Public verification answers in one of four states, computed by comparing the snapshot with the live award (`certificates.controller.js:53-89`): `verified` when the certificate is active and the award is still `approved` at the same amount; `not_current` when the award is no longer approved or its approved amount has changed; `revoked`; and `not_recognised` for any malformed, unknown or manipulated token. A `verified` response discloses the certificate reference, award period, issue date, approval date, award status and the issuing authority, and **not** the student's name, UPI, school or the amount. Revocation is a status change with actor, timestamp and a reason of at least five characters, enforced by a table `CHECK` as well as by the API, and it never alters the award or any payment.

The distinction between the two verification functions is enforced technically, not only conceptually: the endpoints have separate paths, separate token namespaces, separate rate-limit budgets and separate pages, and a certificate token does not verify a student record or the reverse (`tests/bursaryCertificates.test.js`).

5.3.10 Dashboard, reporting and audit

The dashboard is six parallel queries executed with `Promise.all`: beneficiary counts (total, eligible plus highly eligible, synthetic), open duplicate cases, a six-month registration trend, a six-month disbursement trend, the four

most recent audit actions, and an accountability block of nine counters (approved awards, disbursed awards, approved awards with no payment, disbursed amount, awaiting clearance, awaiting receipt, overdue receipts, partial awards and amount discrepancies). The trends use a `generate_series` CTE with a window `SUM`, so a month with no activity appears as a flat point rather than disappearing. The response also carries `acknowledgementOverdueDays` and `lastUpdated`.

Reporting is one endpoint that requires both dates, refuses a reversed range, queries the same accountability snapshot, writes a `reports` row and a `GENERATE_REPORT` audit row, and streams a `pdfkit` document containing a summary, an accountability snapshot and a disbursement detail list. It is an administrator-only, on-demand export; there is no scheduled report generation.

Auditing is a single function, `logAudit(userId, action, tableName, recordId, details, executor)`, called with the transaction client wherever the operation is transactional, so an audit row cannot be committed without its business change or vice versa. The action vocabulary includes `CREATE`, `REVEAL_DUPLICATE_IDENTIFIERS`, `DUPLICATE_CONFIRMED_DUPLICATE`, `DUPLICATE_NOT_DUPLICATE`, `DUPLICATE_NEEDS_VERIFICATION`, `AWARD_VERIFICATION_RECORDED`, `AWARD_APPROVED`, `AWARD_REJECTED`, `PAYMENT_ISSUED`, `PAYMENT_CLEARED`, `RECEIPT_ACKNOWLEDGED_OFFICER_ASSISTED`, `LEGACY_PAYMENT_LINKED_TO_AWARD`, `AWARD_CERTIFICATE_ISSUED`, `AWARD_CERTIFICATE_REVOKED`, `ADMIN_UPDATE_DUPLICATE_THRESHOLDS` and `GENERATE_REPORT`.

5.3.11 Configuration, start-up guards and migration control

Start-up is deliberately strict, because the system's deployment failure modes were configuration failures rather than code faults. Before binding a port the API validates the JWT secret (present, at least 32 bytes, not a known placeholder), resolves `PUBLIC_APP_URL` through the public-origin validator, and asserts that the expected database name matches and that `users` carries the columns the login query needs. Any failure writes one diagnostic line naming the guard and exits with status 1, so a container cannot enter a crash loop silently. `GET /api/health` reports liveness; `GET /api/ready` runs the readiness contract against the required tables and the expected database identity and returns `{"status":"ready"}` or a generic `503`. Neither endpoint creates or migrates anything.

Configuration is read from the environment: `PORT`, `DATABASE_URL`, `SBMS_EXPECTED_DATABASE_NAME`, `JWT_SECRET`, `CORS_ORIGINS`, `PUBLIC_APP_URL`, `PYTHON_EXECUTABLE`, `TRUST_PROXY_HOPS`, `ML_REVIEW_SCORE_THRESHOLD`, `DUPLICATE_REVIEW_THRESHOLD`, `DUPLICATE_HIGH_PRIORITY_THRESHOLD`, `DUPLICATE_AGGREGATION_POLICY` and the optional `ACKNOWLEDGEMENT_OVERDUE_DAYS`. Values are never printed; the README documents how to generate a secret and warns against reusing the production secret in tests.

One configuration behaviour deserves emphasis because it is easy to misread as a defect. Threshold changes made through `PUT /api/settings/threshold` are held in **process memory** and audited, but not persisted: a restart restores the environment defaults. The screen and the audit record do not claim otherwise, but an operator must know it.

5.3.12 Deployment implementation

Deployment is a single container image built from the repository `Dockerfile`, published to `ghcr.io/eggxellent/sbms-api` by a manually dispatched GitHub Actions workflow and run by Azure Container Apps. The build is not a plain `docker build`: it installs Python and creates a virtual environment, installs `xgboost` with `--no-deps` so that a ~350 MB CUDA dependency is never pulled, then asserts that no `nvidia-*` package is present; copies only `predict.py`, `feature_engineering.py` and the single production artefact; and runs an inference gate that loads the artefact, scores a known-duplicate pair above 0.9 and a known-unrelated pair below 0.5, and asserts the exact six-feature order. The workflow repeats the model gate and a secrets gate against the pushed image. An image that cannot score therefore cannot be published.

The frontend is built with `VITE_API_ORIGIN` pointing at the API origin, zipped and published to App Service, where a `web.config` rewrite lets client routes resolve while deliberately leaving `/api/` alone.

Table 5.7: Deployment checks recorded in the live deployment log

Check	Recorded result
Start-up script	PostgreSQL `Ready`; `/api/health` 200; `/api/ready` 200; frontend 200 with `#root`
Field-officer sign-in through the deployed form	200; `/field-officer` renders the officer dashboard
Administrator sign-in	200; `/dashboard` renders the admin dashboard
Admin surfaces	`/api/admin/dashboard`, `/api/duplicate-flags`, `/api/awards`, `/api/disbursements`, `/api/users`, `/api/audit-logs` → 200; `/api/students` → 403 (intended)
Student QR scan through the deployed SPA	Initially **broken** (`POST https://sbms-web.azurewebsites.net/api/verify` → 404) because the page used a relative `fetch`; after routing the call through the axios client, headless-Chrome `qr-smoke` passed **9/9** against the deployed site
Real-browser sign-in for both roles	**9/9**
Officer screens	`/students` rendered the live records (3.2 s warm, measured from `performance.getEntriesByType('resource')`); `/students/register` rendered its inputs
Live screening with shared identifiers	A second student sharing UPI and birth-certificate number produced `exact_identifier_matches: ["UPI","birth certificate number"]`, `birth_certificate_match: 1`, `upi_match: 1`, `model_score: 0.9382`, `priority: high`
Registration-fields smoke against production	**11/11**

The same log records what remains untidy: two probe students created during the live check were still in the production database at the time of that record; the stored administrator password had drifted from the operator secret file, so the admin reveal screen was not re-checked; and the database was left running, with a re-stop due by 8 November 2026 (`docs/azure-deployment-log-2026-10-31.md` §7–§9).

5.4 System testing

5.4.1 Testing strategy and levels

Testing was organised in five levels, each answering a different question, and the levels are reported separately because they carry different weight.

Table 5.8: Testing levels and what each establishes

Level	Instrument	Question answered	Scope in this project
Unit	Jest, `node --test`, Python `unittest`	Does a function or module behave as specified in isolation?	Feature engineering, aggregation policy, priority rules, date normalisation, payment state machine, error sanitisation, JWT validation
Integration (API + database)	Jest + Supertest against a real PostgreSQL database	Do routing, middleware, controllers, services and SQL cohere through the real HTTP surface?	24 suites, 151 tests, including an end-to-end business-workflow suite
Out-of-process smoke	`node --test backend/test/api.smoke.test.js` against a running API	Does a server started the way an operator starts it answer correctly?	3 checks: health, admin authentication with role boundary, report range validation
Browser (UI)	Headless Chrome over the DevTools Protocol	Does the rendered application perform the workflow a human performs?	23 registration/review checks, 9 QR checks, 11 registration-field checks against the deployed site
Deployment	Manual and scripted checks against Azure	Does the deployed configuration behave like the tested configuration?	Health, readiness, both sign-ins, admin surfaces, QR scan through the SPA, live screening

5.4.2 Test environment and provenance of results

Two environments carry the results, and the reader must know which produced what.

Disposable database. Database-backed suites run against `sbms_test` on loopback with role `sbms_test_runner`, under a guard that **fails closed** unless `NODE_ENV=test` and both `DATABASE_URL` and `SBMS_TEST_DATABASE_URL` name that same loopback database (`backend/tests/testDatabaseGuard.js`). The guard makes it impossible for these runs to have touched a production database, and it is why the suites abort rather than skip when the environment is wrong. The recorded state of that database after a full run was 12 base tables, 118 columns, 56 constraints, 21 indexes and 12 applied migrations, containing `users = 4`, `audit_logs = 1` and **zero** students, flags, awards and payments, which also shows that the suites clean up after themselves.

Provenance. The figures below are taken from the project's own records, which name the command, the date and the commit: `docs/FINAL_TEST_REPORT.md` (§1–§10, covering 2026-10-31 and 2026-11-01), `docs/business-workflow-verification-2026-10-30.md`, `docs/frontend-verification-2026-10-30.md`, `docs/qr-verification-2026-10-30.md` and `docs/bursary-certificates-2026-11-01.md`. Those runs were **not re-executed** while this chapter was written; the suites and their assertions were re-read to confirm that the reported counts and behaviours correspond to the code now in the repository. Where records disagree, the newest is cited and the older is identified.

5.4.3 Automated backend suite: test-case inventory

Table 5.9 (part 1 of 2): backend Jest suites and what each establishes

#	Suite	Tests	What it establishes
1	`auth.test.js`	4	Public registration returns 410; field-officer sign-in issues a JWT; invalid tokens and wrong-role access are refused
2	`students.test.js`	9	Authentication required; registry listing; five-school list; unapproved school rejected; admin refused; birth-certificate persistence; reused UPI queues a case; exact birth-certificate agreement prioritised
3	`duplicateFlags.test.js`	9	Review threshold exposed distinctly; queue returns pending cases; reveal audited; paired scoring without persistence; non-admin refused; decision recorded in case history; confirmed duplicate blocks disbursement; pending and needs-verification block until resolved
4	`duplicateAggregation.test.js`	13	`top-1` default; policy read and validated; one flag from many qualifiers; exact identifier above score; deterministic tie-break; below-cutoff candidate flagged only with an exact identifier; `all` retains legacy behaviour; corroboration suppression and its exception; counters; corroboration ignores name and school; missing threshold rejected
5	`mlDuplicateReview.test.js`	8	Batched scoring in order; version reported with no pairs; national-ID exact match as a signal; UPI and birth-certificate signals; exact agreement outranks score; score-only cases use the high-priority cutoff; fallback cutoff; no throw on malformed input
6	`predictorDateNormalization.test.js`	6	`pg` DATE serialised as its calendar date; normalised values unchanged; ISO timestamp truncated; missing values null; unrecognised text passed through; only the date field mutated
7	`duplicateCheck.test.js`	15	Behaviour of the **legacy** rule engine, retained for comparison; not a production path
8	`mlPredictor.integration.test.js`	1	The deployed artefact scores a synthetic pair and reports its version
9	`workflow.test.js`	11	The whole chain: case creation, verification gate, duplicate gates on approval **and** payment, terminal confirmed duplicate, non-re-decision, `needs_verification` clearing, approved-amount ceiling, clearance evidence and reconciliation trail, acknowledgement discrepancy, ten-action audit trail, role boundaries
10	`bursaryAwards.controller.test.js`	5	Approval of a verified award with no open case; rejection with reason; blocked before verification or review; malformed decision refused before a transaction; route guards
11	`disbursements.test.js`	4	Authentication required; invalid amount refused; `issued` payment for an approved verified award; payment blocked while a case is pending
12	`disbursementReconciliation.controller.test.js`	4	Split payments only within the approved balance; clearance requires evidence and is audited; acknowledgement mismatch recorded as an exception; clearing without evidence refused before a transaction

Table 5.9 (part 2 of 2): backend Jest suites and what each establishes

#	Suite	Tests	What it establishes
13	`bursaryRules.test.js`	4	Payment transition legality, cent arithmetic, exception derivation, overdue computation
14	`bursaryCertificates.test.js`	12	Approval as precondition; admin-only issue and revoke; malformed period refused; QR payload is the verification URL; no second certificate; staff read and PDF download; public authenticity without beneficiary data; identical refusal for unknown, malformed and manipulated tokens; token namespaces kept apart; `not_current` after an amount change; revocation leaves the award untouched; audit rows
15	`certificateVerificationRoute.test.js`	5	Handler reached only through the limiter; limiter before the parser; size limit still enforced; only POST routed; separate student and certificate budgets
16	`verification.test.js`	10	64-hex token; QR encodes the fragment URL; token never returned; valid token yields only `found:true`; unknown and malformed tokens indistinguishable; extra fields and empty body refused; budget stops the eleventh request; forged `X-Forwarded-For` does not reset the budget
17	`verificationRoute.test.js`	4	Single registration with the limiter before the parser; size limit enforced; only POST routed; response exposes nothing but `found`
18	`roleAuthorization.test.js`	6	Unauthenticated callers refused; admin lists and creates accounts; separate sign-in endpoints; synthetic counts and admin-only dashboard; student routes stay field-officer-only; field officers cannot manage users or thresholds; forged role claims and role changes take effect
19	`securityMiddleware.test.js`	9	Development default and production fail-closed; origin list parsing; security headers; trusted and untrusted preflight; pre-routing origin rejection; oversized JSON; malformed JSON; no message or stack leakage; limiter on both sign-in routes
20	`schemaReadiness.test.js`	5	Readiness contract lists every required table; built schema reported ready; the legacy table has the columns the model uses; the mounted beneficiaries query executes; a row round-trips
21	`loginRateLimit.test.js`	1	Ten sign-in attempts allowed, the eleventh refused
22	`performance.test.js`	1	A 50-candidate registration is screened within three seconds
23	`kenyanSyntheticDataset.test.js`	4	50 marked records and 21 labelled pairs; typo variants are candidates and reversed order is not assumed exact; shared school and guardian alone do not make a different-age sibling a candidate; missing values are zero evidence
24	`health.test.js`	1	Liveness endpoint returns 200

Recorded result, newest full run (2026-11-01): `24 suites passed, 151 tests passed`, exit code 0 (`docs/FINAL_TEST_REPORT.md` §10; `docs/bursary-certificates-2026-11-01.md` §7). An earlier run on the same branch (2026-10-31, before the certificate feature) recorded `21 suites / 128 tests` with wall-clock times of 93.6 s and 73.5 s; it is retained because it shows the count depends on the revision, not the machine.

An honest note on stability. During the documentation work that preceded this chapter, three repeated `--runInBand` sweeps each completed 149 or 150 of 151 tests, with the failing suite varying between runs and always being one that reaches the Python predictor (`duplicateFlags`, `mlPredictor.integration`, `students`). The failure surfaced as `Python predictor process exited` → HTTP 503 `DUPLICATE_SCREENING_UNAVAILABLE`, and each of those suites passed alone (`duplicateFlags` 9/9, `duplicateCheck` + `mlDuplicateReview` 23/23, `mlPredictor.integration` 1/1). The cause identified was host memory pressure — about 0.5 GB free of 7.4 GB — combined with the design fact that `duplicateCheck.js` keeps an unreferenced predictor child process, with XGBoost and pandas resident, alive for the lifetime of each test file, so several accumulate during a sequential sweep and the operating system terminates one. This is an environment-dependent instability, not a code defect, and it is also a genuine operational limitation of the single-process predictor design (§5.6.5). The green 151/151 sweep in the project report remains the authoritative result for the revision.

5.4.4 Native, smoke and Python testing

Native in-process suite — 43/43. Seven files run under `node --test` without a live database: `databaseReadiness.node.js`, `databaseStartup.node.test.js`, `databaseBootstrap.node.test.js`, `errorHandler.unit.test.js`, `verification.unit.test.js`, `migrate.unit.test.js` and `jwtSecret.validation.node.cjs`. Recorded: `# tests 43 # pass 43 # fail 0`, 0.79 s, exit 0. Coverage includes the readiness contract (missing or shadowed table, wrong database, unset expected name, malformed catalogue rows, error propagation), start-up validation, the fail-closed empty-schema bootstrap (approval gate, wrong database, pre-existing relation, atomic rollback, bcrypt admin hash, e-mail collision), error-handler sanitisation, JWT-secret validation, migration target, inventory and ledger adoption, and QR verification including the rate-limit boundary.

Out-of-process API smoke — 3/3. `backend/test/api.smoke.test.js` fetches a live API; run in-process it fails on its prerequisites (`fetch failed`), which is expected and documented. Run against a server started the operator's way on `sbms_test`, it recorded `# tests 3 # pass 3 # fail 0`, exit 0, covering health, admin authentication with the field-officer boundary, and report range validation, with `/api/ready` returning `{"status":"ready"}`.

Python ML tests — 33/33 and 7/7. `python -m unittest discover -s ml-service/tests` recorded 33 passing tests in 1.96 s, covering feature engineering (shared training/serving features, missing-value handling), operating-point and threshold metrics, registration simulation, artefact loading (only the production artefact; missing or corrupt artefacts fail closed), training-integrity reporting (group holdout excludes pair families, the model does not memorise training rows, no single feature determines the label) and reliability analysis. `python -m unittest discover -s ml-service/experiments/sbms_dataset -p test_experiment.py` recorded 7 passing tests in 0.47 s. `pytest` was **not** run because it is not installed in that environment (`importlib.util.find_spec('pytest') -> None`), and no package was installed to change that.

5.4.5 Database, security and performance testing

Database integration. The Jest suites run against PostgreSQL 17.11, not mocks, and are gated so they cannot touch another database. Their value is not only that they exercise SQL but that they exercise the constraints: the workflow suite proves that a database `CHECK` refuses an approval without verification, a clearing event without evidence and an acknowledgement without a method, and the certificate suite proves that the revocation and token-format `CHECK`s behave. Database recovery was separately rehearsed end to end — custom-format dump, restore into an independent database, readiness check, schema and row-digest comparison, and an application smoke test against the restored copy (`docs/database-recovery-rehearsal-2026-10-31.md`). No backup schedule was configured, so this is a rehearsed procedure rather than an operating control.

Security testing. `securityMiddleware.test.js` (9) and `roleAuthorization.test.js` (6) carry the security evidence: origin allowlist enforced before routing, trusted and untrusted preflight, security headers, oversized and malformed JSON handled without leakage, no exception message or stack returned, both sign-in routes limited, and the role matrix enforced including the "forged claim" case. Rate limiting is verified at its boundary in three suites, including the check that a forged `X-Forwarded-For` header does not create a fresh budget.

Performance. `performance.test.js` records that a 50-candidate registration is screened within the three-second requirement, and the 2026-10-30 record measured 22.67 ms for 50 pairs with a warm predictor. This is a single-point measurement at a small registry size. There is **no** measurement at 500 or 1 000 candidates, no hosted load test and no concurrency benchmark, so the requirement is verified only for the case tested, and behaviour under concurrent registration is asserted by the advisory lock rather than measured. Screening time also grows with registry size, because every registration is compared against every student (§5.6.5).

5.4.6 Browser-based (user-interface) testing

Three browser harnesses drive real Chrome through the DevTools Protocol, signing in through the real forms and reading the rendered DOM.

Table 5.10: Browser test results by harness

Harness	Checks	Recorded result	What it establishes
`scripts/browser-smoke.mjs`	23	**23/23 pass** (2026-10-30)	Field-officer sign-in; normal registration; screening sentence shown; no model internals to the officer; re-registration created rather than rejected; duplicate warning; exact identifier named; red styling for a suspicious case; one case from several qualifying candidates; blocked API surfaces an error and no confirmation panel; admin sign-in; queue shows the case as pending; both cutoffs and the aggregation policy visible; model version and cutoff in case detail; score labelled as not a decision; audit counters rendered readably; admin redirected away from the registration page
`scripts/qr-smoke.mjs`	9	**9/9** locally (2026-10-30) and **9/9** against the deployed site after the fix (2026-11-01)	Valid code opens the page and reports the record; token removed from the visible URL; no student data displayed; unknown token refused; missing token refused; query-string token not accepted; second scan in the same tab re-verifies; no uncaught page errors
`scripts/registration-fields-smoke.mjs`	11	**11/11** against the deployed site (2026-11-01)	Both optional identifier inputs present and not required; the six original fields still posted; the payload carries both markers; a blank identifier omitted; a blocked request shows the error panel

Two defects were found at this level and would not have been found by unit tests: the stale-result behaviour of the verification page across two scans in one tab, and — in production — the relative `fetch("/api/verify")` that made every scan fail on the App Service host while the same endpoint answered correctly when called directly on the container app.

5.4.7 Defects found through testing and their resolution

Testing was not only confirmatory. The following defects were found by executing the system, and each is included because it is evidence about the system's quality rather than against it.

Table 5.11: Defects found and fixed

#	Defect	How it was found	Root cause	Resolution
D-01	Payment clearance returned 500 for every status change, so reconciliation was unreachable	The new end-to-end workflow suite exercised the endpoint for the first time	PostgreSQL `42P08`: one parameter reused in three type positions (`VARCHAR(20)`, `VARCHAR(50)` and a text comparison)	Explicit casts with a comment recording why they must stay (`docs/business-workflow-verification-2026-10-30.md` §2)
D-02	Every model-created case was `high` priority and the high-priority threshold was dead configuration; the queue rendered "high 0.00"	Browser verification of the review screen	Priority computed from the review threshold instead of the high-priority threshold; the policy block published `null`	One shared `reviewPriority` function used by both the registration response and the queue; the real value published (`docs/frontend-verification-2026-10-30.md` §1)
D-03	The `risk_level: 'critical'` styling path was unreachable	Browser verification	The API only ever emitted `'review'`	The API keeps `risk_level: 'review'` and publishes `priority` and `exact_identifier_matches` instead
D-04	A second QR scan in the same tab showed the previous result	Headless-Chrome QR harness	The page verified once on mount; a hash-only change does not remount	Re-verify on `hashchange`, with the token captured during render so React StrictMode's double effect does not break it
D-05	Every QR scan failed on the deployed site (404)	Production re-check through the deployed SPA	`fetch("/api/verify")` relative to the static host, which deliberately does not proxy `/api/`	Route the call through the axios client that honours `VITE_API_ORIGIN`
D-06	Stored dates of birth scored as non-matching	Integration testing of screening against the database	`pg` returns `DATE` at local midnight, which serialises to the previous calendar day east of UTC	Normalise to `YYYY-MM-DD` at the single predictor boundary (six tests)
D-07	A legitimate registration could be flagged with near certainty at scale	Measured, then simulated	One flag per qualifying pair against the whole registry	Aggregation policy with `top-1` default and counters for suppressed candidates

#	Defect	How it was found	Root cause	Resolution
D-08	`/api/verify` was registered twice, so the limiter and handler could be detached by an edit	Code review recorded as observation O11	Two independent route registrations	One chain — limiter, parser, handler — with a regression test asserting the ordering (`SBMS_O10_O11_IMPLEMENTATION_REPORT.md`)
D-09	Public rate limiting would put all users in one bucket behind a proxy	Code review during the QR phase	Limiter keyed on the socket address	`TRUST_PROXY_HOPS` sets Express `trust proxy`; the default preserves direct-socket behaviour

5.4.8 Proposed and outstanding tests

The following tests have **not** been executed. They are listed separately, as required, and none is reported as a pass anywhere in this chapter.

Table 5.12: Outstanding tests

#	Test	Why it is outstanding	What it would establish
O-01	Phone-camera scan of a printed student QR code and of a printed certificate	Requires a public HTTPS origin and a physical device; the deployment exists but no camera run is recorded	That the artefact works as a physical document, which is the project's stated identification mechanism
O-02	Load test at 500 and 1 000 candidates, and concurrent registrations	No hosted load test or concurrency harness exists	Whether NFR-01 and NFR-13 hold beyond a 50-candidate single-threaded case
O-03	Concurrency test of two simultaneous registrations of the same identifiers	The advisory lock exists; no test exercises it concurrently	That the lock prevents interleaved comparisons
O-04	Print-output verification of the certificate PDF and of the exported report	Only generated bytes are covered	That the printed documents are legible and correctly laid out
O-05	Accessibility audit of the public pages and staff screens	No automated or manual accessibility review exists	Whether the interface meets a stated accessibility standard
O-06	Cross-browser testing (Firefox, Safari, mobile browsers)	Only Chrome was used	NFR-19 beyond Chrome
O-07	Tests asserting the arithmetic of every dashboard counter and report figure	Counters are exercised for authorisation, not arithmetic	That each reported figure matches the underlying data
O-08	Automated test of the eligibility scoring arithmetic	No such test exists	FR-12 beyond the route and role check
O-09	Threshold validation against adjudicated pairs	Requires reviewer-labelled ground truth (§5.5.5)	Whether 0.50 is an appropriate operating point for real records
O-10	Disaster-recovery drill against the **hosted** database	The rehearsal used a local disposable database	That the production recovery procedure works against the managed service
O-11	Behavioural test of the legacy `beneficiaries` module	The router is mounted and readiness depends on it, but no behavioural test exists	Whether the legacy module still does anything useful
O-12	Certificate verification page in a real browser	No script exists for it; covered by API tests only	That the page renders its four states correctly

---

5.5 System evaluation

5.5.1 Evaluation approach

The system was evaluated against three references: the four specific objectives in Chapter One (§1.3.1), the requirement register in Chapter Four, and the problem statement itself. For each reference the evaluation distinguishes four outcomes — *demonstrated*, *partially demonstrated*, *not demonstrated* and *not evaluable with the*

evidence available. A requirement is "demonstrated" only where a test or a recorded observation establishes it; implementation alone is not demonstration.

5.5.2 Evaluation against the specific objectives

Table 5.13: Achievement of the specific objectives

Objective (Chapter One §1.3.1)	Evidence of achievement	Verdict
(i) Investigate the existing challenges of manual registration and tracking in Kenya's government-funded welfare programmes	Document analysis of Auditor-General, Controller of Budget and NG-CDF sources in Chapter Two, and the requirements derived from it in Chapter Four §4.2.1. No primary data collection was undertaken	**Partially achieved** — the secondary evidence base is documented, but the investigative dimension rests on published audit findings rather than field research
(ii) Design the architecture, database schema and machine-learning pipeline, incorporating a Random Forest classifier	Chapter Four §4.4–§4.7 presents the delivered architecture, schema (12 tables, 56 constraints) and pipeline. The classifier **shipped as XGBoost over six features**, not Random Forest over four; the Random Forest exists only as an unevaluated research candidate	**Achieved with a documented design change** — the substitution must be acknowledged rather than explained away
(iii) Implement a web-based system supporting digital registration, unique identifier generation, QR-based identification and role-based access control	Registration (FR-05), server-generated UUID keys and 64-hex QR tokens (FR-20), two-role access control enforced in middleware and pinned by tests (FR-04), and a deployed instance at a public HTTPS origin — covered by automated tests plus browser and deployment checks	**Achieved**
(iv) Validate and test the system to evaluate its effectiveness in detecting duplicate records and improving beneficiary tracking	151 backend tests, 43 native tests, 3 smoke checks, 33 + 7 Python tests, 43 browser checks and deployment checks. Detection effectiveness **against real records is not established**: labels are synthetic and no adjudicated pairs exist. Tracking is demonstrated through the award, payment, reconciliation and certificate chain, covered by an eleven-test end-to-end suite	**Partially achieved** — validated as an artefact and the tracking chain demonstrated; detection effectiveness demonstrated only against synthetic data

The honest reading is that two objectives are achieved, one is achieved with a documented design change, and one is partially achieved for a reason that cannot be removed within scope: no legitimately obtainable, adjudicated set of real duplicate/non-duplicate beneficiary pairs exists for this project, so no statement about real-world detection effectiveness follows from its evidence.

5.5.3 Evaluation against functional requirements

Of the thirty delivered functional requirements, **twenty-eight** are demonstrated by at least one executed test that asserts the behaviour, and two are implemented but not demonstrated at the level the requirement specifies:

- **FR-12 (eligibility scoring)** — the route, validation and role restriction are tested, but the point arithmetic and category boundaries are not. Demonstration would need a table-driven test of `calculateEligibility`.
- **FR-25 (dashboard)** — authorisation and synthetic-count reporting are tested; the correctness of each of the nine accountability counters and of the two trend series is not asserted.

Three further requirements are only partially demonstrated: FR-26 (PDF export) is tested for date-range validation but its rendered content is verified by inspection; FR-10 (masking) is tested for the queue response, while the reveal endpoint's audit row is asserted rather than the disclosure workflow exercised in a browser; FR-19 (exceptions) is tested through the discrepancy case in the workflow suite, not for all eleven reason codes.

Two requirements are **not implemented by design** (FR-32 NEMIS integration, FR-33 offline and native mobile) and four recorded gaps remain open (FR-34 candidate blocking, FR-35 QR re-issue, FR-36 token expiry, FR-37 financial transfer). None is a defect against the approved scope: the first two were excluded in the proposal, and the latter two were never claimed.

5.5.4 Evaluation against non-functional requirements

Table 5.14: Non-functional requirement outcomes

NFR	Outcome	Basis
01 Performance (≤ 3 s screening)	Satisfied at the tested size; **not evaluated at registry scale**	22.67 ms for 50 pairs; no larger measurement
02–12 Security, privacy, integrity	Satisfied	Fifteen named tests across `securityMiddleware`, `roleAuthorization`, `verification*`, transaction and certificate suites; masking, redaction and rate limits are asserted
13 Concurrency	**Not evaluated**	Advisory lock present; no concurrent test
14 Usability	**Not evaluated with users**	Implemented and browser-checked; no officer session
15 Maintainability	Satisfied	Shared feature module, ordered migrations, three test frameworks, pinned versions
16 Cost	Satisfied as configuration	Deployed topology inside the approved cost envelope
17 Human oversight	Satisfied	Every decision path is role-gated and audited
18 Recovery	Procedure rehearsed; **no scheduled backup**	`docs/database-recovery-rehearsal-2026-10-31.md`
19 Browser compatibility	Chrome only	No cross-browser record
20 Predictable degradation	Satisfied	503 with no student created when the predictor is unavailable
21 Audit completeness	Satisfied for tested workflows; **no immutability or retention control**	Ten-action audit sequence asserted

5.5.5 Machine-learning evaluation

The model was evaluated on a **synthetic holdout**, and this subsection reports those figures with the limits that make them interpretable. Nothing here is an accuracy claim about Kenyan beneficiary records.

Training and evaluation design. Two independently generated datasets of 12 000 pairs each (6 000 duplicate groups, balanced 6 000/6 000) were used: an internal training set split 80/20 by identity group, and an independent holdout generated from new latent identities and a separate random seed. The holdout was not used for fitting or for parameter tuning beyond model selection between candidate algorithms, and the artefact reports this explicitly (`holdout_used_for_training_or_tuning: false`, `holdout_used_for_model_selection: true`). The selected configuration was `n_estimators 250`, `max_depth 4`, `learning_rate 0.05`, `subsample 0.9`, `colsample_bytree 0.9`, `reg_lambda 1.0`, evaluated at a threshold of 0.50.

Table 5.15: Pair-level performance on the synthetic holdout at the live 0.50 cut-off

Metric	Value
Accuracy	0.9078
Precision	0.9204
Recall	0.8927
F1	0.9063
Average precision (PR-AUC)	0.9727
ROC-AUC	0.9685
Confusion matrix (TP / FP / TN / FN)	5 356 / 463 / 5 537 / 644
Miss rate	10.73%
False-positive rate on the hard-negative pool	7.72%

Threshold behaviour. Because a single threshold is a policy choice rather than a property of the model, the operating point was measured across a range.

Table 5.16: Operating-point trade-off (synthetic holdout)

Cut-off	Precision	Recall	F1	Miss rate	FPR (hard negatives)	Review share
0.30	0.8694	0.9352	0.9011	6.48%	14.05%	53.78%
0.40	0.8985	0.9132	0.9058	8.68%	10.32%	50.82%
0.50 (live)	**0.9204**	**0.8927**	**0.9063**	**10.73%**	**7.72%**	**48.49%**
0.60	0.9404	0.8647	0.9009	13.53%	5.48%	45.97%
0.70	0.9607	0.8270	0.8888	17.30%	3.38%	43.04%
0.80	0.9791	0.7797	0.8681	22.03%	1.67%	39.82%
0.90	0.9899	0.7037	0.8226	29.63%	0.72%	35.54%

F1 is essentially flat between 0.30 and 0.60 (0.9011–0.9065), the best value is 0.9065 at 0.35, and the live 0.50 is 0.0002 below it while having the highest Youden J (0.8155). The measurement therefore does **not** justify moving the production cut-off, and equally does not justify the claim that 0.50 is optimal — it says only that this data cannot distinguish the options in that band.

Where the flags actually come from. The negatives were deliberately hard, and the per-family rates show what kind of case reaches a reviewer.

Table 5.17: Negative families at two cut-offs (synthetic holdout)

Negative family	Pairs	Flagged at 0.50	Flagged at 0.70
`identifier_collision`	1 202	22.21%	11.98%
`common_name_same_school`	1 492	7.84%	2.41%
`twins_or_close_age`	1 150	5.57%	1.57%
`siblings_shared_household`	1 532	0.98%	0.33%
`unrelated_candidate`	624	**0.00%**	**0.00%**
`repeat_application` (positives)	6 000	89.27% recalled	82.70% recalled

The model does not emit noise: genuinely unrelated synthetic pairs never reach the cut-off. What it flags are cases a human would want to see — reused identifiers, twins, same-age siblings, identical common names in one school. That is correct screening behaviour, and it is the reason queue volume has to be governed by candidate selection rather than by declaring the model wrong.

Registration-level consequences. Pair-level precision is not what an administrator experiences, because a registration compares one record against *every* student. The probability of at least one nuisance flag per legitimate registration is `1 - (1 - p)^N`; with `p = 0.35%` (the measured rate on 2 000 random unrelated pairs, `docs/ml-evaluation-baseline-2026-10.md`) at `N = 1 000` that is 96.98%, and the simulation reproduced 97.00%. At cut-off 0.50 the simulation reported, for hard negatives, 970 nuisance flags per 1 000 legitimate registrations at `N = 1 000` and 1 000 at `N = 5 000`; at 0.90 the random-unrelated column collapses to zero but the hard-negative column remains at 999–1 000. Raising the cut-off therefore cannot fix queue volume while the comparison set is the whole registry, because recall falls faster than the queue shrinks (0.8927 → 0.7037 between 0.50 and 0.90).

The `top-1` policy also has a second cost, which the aggregation tests do not measure: if a nuisance candidate outranks the true duplicate, the reviewer is shown the wrong pair and the real duplicate is suppressed. The simulation reports clean detection falling from 67.6% at `N = 100` to 36.6% at `N = 5 000`, with masking rising from 23.8% to 52.6%; the controller ranks exact identifier agreement above score, which rescues identifier-verified duplicates and is not modelled in the simulation, so those masking figures are pessimistic but not negligible.

Table 5.18: Candidate-generation projection (only lever that changes the order of magnitude)

Candidate key	Block size at N = 1 000	Block size at N = 5 000	Duplicates retained
Same surname **and** same school	0.99	4.95	**16.47%**
Any shared name token	29.85	149.25	**80.52%**

The surname-and-school key looks excellent on volume and is unusable in practice: it silently discards 83.5% of duplicates. Name-token blocking retains 80.52% while cutting the comparison set by roughly 33× at `N = 1 000`, and the projected nuisance rate behind it ranges from 9.94% to 90.90% of registrations at that size — a real improvement, and an honest measure of how much remains unknown.

Class imbalance and the cost of errors. The evaluated label mix is balanced 50/50, which is not the prevalence expected operationally; average precision therefore depends on the synthetic 25% positive prevalence in the research dataset and on 50% in the production holdout. The asymmetry that matters is operational: a false positive costs reviewer time bounded by the aggregation policy, while a false negative leaves a duplicate beneficiary unflagged. The project did not fix a miss-tolerance policy (decisions D1–D5 remain unanswered), so the choice between those costs was deferred rather than made — recorded in Chapter Six as a required decision.

What this evaluation does not establish. It does not establish precision, recall or miss rate on real beneficiary records; it does not establish the false-positive rate of the applicant population; it does not establish that 0.50 is the right cut-off; it does not cover duplicate patterns other than `repeat_application`; and it does not establish generalisation beyond the simulator. Those exclusions are the project's own, stated in `docs/ml-decision-sheet-2026-10-30.md` §8, and they are the reason the chapter reports a pipeline result rather than a detection capability.

5.5.6 Usability and user acceptance

User acceptance testing was **not conducted with real users**. A UAT plan exists (`docs/uat-test-plan.md`) with five scenarios — login, registration with duplicate detection, duplicate review, disbursement control, and dashboard/reporting — each marked `PASS`, and an audit-trail check. That document cannot be presented as participant-based acceptance evidence, for three reasons that must be recorded rather than smoothed over: it names no participants, roles, dates or locations; it records no observations or free-text feedback; and one of its scenarios still states that the backend does not call the Python predictor, which the implementation clearly does (Chapter Four §4.4.4). It is best read as an author-executed scripted walkthrough of the happy paths, superseded in evidential value by the browser harnesses in §5.4.6, which record what the rendered application actually did.

Consequently NFR-14 (usability) and any claim about user acceptance are **REQUIRES INPUT**: participants, tasks, findings and consent arrangements must be supplied by the author if an acceptance study is to be reported.

5.6 Discussion of results and limitations

5.6.1 What the implementation achieved

The system built is a working, deployed, role-separated web application in which a beneficiary is registered once, screened against the existing registry and issued a QR-verifiable identity; a possible duplicate becomes a review case that only an administrator can close; and money cannot be approved or paid to a student whose case is open or confirmed. Around that spine sit award submission with mandatory verification evidence, payment records that distinguish issuance from settlement, receipt acknowledgement that preserves a discrepancy instead of hiding it, an exception list derived from the data, an audited disclosure path for beneficiary identifiers, and a QR-verifiable award certificate that reports its own staleness.

Three capabilities deserve emphasis because they exceed what the proposal promised and make the artefact believable as an accountability tool rather than a record store: the certificate snapshot that survives a later award change; the derived exception rules that make under-payment and missing acknowledgement visible; and the masking-plus-audit design that lets an administrator see enough to judge a case without turning the queue into a personal-data browser.

5.6.2 How the system addresses the problem in Chapter One

The problem was that beneficiary records are held on paper in independent offices, that the same learner can appear more than once, and that payments cannot be traced to a verified student. Against that, the delivered system contributes a single registry with a unique identifier per student; a screening step that surfaces near-

duplicates exact matching would miss, with a human decision required; a payment chain in which every step is attributable to a named user and carries evidence; and a verification artefact a school or guardian can check without gaining access to the beneficiary's data.

It does **not** address the root cause of cross-constituency duplication, because the system is single-tenant: it screens within one registry, and a student registered in two different constituency deployments would still not be detected. That limitation is inherent to the scope (no government data-sharing agreement, no NEMIS integration) and must not be glossed: within-registry detection is a genuine improvement over paper, but it is not national deduplication.

5.6.3 What the testing establishes, and what it does not

The executed evidence establishes that: the access-control model behaves as specified for every role-sensitive route tested; the two payment gates cannot be bypassed through the documented API, including when the student appears as the *matched* rather than the *new* record; database constraints independently enforce the same rules; the public endpoints disclose only what they claim, and refuse without distinguishing "unknown" from "malformed"; the predictor is correctly wired, versioned and fail-closed; and the deployed configuration behaves like the tested configuration for the checks run, after one genuine production defect was found and fixed.

It does **not** establish that the deployed application behaves correctly for a real officer working with real documents: there is no usability observation, no phone-camera scan, no print check, no load test and no cross-browser evidence. Nor does it establish that the model's performance on real records resembles its performance on the simulator.

5.6.4 Technical and operational limitations

1. **Candidate generation is unindexed.** Every registration is scored against every existing student, and the administrative rescan is quadratic. This bounds the registry size at which the system is usable in practice.
2. **The predictor is a single child process.** One artefact, one process, one lifetime bound to the API; a crash fails current registrations and momentarily disables registration, and it accumulates memory per test file, which is the documented cause of the intermittent failures in §5.4.3.
3. **Rate limiting is per process.** Counters reset on restart and are not shared between instances, so horizontal scaling weakens the protection.
4. **Threshold changes are not durable.** `PUT /api/settings/threshold` alters in-memory values that a restart discards, with no persisted configuration record beyond the audit entry.
5. **No backup schedule or retention policy.** Recovery is a rehearsed manual procedure; audit rows are append-only by convention only; nothing is ever deleted, which is a privacy liability as well as a storage one.
6. **Two partially overlapping notions of "beneficiary"** exist (the `students` model and the legacy `beneficiaries` table), with no reconciliation and an unresolved `scholarship_amount` meaning.
7. **The dead rule engine** in `duplicateCheck.js` remains exported and is exercised only by tests — a maintenance hazard in a codebase whose central concept is duplicate scoring.
8. **No individual session revocation** and no multi-factor authentication.

5.6.5 Data and machine-learning limitations

The central limitation is that **no adjudicated real-world label set exists**. Everything measured is a property of a simulator: the positives are one duplicate pattern (`repeat_application`); the negatives are designed rather than observed; the balance is not operational prevalence; and one artefact at one seed produced every number, so a retrain invalidates the report. The decisions that would convert these measurements into a defensible operating point — flag budget, review capacity, miss tolerance, cross-school comparison, adjudication owner — are recorded as unanswered in `docs/ml-evaluation-policy.md` and remain unanswered.

A second limitation is specific to the data the system can collect: the registry uses five school *names* as identifiers because no authoritative school-code list exists, so the model's `school_match` feature compares names. Until an

authoritative code list and a name-to-code mapping are supplied, the feature is weaker than its name suggests (`docs/ml-integration-and-school-identifiers.md`).

5.6.6 Security, deployment and integration limitations

- The deployment is **not continuously available**: the managed database is stopped between demonstrations to stay inside a student credit budget, so availability is a scheduled property, not a service level.
- The system is **not integrated** with NEMIS, a national identifier service, a bank or a mobile-money operator, and it neither executes nor confirms a financial transfer.
- Receipt acknowledgement is **officer-assisted**; independent school confirmation does not exist.
- Certificate verification is live and therefore depends on the database being up; a printed certificate cannot be checked while the deployment is stopped, a trade-off that favours truthfulness over offline convenience.
- The public rate limits are in-process, and `TRUST_PROXY_HOPS` must be configured correctly behind a proxy or all public users share one budget (defect D-09).

5.6.7 Reliability of the evidence itself

Two project records conflict with the implementation and are named here so that the report does not inherit their errors. `docs/verification.md` states that "a repository reference check found no current callers" of the predictor and that controller decisions still use their own scoring; that was true of an earlier revision and is false now (`students.controller.js:69`). `docs/uat-test-plan.md` both asserts that the predictor is not called and marks all five scenarios `PASS` without participant records. Both documents are superseded by `docs/FINAL_TEST_REPORT.md`, `docs/business-workflow-verification-2026-10-30.md` and the suite inventory in §5.4.3. `docs/FINAL_TEST_REPORT.md` itself contains two generations of result (128 tests on 2026-10-31, 151 on 2026-11-01), which is why this chapter always names the date of a count.

5.7 Chapter summary

This chapter reported an implementation whose technical spine is sound and whose evidence has clear edges. On the implementation side: a three-tier system with a supervised Python scorer, a two-role access model re-validated against the database on every request, database-enforced integrity rules around awards and payments, two independent payment gates, QR and certificate verification separated technically as well as conceptually, and a deployment whose image cannot be published if the model cannot score.

On the evidence side: 151 automated backend tests, 43 native tests, 3 out-of-process smoke checks, 40 Python tests, 43 browser checks and a set of live deployment checks, with nine defects found and fixed during testing — one of them, payment clearance, a total functional failure that no earlier test had covered, and one a production-only routing defect that made every QR scan fail on the deployed site.

On the evaluation side: two objectives achieved, one achieved with a documented classifier substitution, and one partially achieved because the evidence needed to establish real-world duplicate-detection effectiveness does not exist within the project's scope. The model's measured performance (F1 0.9063, precision 0.9204, recall 0.8927 at 0.50 on a synthetic holdout) is a property of the simulator, and the operating point is unvalidated because there are no adjudicated labels; the more consequential finding is that candidate generation, not the threshold, governs the practical review load.

Chapter Six draws conclusions from exactly this evidence, and confines its recommendations to what the limitations above make necessary.

CHAPTER 6: CONCLUSIONS, RECOMMENDATIONS AND FUTURE WORKS

> **Status of this file.** Report-ready content for Chapter 6. Every conclusion in §6.2 is derived from evidence presented in Chapters Four and Five and introduces no new results, measurements, citations or claims. Every recommendation in §6.3 is tied to a stakeholder, an action, a reason and the limitation that motivates it. Every item in §6.4 addresses a limitation that Chapter Five identified.
>
> **Revision described.** Branch `fix/sbms-completion-work` at commit `d6b5032`, the same revision as Chapters 4 and 5. Every fact in this chapter traces to the dossier in `SBMS_COMPREHENSIVE_CHAPTERS_4_5_6_INFORMATION_REPORT.md` §28 and to the chapter sections named in §6.2.

6.1 Introduction

This chapter closes the study. Section 6.2 states the conclusions in the same order as the specific objectives in Chapter One, and answers what the project set out to do, what it achieved, what its testing established and where the boundaries of the achieved solution lie. Section 6.3 sets out recommendations for the stakeholders who would operate or maintain the system, each tied to the evidence that motivates it. Section 6.4 identifies future work arising from the limitations recorded in Chapter Five, explaining for each item the limitation addressed, what would have to change, and how the improvement could be evaluated.

The chapter deliberately does not claim that the system eliminates fraud, prevents all duplicate registrations or guarantees correct allocation of bursary funds. The system provides a controlled registration record, a screening signal that a human adjudicates, and an evidence-bearing payment chain; what it achieves is bounded by the quality of the data entered into it, by the decisions its human reviewers make, and by the fact that it operates on one registry at a time.

6.2 Conclusions

6.2.1 The problem the project addressed

The project addressed a specific and documented failure of public bursary administration in Kenya: beneficiary records are kept on paper in independent constituency offices, so a learner can be registered more than once without any office being able to detect it, and payments are frequently not traceable to a verified student. Chapter One grounded this in published audit findings, and Chapter Two established that existing beneficiary-management systems either address a different domain or rely on exact matching that cannot identify near-duplicates caused by spelling variation, name reordering or partial identifiers. The problem was therefore not a general claim about inefficiency; it was a specific claim about the absence of a shared, verifiable record and of any mechanism for identifying the same learner twice.

6.2.2 The purpose of the project

The purpose, as stated in the general objective, was to design and develop a web-based student beneficiary management system with machine-learning-driven duplicate detection, to digitise and streamline registration, identification and benefit-distribution tracking. That purpose was pursued through Design Science Research: an artefact was built, deployed and evaluated, with the evaluation reported in Chapter Five as far as the available evidence allows.

6.2.3 Achievement of the first objective: investigating existing challenges

The first objective — to investigate the challenges associated with manual registration and tracking — was **partially achieved**. The investigation was conducted by document analysis of Auditor-General, Controller of Budget and NG-CDF material, and it produced a requirements set that is traceable into the implemented system (Chapter Four

§4.2.1). What it did not include is primary investigation: no bursary officer was interviewed, no constituency office was observed, and no manual register was examined. The conclusions about practice therefore rest on published audit evidence rather than on the researcher's own fieldwork, and this should be stated wherever those conclusions are relied upon.

6.2.4 Achievement of the second objective: architecture, schema and ML pipeline design

The second objective — to design the architecture, database schema and machine-learning pipeline — was **achieved with one documented design change**. Chapter Four presents a three-tier architecture in which the machine-learning component is a supervised child process rather than a separate network service; a relational schema of twelve tables with fifty-six constraints and twenty-one indexes; and a pipeline in which one feature-engineering module serves both training and inference, which is what makes the served score the same quantity the evaluation measured.

The design change is that the classifier specified in the proposal as a Random Forest over four features was delivered as an XGBoost classifier over six features. This is not a cosmetic substitution: the two additional features (UPI and birth-certificate agreement) change what the model can see, and the algorithm change alters both performance and interpretability. The honest conclusion is that the design objective was met in substance — an architecture, a schema and an ML pipeline were designed, built and documented — while the specific classifier named in the approved proposal was not the classifier delivered. That divergence must be reported as a deviation, not silently reconciled.

6.2.5 Achievement of the third objective: implementation of the web-based system

The third objective — to implement a system supporting digital registration, unique identifier generation, QR-based identification and role-based access control — was **achieved**. The evidence is not merely that the code exists: the registration path, the server-generated identifier and QR token, and the two-role access matrix are each covered by executed tests, and the deployed instance was checked with real browser sessions for both roles and with a live screening request that produced an exact identifier match (Chapter Five §5.3.12). The system additionally delivered capabilities beyond the original scope — award verification, payment reconciliation and QR-verifiable award certificates — which are relevant to the accountability problem even though they were not in the proposal.

Two boundaries must be stated with that conclusion. First, the system identifies a student by a random token whose possession proves only that a record exists; it does not establish identity, eligibility, approval or payment, and it says so on the page a verifier sees. Second, the system records payments; it does not make them, and it does not receive independent confirmation from a school that money arrived.

6.2.6 Achievement of the fourth objective: validation and evaluation

The fourth objective — to validate and test the system, and to evaluate its effectiveness in detecting duplicate records and improving beneficiary tracking — was **partially achieved**, and the split matters.

The system was validated as an artefact to a degree that is unusual for a project of this size: 151 automated backend tests against a real PostgreSQL database, 43 native tests, 3 out-of-process smoke checks, 40 Python tests, 43 browser-driven checks and a series of live deployment checks, with nine defects found and fixed during testing (Chapter Five §5.4). Those defects include a total functional failure — payment clearance returned an error for every request — that no earlier test had covered, and a production-only routing fault that made every QR scan fail on the deployed site while succeeding when the same endpoint was called directly. Finding such defects is evidence that the validation was real rather than ceremonial.

Effectiveness in *detecting duplicate records* was **not** established in the sense the objective implies. The model was measured on a synthetic holdout and achieved F1 0.9063, precision 0.9204 and recall 0.8927 at the live cut-off of 0.50, with false-positive rates that vary by negative family from 0.00% for unrelated candidates to 22.21% for identifier collisions. Those figures demonstrate that the pipeline is correctly constructed and that the model separates the simulated classes well. They do not demonstrate detection performance on real beneficiary records,

because no adjudicated real-world labels exist, and the project says so rather than presenting synthetic figures as an accuracy claim.

Improvement in *beneficiary tracking* is better supported, because it does not depend on the model: the award, payment, reconciliation and certificate chain is demonstrated end to end, including the rule that a student with an open or confirmed duplicate cannot be approved or paid, and that a duplicate raised after approval still blocks payment. That is a demonstrable improvement in traceability over a paper register, and it is the part of the objective that can be concluded without qualification.

6.2.7 What the testing established about the system's quality

Beyond the objectives, the testing supports four conclusions about the system's engineering quality.

First, the system's integrity does not rest on application code alone. The rules that matter most — an approval requires a verified beneficiary, a cleared payment requires evidence, a revoked certificate requires a reason, a certificate token must be 64 hexadecimal characters, a school must be one of the five approved — are enforced by database constraints as well as by controllers, and the tests exercise the constraints. That redundancy is why a mistake in one controller does not become a financial irregularity.

Second, the access-control model is sound in a specific and testable way: the role used for authorisation is re-read from the database on every request, so a forged token claim confers nothing and a deactivation takes effect immediately. That property was verified rather than assumed.

Third, the public verification surfaces disclose what they claim and nothing more. The student endpoint returns a boolean; the certificate endpoint returns authenticity facts without the student's identity or the amount; both return the same answer for a malformed token as for an unknown one; and both are rate limited with separate budgets. These properties are asserted by eleven tests between them.

Fourth, the system is honest about its own uncertainty where it matters most. The review queue declares `thresholds_validated: false`; the model version is published with every case so a decision can be traced to an artefact; the officer's screen states that an administrator must review the case; and both public pages state what they do not prove. A system that over-claimed here would be worse than one that is incomplete.

6.2.8 What the project contributes

The contribution of the work is threefold. To practice, it contributes a working, deployable design for a beneficiary-management system in which machine-learning-assisted duplicate screening is embedded in an accountability chain rather than presented as an end in itself, and in which the model's output is explicitly a review signal rather than a decision. To the specific problem in Chapter One, it contributes a concrete mechanism — an evidence-bearing award and payment chain guarded by duplicate review — that addresses the traceability failures documented by the Auditor-General within a single registry. To the research question about near-duplicate detection, it contributes a reproducible pipeline, a measured operating-point analysis on synthetic data, and a documented, quantified finding that the aggregation policy and candidate generation, not the classifier threshold, dominate the practical review load — a result that is transferable to any registry-scale record-linkage deployment.

6.2.9 The boundaries of the achieved solution

The solution achieved is bounded in five ways that the report must carry explicitly.

1. **Single registry.** Deduplication operates within one deployment's students. Cross-constituency duplication — the core of the problem in Chapter One — is not addressed, because that requires shared infrastructure and a data-sharing agreement that are outside the project's scope.
2. **Synthetic evidence.** All model performance figures describe a simulator. No claim about real detection performance is made, and none can be made from this evidence.
3. **Human-dependent.** Every duplicate decision, approval, clearance and acknowledgement is a human act. The system's guarantees are of the form "this cannot happen without a named person and a recorded reason", not "this cannot happen".

4. **No financial execution or independent confirmation.** The system records that a payment was issued and, separately, that it was reported as cleared; it does not move money and does not obtain a school's independent confirmation of receipt.
5. **Operational scope.** The deployment is a demonstration instance with a stopped database outside demonstration windows, no automated backup schedule, no load testing and no acceptance study. It is a validated artefact, not a production service.

6.3 Recommendations

Each recommendation names the stakeholder it is for, the action, the reason, the evidence or limitation that motivates it, and a priority. Priorities are **P1** (before any use with real beneficiary data), **P2** (before routine operational use) and **P3** (improves the system but is not blocking).

6.3.1 Recommendations for the implementing organisation (programme owner)

Table 6.1: Recommendations for the implementing organisation

ID	Action	Reason	Evidence / limitation	Priority
R-01	Answer decisions D1–D5 before the system is used with real records: flag budget per registration, weekly review capacity, acceptable miss rate, whether cross-school comparison is permitted, and who adjudicates	These choices determine the operating point, the review workload and the meaning of "duplicate"; until they are answered the cut-off is unvalidated and the queue cannot be sized	`docs/ml-evaluation-policy.md`; Chapter Five §5.5.5	**P1**
R-02	Commission an adjudication exercise of 200–300 stratified pairs and adopt it as the reference label set	Every real-world quality claim depends on it; without it the measured performance describes only a simulator	Chapter Five §5.5.5; `docs/ml-decision-sheet-2026-10-30.md` §7 (D5)	**P1**
R-03	Approve a data-protection basis, retention period and access policy before real beneficiary data is entered	The system stores names, dates of birth, national identifiers, UPI, birth-certificate numbers and guardian contacts, and has no retention or deletion mechanism	Chapter Four §4.2.4; Chapter Five §5.6.4	**P1**
R-04	Supply an authoritative school-code list and a name-to-code mapping	The registry uses five school names as identifiers, so the model's school feature compares names	`docs/ml-integration-and-school-identifiers.md`; Chapter Five §5.6.5	**P2**
R-05	Define the acknowledgement service standard and set `ACKNOWLEDGEMENT_OVERDUE_DAYS` accordingly	Overdue-acknowledgement reporting is disabled while the setting is unset, so a central indicator is silently inactive	Chapter Five §5.3.7; `docs/business-workflow-verification-2026-10-30.md` §4	**P2**
R-06	Decide whether independent school confirmation of receipt is required, and if so fund the interface for it	Officer-assisted acknowledgement is not independent verification, and the audit problem in Chapter One is precisely about unconfirmed school receipts	NFR-24; Chapter Five §5.6.6	**P2**
R-07	Treat the current deployment as a demonstration instance and budget for a continuously available environment before any operational pilot	The database is stopped between demonstrations and there is no backup schedule, so availability is scheduled rather than assured	`docs/azure-deployment-log-2026-10-31.md` §1–§4; Chapter Five §5.6.6	**P2**

6.3.2 Recommendations for system administrators

Table 6.2: Recommendations for administrators

ID	Action	Reason	Evidence	Priority
R-08	Work the review queue to closure before approving awards; never approve while a case is open	The gate is the system's central control	FR-15; `tests/workflow.test.js`	**P1**

ID	Action	Reason	Evidence	Priority
R-09	Record a substantive reason on every decision, and use `needs_verification` rather than `not_duplicate` when evidence is incomplete	`confirmed_duplicate` is terminal and cannot be reversed through the API, so a mistaken confirmation can only be corrected in the database	Chapter Four §4.4.5; Chapter Five §5.3.5	**P1**
R-10	Reveal identifiers only when comparing source documents, and expect the reveal to be audited	The reveal writes an audit row naming the fields disclosed and the purpose; routine use defeats the privacy design	NFR-05; `tests/duplicateFlags.test.js`	**P2**
R-11	Do not change the thresholds expecting the change to persist	Threshold updates are held in process memory and are lost on restart	Chapter Five §5.3.11	**P2**
R-12	Close every record flagged as `discrepancy`, partial payment or awaiting acknowledgement, and review the exception list routinely	The exceptions are derived from data and are the system's principal accountability output	FR-19; Chapter Five §5.3.7	**P2**

6.3.3 Recommendations for field officers

Table 6.3: Recommendations for field officers

ID	Action	Reason	Evidence	Priority
R-13	Record UPI and birth-certificate number whenever the guardian presents the documents	These are the identifiers whose exact agreement raises a case regardless of the model score; leaving them blank removes the strongest human-checkable signal	`docs/azure-deployment-log-2026-10-31.md` §9; FR-05	**P1**
R-14	Treat a "possible duplicate match" message as a normal outcome, not a rejection, and tell the guardian an administrator will review it	Registration is never blocked and the student is registered; the case concerns a later payment decision	FR-08; `docs/business-workflow-verification-2026-10-30.md` §4	**P2**
R-15	Record beneficiary verification with a real evidence reference rather than a placeholder	Approval is impossible without it, and a placeholder defeats the traceability the system exists to provide	FR-14; `bursary_awards_verification_fields_check`	**P2**
R-16	Print or display the student QR at registration and confirm the student can present it later	The token is returned only in that response, and there is no re-issue path in the current API	Chapter Five §5.3.8; FR-35	**P2**

6.3.4 Recommendations for technical maintainers

Table 6.4: Recommendations for technical maintainers

ID	Action	Reason	Evidence	Priority
R-17	Implement candidate generation (blocking) before the registry exceeds a few hundred students	Screening compares against the whole registry; review volume rises with registry size and no threshold setting repairs it	Chapter Five §5.5.5, §5.6.4; FR-34	**P1**
R-18	Remove or clearly quarantine the legacy rule engine, and reconcile or retire the `beneficiaries` table	Both are reachable-looking dead weight in a codebase whose central concept is duplicate scoring, and one is required by the readiness check	Chapter Four §4.4.2; Chapter Five §5.6.4	**P2**
R-19	Persist threshold configuration in the database and load it at start-up	In-memory settings are lost on restart, which makes an audited change appear to have been reverted	Chapter Five §5.3.11	**P2**
R-20	Replace the in-process rate-limit counters with a shared store, and configure `TRUST_PROXY_HOPS` per deployment	Counters are per process and, behind a proxy without the setting, all public users share one budget	NFR-07; defect D-09	**P2**
R-21	Add automated tests for the eligibility arithmetic, the dashboard counters and the report content	These are the implemented areas with the weakest automated coverage	Chapter Five §5.4.8 (O-07, O-08)	**P2**
R-22	Add a scheduled backup, a restore verification step and a retention/deletion procedure	Recovery is a rehearsed manual procedure and no data is ever deleted	NFR-18; Chapter Five §5.6.4	**P2**

ID	Action	Reason	Evidence	Priority
R-23	Add a QR re-issue path and a token-rotation policy for student records	A student whose token is missing or compromised has no remedy through the API, and possession of a token is permanent	FR-35, FR-36	**P3**
R-24	Run the outstanding tests: phone-camera scan, load and concurrency, print output, accessibility, cross-browser	These are the areas where behaviour is asserted by reasoning rather than by observation	Chapter Five §5.4.8	**P2**
R-25	Correct the superseded project documents (the claim in `docs/verification.md` that the predictor is uncalled, and the UAT plan's `PASS` summary) before submission	A report that cites documents contradicting its own code is weaker than one that cites fewer documents	Chapter Five §5.6.7	**P1**

6.3.5 A note on the limits of these recommendations

None of the recommendations above proposes a change to the system's scoring model, and that is deliberate. The evidence in Chapter Five does not support a model change: the measured performance is adequate for the screening role, the threshold analysis shows no defensible alternative in the band 0.30–0.60 on the available data, and the dominant problem is the volume and attribution of review work rather than the ranking quality of the classifier. Recommending a different algorithm without adjudicated labels would replace one unvalidated choice with another.

6.4 Future works

Each item states the limitation it addresses, why the improvement matters, what would have to change, what additional data or infrastructure is required, and how the improvement could be evaluated.

6.4.1 Adjudicated ground truth and real evaluation

Limitation addressed: all model evidence is synthetic; no adjudicated pairs exist (Chapter Five §5.5.5).

Why it matters: without reviewed labels, precision, recall, miss rate and the false-positive rate of the applicant population cannot be estimated, so the operating point cannot be defended and no claim about real detection performance can be made.

What would change: a sample of 200–300 pairs stratified by score band, labelled independently by two reviewers blind to the score with disagreements adjudicated; pseudonymous entity identifiers; a tuning/holdout split by entity so no person appears in both; and the reviewer, date and data-protection basis recorded.

Additional requirements: named reviewers with authority to judge identity, a data-protection basis for processing real beneficiary records, and a labelling instrument that preserves the score band.

Evaluation: re-run `evaluate_reviewed_pairs.py` against the adjudicated split and report precision, recall, F1, PR-AUC and the confusion matrix with confidence intervals, comparing them with the synthetic figures.

6.4.2 Candidate generation (blocking)

Limitation addressed: every registration is compared against every existing student, and the rescan is quadratic (Chapter Five §5.6.4, limit 1).

Why it matters: it is the only lever that changes the order of magnitude of review work; raising the cut-off reduces the queue more slowly than it loses true duplicates.

What would change: replace "compare against every student" with name-token candidate generation; keep `school_match` as a model feature; treat a missing school as a wildcard; cap the candidate list; and measure the duplicates lost by the key. The surname-and-school key must not be used, because the project's own measurement shows it discards 83.5% of duplicates.

Additional requirements: a name-tokenisation routine shared with the feature builder, an index to support the token lookup, approval for a change in which pairs are compared, and a migration that leaves existing cases untouched.

Evaluation: replay a fixed synthetic registry through both strategies and compare duplicates retained, flags per registration and screening latency at `N = 100`, `500`, `1 000` and `5 000`; then repeat on the adjudicated set from §6.4.1 to confirm that retained duplicates are still detected.

6.4.3 A calibrated, explainable score

Limitation addressed: the score is uncalibrated and has no explanation surface (Chapter Five §5.3.4).

Why it matters: a reviewer told "0.63" cannot check it; a calibrated score with per-feature contributions would show that the agreement is on name and school but on no identifier, which is the judgement the workflow asks the reviewer to make.

What would change: calibration (for example isotonic or Platt scaling) fitted on a held-out set and persisted with the artefact; a per-case explanation derived from the six stored features; and a reviewer-facing display of feature agreement alongside the score, shown to administrators only, since the field-officer screen deliberately shows no scores.

Additional requirements: a labelled subset reserved for calibration, and a display decision about how much model detail a reviewer should see.

Evaluation: calibration curves and Brier score before and after; and a reviewer study measuring whether explanations change agreement between the model's suggestion and the human decision.

6.4.4 Flag budget and review capacity

Limitation addressed: masking of true duplicates behind nuisance candidates reaches 52.6% at `N = 5 000` under `top-1`, and decisions D1–D3 remain unanswered (Chapter Five §5.5.5).

Why it matters: the flag budget is a policy choice with a cost on both sides — queue volume against the risk of showing the reviewer the wrong pair.

What would change: implement a bounded `top-k` policy (for example `top-3`) with the same audit counters and make the budget configurable, keeping `top-1` as the default until the registry grows.

Additional requirements: owner decisions D1–D3 and a measured weekly review capacity.

Evaluation: re-run the registration-level simulation with `k = 1, 2, 3` and compare masking, queue volume and the share of detections attributed to the correct candidate, then confirm on adjudicated data.

6.4.5 Multi-registry operation and shared identity

Limitation addressed: the system is single-tenant, so cross-constituency duplication — the core of the problem in Chapter One — is not addressed (Chapter Five §5.6.2).

Why it matters: a learner can still be registered in two deployments without either detecting it.

What would change: a tenant identifier on students and dependent records, an authenticated inter-instance query for a hashed identity probe, and a policy for what one constituency may learn about another's records. The first step, which is technically modest, is to compute and store a keyed hash of the normalised identity tuple, so that a future shared service can match without exchanging records.

Additional requirements: a data-sharing agreement, a governance body to authorise cross-tenant queries, and infrastructure beyond a single student subscription.

Evaluation: a two-instance simulation in which the same learner is registered in both, measured by detection rate, false-match rate and the number of records exchanged — which should be zero under a hashed-probe design.

6.4.6 Identity assurance and token lifecycle

Limitation addressed: a QR token proves only that a record exists, and there is no expiry, rotation, re-issue or revocation for student tokens (Chapter Five §5.3.8; FR-35, FR-36).

Why it matters: a permanently valid token that can never be replaced is a weak identity artefact, and a lost or misdirected printout cannot be invalidated.

What would change: an administrative re-issue endpoint that replaces `qr_token` and audits the change; optional expiry with re-issue on renewal; and a statement on the printed material of what the token does and does not prove.

Additional requirements: an administrative UI action, a migration for token metadata, and a policy decision about expiry, because expiring tokens would invalidate already-printed codes.

Evaluation: tests asserting that a re-issued token invalidates its predecessor, that the audit row records the re-issue, and that verifying the superseded token returns `found:false`.

6.4.7 Independent confirmation of receipt

Limitation addressed: acknowledgement is officer-assisted, not independent school confirmation (Chapter Five §5.6.6).

Why it matters: the audit finding that motivated the project concerns payments schools did not acknowledge, so a system that records only the officer's own acknowledgement cannot close that gap.

What would change: a limited school-facing confirmation route — for example a per-payment single-use link issued to a named school contact — recording an independent confirmation or dispute without granting access to the register.

Additional requirements: a verified school contact register, a delivery channel (SMS or e-mail), and a decision about what a school may see.

Evaluation: a two-party exercise in which an officer records a payment and a school confirms it independently, with the exception list measured before and after to show that unconfirmed payments become detectable.

6.4.8 Operational hardening and scale

Limitation addressed: no backup schedule, in-process rate limiting, a single predictor process, no load test, and an availability model based on a stopped database (Chapter Five §5.6.4, §5.6.6).

Why it matters: these are the conditions under which a working prototype becomes an unreliable service.

What would change: a scheduled managed backup with a tested restore; a shared counter store for rate limits; a predictor pool or an HTTP inference service with health checks; and a deployment profile with the database always available. Each is an infrastructure change rather than a redesign.

Additional resources: a hosting budget beyond the student credit, a monitoring destination (the current environment intentionally has no log workspace), and an error-reporting channel.

Evaluation: a load test at 500 and 1 000 candidates with concurrent registrations measuring p95 latency and error rate; a restore drill against the managed server; and a rate-limit test across two replicas proving that the budget is shared.

6.4.9 Accessibility, usability and acceptance

Limitation addressed: no accessibility audit, no cross-browser testing and no user acceptance study (Chapter Five §5.4.8, §5.5.6).

Why it matters: the system's users are officers working at a counter with documents in hand; a screen that is correct but slow or confusing to them fails the requirement it was built for, and NFR-14 is currently unverified.

What would change: an accessibility audit against a stated standard with remediation; cross-browser and small-screen testing; and a structured acceptance session with serving officers using synthetic data, followed by remediation of the findings.

Additional requirements: ethics approval and consent for a user study, access to serving officers, and a facilitator.

Evaluation: task success rate, time on task and error rate per task across participants, plus the number and severity of accessibility findings before and after remediation.

6.4.10 Documentation and evidence hygiene

Limitation addressed: superseded project records contradict the implementation (`docs/verification.md`, `docs/uat-test-plan.md`), and `docs/FINAL_TEST_REPORT.md` contains two generations of results (Chapter Five §5.6.7).

Why it matters: the report's credibility rests on its evidence being traceable and self-consistent; an examiner who finds a document claiming the predictor is uncalled will discount the rest.

What would change: retire or clearly mark superseded documents, keep one current test report whose header names the commit and command, and record the date and revision with every count.

Evaluation: a reference check that no current document contradicts the code, and that every test count in the chapters is traceable to a dated record.

6.5 Chapter summary

This chapter concluded that the project met its general objective in the form of a working, deployed artefact, and met three of its four specific objectives fully or with a documented deviation, while the fourth — evaluating effectiveness in detecting duplicate records — is only partially met because no adjudicated real-world labels exist. The delivered system registers beneficiaries, screens them for near-duplicates through a versioned XGBoost model, routes candidates to an administrator who alone can adjudicate, blocks approval and payment while a case is open or confirmed, records payments as issued and separately as cleared with evidence, derives an exception list, verifies students and certificates by QR without disclosing beneficiary data, and audits every state-changing action.

It also concluded, explicitly, what the system does not do: it does not operate across registries, does not claim real-world detection performance, does not move money, does not obtain independent school confirmation, and is not a production service. Its recommendations are directed at the programme owner (four decisions before real data is used), administrators (work the queue, record reasons), field officers (capture the identifiers that make a case checkable), and technical maintainers (implement blocking, persist configuration, harden operations). Its future work is confined to the limitations that Chapter Five established, with candidate generation and adjudicated ground truth first, because every other improvement is less valuable without them.

SBMS REQUIREMENTS AND SCOPE CATALOGUE

Prepared: 2026-11-02 **System:** Student Beneficiary Management System (SBMS) **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Evidence basis:** source, migrations, tests and recorded run documents in the repository. Nothing in this catalogue is a plan; every row describes what exists at the revision above. **Companion files:** `SBMS_API_ENDPOINT_CATALOGUE.md`, `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md`, `SBMS_TEST_EVIDENCE_CATALOGUE.md`, `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md`.

---

1. Purpose

This file is the single requirements register for the written report. It exists so that Chapters Four, Five and Six can cite one authoritative numbered list instead of restating requirements in prose that drifts between chapters. It states, for each requirement: the identifier, the requirement as a testable statement, the actor, the priority, the implementation location, the evidence that establishes the behaviour, and the verification status.

The identifiers are those already used in `CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md` §4.2.2 (FR-01 … FR-37) and §4.2.3 (NFR-01 … NFR-21). They are reproduced here, not renumbered, so cross-references in the chapters remain valid.

2. Status vocabulary

Code	Meaning	Test applied
V	Verified	An executed automated test or a recorded observation establishes the behaviour at this revision
P	Partially verified	Some behaviour is established; a named part is not (the row says which)
N	Implemented, not tested	Code path exists and was read; no executed test exercises it
X	Not implemented	Absent by scope decision or recorded as a gap

A requirement is called *implemented* only where a route, a service, a database object and a user-facing affordance all exist. Where the interface exists but no executed test covers the behaviour, the row is marked **N**, never **V**.

3. System identity and component inventory

Layer	Technology at this revision	Entry point
Presentation	React 18 single-page application, Vite build, `react-router-dom`	`frontend/src/main.jsx`, `frontend/src/App.jsx`
Application	Node.js 22 + Express 4 REST API, 13 routers	`backend/src/index.js`
Domain services	10 service modules (screening, aggregation, review policy, eligibility, bursary rules, certificates, audit, readiness)	`backend/src/services/`
Persistence	PostgreSQL 17, 13 tables, 14 forward migrations + 1 rollback script	`backend/src/db/schema.sql`, `backend/src/db/migrations/`
Machine learning	Python 3.11, scikit-learn + XGBoost 3.2.0, one six-feature model, one long-running scoring subprocess	`ml-service/predict.py`, `ml-service/feature_engineering.py`
Delivery	Single container image (API + ML runtime), GitHub Actions build, Azure Container Apps	`Dockerfile`, `.github/workflows/publish-api-image.yml`

4. Actors and interests

Actor	Description	Where they act
Field Officer	Registers beneficiaries at a school, screens them, records verification evidence, submits award applications, records payments	`/students`, `/students/register`, `/eligibility`, `/disbursements`, `/field-officer`

Actor	Description	Where they act
Administrator	Adjudicates duplicate cases, approves or rejects awards, issues and revokes certificates, manages users and cut-offs, reads the audit log and reports	`/dashboard`, `/duplicate-review`, `/disbursements`, `/reports`, `/settings`, `/audit`, `/users`
Public verifier	Student, guardian, school or employer holding a printed QR code	`/verify`, `/verify-certificate` (unauthenticated)
Operator / deployer	Applies migrations, sets secrets, starts and stops the deployment, restores a backup	`backend/scripts/migrate.js`, `docs/deployment.md`, `deploy/azure/**`
Programme owner	Owns the open policy decisions (cut-off choice, adjudication protocol, acknowledgement deadline)	Not represented in the interface; see `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §6

5. Scope boundary

In scope and delivered: authentication and role separation; beneficiary registration with a five-school constraint; six-feature ML-assisted duplicate screening with human adjudication; rule-based eligibility scoring; bursary award submission, verification, approval and rejection; payment issue, clearance and officer-assisted receipt acknowledgement with exception reporting; QR student identification (existence only); QR-verifiable award certificates with revocation; dashboard, PDF export, audit log, cut-off settings and user administration; migrations, readiness guards and a containerised deployment.

Delivered beyond the approved proposal: award verification evidence, payment reconciliation (issue/clearance/acknowledgement with discrepancy handling), and award certificates. Recorded as expansion, not as a proposal obligation.

Excluded and stated as excluded: NEMIS or any national identifier verification service (FR-32); offline operation and native mobile applications (FR-33); financial transfer execution (FR-37 — SBMS records payments, it does not move money); multi-tenancy; token expiry and re-issue for student QR codes (FR-35, FR-36).

6. Functional requirements register (FR-01 … FR-30)

Priority: **M** must have, **S** should have, **C** could have. Verification names the file in `backend/tests/` unless a full path is given.

ID	Requirement (abbreviated statement)	Actor	Pri	Implementation	Verification	Status
FR-01	Authenticate against a stored bcrypt hash and issue a JWT valid for 8 hours	Admin, Officer	M	`auth.controller.js`	`auth.test.js`, `roleAuthorization.test.js`	V
FR-02	Separate administrator and field-officer sign-in; refuse the wrong endpoint with 403	Both	M	`auth.controller.js:44-60`	`auth.test.js`, `roleAuthorization.test.js`	V
FR-03	Refuse public self-registration (410); accounts created only by an administrator	Admin	M	`auth.routes.js:9`	`auth.test.js`	V
FR-04	Enforce the role matrix on every endpoint; 403 otherwise	All	M	`middleware/requireRole.js`; every router	`roleAuthorization.test.js` (6), `workflow.test.js`	V
FR-05	Register a beneficiary with validated name, DOB, school, national ID, phone and optional UPI / birth certificate	Officer	M	`students.controller.js:15-24`	`students.test.js` (9)	V
FR-06	Reject a school outside the five approved schools	Officer	M	`config/schools.js`; `students_school_supported_check`	`students.test.js`	V

ID	Requirement (abbreviated statement)	Actor	Pri	Implementation	Verification	Status
FR-07	Score a new registration against every existing record and raise review cases per policy	System	M	`students.controller.js:69-124`; `services/duplicateCheck.js`	`duplicateFlags.test.js`, `workflow.test.js`, `mlPredictor.integration.test.js`	V
FR-08	Never reject, delete or amend an existing record because of screening; the flagged student is still created	System	M	`students.controller.js:101`	`workflow.test.js` #1; `docs/business-workflow-verification-2026-10-30.md` §4	V
FR-09	Store score, model version, review threshold and feature vector per case	System, Admin	M	`students.controller.js:103-109`; migration `20261022_ml_duplicate_provenance.sql`	`duplicateFlags.test.js`	V
FR-10	Mask identifiers in the queue; reveal only on an explicit audited request	Admin	M	`duplicateFlags.controller.js:37-42,106-107`	`duplicateFlags.test.js`	V
FR-11	Record a reviewer decision with a mandatory reason, appended to case history	Admin	M	`duplicateFlags.controller.js:452-503`; `duplicate_review_events`	`duplicateFlags.test.js`, `workflow.test.js`	V
FR-12	Compute eligibility from five weighted factors and assign one of four categories	Officer	S	`services/eligibility.js`; `eligibility.controller.js`	Route and validation only; no automated test of the arithmetic	P
FR-13	Submit a bursary award application for an existing student with a requested amount	Officer	S	`bursaryAwards.controller.js:64-101`	`bursaryAwards.controller.test.js` (5)	V
FR-14	Record a verification outcome with an evidence reference; `verified` requires one	Officer	S	`bursaryAwards.controller.js:104-148`	`bursaryAwards.controller.test.js`; `workflow.test.js` #2	V
FR-15	Approve or reject with a reason; approval refused unless verified and no open or confirmed duplicate exists	Admin	M	`bursaryAwards.controller.js:151-209`	`bursaryAwards.controller.test.js`; `workflow.test.js` #2-#6	V
FR-16	Record payment as `issued`; refuse unapproved, unverified, duplicate-blocked or over-amount payments	Admin, Officer	M	`disbursements.controller.js:65-136`	`disbursements.test.js` (4); `workflow.test.js` #7	V
FR-17	Record clearance separately from issue, with a mandatory evidence reference	Admin, Officer	S	`disbursements.controller.js:138-191`	`disbursementReconciliation.controller.test.js` (4); `workflow.test.js` #8	V
FR-18	Record officer-assisted acknowledgement; classify a shortfall as `discrepancy` without overwriting the recorded amount	Admin, Officer	S	`disbursements.controller.js:193-260`; `bursaryRules.receiptStatus`	`workflow.test.js` #9; `bursaryRules.test.js` (4)	V
FR-19	Report reconciliation exceptions (unlinked legacy, unverified, partial, awaiting, overdue, failed, cancelled, discrepancy)	Admin, Officer	S	`bursaryRules.exceptionReasons`; `disbursements.controller.js:53-63`	`workflow.test.js` covers exercised paths; not every reason code is individually asserted	P
FR-20	Generate a cryptographically random QR token per student and return a QR image	System, Officer	M	`students.controller.js:58-60`; `config/publicAppUrl.js`	`verification.test.js` (10)	V
FR-21	Public endpoint reports only whether a matching record exists	Public	M	`verification.controller.js`; `index.js:65`	`verification.test.js`; `verificationRoute.test.js` (4)	V
FR-22	Issue a QR-verifiable certificate for an approved award; refuse a second certificate	Admin	C	`certificates.controller.js:95-181`; `bursary_certificates` UNIQUE `award_id`	`bursaryCertificates.test.js` (12)	V

ID	Requirement (abbreviated statement)	Actor	Pri	Implementation	Verification	Status
FR-23	Revoke a certificate with a reason; public verification then reports `revoked`	Admin	C	`certificates.controller.js:213-249`	`bursaryCertificates.test.js`	V
FR-24	Report `not_current` when the award is no longer approved or its amount changed since issue	System	C	`certificates.controller.js:251-290`	`bursaryCertificates.test.js`	V
FR-25	Dashboard: registry, duplicate, award and payment totals, six-month trend, recent audit activity	Admin	S	`adminDashboard.controller.js`; `pages/AdminDashboard.jsx`	Authorisation and synthetic counts in `roleAuthorization.test.js`; counter arithmetic not individually tested	P
FR-26	Date-range PDF export of registration, disbursement, duplicate and accountability summaries	Admin	S	`reports.controller.js`; `routes/reports.routes.js:8-13`	Range validation tested; PDF content verified by inspection only	P
FR-27	Audit every state-changing operation with actor, action, table, record and details	System	M	`services/auditLog.js`; every controller	`workflow.test.js` #10 (ten-action sequence)	V
FR-28	Read and update the duplicate, high-priority and model cut-offs within 0-1, and audit the change	Admin	S	`settings.controller.js`; `config/settings.js`	Denial tested in `roleAuthorization.test.js`; validation by inspection	P
FR-29	List users and create administrator or field-officer accounts	Admin	S	`users.controller.js`	`roleAuthorization.test.js`; last-admin guard by inspection	P
FR-30	Report liveness and readiness separately; readiness fails closed on a missing table or wrong database identity	Operator	S	`index.js:69-87`; `services/databaseReadiness.js`	`test/databaseReadiness.node.js`; `schemaReadiness.test.js` (5); `health.test.js`	V

6.1 Proposal requirements recorded for completeness

ID	Proposal requirement	Delivered state	Classification
FR-31	Random Forest over four similarity features	XGBoost over six features in production; Random Forest retained only as an unevaluated research artefact	Implemented with a documented design change (G-21)
FR-32	NEMIS / national identifier verification integration	Not implemented; excluded in the proposal	Not implemented, as scoped
FR-33	Offline operation and native mobile applications	Not implemented; responsive web only	Not implemented, as scoped

6.2 Functional gaps recorded as gaps

ID	Gap	Consequence	Evidence
FR-34	No candidate blocking: each registration is compared with the whole registry; the administrator rescan is quadratic	Model calls grow linearly with registry size; review cost grows with N	`students.controller.js:69-73`; `docs/ml-decision-sheet-2026-10-30.md` §6
FR-35	No re-issue path for a student QR token	A row created without a token cannot be given one through the API	`docs/qr-verification-2026-10-30.md` §6
FR-36	No token expiry or rotation	A token remains valid for the life of the student row	`docs/qr-verification-2026-10-30.md` §6
FR-37	Payments are recorded, not executed	No bank, mobile-money or treasury integration	`disbursements.controller.js`

7. Non-functional requirements register (NFR-01 … NFR-21)

ID	Requirement	Measure or constraint implemented	Evidence	Status
NFR-01	Screening completes within three seconds	<= 3 000 ms for a 50-candidate registration; 22.67 ms measured for 50 pairs (2026-10-30)	`tests/performance.test.js`	P: satisfied at 50 candidates; no measurement at registry scale
NFR-02	Password storage resists offline attack	bcrypt, cost factor 12	`auth.controller.js:26`	V as implementation; cost not benchmarked on the host
NFR-03	Session credentials are signed and time-bounded	HMAC JWT, `expiresIn: 8h`, secret >= 32 bytes, placeholder rejected at start-up	`auth.controller.js:55-59`; `config/jwtSecret.js`	V
NFR-04	Authorisation does not trust a client-supplied claim	Role re-read from the `users` row on every request; inactive users refused	`middleware/auth.js:20-24`	V
NFR-05	Identifiers are not exposed by default in the queue	Last-three-character masking plus an audited reveal endpoint	`duplicateFlags.controller.js:37-42,106-107`	V
NFR-06	Public verification discloses no beneficiary data	Student endpoint returns `{found}` only; certificate endpoint returns authenticity facts only	`verification.controller.js`; `certificates.controller.js:53-89`	V
NFR-07	Public endpoints are rate limited	10 requests / IP / 15 min (student); 20 / IP / 15 min (certificate); separate counters	`middleware/verificationRateLimit.js`; `certificateRateLimit.js`; `rateLimitWindow.js`	V in-process only; not shared across instances
NFR-08	Sign-in is rate limited	10 attempts / IP / 15 min	`middleware/loginRateLimit.js`	V
NFR-09	Request bodies are bounded	JSON limit 100 kB	`index.js:64`	V
NFR-10	Errors do not leak internals	5xx returns a generic message; logs carry no stack, message or credential	`middleware/errorHandler.js:59-78`	V
NFR-11	Browser origins restricted by allowlist	Unlisted origin rejected before routing; malformed configuration stops start-up	`config/cors.js`; `index.js:40-47`	V
NFR-12	Integrity enforced by the database, not only the application	CHECK constraints for roles, school list, award decision, verification, payment and receipt status, clearance and acknowledgement completeness, certificate status and token format; foreign keys throughout	`db/schema.sql`; migrations	V
NFR-13	Concurrent registrations do not interleave	Transaction plus `pg_advisory_xact_lock`; candidates selected `FOR UPDATE`	`students.controller.js:67-73`	P: lock exists and is used; no concurrency test exists
NFR-14	Usable by non-technical officers on varied devices	Responsive layout, role navigation, plain-language screening messages, single-line status pages	`components/AppLayout.jsx`; `pages/StudentRegistration.jsx`; `pages/Verification.jsx`	P: browser-checked; no usability study or acceptance session
NFR-15	Maintainable and reproducible	One feature-schema module shared by training and serving; ordered migrations; Jest, native `node --test` and Python `unittest`; pinned requirements	`ml-service/feature_engineering.py`; `db/migrations/`; `ml-service/requirements.txt`	V

ID	Requirement	Measure or constraint implemented	Evidence	Status
NFR-16	Operable within a zero-cost student budget	Container with `minReplicas 0`; free App Service tier; PostgreSQL stopped between demonstrations; no log workspace	`Dockerfile`; `docs/azure-deployment-log-2026-10-31.md` §1-§4	P: the zero-cost configuration is recorded, but no billed month has been reconciled against the budget and the cost model is an owner decision
NFR-17	A human makes every duplicate and payment decision	Scores create cases only; approval, disbursement and issue are role-gated human actions	`duplicateFlags.controller.js:452-503`; `bursaryAwards.controller.js:151-209`	V
NFR-18	Recovery from database loss with verified evidence	Dump, restore into an independent database, readiness and row-digest comparison rehearsed	`docs/database-recovery-rehearsal-2026-10-31.md`	P: local rehearsal; no scheduled backup job or retention policy in the deployment
NFR-19	Compatible with ordinary modern browsers	SPA over HTTPS using `fetch`, `history.replaceState` and router hash handling only	`frontend/src/**`; `frontend/public/web.config`	P: exercised in Chrome only; Firefox, Safari and mobile browsers unrecorded
NFR-20	Predictable degradation when the predictor is unavailable	Registration aborts with 503 instead of registering an unscreened beneficiary	`students.controller.js:74-96`; `services/duplicateCheck.js`	V by inspection and the 503 assertion path
NFR-21	Audit records attributable and complete for state changes	Every controller writes an `audit_logs` row with actor, action, table, record and JSON details	`services/auditLog.js`; `workflow.test.js` #10	P: attributable and complete in every tested workflow; append-only is convention only and is not enforced by the schema

8. Counts and status summary

Group	Count	V	P	N	X
Functional requirements delivered (FR-01 … FR-30)	30	24	6	0	0
Proposal requirements carried as deviations (FR-31 … FR-33)	3	0	0	0	3
Functional gaps reported as gaps (FR-34 … FR-37)	4	0	0	0	4
Non-functional requirements (NFR-01 … NFR-21)	21	14	7	0	0

The six partially verified functional requirements are FR-12 (eligibility arithmetic), FR-19 (not every reason code individually asserted), FR-25 (counter arithmetic), FR-26 (PDF content), FR-28 (cut-off validation) and FR-29 (last-administrator guard); every other delivered functional requirement carries a route-level or workflow-level test. The seven partially verified non-functional requirements are NFR-01, NFR-13, NFR-14, NFR-16, NFR-18, NFR-19 and NFR-21. These counts are obtained by counting the status cells in §6 and §7, not maintained as a separate figure, so a change to a row changes the count.

Two **V** verdicts rest partly on inspection and say so in the row: NFR-02 (bcrypt cost factor is set but not benchmarked on the host) and NFR-20 (the 503 path is asserted and the surrounding branch was read). Every other **V** rests on an executed test or a recorded observation. The **N** column is empty because every implemented requirement has at least a route-level test, which is not a claim that every branch is covered; branch-level coverage is stated in `SBMS_TEST_EVIDENCE_CATALOGUE.md` §7.

9. Traceability entry points

Question	File that answers it
Which endpoint implements a requirement?	`SBMS_API_ENDPOINT_CATALOGUE.md` §3
Which table and column stores the result?	`SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` §4-§8
Which test proves it, and on what date?	`SBMS_TEST_EVIDENCE_CATALOGUE.md` §3-§6

Question	File that answers it
Which claim is unsupported, and why?	`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §4-§5
Which objective does it serve?	`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §1
Which gap is still open?	`DOCUMENTATION_GAP_REGISTER.md` §1-§3

Statement of provenance. Every row above was read from the revision named in the header, not from an earlier draft. Where the repository contradicts a chapter, the contradiction is recorded in `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §3 rather than smoothed away here.

SBMS SYSTEM ARCHITECTURE AND MODULE MAP

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `backend/src/index.js`, `backend/src/routes/*`, `backend/src/controllers/*`, `backend/src/services/*`, `backend/src/models/*`, `backend/src/config/*`, `backend/src/middleware/*`, `frontend/src/**`, `ml-service/predict.py`, `ml-service/feature_engineering.py`, `Dockerfile`.

---

1. Architecture as built

SBMS is a three-tier web application with a fourth, in-process inference component:

```
Browser (React SPA, Vite build)
   |  HTTPS + JSON, Bearer JWT
   v
Express API (Node 22)  ->  13 routers -> 13 controllers -> 10 services -> 5 models
   |  stdio JSON lines
   +--> Python predictor (predict.py --server, joblib artifact)
   |
   v  TCP, TLS in deployment
PostgreSQL 17 (13 tables, 14 forward migrations)
```

Tier	Responsibility	Deliberately not responsible for
Presentation	Rendering, routing by role, form state, plain-language messaging	Business rules, authorisation decisions, any scoring
Application	Authentication, authorisation, validation, workflow transitions, audit, response shaping	Rendering, model training
Domain services	Screening and aggregation, review policy, eligibility arithmetic, bursary and payment rules, certificate rendering, readiness checks	HTTP concerns
Persistence	Constraints, referential actions, uniqueness, transactional integrity	Rules that need more than one row in context
Inference	Feature construction and scoring with one artifact	Deciding what a score means, creating cases, storing anything

The ML component lives inside the API container and is reached only through the Node process's stdin. There is no separate ML service, no message broker and no job queue anywhere in the system.

2. Request lifecycle and middleware order

Order	Middleware	Purpose	Failure behaviour
1	Security headers	`nosniff`, `DENY`, `no-referrer`, restricted permissions	—
2	Origin allowlist check	Refuse a disallowed browser origin before routing	403 `Origin is not allowed`
3	`cors`	Reflect allowlisted origins only	—
4	Proxy trust (conditional)	Honour `TRUST_PROXY_HOPS` when set	—
5	`POST /api/verify` chain	Rate limit, then JSON parse, then handler	429, 400 or 503
6	`POST /api/certificates/verify` chain	Same, with its own limiter	429, 400 or 503
7	Shared JSON parser	100 kB body limit for all other routes	`Request body is too large`
8	`/api/health`, `/api/ready`	Liveness and readiness	503 when not ready
9	Eleven mounted routers	Feature endpoints with their own guards	401, 403 or 4xx

Order	Middleware	Purpose	Failure behaviour
10	`errorHandler`	Sanitise every thrown error	Generic 5xx; `Invalid request` for 4xx

The order matters in three places: the public verifiers are mounted before the shared parser so an abusive client is rejected before its body is read; the origin check precedes routing so no route can be reached from a disallowed origin; the error handler is last so every controller can simply call `next(error)`.

3. Backend module map

3.1 Routers (13) and their guards

Router	Mount	Guard
`auth.routes.js`	`/api/auth`	per route: limiter for sign-in, `authenticateToken` for `/me`
`students.routes.js`	`/api/students`	router-level `authenticateToken` + `requireRole('field_officer')`
`duplicateFlags.routes.js`	`/api/duplicate-flags`	router-level `authenticateToken` + `requireRole('admin')`
`bursaryAwards.routes.js`	`/api/awards`	router-level `authenticateToken`; per-route roles
`disbursements.routes.js`	`/api/disbursements`	router-level `authenticateToken`; per-route roles
`eligibility.routes.js`	`/api/eligibility`	`authenticateToken` + `requireRole('field_officer')`
`settings.routes.js`	`/api/settings`	`authenticateToken` + `requireRole('admin')`
`adminDashboard.routes.js`	`/api/admin/dashboard`	`authenticateToken` + `requireRole('admin')`
`reports.routes.js`	`/api/reports`	`authenticateToken` + `requireRole('admin')`
`audit.routes.js`	`/api/audit-logs`	`authenticateToken` + `requireRole('admin')`
`users.routes.js`	`/api/users`	router-level `authenticateToken` + `requireRole('admin')`
`beneficiaries.routes.js`	`/api/beneficiaries`	router-level `authenticateToken`; officer for writes
inline routes	`/api/health`, `/api/ready`, `/api/verify`, `/api/certificates/verify`	none, or a dedicated limiter

3.2 Services (10) and what each owns

Service	Owns	Called from
`duplicateCheck.js`	Spawning and supervising the predictor, batching pairs, mapping failure to a 503	`students.controller.js`, `duplicateFlags.controller.js`
`duplicateAggregation.js`	`top-1` / `all` policy, candidate ranking, suppression counts, policy normalisation	`students.controller.js`
`mlDuplicateReview.js`	Exact-identifier detection, priority banding, the shared `configuredThreshold` accessor	Registration and queue
`eligibility.js`	The five-factor arithmetic and category banding	`eligibility.controller.js`
`bursaryRules.js`	Payment transitions, cents comparison, receipt status, overdue arithmetic, exception reasons	`disbursements.controller.js`
`certificateDocument.js`	PDF layout with pdfkit and the QR code	`certificates.controller.js`
`auditLog.js`	Writing an `audit_logs` row inside the caller's transaction	Every mutating controller

Service	Owns	Called from
`studentRecordFormat.js`	Identifier and phone normalisation, ISO date validation	`students.controller.js` and others
`databaseReadiness.js`	Required-table and database-identity contract	`/api/ready`, tests
`databaseStartup.js`	Start-up assertion on the login schema	`index.js`

3.3 Models (5) — thin data access

Model	Tables it touches
`student.model.js`	`students` (create, list, read; strips `qr_token` from responses)
`beneficiary.model.js`	`beneficiaries` (legacy)
`bursaryCertificate.model.js`	`bursary_certificates`
`disbursement.model.js`	`disbursements`
`audit.model.js`, `auth.model.js`	`audit_logs`, `users`

Several controllers also issue SQL directly (awards, duplicate flags, eligibility persistence), so the model layer is a convention rather than a mandatory boundary. That is a real design inconsistency: a reader should not assume every query passes through a model.

3.4 Configuration modules

Module	Responsibility	Fail-fast behaviour
`config/db.js`	Shared `pg` pool	Connection errors surface through the handler
`config/jwtSecret.js`	Validate and hold the signing secret	Throws on length, placeholder or low diversity
`config/cors.js`	Parse and validate the origin allowlist	Throws on a malformed origin
`config/publicAppUrl.js`	Resolve and validate the public frontend origin used in QR links	Throws on a non-HTTPS production origin or a LAN address
`config/schools.js`	The five approved schools	—
`config/settings.js`	The three duplicate cut-offs in process memory	Falls back to a valid default when the environment value is out of range
`config/reconciliation.js`	`ACKNOWLEDGEMENT_OVERDUE_DAYS`	Throws on a non-integer or out-of-range value

4. Frontend module map

Layer	Files	Responsibility
Shell	`main.jsx`, `App.jsx`	Mount, route table, role guards, role-based home redirect
Layout	`components/AppLayout.jsx`, `components/PublicVerificationLayout.jsx`	Authenticated shell with navigation; minimal public shell
Context	`context/AuthContext.jsx`	Session bootstrap from `GET /auth/me`, sign-in, sign-out
Hooks	`hooks/useHashToken.js`	Fragment token capture, URL sanitisation, re-verification on `hashchange`
Transport	`services/api.js`	Axios instance, base URL, bearer interceptor
Feature panel	`components/AwardCertificatePanel.jsx`	Certificate issue, read, print, revoke
Pages	14 files in `pages/`	One screen per route in the UI inventory

5. End-to-end flows across modules

Flow	Sequence
Registration and screening	`StudentRegistration` → `POST /students` → Zod validation → advisory lock → candidate select `FOR UPDATE` → `duplicateCheck.scorePairs` → `duplicateAggregation.summarizeAggregation` → `studentModel.create` → `duplicate_flags` inserts → `auditLog` → response with QR image and screening block
Duplicate adjudication	`DuplicateReview` → `GET /duplicate-flags` (masked, prioritised) → optional audited reveal → `PATCH /duplicate-flags/:id` → transaction: status update, `duplicate_review_events` insert, audit row
Award and payment	`POST /awards` → verification event → `PATCH /awards/:id/decision` (gated on verification and duplicate state) → `POST /disbursements` (gated on approval, verification, duplicate state and amount) → `PATCH /disbursements/:id/payment-status` → `PATCH /disbursements/:id/acknowledgement` → reconciliation events and audit rows
Student verification	QR image → `/verify#token=...` → `useHashToken` → `POST /api/verify` → token format check → existence query → `{found}`
Certificate	`POST /awards/:id/certificate` → snapshot row, reference, token, QR → `/verify-certificate#token=...` → `POST /api/certificates/verify` → snapshot compared with the live award → status

6. Architectural decisions worth recording in Chapter Four

Decision	Rationale	Cost
ML inside the API container	One deployable unit, no extra port, no second authentication boundary	A predictor crash affects the API process; scaling scales the model with it
Feature construction in Python only	Training and serving cannot drift	The Node side cannot score without the subprocess
Advisory lock around registration	Serialises candidate comparison so two registrations cannot interleave	Registrations queue behind one another
Snapshot table for certificates	A printed document cannot be rewritten by a later award edit	Duplicated facts to keep consistent in the verification comparison
Human decision gates at the database	`CHECK` constraints make an unverified approval impossible, not merely discouraged	Changing policy requires a schema change
`top-1` case aggregation	Stops the queue growing with registry size	Suppressed candidates are only counted, never reviewed
Thresholds in process memory	Runtime tuning without a redeploy	Values are lost on restart
Migrations never automatic	A container restart cannot change the schema	An operator must remember to apply them

7. Architectural weaknesses to report honestly

Weakness	Consequence	Recorded as
No candidate blocking	Each registration is O(N) model calls; the rescan is quadratic	FR-34
Single process, single container	In-process rate limits, thresholds and the ML subprocess all vanish on restart and are not shared across replicas	NFR-07, §6
No queue or worker	A slow registration blocks the request; there is no background processing	Design note
SQL in both controllers and models	Two ways to reach the database; no single data-access boundary	§3.3
Legacy `beneficiaries` module still mounted	Two registries with no relationship; a source of confusion for a reader	§8
Dead `auth.controller.register`	Suggests a capability that does not exist	`SBMS_API_ENDPOINT_CATALOGUE.md` §7.1
No observability	Logging destination none in the deployment and no metrics	Deployment guide §13
No API versioning	A breaking change cannot be introduced without breaking the deployed frontend	Design note

8. Dead and legacy code inventory

Item	State	What the report should say
`beneficiaries` router, model and table	Mounted and reachable; not used by any page	A legacy module retained for compatibility
`auth.controller.register` and `registerSchema`	Unreachable behind a 410 route	Dead code, not a feature
`render.yaml`	Superseded by Azure	Historical artefact
`disbursements.status`	Legacy duplicate of `payment_status`	Kept in step, not authoritative
`duplicate_flags.probability`	Legacy column name for an uncalibrated score	Documented by a column comment
Random Forest research artifact	Never loaded by the application	Research candidate only

9. Technology stack with version evidence

Layer	Technology	Version evidence
Runtime	Node.js	v22.11.0 (test report)
API framework	Express	4.x per `backend/package.json`
Validation	Zod	Per-controller schemas
Database	PostgreSQL	17.11
Driver	`pg`	Pool in `config/db.js`
Auth	`jsonwebtoken`, `bcrypt`	Cost factor 12
QR and PDF	`qrcode`, `pdfkit`	Registration and certificate rendering
Frontend	React, Vite, `react-router-dom`, `axios`	2 006 modules in the production build
ML	Python 3.11.9, scikit-learn 1.9.0, XGBoost 3.2.0, pandas 3.0.5, jellyfish 1.2.1, joblib 1.6.0	`ml-service/requirements.txt`, experiment report
Tests	Jest, `node --test`, `unittest`	Test evidence catalogue §1
Delivery	Docker, GitHub Actions, Azure Container Apps, Azure App Service	Deployment guide §2-§5

10. What this architecture does not include

No message broker, cache, search index, service mesh, API gateway, feature store, model registry, experiment tracker, or infrastructure-as-code. Deployment was performed with CLI commands recorded in the deployment log, not from a declarative template; a reader who expects `main.bicep` or a `terraform/` directory will not find one. That absence is the honest answer to "how is this deployment reproducible" — the runbook is a document, not code.

SBMS DATABASE SCHEMA AND DATA DICTIONARY

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `backend/src/db/schema.sql` (baseline), all 14 forward migrations in `backend/src/db/migrations/`, `backend/scripts/migrate.js`, and the table list measured during the backup rehearsal (`docs/database-recovery-rehearsal-2026-10-31.md` §2). **Machine:** PostgreSQL 17.11, database `sbms`, extension `pgcrypto` enabled for `gen_random_uuid()`.

---

1. Purpose and method

This file is the reference data dictionary for the report. It states, for every table and column that exists at this revision: the type, nullability, default, the constraint that governs it and what it means in the bursary workflow. It also records the state machines the schema enforces and, separately, the integrity rules the schema does **not** enforce, so a reader cannot mistake an application convention for a database guarantee.

Column definitions were read from `schema.sql` and from `ALTER TABLE ... ADD COLUMN` statements in the migrations. A column added by a migration is attributed to that migration rather than to the baseline file, because a database built from `schema.sql` alone will not contain it.

2. Schema organisation and migration policy

Property	Value
Baseline file	`backend/src/db/schema.sql` — creates `users`, `students`, `duplicate_flags`, `duplicate_review_events`, `disbursements`, `audit_logs`, `reports`, `beneficiaries` and applies idempotent `ALTER`s
Migration files	14 forward migrations, dated `20261019` to `20261031`, plus one rollback script `20261022_ml_duplicate_provenance.down.sql`
Application mechanism	`backend/scripts/migrate.js` applies unapplied files in filename order and records each in `sbms_schema_migrations`
Automatic application	**None.** The API never applies a migration at start-up; an operator applies them after backup and a verified restore
Fail-closed guard	`GET /api/ready` reports not ready when a required table is missing, including `bursary_certificates`
Migration count applied on the test database	12 on 2026-10-31 (before `20261023_retain_primary_accounts.sql` and `20261031_bursary_certificates.sql`); 14 after those were applied for the certificate work

Forward migration inventory

File	What it adds or changes
`20261019_admin_field_officer_roles.sql`	Constrains `users.role` to `admin` / `field_officer`
`20261019_duplicate_review_workflow.sql`	Creates `duplicate_review_events`; adds review columns to `duplicate_flags`
`20261019_enforce_supported_schools.sql`	Adds `students_school_supported_check`
`20261019_field_officer_registration.sql`	Supports officer-created accounts
`20261019_limit_synthetic_schools.sql`	Restricts synthetic seeds to the five schools
`20261019_remove_duplicate_case_assignment.sql`	Drops case assignment to a person; review is unassigned
`20261019_standardize_student_records.sql`	Normalises student identifier columns

File	What it adds or changes
`20261019_synthetic_student_marker.sql`	Adds `students.is_synthetic`
`20261022_ml_duplicate_provenance.sql`	Adds `model_version`, `review_threshold`, `comparison_features` to `duplicate_flags`
`20261023_bursary_award_reconciliation.sql`	Creates `bursary_awards`, `bursary_award_verification_events`, `disbursement_reconciliation_events`; adds 14 reconciliation columns to `disbursements` and six CHECK constraints
`20261023_retain_primary_accounts.sql`	Keeps primary (non-duplicate) accounts
`20261024_add_user_activation_flag.sql`	Adds `users.is_active`
`20261025_legacy_beneficiaries_baseline.sql`	Establishes the legacy `beneficiaries` baseline
`20261031_bursary_certificates.sql`	Creates `bursary_certificates` with snapshot and revocation columns

3. Table inventory

#	Table	Purpose	Primary key	Rows on the test database, 2026-10-31
1	`users`	Staff accounts: administrators and field officers	`id` UUID	4
2	`students`	Beneficiary register	`id` UUID	0
3	`duplicate_flags`	One review case per student pair, with model provenance	`id` UUID; unique unordered pair	0
4	`duplicate_review_events`	Append-only adjudication history per case	`id` UUID	0
5	`bursary_awards`	Award application, verification and decision	`id` UUID	0
6	`bursary_award_verification_events`	Append-only verification history per award	`id` UUID	0
7	`disbursements`	Payments issued against an award, with clearance and receipt fields	`id` UUID	0
8	`disbursement_reconciliation_events`	Append-only payment and receipt events	`id` UUID	0
9	`bursary_certificates`	Issued certificate snapshot with token and revocation	`id` UUID; `award_id` UNIQUE	0
10	`audit_logs`	Action trail across all controllers	`id` UUID	1
11	`reports`	Export metadata (type, range, file path)	`id` UUID	0
12	`beneficiaries`	Legacy module, serial key, no foreign keys	`id` SERIAL	0
13	`sbms_schema_migrations`	Applied-migration ledger	filename	12 then 14

The database therefore holds **13 tables** at this revision. The rehearsal measured 12 tables and 118 columns before `bursary_certificates` was added; the certificate table adds 15 more columns, giving 133 by arithmetic — a derived figure, not a re-measured one.

4. Data dictionary

4.1 `users` — staff accounts

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Account identifier; the subject of the JWT and the actor on every audit row

Column	Type	Null	Default	Constraint	Meaning
`full_name`	VARCHAR(255)	no	—	—	Name shown on staff screens
`email`	VARCHAR(255)	no	—	UNIQUE	Sign-in identity; lower-cased by the API before storage
`password_hash`	TEXT	no	—	—	bcrypt hash, cost 12; the plaintext is never stored or logged
`role`	VARCHAR(50)	no	`field_officer`	`users_role_supported_check` IN (`admin`, `field_officer`)	Authorisation role; re-read on every request, never trusted from the token
`is_active`	BOOLEAN	no	TRUE	—	Deactivation flag (`20261024_add_user_activation_flag.sql`); an inactive user cannot authenticate
`created_at`	TIMESTAMP	yes	`NOW()`	—	Registration time

4.2 `students` — beneficiary register

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Student identifier used by every downstream table
`full_name`	VARCHAR(255)	no	—	—	Name as recorded at the counter; whitespace collapsed to single spaces
`date_of_birth`	DATE	yes	—	—	ISO date; feeds the `dob_match` feature
`school_code`	VARCHAR(100)	yes	—	`students_school_supported_check` IN the five approved schools	School membership; feeds `school_match` and gates award submission
`guardian_id`	VARCHAR(100)	yes	—	—	Guardian identifier; feeds `guardian_match`
`upi`	VARCHAR(100)	yes	—	not unique (uniqueness deliberately removed)	Unique Personal Identifier; feeds `upi_match` and the critical identifier rule
`national_id_number`	VARCHAR(100)	yes	—	—	National identifier held for disclosure control and review evidence; not a model feature
`primary_phone`	VARCHAR(50)	yes	—	API constrains to `+254[17]` plus eight digits	Contact number
`qr_token`	VARCHAR(100)	yes	—	`students_qr_token_unique` partial unique index	64 lowercase hex characters generated at registration; never returned by the API
`is_synthetic`	BOOLEAN	no	FALSE	—	Marks seeded demonstration rows so they can be separated from real registrations
`birth_certificate_number`	VARCHAR(100)	yes	—	—	Birth certificate entry number; feeds `birth_certificate_match`
`eligibility_score`	INTEGER	yes	—	—	Last computed score (0-100)
`eligibility_category`	VARCHAR(50)	yes	—	—	`highly_eligible`, `eligible`, `partially_eligible`, `not_eligible`
`eligibility_assessed_at`	TIMESTAMP	yes	—	—	When the score was computed
`created_at`	TIMESTAMP	yes	`NOW()`	—	Registration time; used as the candidate ordering key during screening

Note: the baseline file renames `birth_certificate_entry_number` to `birth_certificate_number` if the old column exists, and drops the historical `students_upi_key` uniqueness constraint. Both actions are idempotent and are recorded in `schema.sql:47-72`.

4.3 `duplicate_flags` — review cases

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Case identifier
`student_id`	UUID	no	—	FK `students(id)` ON DELETE CASCADE	The newly registered student
`matched_student_id`	UUID	no	—	FK `students(id)` ON DELETE CASCADE	The existing student it was compared with
`probability`	NUMERIC(5,4)	no	—	—	**Legacy name**: holds the uncalibrated positive-class model score, as documented in the column comment
`status`	VARCHAR(50)	no	`pending`	—	`pending`, `confirmed_duplicate`, `not_duplicate`, `needs_verification`
`model_version`	TEXT	yes	—	—	Artefact identity, e.g. `xgboost:<sha256>`; proves which model produced the score
`review_threshold`	NUMERIC(5,4)	yes	—	—	Cutoff in force at scoring time
`comparison_features`	JSONB	yes	—	—	The six feature values used, retained as evidence
`resolved_by`	UUID	yes	—	FK `users(id)`	Administrator who decided the case
`resolved_at`	TIMESTAMP	yes	—	—	Decision time
`updated_at`	TIMESTAMP	no	`NOW()`	—	Last modification
`created_at`	TIMESTAMP	yes	`NOW()`	—	Case creation

A unique index on `LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id)` (`duplicate_flags_unordered_pair_unique`) makes a pair unique regardless of direction, so the same two students cannot accumulate duplicate cases in both orientations.

4.4 `duplicate_review_events` — adjudication history

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Event identifier
`duplicate_flag_id`	UUID	no	—	FK `duplicate_flags(id)` ON DELETE CASCADE	Case the event belongs to
`reviewer_id`	UUID	no	—	FK `users(id)`	Acting administrator
`decision`	VARCHAR(30)	no	—	CHECK IN (`confirmed_duplicate`, `not_duplicate`, `needs_verification`)	Decision recorded
`reason`	TEXT	no	—	CHECK `length(btrim(reason)) > 0` at database level; API enforces >= 5 characters	Justification
`notes`	TEXT	yes	—	—	Free-text detail
`created_at`	TIMESTAMP	no	`NOW()`	—	When the decision was appended

Index `duplicate_review_events_flag_created_idx` on `(duplicate_flag_id, created_at DESC)` supports the per-case history view.

4.5 `bursary_awards` — application, verification and decision

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Award identifier

Column	Type	Null	Default	Constraint	Meaning
`student_id`	UUID	no	—	FK `students(id)` ON DELETE RESTRICT	Beneficiary the award belongs to
`school_code`	VARCHAR(100)	no	—	—	Copied from the student at submission so the award records the school it was granted under
`requested_amount`	NUMERIC(12,2)	no	—	CHECK `> 0`	Amount applied for
`approved_amount`	NUMERIC(12,2)	yes	—	CHECK NULL or `> 0`	Amount approved; also the ceiling for total disbursement
`status`	VARCHAR(20)	no	`submitted`	CHECK IN (`submitted`, `approved`, `rejected`)	Decision state
`verification_status`	VARCHAR(30)	no	`not_checked`	CHECK IN (`not_checked`, `verified`, `not_found`, `needs_investigation`)	Beneficiary verification state
`verification_notes`	TEXT	yes	—	—	Verification narrative
`verification_evidence_ref`	VARCHAR(500)	yes	—	Required when `verification_status = 'verified'`	Pointer to the evidence seen; explicitly **not** an upload
`verified_by`	UUID	yes	—	FK `users(id)` ON DELETE RESTRICT	Officer who verified
`verified_at`	TIMESTAMPTZ	yes	—	—	Verification time
`decision_reason`	TEXT	yes	—	CHECK >= 5 trimmed characters when decided	Why approved or rejected
`decided_by`	UUID	yes	—	FK `users(id)` ON DELETE RESTRICT	Administrator who decided
`decided_at`	TIMESTAMPTZ	yes	—	—	Decision time
`submitted_by`	UUID	yes	—	FK `users(id)` ON DELETE RESTRICT	Officer who submitted
`created_at`	TIMESTAMPTZ	no	`NOW()`	—	Submission time
`updated_at`	TIMESTAMPTZ	no	`NOW()`	—	Last modification

Table-level CHECK constraints (`bursary_awards_*`): a submitted row must have no decision fields; an approved row must have an amount, a reason, a decider and a time; a rejected row must have no amount but must have a reason, a decider and a time; **`status <> 'approved' OR verification_status = 'verified'`** — approval without verification is impossible at database level, not only in the controller; and a verified row must carry verifier, time and a non-empty evidence reference.

4.6 `bursary_award_verification_events` — verification history

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Event identifier
`award_id`	UUID	no	—	FK `bursary_awards(id)` ON DELETE RESTRICT	Award verified
`outcome`	VARCHAR(30)	no	—	CHECK IN (`verified`, `not_found`, `needs_investigation`)	Outcome of the check
`notes`	TEXT	no	—	CHECK >= 5 trimmed characters	What was checked
`evidence_ref`	VARCHAR(500)	yes	—	Required when `outcome = 'verified'`	Evidence pointer
`actor_id`	UUID	no	—	FK `users(id)` ON DELETE RESTRICT	Officer
`created_at`	TIMESTAMPTZ	no	`NOW()`	—	When recorded

4.7 `disbursements` — payments

Baseline columns:

Column	Type	Null	Default	Meaning
`id`	UUID	no	`gen_random_uuid()`	Payment identifier
`student_id`	UUID	no	—	FK `students(id)` ON DELETE CASCADE; payer-facing student
`amount`	NUMERIC(12,2)	no	—	Amount issued
`disbursement_date`	DATE	no	`CURRENT_DATE`	Issue date; basis of the overdue-acknowledgement calculation
`status`	VARCHAR(50)	no	`completed`	Legacy status, kept in step with `payment_status` by the update statement
`created_by`	UUID	yes	—	FK `users(id)`
`created_at`	TIMESTAMP	yes	`NOW()`	Record creation

Reconciliation columns added by `20261023_bursary_award_reconciliation.sql`:

Column	Type	Constraint	Meaning
`award_id`	UUID	FK `bursary_awards(id)` ON DELETE RESTRICT	Award the payment settles; NULL for a legacy payment
`payment_method`	VARCHAR(80)	—	How it was paid (API requires 2-80 characters)
`payment_reference`	VARCHAR(150)	—	Bank or mobile-money reference
`payment_status`	VARCHAR(20)	CHECK `issued` | `cleared` | `failed` | `cancelled`	Settlement state
`cleared_by`	UUID	FK `users(id)`	Who confirmed settlement
`cleared_at`	TIMESTAMPTZ	—	When settlement was confirmed
`clearance_evidence_ref`	VARCHAR(500)	—	Evidence for clearance
`receipt_status`	VARCHAR(30)	CHECK `awaiting_acknowledgement` | `acknowledged` | `discrepancy`	Receipt state
`acknowledged_amount`	NUMERIC(12,2)	CHECK `>= 0`	Amount the officer recorded as received
`acknowledged_by`	UUID	FK `users(id)`	Who recorded the receipt
`acknowledged_at`	TIMESTAMPTZ	—	When it was recorded
`acknowledgement_method`	VARCHAR(30)	CHECK must equal `officer_assisted` when acknowledged	Records that the acknowledgement is officer-assisted, not school-confirmed
`acknowledgement_evidence_ref`	VARCHAR(500)	—	Evidence for the acknowledgement
`acknowledgement_notes`	TEXT	—	Free text

Two table CHECK constraints make the states self-consistent: `disbursements_clearance_fields_check` requires `cleared_by`, `cleared_at` and a non-empty `clearance_evidence_ref` if and only if `payment_status = 'cleared'`; `disbursements_acknowledgement_fields_check` requires amount, actor, time, method `officer_assisted` and evidence if and only if `receipt_status` is `acknowledged` or `discrepancy`. A half-recorded clearance or receipt is therefore impossible.

4.8 `disbursement_reconciliation_events` — payment and receipt trail

Column	Type	Null	Constraint	Meaning
`id`	UUID	no	PK	Event identifier

Column	Type	Null	Constraint	Meaning
`disbursement_id`	UUID	no	FK `disbursements(id)` ON DELETE RESTRICT	Payment the event belongs to
`event_type`	VARCHAR(30)	no	CHECK IN (`payment_issued`, `payment_cleared`, `payment_failed`, `payment_cancelled`, `payment_linked_to_award`, `receipt_acknowledged`)	What happened
`payment_status`	VARCHAR(20)	yes	CHECK as above	Status after the event
`receipt_status`	VARCHAR(30)	yes	CHECK as above	Receipt state after the event
`amount`	NUMERIC(12,2)	yes	CHECK `>= 0`	Amount associated with the event
`evidence_ref`	VARCHAR(500)	yes	—	Evidence pointer
`notes`	TEXT	yes	—	Reason or note
`actor_id`	UUID	no	FK `users(id)` ON DELETE RESTRICT	Who acted
`created_at`	TIMESTAMPTZ	no	`NOW()`	When

4.9 `bursary_certificates` — issued certificate snapshot

Column	Type	Null	Constraint	Meaning
`id`	UUID	no	PK	Certificate row identifier
`award_id`	UUID	no	FK `bursary_awards(id)` ON DELETE RESTRICT, **UNIQUE**	One certificate per award; a second issue is refused with 409
`certificate_reference`	VARCHAR(40)	no	UNIQUE	`SBMS-CERT-<year>-<8 uppercase hex>`; the hex part is random, not a sequence
`verification_token`	VARCHAR(100)	no	UNIQUE, CHECK `^[a-f0-9]{64}$`	Public token; lives in the URL fragment and is never logged
`award_period`	VARCHAR(20)	no	CHECK `^[0-9]{4}(/[0-9]{4})?$`	`2026` or `2025/2026`
`student_name`	VARCHAR(255)	no	—	Snapshot of the printed name
`student_upi`	VARCHAR(100)	yes	—	Snapshot of the UPI
`school_code`	VARCHAR(100)	no	—	Snapshot of the school
`approved_amount`	NUMERIC(12,2)	no	CHECK `> 0`	Snapshot of the amount; verification compares it with the live award
`award_approved_at`	TIMESTAMPTZ	no	—	Snapshot of the approval time
`status`	VARCHAR(20)	no	CHECK IN (`active`, `revoked`)	Certificate state
`issued_by`	UUID	no	FK `users(id)` ON DELETE RESTRICT	Issuing administrator
`issued_at`	TIMESTAMPTZ	no	—	Issue time
`revoked_by`	UUID	yes	FK `users(id)` ON DELETE RESTRICT	Revoking administrator
`revoked_at`	TIMESTAMPTZ	yes	—	Revocation time
`revocation_reason`	TEXT	yes	CHECK >= 5 trimmed characters when revoked	Why revoked

`bursary_certificates_revocation_check` enforces that an active row has no revocation fields and a revoked row has all three. The table exists so that a later edit to the award cannot silently rewrite what a printed document says; the verification endpoint compares the snapshot with the live award and answers `not_current` on a mismatch.

4.10 `audit_logs`, `reports`, `beneficiaries`, `sbms_schema_migrations`

Table	Column	Type	Null	Constraint	Meaning
`audit_logs`	`id`	UUID	no	PK	Event identifier
	`user_id`	UUID	yes	FK `users(id)`	Actor; NULL only for a system action
	`action`	VARCHAR(100)	no	—	Verb, e.g. `CREATE`, `AWARD_APPROVED`, `PAYMENT_CLEARED`, `AWARD_CERTIFICATE_ISSUED`
	`table_name`	VARCHAR(100)	yes	—	Affected table
	`record_id`	UUID	yes	—	Affected record
	`details`	JSONB	yes	—	Context: model version, thresholds, counters, reasons
	`created_at`	TIMESTAMP	yes	`NOW()`	When
`reports`	`id`	UUID	no	PK	Export identifier
	`generated_by`	UUID	yes	FK `users(id)`	Administrator who exported
	`report_type`	VARCHAR(100)	no	—	Export kind
	`start_date`, `end_date`	DATE	yes	—	Range requested
	`file_path`	TEXT	yes	—	Path recorded at generation; the export is streamed, not persisted
	`created_at`	TIMESTAMP	yes	`NOW()`	When
`beneficiaries`	`id`	SERIAL	no	PK	Legacy integer key
	`name`	VARCHAR(255)	no	—	Legacy name
	`school`	VARCHAR(255)	yes	—	Legacy school text; not a foreign key to `students.school_code`
	`grade_level`	VARCHAR(50)	yes	—	Legacy field with no counterpart in `students`
	`scholarship_amount`	NUMERIC	yes	default 0	Legacy amount; meaning unresolved and deliberately not mapped to any award or payment
	`created_at`	TIMESTAMP	yes	`NOW()`	When
`sbms_schema_migrations`	`filename`	text	no	PK	Migration applied
	`applied_at`	timestamp	—	—	When `migrate.js` applied it

5. Relationships and referential actions

Parent	Child	Column	On delete	Consequence
`students`	`duplicate_flags`	`student_id`	CASCADE	Deleting a student removes its cases
`students`	`duplicate_flags`	`matched_student_id`	CASCADE	Cases disappear when either party is deleted
`duplicate_flags`	`duplicate_review_events`	`duplicate_flag_id`	CASCADE	Review history dies with the case
`users`	`duplicate_flags`	`resolved_by`	no action	A deciding administrator cannot be deleted while cases reference them
`users`	`duplicate_review_events`	`reviewer_id`	no action	Same
`students`	`bursary_awards`	`student_id`	RESTRICT	A student with an award cannot be deleted
`bursary_awards`	`bursary_award_verification_events`	`award_id`	RESTRICT	Verification history blocks deletion
`bursary_awards`	`disbursements`	`award_id`	RESTRICT	A paid award cannot be deleted

Parent	Child	Column	On delete	Consequence
`bursary_awards`	`bursary_certificates`	`award_id`	RESTRICT, UNIQUE	A certified award cannot be deleted and cannot be certified twice
`disbursements`	`disbursement_reconciliation_events`	`disbursement_id`	RESTRICT	Payment history blocks deletion
`students`	`disbursements`	`student_id`	CASCADE	Legacy behaviour retained; the RESTRICT award link still applies when `award_id` is set

The asymmetry is deliberate: review data cascades because it is derived from the register, while money, certification and verification data restrict deletion because they are accountability records. Deleting a student who holds an award therefore fails at the database, not only in the application.

6. Indexes and their purpose

Index	Table (columns)	Purpose
`students_qr_token_unique`	`students(qr_token)` WHERE NOT NULL	Public verification lookup by token; partial so rows without a token do not collide
`duplicate_flags_unordered_pair_unique`	`duplicate_flags(LEAST(...), GREATEST(...))`	One case per unordered pair
`duplicate_review_events_flag_created_idx`	`duplicate_review_events(duplicate_flag_id, created_at DESC)`	Case history, newest first
`bursary_awards_status_created_idx`	`bursary_awards(status, created_at DESC)`	Queue of submitted awards and their age
`bursary_awards_student_created_idx`	`bursary_awards(student_id, created_at DESC)`	A student's award history
`bursary_award_verification_events_award_idx`	`(award_id, created_at DESC)`	Verification history per award
`disbursements_award_status_idx`	`disbursements(award_id, payment_status, receipt_status)`	Reconciliation views and exception queries
`disbursement_reconciliation_events_disbursement_idx`	`(disbursement_id, created_at DESC)`	Payment event trail
`bursary_certificates_status_issued_idx`	`(status, issued_at DESC)`	Active versus revoked certificate listing

No index exists on `students.full_name`, `students.school_code` or `students.date_of_birth`. That is consistent with FR-34: screening reads the whole table, so no index would be used. It is also why a blocking strategy (D4 in the ML decision sheet) would require new indexes.

7. State machines enforced by the schema

Entity	Transition	Enforced by	Cannot happen
Duplicate case	`pending` to `confirmed_duplicate` / `not_duplicate` / `needs_verification`, then terminal	Controller refuses a second decision; `duplicate_review_events` appends history	Re-deciding a terminal case returns 404
Award	`submitted` to `approved` or `rejected`	`bursary_awards_decision_fields_check`; approval additionally requires `verification_status = 'verified'`	Approval of an unverified award; a decision without a reason; an amount on a rejected award
Verification	`not_checked` to `verified` / `not_found` / `needs_investigation`	`bursary_awards_verification_fields_check`; the controller closes verification after a decision	A `verified` outcome without evidence, actor and time

Entity	Transition	Enforced by	Cannot happen
Payment	`issued` to `cleared` / `failed` / `cancelled`, with no further change	`bursaryRules.canTransitionPayment`; `disbursements_clearance_fields_check`	Clearing twice; failing a payment that already has a receipt status
Receipt	NULL to `awaiting_acknowledgement` to `acknowledged` or `discrepancy`	`bursaryRules.receiptStatus` compares cents; `disbursements_acknowledgement_fields_check`	An acknowledged receipt without amount, actor, time and evidence; any method other than `officer_assisted`
Certificate	`active` to `revoked`	`bursary_certificates_revocation_check`	Revoking twice; reissuing for the same award (`award_id` UNIQUE)

Amount comparison is performed in integer cents (`bursaryRules.cents`), so a floating-point difference cannot turn an exact receipt into a discrepancy, or the reverse.

8. Legacy and dead schema objects

Object	State	Handling
`duplicate_flags.probability`	Legacy name for an uncalibrated model score	Documented by a column comment; the API exposes the value as `model_score`
`disbursements.status`	Legacy duplicate of `payment_status`	Kept in step by the update statement; not the authoritative field
`beneficiaries` table	Legacy module, serial key, no foreign keys, `scholarship_amount` with unresolved meaning	Retained because `/api/beneficiaries` and the readiness contract require it; explicitly not mapped to awards or payments
Historical `students_upi_key` uniqueness	Dropped in `schema.sql:47-49`	Duplicate UPI values are now permitted, because duplicate detection must be able to observe them as a signal
`birth_certificate_entry_number`	Renamed to `birth_certificate_number` where it exists	Idempotent rename block in `schema.sql:59-72`
Case-assignment column	Removed by `20261019_remove_duplicate_case_assignment.sql`	Review is unassigned by design; any administrator may take a case

9. Data provenance and synthetic data

Marker or record	Meaning
`students.is_synthetic = TRUE`	Row created by a seed script, not by a registration; separates demonstration data from registrations
`docs/repository-cleanup-2026-10-31.md`	Records the cleanup that limited synthetic schools to the five approved names
Deployment demonstration data	Contains records created during live checks (`docs/azure-deployment-log-2026-10-31.md` §7-§9), including two `E2E Probe` students and one case the log instructs the operator to delete
No real beneficiary data	The repository and the test database contain no real student data; every figure in this documentation set must be produced from synthetic rows (G-05)

10. What the schema does not enforce

Gap	Consequence	Recorded as
No append-only protection on `audit_logs`, `duplicate_review_events`, `bursary_award_verification_events`, `disbursement_reconciliation_events`	Immutability is by convention; a database user with UPDATE or DELETE rights can rewrite history	NFR-21; `docs/business-workflow-verification-2026-10-30.md` §5
No retention or archival policy	Rows are never aged out; no partition or purge strategy exists	G-19

Gap	Consequence	Recorded as
No uniqueness on `students.upi`	By design, but a UPI is therefore not a key and cannot be used to join reliably	§8
No row-level security	Access control is entirely in the application; any client holding database credentials bypasses it	`SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md` §8
No constraint tying `disbursements.student_id` to `bursary_awards.student_id`	The controller checks the pairing; the database does not	Future work
`beneficiaries` has no relationship to any other table	Two parallel registries exist	§8
No trigger maintaining `updated_at`	Every update path must set it; a direct `UPDATE` leaves it stale	Inspection finding

11. Reproduction

```
# schema only (no data), against a disposable database
psql -h localhost -U <owner> -d <disposable> -f backend/src/db/schema.sql
node backend/scripts/migrate.js          # applies unapplied files in filename order
psql -h localhost -U <owner> -d <disposable> -c "\dt"        # 13 tables expected
psql -h localhost -U <owner> -d <disposable> -c "SELECT * FROM sbms_schema_migrations ORDER BY filename"
```

Applying migrations to the development or deployment database is an operator action with backup and a verified restore first; the application never does it automatically.

SBMS API ENDPOINT CATALOGUE

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Scope:** every HTTP route the Express application mounts, as read from `backend/src/routes/*.js` and `backend/src/index.js:65-99`, with the guard chain, request contract, success shape, refusal conditions and the test that covers it. **Not in scope:** request or response bodies containing real beneficiary data (none is reproduced here), and any handler that exists in code but is not mounted (see §7.1).

---

1. Method and conventions

Routes were extracted mechanically from the twelve router files and cross-checked against the mount table in `index.js`. Controller contracts were read from the Zod schema in each controller and from the explicit `res.status(...)` calls. Where this catalogue states a refusal condition, that condition exists as an explicit branch in the controller; nothing here is inferred from naming.

Convention	Value
Base URL	`http://localhost:5000/api` in development; `https://<api-host>/api` in deployment
Transport	HTTPS in deployment; JSON request and response bodies
Body limit	100 kB (`index.js:64`); oversize bodies are refused before routing
Authentication	`Authorization: Bearer <JWT>`; token signed HS256, `expiresIn: 8h`
Identity resolution	`decoded.id` or `decoded.sub`; the role is re-read from the `users` row on every request, and an inactive account is refused (`middleware/auth.js:20-24`)
Success envelope	Object or array as documented; list endpoints return a bare array unless stated
Error envelope	`{ message }` or `{ error }` depending on the router family; a 5xx never carries an internal message
Roles	`admin`, `field_officer`; `requireRole` refuses with 403, a missing or expired token with 401

2. Public, unauthenticated endpoints

#	Method and path	Guard	Success	Failure	Controller
1	`GET /api/health`	none	200 liveness object	—	inline, `index.js:69-72`
2	`GET /api/ready`	none	200 `{status: ready}`	503 generic body when a required table or the expected database identity is absent	inline + `services/databaseReadiness.js`
3	`POST /api/verify`	`verificationRateLimit` (10 / IP / 15 min), JSON parser	200 `{found: true or false}` and nothing else	400 for a body that is not exactly `{token}` of 64 lowercase hex; 429 when the budget is exhausted; 503 `{error: temporarily_unavailable}` on lookup failure	`verification.controller.js`
4	`POST /api/certificates/verify`	`certificateRateLimit` (20 / IP / 15 min), JSON parser	200 authenticity facts only: status, certificate reference, award period, issue and approval dates, issuing authority	400 malformed body; 429; 503	`certificates.controller.js:251-290`

Endpoints 3 and 4 are mounted before the shared JSON body parser, so an abusive client is refused before its body is parsed (`index.js:65-67`). Neither returns a student name, school, UPI or national identifier; the certificate endpoint additionally never returns the award amount.

3. Authenticated endpoint catalogue

3.1 Authentication — `/api/auth`

Method and path	Roles	Request	Success	Failure
`POST /api/auth/register`	none (public)	any	—	410 always: the route returns a fixed refusal and the `register` controller beneath it is unreachable (see §7.1)
`POST /api/auth/login/admin`	none; `loginRateLimit` 10 / IP / 15 min	`{email, password}`	200 `{message, token, user}`	400 invalid body; 401 invalid credentials; 403 when the account is a field officer
`POST /api/auth/login/field-officer`	none; same limiter	`{email, password}`	as above	400; 401; 403 when the account is an administrator
`GET /api/auth/me`	any authenticated	—	200 current user row	401 when the account no longer exists or is inactive

The two sign-in routes share one limiter instance, so ten attempts across both endpoints exhaust the budget for that client IP.

3.2 Users — `/api/users` (administrator only, router-level guard)

Method and path	Request	Success	Failure
`GET /api/users`	—	200 array of users	403 for a field officer
`POST /api/users`	`{full_name, email, password (8-72), role}`	201 created user	400 validation; 409 duplicate email
`PATCH /api/users/:id/role`	`{role}`	200 updated user	400 unsupported role; 404 unknown user; 409 self-demotion and last-administrator protection

3.3 Students — `/api/students` (field officer only, router-level guard)

Method and path	Request	Success	Failure
`GET /api/students/schools`	—	200 the five approved school names	403 for an administrator
`GET /api/students`	—	200 array of student records	403
`GET /api/students/:id`	—	200 student record	404 `Student not found`
`POST /api/students`	`full_name`; `date_of_birth` (YYYY-MM-DD); `school_code` (enum of five); `guardian_id`; `upi`; `birth_certificate_number`; `national_id_number`; `primary_phone` (+254[17] plus eight digits)	201 `{message, student, qr_code, duplicate_flags[], duplicate_screening{}}`	400 `Invalid student data` with a Zod `errors` block; 503 when screening is unavailable

The 201 body is the richest contract in the system. It returns the created student, a `data:` URL QR image, every review case created, and a `duplicate_screening` block carrying `model_version`, `review_threshold`, `high_priority_threshold`, `thresholds_validated: false`, `score_is_calibrated_probability: false`, `aggregation_policy`, `candidates_considered`, `candidates_qualifying` and `candidates_suppressed`.

3.4 Duplicate review — `/api/duplicate-flags` (administrator only)

Method and path	Request	Success	Failure
`GET /api/duplicate-flags`	—	200 queue with identifiers masked, `priority` band, model version, cutoff and policy block	403
`GET /api/duplicate-flags/:id/identifiers`	—	200 revealed identifiers; the disclosure is written to the audit log	404 `Duplicate flag not found`
`POST /api/duplicate-flags/evaluate`	`{pairs: [{label, first, second}]}`, 1-30 pairs, `upi` required on each side	200 score and feature vector per labelled pair	400 validation
`POST /api/duplicate-flags/scan`	—	200 rescan of the existing registry	controller-specific 4xx/5xx paths
`PATCH /api/duplicate-flags/:id`	`{decision: confirmed_duplicate | not_duplicate | needs_verification, reason (>=5 chars), notes}`	200 `Review decision recorded` plus a `duplicate_review_events` row	400 `Select a decision and provide a reason of at least 5 characters.`; 404; 409 when the case is already terminal

3.5 Bursary awards and certificates — `/api/awards`

Method and path	Roles	Request	Success	Failure
`GET /api/awards`	admin, field_officer	—	200 `{awards: []}`	403
`POST /api/awards`	field officer	`{student_id (uuid), requested_amount (>0)}`	201 `{message: Award submitted., award}`	400 `Choose a student and enter a positive requested amount.`; 404 `Student not found.`; 409 `Add the student's school before submitting an award.`
`POST /api/awards/:id/verification`	field officer	`{outcome: verified | not_found | needs_investigation, notes (>=5), evidence_ref}`; `evidence_ref` mandatory when `outcome = verified`	201 `Verification recorded.` plus a `bursary_award_verification_events` row	400 `Provide an outcome, notes, and evidence for verified cases.`; 404 `Award not found.`; 409 `Verification is closed after an award decision.`
`PATCH /api/awards/:id/decision`	admin	`{decision: approved | rejected, approved_amount (required when approved), reason (>=5)}`	200 `Award approved.` or `Award rejected.`	400 `Choose approve or reject and provide a reason.`; 404; 409 `This award already has a decision.`, `Verify the beneficiary before approving this award.`, `Resolve duplicate review cases before approving this award.`
`POST /api/awards/:id/certificate`	admin	`{award_period}` optional, `2026` or `2025/2026`	201 `Certificate issued.` with reference, verification URL and QR data URL	400 malformed period; 404 `Award not found.`; 409 not approved, already issued, or student record missing
`GET /api/awards/:id/certificate`	admin, field_officer	—	200 staff view with QR and verification URL	404 `No certificate has been issued for this award.`

Method and path	Roles	Request	Success	Failure
`GET /api/awards/:id/certificate.pdf`	admin, field_officer	—	200 `application/pdf`, A4 landscape, `Content-Disposition: attachment`	404
`POST /api/awards/:id/certificate/revoke`	admin	`{reason (>=5)}`	200 `Certificate revoked.`	400 `Enter a reason of at least 5 characters for revoking the certificate.`; 404; 409 `This certificate has already been revoked.`

3.6 Disbursements — `/api/disbursements`

Method and path	Roles	Request	Success	Failure
`GET /api/disbursements`	authenticated (router-level `authenticateToken` only)	—	200 array with `exception_reasons[]` and `acknowledgement_overdue_days` per row	401
`GET /api/disbursements/exceptions`	admin, field_officer	—	200 `{deadline_days, exceptions[]}`	403
`POST /api/disbursements`	admin, field_officer	`{award_id, amount (>0), payment_method (2-80), payment_reference, disbursement_date}`	201 `Payment recorded as issued; clearance is not confirmed.`	400; 404 `Award not found.`; 409 `Only verified, approved awards can be disbursed.`, `Resolve duplicate review cases before disbursing this award.`, over-amount and duplicate-block refusals
`PATCH /api/disbursements/:id/payment-status`	admin, field_officer	`{payment_status: cleared | failed | cancelled, evidence_ref (required for cleared), notes (>=5 for failed/cancelled)}`	200 `Payment marked <status>.` plus a `disbursement_reconciliation_events` row	400; 404 `Payment not found.`; 409 `This payment status cannot be changed.`, `A payment with a receipt record cannot be failed or cancelled.`
`PATCH /api/disbursements/:id/acknowledgement`	admin, field_officer	`{acknowledged_amount (>=0), evidence_ref (>=1), notes}`	200 recorded receipt, `acknowledged` or `discrepancy` derived by comparing cents	400 `Enter the received amount and evidence reference.`; 404; 409
`PATCH /api/disbursements/:id/award`	admin	`{award_id, reason (>=5)}`	200 legacy payment linked to an approved award	400; 404; 409 when the award does not match the student, is not approved, or is already linked

3.7 Eligibility, reports, settings, dashboard, audit, beneficiaries

Method and path	Roles	Request	Success	Failure
`POST /api/eligibility`	field officer	`{student_id, fee_balance, household_income, vulnerability_status, siblings_in_school, prior_bursary_history}`	200 `{score, category, breakdown{}, reasons[]}` and the result is persisted on the student row	400 `Fee balance, household income, and siblings must be non-negative numbers`, `Invalid vulnerability status`, `Invalid prior bursary history`; 404 `Student not found`

Method and path	Roles	Request	Success	Failure
`GET /api/reports/export`	admin	query `start_date`, `end_date`	200 PDF stream	400 `start_date and end_date are required`, `start_date cannot be after end_date`
`GET /api/settings/threshold`	admin	—	200 the three cut-offs in force	403
`PUT /api/settings/threshold`	admin	`{threshold, reviewThreshold, mlReviewThreshold}` at least one	200 `Duplicate review settings updated successfully`	400 each out-of-range field and `Duplicate threshold must be greater than or equal to the review threshold`
`GET /api/admin/dashboard`	admin	—	200 totals, six-month trend series, recent audit activity	403
`GET /api/audit-logs`	admin	—	200 audit rows	403
`GET /api/beneficiaries`	any authenticated	—	200 legacy beneficiary rows	401
`GET /api/beneficiaries/:id`	any authenticated	—	200 one legacy row	404 `Not found`
`POST /api/beneficiaries`	field officer	legacy beneficiary fields	201 created row	403
`PUT /api/beneficiaries/:id`	field officer	legacy beneficiary fields	200 updated row	404
`DELETE /api/beneficiaries/:id`	field officer	—	204 no content	404

`/api/beneficiaries` is the legacy module retained for compatibility. It is deliberately not aligned with `students`, and its `scholarship_amount` column is **not** mapped to any award or payment amount (see `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` §8).

4. Endpoint counts

Family	Endpoints	Public	Admin only	Officer only	Both roles	Authenticated, any role
health / ready / verify / certificate verify	4	4	—	—	—	—
auth	4	3	—	—	—	1
users	3	—	3	—	—	—
students	4	—	—	4	—	—
duplicate-flags	5	—	5	—	—	—
awards + certificates	8	—	3	1	4	—
disbursements	6	—	1	—	4	1
eligibility	1	—	—	1	—	—
reports / settings / dashboard / audit	5	—	5	—	—	—

Family	Endpoints	Public	Admin only	Officer only	Both roles	Authenticated, any role
beneficiaries (legacy)	5	—	—	3	—	2
Total	**45**	**7**	**17**	**9**	**8**	**4**

The `auth` public count includes the 410 registration stub and the two sign-in routes.

5. Error catalogue

Status	Meaning in SBMS	Typical body	Where produced
400	Request failed field-level or cross-field validation	`{message}` or `{error, errors}`	Zod `safeParse` in every controller
401	Missing, malformed or expired token; account deleted or deactivated	`Authentication token required`, `Invalid or expired token`, `User account no longer exists`	`middleware/auth.js`
403	Authenticated but not permitted, or account presented at the wrong sign-in endpoint	`You do not have permission to access this resource`	`middleware/requireRole.js`; `auth.controller.js:52`
404	Referenced record does not exist, or a terminal case is re-decided	entity-specific message	controllers
409	State transition refused: award not verified, duplicate open, already decided, already issued or revoked, amount exceeded	state-specific message	controllers, backed by database CHECK constraints
410	Public registration disabled	`Public registration is disabled.`	`auth.routes.js:9-11`
429	Rate-limit budget exhausted for this client IP	generic message unique per limiter	`middleware/rateLimitWindow.js:29,38`
503	Screening unavailable, database not ready, or public lookup failed	registration error sentence; `{status: not-ready}`; `{error: temporarily_unavailable}`	`students.controller.js:78`; `index.js:73-87`; `verification.controller.js:22`
500	Unexpected failure; the message is always `Internal server error` and the stack never leaves the process	`Internal server error`	`middleware/errorHandler.js:59-78`

6. Rate limits and budgets

Limiter	Budget	Window	Scope	Endpoints	Limitation
`loginRateLimit`	10 requests	15 min	per client IP, shared by both sign-in routes	`/auth/login/admin`, `/auth/login/field-officer`	Process-local; resets on restart
`verificationRateLimit`	10 requests	15 min	per client IP, own counter map	`/api/verify`	Process-local; not shared across instances
`certificateRateLimit`	20 requests	15 min	per client IP, own counter map	`/api/certificates/verify`	Process-local
JSON body limit	100 kB	—	per request	all parsed routes	—
Tracked-IP ceiling	10 000 IPs per limiter	—	per limiter instance	all three limiters	A new IP is refused with 429 once the map is full (`rateLimitWindow.js:28-30`)

Client IP resolution uses the socket address unless `TRUST_PROXY_HOPS` is set, in which case Express resolves the forwarded client. Forwarded headers are never trusted by default, and a forged `X-Forwarded-For` does not create a fresh budget (asserted by `verification.test.js`).

7. Contract notes and honest limitations

7.1 Handlers that exist but are not reachable

Item	State	Evidence
`auth.controller.register`	Implemented with validation, hash and duplicate-email handling, but the route returns 410 before it is called	`auth.routes.js:9-11` versus `auth.controller.js:14-35`
Public self-registration	Intentionally impossible; accounts are created through `POST /api/users` by an administrator	FR-03

These are reported as dead code, not as features. A reader should not infer from the existence of `registerSchema` that public sign-up works.

7.2 Role asymmetry between the API and one shared screen

`/api/students` is guarded at router level for `field_officer` only, so an administrator calling `GET /api/students` receives 403. The disbursement screen is rendered for both roles and calls `GET /students` to populate the award-application picker (`frontend/src/pages/Disbursements.jsx:54`), which means that picker has no data for an administrator. This was found by reading the guard chain against the page; it has **not** been exercised in a browser for the administrator role, so it is recorded as an inspection finding rather than a reproduced defect.

7.3 What the API deliberately does not expose

- No endpoint returns `students.qr_token`; the value is stripped from every student response and only the QR image and verification URL are returned.
- No endpoint reveals a beneficiary identifier without an administrator request that is written to the audit log.
- No endpoint reports eligibility, award, payment or receipt status to an unauthenticated caller.
- No endpoint deletes a student, a duplicate case, an award, a payment, an audit row or a certificate. Certificate revocation is a status change, not a deletion.

7.4 Coverage of this catalogue by tests

Every route in §3 has at least one assertion in the backend suite **except**: `POST /api/duplicate-flags/scan`, the plain `GET /api/disbursements` list, the success-path arithmetic of `POST /api/eligibility`, the PDF bytes of `GET /api/reports/export`, `GET /api/settings/threshold`, `PATCH /api/beneficiaries/:id`, `DELETE /api/beneficiaries/:id`, and the dashboard counters. Those exclusions match the partial ratings in `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §8 and are re-listed with the test inventory in `SBMS_TEST_EVIDENCE_CATALOGUE.md` §7.

7.5 Revision discipline

The endpoint set was extracted from the working tree at `d6b5032`. Two root-level reports cited elsewhere in this documentation set are untracked at that revision (`SBMS_O10_O11_IMPLEMENTATION_REPORT.md`, `SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md`), and `.gitignore` still lists `ml-service/model/*.joblib` although the artefact is tracked. Neither affects the API contracts above, but both affect what a future reader can reproduce from a clean clone; they are carried as G-27 and G-28 in the gap register.

SBMS SECURITY AND ACCESS CONTROL CATALOGUE

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `backend/src/index.js`, `backend/src/middleware/*`, `backend/src/config/{cors,jwtSecret,publicAppUrl,settings,reconciliation}.js`, all twelve routers, `backend/src/controllers/*`, `docs/bursary-certificates-2026-11-01.md` §6, `docs/qr-verification-2026-10-30.md` §1-§5.

---

1. Summary of the control set

Control	Implemented as	Where
Password storage	bcrypt, cost factor 12	`auth.controller.js:26`
Session credential	HMAC-signed JWT, `expiresIn: 8h`, secret validated at start-up	`auth.controller.js:55-59`; `config/jwtSecret.js`
Role separation	Two sign-in endpoints; wrong endpoint refused with 403	`auth.routes.js:12-13`; `auth.controller.js:52`
Authorisation	`requireRole(...)` per router or per route; role re-read from the database on every request	`middleware/auth.js:20-24`; `middleware/requireRole.js`
Account state	`users.is_active` must be TRUE; a deleted user is refused	`middleware/auth.js:22-23`
Origin control	Configured allowlist checked before routing, plus the `cors` middleware	`index.js:40-47`; `config/cors.js`
Security headers	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy: microphone=(), geolocation=()`	`index.js:33-39`
Body bound	`express.json({limit: '100kb'})`	`index.js:64`
Error sanitation	5xx returns `Internal server error`; only allowlisted metadata is logged	`middleware/errorHandler.js:59-78`
Rate limits	10 sign-ins, 10 student verifications, 20 certificate verifications per IP per 15 minutes	`middleware/*RateLimit.js`; `rateLimitWindow.js`
Public disclosure control	`{found}` only; authenticity facts only; no identifiers	`verification.controller.js`; `certificates.controller.js:251-290`
Identifier masking	Last three characters until an audited reveal	`duplicateFlags.controller.js:37-42,106-107`
Audit trail	`audit_logs` row for every state change	`services/auditLog.js`
Start-up guards	Missing or weak `JWT_SECRET`, unsafe `PUBLIC_APP_URL`, wrong database identity, missing login columns all stop the process	`index.js:102-125`; `config/publicAppUrl.js`; `services/databaseStartup.js`

2. Authentication

Property	Value	Notes
Hash	bcrypt, cost 12	Cost factor is not benchmarked on the deployment host; a slower host would raise sign-in latency
Token	JWT, HS256, 8-hour expiry	No refresh token and no server-side revocation list; a stolen token is valid until expiry or until the account is deactivated
Token transport	`Authorization: Bearer` header	The frontend keeps the token in `localStorage` (`frontend/src/services/api.js:13`)
Secret minimum	32 bytes	Enforced at start-up; also rejected if it contains a placeholder word or has fewer than 8 distinct characters

Property	Value	Notes
Secret generation instruction	`node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"`	README; the value must never be committed
Sign-in separation	`/api/auth/login/admin` refuses a field officer with 403; `/api/auth/login/field-officer` refuses an administrator with 403	Prevents a role being used at the wrong door, though the role is drawn from the database either way
Failed sign-in	`401 Invalid email or password`	Identical for unknown email and wrong password, so the endpoint does not confirm account existence
Account enumeration at sign-up	Public registration is 410; accounts are created by an administrator	No sign-up oracle exists
Lockout	Per client IP only (10 attempts / 15 min)	No per-account lockout and no progressive delay

3. Authorisation matrix

Capability	Field Officer	Administrator	Unauthenticated
Sign in	yes (officer endpoint)	yes (admin endpoint)	yes
Register / list / read students	yes	**no (403)**	no
Compute eligibility	yes	no	no
Submit award application	yes	no	no
Record beneficiary verification	yes	no	no
Approve or reject award	no (403)	yes	no
Issue, read, print, revoke certificate	read and print only	yes (issue and revoke)	no
Record, clear, acknowledge payment	yes	yes	no
Link a legacy payment to an award	no	yes	no
Read duplicate queue, reveal identifiers, decide case	no (403)	yes	no
Rescan the registry	no	yes	no
Read/update cut-offs	no	yes	no
Dashboard, reports export, audit log, users	no	yes	no
Student QR verification	—	—	yes, rate limited, `{found}` only
Certificate verification	—	—	yes, rate limited, authenticity facts only
Health and readiness	—	—	yes

The four role boundaries proven by `backend/tests/workflow.test.js` #11 are: an officer cannot adjudicate a case (403) or approve an award (403); an administrator cannot register a student (403) or create an award (403).

4. Identity resolution and role freshness

`authenticateToken` verifies the signature, then presents `decoded.id || decoded.sub` to the database and rebuilds `req.user` from the stored row:

```
SELECT id, role FROM users WHERE id = $1 AND is_active = TRUE
```

Three consequences worth stating in the report. First, a role change takes effect on the next request without re-issuing a token, because the signed role claim is ignored. Second, deactivating an account is an immediate revocation mechanism, and it is the only one the system has. Third, a token whose subject no longer exists answers `401 User account no longer exists` rather than a 500, so a deleted account cannot be mistaken for a server fault.

5. Secrets and configuration

Secret or setting	Where it lives	Guard
`JWT_SECRET`	Environment variable, never in the repository	>= 32 bytes; placeholder patterns rejected; start-up aborts otherwise
`DATABASE_URL`	Environment variable	Database identity asserted at start-up and on `/api/ready`
`SBMS_EXPECTED_DATABASE_NAME`	Environment variable	Readiness fails closed when the connected database name differs
`PUBLIC_APP_URL`	Environment variable	Must be a public HTTPS origin in production; LAN and plain-HTTP origins rejected
`CORS_ORIGINS`	Environment variable	Comma-separated bare origins; a malformed value throws and stops the API
`TRUST_PROXY_HOPS`	Environment variable	Default 0; forwarded headers are never trusted unless set
`ACKNOWLEDGEMENT_OVERDUE_DAYS`	Environment variable	Integer 1-3650; unset disables overdue reporting rather than inventing a deadline
Deployment credentials	`deploy/azure/.secrets.json`	Git-ignored and never printed; records the administrator and JWT secrets used for the demonstration deployment

The start-up failure message deliberately appends the cause (`index.js:117-124`), because a Container Apps environment with logging disabled would otherwise show an unexplained crash loop. The comment states that the message is authored text and carries no credentials; stacks are still withheld.

6. Origin control, proxy trust and headers

Control	Behaviour	Limitation
Pre-routing origin check	A request carrying a disallowed `Origin` header is refused with 403 before any route matches	A non-browser client can omit the header; CORS is a browser policy, not authentication
`cors` middleware	Reflects only allowlisted origins	Same as above
Default allowlist	`http://localhost:4174` in development, empty in production unless configured	A missing `CORS_ORIGINS` in production denies browser origins rather than allowing all
Proxy trust	Off by default; `TRUST_PROXY_HOPS` opts in	Mis-set hops would let a client spoof `X-Forwarded-For` and evade the per-IP limiter
Security headers	`nosniff`, `DENY`, `no-referrer`, restricted permissions	**No `Content-Security-Policy` and no `Strict-Transport-Security` header is set by the API**; TLS and HSTS are properties of the hosting platform

7. Input validation and error handling

Layer	Mechanism	Coverage
Request bodies	Zod schemas per controller with cross-field refinements (evidence required for clearance or verification, amount required for approval, reason length)	Every mutating endpoint
Identifiers	Uppercased and normalised; blank strings dropped rather than stored as empty	`services/studentRecordFormat.js`
Phone numbers	Must normalise to `+254[17]` plus eight digits	`students.controller.js:23`
Dates	ISO validation in the API; day-first parsing in the ML feature builder	`studentRecordFormat.validIsoDate`; `feature_engineering._dob_similarity`
Enum values	`school_code` one of five; `role`, `decision`, `payment_status`, `outcome` constrained	Controllers and database CHECK constraints
Quantities	Amounts coerced to finite positive numbers bounded at 9 999 999 999.99; acknowledgement allows zero	Award and disbursement schemas

Layer	Mechanism	Coverage
Unparseable JSON	`Invalid request`	`errorHandler.js:73-77`
Oversize body	`Request body is too large`	`errorHandler.js:75`
Any 5xx	`Internal server error`, stack never serialised	`errorHandler.js:73-77`

8. Data protection and privacy

Property	Implementation	Consequence
QR token placement	Token in the URL **fragment** (`/verify#token=...`), removed from the visible URL with `history.replaceState`, then POSTed	The token never reaches a web server in a URL, never appears in access logs and is not sent as a `Referer`
Token format	Exactly 64 lowercase hex characters; exactly one body key accepted	Malformed, unknown and well-formed-but-unknown tokens all answer `{found: false}`, so the endpoint is not a validity oracle
Student endpoints	No name, school, date of birth, identifier or phone in any response	Verification confirms existence only, never identity, eligibility, approval or payment
Certificate endpoint	Authenticity facts only; no student name, UPI, school or amount	A holder of the QR learns that a certificate exists and is current, not whose it is
Certificate storage	Snapshot row, `award_id` UNIQUE, revocation separate from the award	A printed document cannot be rewritten by a later award edit
Review queue	Identifiers masked to the last three characters	Disclosure requires an explicit, audited request
Evidence references	Free-text pointers, not uploads	No document is stored by SBMS, so there is no document store to breach, and no document integrity guarantee either
Transport	HTTPS in deployment	Local development runs over HTTP by design
Encryption at rest	Provided by the managed PostgreSQL service, not the application	No column-level encryption for identifiers
Personal data minimisation	Only fields needed for screening, identification and payment are stored	The national identifier is stored but is never a model feature
Retention	None implemented	Audit, review and payment rows are kept indefinitely (G-19)

9. Audit trail

Property	Value
Writer	`services/auditLog.js`, called inside the same transaction as the change
Fields	`user_id`, `action`, `table_name`, `record_id`, `details` (JSONB), `created_at`
Actions recorded	`CREATE` on registration and award submission; `AWARD_VERIFICATION_RECORDED`; duplicate confirm and reject actions; `AWARD_APPROVED` / `AWARD_REJECTED`; `PAYMENT_ISSUED`, `PAYMENT_CLEARED`, `PAYMENT_FAILED`, `PAYMENT_CANCELLED`; `RECEIPT_ACKNOWLEDGED_OFFICER_ASSISTED`; `AWARD_CERTIFICATE_ISSUED`, `AWARD_CERTIFICATE_REVOKED`; threshold updates; identifier reveals
Screening provenance	`duplicate_model_version`, `duplicate_review_threshold`, `duplicate_aggregation_policy`, candidate counts, flags created
Ten-action sequence proven	`tests/workflow.test.js` #10
Read access	Administrator only (`GET /api/audit-logs`), rendered readably rather than as raw JSON
Immutability	**By convention only.** No trigger, no permission restriction, no append-only table, no retention policy

10. Rate limiting, and exactly how far it reaches

Limiter	Budget	Reach	Honest limitation
Sign-in	10 / IP / 15 min	Both sign-in routes share one counter map	Per IP only; a distributed attacker is not slowed, and a shared-NAT school could be locked out until the window expires
Student verification	10 / IP / 15 min	`/api/verify`	Counters are process-local: they reset on restart and are not shared between container replicas
Certificate verification	20 / IP / 15 min	`/api/certificates/verify`	Same
IP map ceiling	10 000 IPs per limiter	All three	Once full, new IPs are refused with 429 until cleanup runs

Rate limiting is mounted before the JSON parser on the two public verifiers, so an abusive client is rejected before its body is parsed.

11. Residual security risks to report

Risk	Severity for a demonstration	Reason it cannot be closed inside this project
JWT in `localStorage`	Medium	Mitigating it properly needs httpOnly cookies, CSRF handling and a same-site frontend and API origin; the frontend is served from a different host than the API
No token revocation list	Medium	Needs a store and a sign-out endpoint; the only revocation today is deactivating the account
No per-account lockout	Medium	Needs a counter keyed on the account as well as the IP
No multi-factor authentication	Medium	Out of scope for the proposal; administrator actions are high-value
In-process rate limiter	Medium	Needs a shared store to survive scaling and restarts
No CSP or HSTS header from the API	Low to medium	Static frontend hosting controls these; the API serves JSON only
No row-level security in PostgreSQL	Medium	Any client with database credentials bypasses all application control
Audit trail is not immutable	Medium	Needs a restricted role, a trigger or an append-only store
Route-scoped admin-login diagnostic logs error name, PostgreSQL code and file:line	Low	Marked temporary in the source; it logs no credential, body or stack, but it is a deliberate diagnostic left in the code
No security review or penetration test	High for any production claim	Never performed; nothing in this catalogue is a substitute
No dependency-vulnerability gate in CI	Medium	No `npm audit` or Python audit gate is part of the recorded pipeline

12. How to describe the security position in Chapters 4-6

1. State the controls that exist and are tested (§1, §3, §9, §10) together with the test that proves each.
2. State the risks in §11 explicitly, as residual risks of a demonstration system, not as future work that has been done.
3. Do not describe the system as production-ready, hardened or secure without a review; no penetration test, no dependency audit and no immutable audit store exist.
4. Keep the distinction between *authenticated* (who you are), *authorised* (what role you hold) and *accountable* (what was recorded). SBMS implements all three, and the audit trail is the weakest, because it is only conventionally immutable.

SBMS ML SUBSYSTEM DOCUMENTATION

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `ml-service/feature_engineering.py`, `ml-service/predict.py`, `backend/src/services/duplicateCheck.js`, `backend/src/services/mlDuplicateReview.js`, `backend/src/services/duplicateAggregation.js`, `backend/src/controllers/students.controller.js`, `docs/ml-decision-sheet-2026-10-30.md`, `docs/ml-evaluation-baseline-2026-10.md`, `docs/ml-training-integrity-2026-10-30.md`, `ml-service/reports/sbms_dataset_experiment.md`.

---

1. What is deployed, what is research

Item	Status	Identity
Production classifier	**Deployed and called on every registration**	`XGBClassifier` inside a scikit-learn pipeline, six features, artifact `ml-service/model/xgb_duplicate_model.joblib`, model version `xgboost:6ec9f2a3cbe6ed0dcf80c79b15f47364d9f4f7bd3e27da25c690d0436bb0a6f9`
Random Forest	**Not deployed.** Research candidate only	`ml-service/model/research/rf_attachment_style_candidate.joblib`; the application never loads it
Research ten-feature model on the supplied archive	**Not deployed and structurally incompatible with the serving path**	`ml-service/experiments/sbms_dataset/artifacts/sbms_dataset_xgb_research_only.joblib` (183 445 bytes, separate hash)
Review cut-off	**Deployed, provisional**	0.50 (`ML_REVIEW_SCORE_THRESHOLD`), `thresholds_validated: false`
High-priority cut-off	**Deployed, provisional, presentation only**	0.70 (`DUPLICATE_HIGH_PRIORITY_THRESHOLD`)
Human adjudication	No adjudicated pair exists	`ml-service/evaluation/reviewed_pairs_template.csv` is header-only

The approved proposal specified a Random Forest over four features. What runs is an XGBoost classifier over six. That is a design deviation (G-21), reported as a deviation rather than reconciled, because a reader must not be told the proposal was followed when it was not.

2. Integration architecture

Element	Implementation	Consequence
Where scoring happens	Node service `backend/src/services/duplicateCheck.js` spawns `python predict.py --server` and exchanges newline-delimited JSON over stdio	No HTTP hop, no extra port, one container
Request shape	`{pairs: [{first, second}, ...]}` with raw student records	The feature builder runs in Python, so training and serving share one implementation
Response shape	`{model_version, results: [{score, features}]}`	Score and the six feature values come back together and are stored with the case
Model version	SHA-256 of the artifact file, prefixed `xgboost:`	A stored case names the exact bytes that scored it
Failure mode	Predictor unavailable or timed out → registration aborts with **503** and no student row is created	An unscreened beneficiary cannot be registered silently (NFR-20)
Batching	All candidates for one registration are sent in a single request	One subprocess round trip per registration, not per pair
Training calls in the serving path	None: `predict.py` contains no `.fit(`, `GridSearchCV`, `joblib.dump` or `train_test_split`, and the assertion is enforced by `ml-service/tests/test_training_integrity.py`	Live registrations cannot change the model

The Python process is long-lived for the life of the API process and is restarted with it. It is not a REST service, and it has no authentication of its own, because it is reachable only through the Node process's stdin.

3. Feature specification (the six model inputs)

`ml-service/feature_engineering.py` is the single source of truth; the training scripts and `predict.py` both import it.

#	Feature	Type and range	Definition	Missing-value behaviour
1	`jaro_winkler`	float 0-1	**Maximum** of Jaro-Winkler similarity on the concatenated name tokens and on the alphabetically sorted tokens	0.0 if either side is missing
2	`dob_match`	float in {0, 0.5, 0.8, 1.0}	1.0 same date; 0.8 one day apart; 0.5 within seven days; 0.0 otherwise	0.0 if either side is missing or unparseable
3	`school_match`	int {0, 1}	Exact match after removing non-alphanumerics and case-folding	0
4	`guardian_match`	float 0-1	Identifier similarity after stripping shared prefixes (`guardian`, `guard`, `upi`, `student`, `bc`, `g`); 1.0 when equal	0.0
5	`upi_match`	float 0-1	As above for the UPI	0.0
6	`birth_certificate_match`	float 0-1	As above for the birth certificate number	0.0

Accepted date patterns are `%Y-%m-%d`, `%Y/%m/%d`, `%d/%m/%Y`, `%d-%m-%Y` and `%d.%m.%Y`. Kenyan day-first convention is assumed; an ambiguous US-style date is deliberately not treated as a confident match, which is the behaviour asserted by `backend/tests/predictorDateNormalization.test.js` (6 tests).

"Missing" is a defined set, not merely an empty string: `''`, `?`, `NA`, `N/A`, `NONE`, `NULL`, `UNKNOWN`, `NOT PROVIDED`, `NOT AVAILABLE`, any value that is all zeros with length four or more, and any value with no alphanumeric characters. A shared placeholder such as `NOT PROVIDED` on both rows therefore produces similarity 0, not 1 — the defect class the integrity checker explicitly tests for.

4. Prediction contract

Function	Input	Output	Used by
`predict(data)`	one dict of the six named features	`{probability}` rounded to four decimals	Unit tests, one-off scoring
`predict_many(list)`	list of feature dicts	list of `{probability}`	Batch paths
`predict_pairs(pairs)`	`[{first, second}]` raw records	`[{score, features}]`	The Node service, via the `--server` protocol
`predict_duplicate(first, second, threshold=0.5)`	two raw records	`{duplicate_probability, is_duplicate, features}`	Not called by the API; a convenience wrapper used in the service tests
`model_version()`	—	`xgboost:<sha256 of artifact bytes>`	Every stored case and every registration response

`probability` is the raw positive-class score of the classifier. It is **not** a calibrated probability, and the registration response says so explicitly with `score_is_calibrated_probability: false`. A score of 0.6 does not mean "60% chance of being a duplicate"; it means "above the provisional review cut-off".

5. Thresholds in force

Threshold	Value	Environment variable	Effect	Validation status
Model review cut-off	0.50	`ML_REVIEW_SCORE_THRESHOLD`	A pair at or above this score becomes a review case	`thresholds_validated: false`; F1 is flat from 0.30 to 0.60, so no value in that band is defensible over another on the available evidence

Threshold	Value	Environment variable	Effect	Validation status
High-priority cut-off	0.70	`DUPLICATE_HIGH_PRIORITY_THRESHOLD`	Presentation only: the case is badged `high`; it cannot approve, reject or pay anything	Provisional; changes ordering, not decisions
Legacy review threshold setting	0.30	`DUPLICATE_REVIEW_THRESHOLD`	Stored with the case and shown to the administrator as the `standard` band	Kept for compatibility with the settings screen

All three are readable and updatable at runtime through `GET`/`PUT /api/settings/threshold` (administrator only), and an update is written to the audit log. The values live in process memory (`backend/src/config/settings.js`), so a restart restores the environment defaults. That is a deliberate, documented limitation: a change made through the screen does not survive a redeploy.

6. Training data and provenance

Item	Value
Training file	`ml-service/training/realistic_pairs_train.csv` — 12 000 labelled pairs over 6 000 groups
Holdout file	`ml-service/evaluation/realistic_pairs_holdout.csv` — 12 000 pairs / 6 000 groups
Generator	`ml-service/generate_realistic_pairs.py`, seeded, with scripted corruption patterns
Positives	`repeat_application` only — one duplicate pattern
Negatives	Deliberately hard: siblings, twins and close ages, identifier collisions, common names, unrelated
Overlap between splits	0 shared groups, 0 shared identities, 0 identical rows (measured)
Holdout used for fitting	`false`, asserted by the training script
Holdout used for model selection	`true` — the same holdout chose XGBoost over Random Forest, so the reported numbers are selection-optimistic
Library versions	Python 3.11.9, XGBoost 3.2.0, scikit-learn 1.9.0, pandas 3.0.5, jellyfish 1.2.1, joblib 1.6.0
Seed	42

7. Measured performance (synthetic holdout — not an accuracy claim)

Per-pair results on the 12 000-pair holdout at each cut-off, from `ml-service/evaluation/operating_point_2026-10-30.json`:

Cut-off	Precision	Recall	F1	Miss rate	FPR on hard negatives	Review share
0.30	0.8694	0.9352	0.9011	6.48%	14.05%	53.78%
0.40	0.8985	0.9132	0.9058	8.68%	10.32%	50.82%
0.50 (live)	**0.9204**	**0.8927**	**0.9063**	**10.73%**	**7.72%**	**48.49%**
0.60	0.9404	0.8647	0.9009	13.53%	5.48%	45.97%
0.70	0.9607	0.8270	0.8888	17.30%	3.38%	43.04%
0.80	0.9791	0.7797	0.8681	22.03%	1.67%	39.82%
0.90	0.9899	0.7037	0.8226	29.63%	0.72%	35.54%

Ranking quality: ROC-AUC 0.9685, PR-AUC 0.9727. Best F1 is 0.9065 at 0.35; the live 0.50 scores 0.9063 and has the highest Youden J (0.8155). F1 is flat across 0.30-0.60, which is why the cut-off is described as provisional rather than tuned.

Overfitting checks (`docs/ml-training-integrity-2026-10-30.md`):

Check	Result
In-sample F1 versus holdout F1	0.9196 versus 0.9063 — a 1.33-point gap
5-fold grouped cross-validation inside the training file	0.9067 mean F1, within 0.0004 of the holdout

Check	Result
Learning curve (10% → 100% of groups)	0.8886 → 0.9063 F1, close to flat
Per-feature ROC-AUC	0.34-0.71; no feature approaches 0.95, so there is no generator shortcut

The independent per-pair false-positive baseline on 2 000 synthetic unrelated pairs (`docs/ml-evaluation-baseline-2026-10.md`) is **0.35% at cut-off 0.50** (7 pairs), with 0.15% when names are distinct. The 7.72% figure in the table above is measured on a negative set enriched with confusable cases and is **not** interchangeable with 0.35%; both must be reported together or neither.

8. From score to case: the registration decision path

Registration never compares only a candidate subset; it scores the new record against every existing student and then applies an aggregation policy.

Step	Implementation	Output
1. Lock	`pg_advisory_xact_lock(hashtext('sbms.student.registration'))` inside a transaction	Serialises concurrent registrations so two registrations cannot interleave their comparisons
2. Candidate set	`SELECT ... FROM students ORDER BY created_at ASC, id ASC FOR UPDATE`	Every existing record, frozen for the transaction
3. Scoring	`scorePairs([{first: submitted, second: candidate}])` in one Python call	A score and feature vector per candidate
4. Exact identifier evidence	`exactReviewIdentifierMatches` compares UPI, birth certificate and national ID	`exact_identifier_matches[]` per case
5. Qualification	Score >= `ML_REVIEW_SCORE_THRESHOLD` **or** an exact identifier agreement	Qualifying candidates
6. Aggregation	`summarizeAggregation` with policy `top-1` (default) or `all`	At most one case, or one case per pair
7. Persistence	`INSERT INTO duplicate_flags ... ON CONFLICT DO NOTHING`	One row per surviving candidate
8. Audit	`duplicate_candidates_considered`, `_qualifying`, `_suppressed`, `_flags_created`, model version, threshold, policy	Audit row plus the `duplicate_screening` response block

The `top-1` policy ranks exact identifier agreement above score, then score. Its purpose is to stop the review queue growing with registry size: with one case per registration the queue grows linearly in *registrations* rather than in *registrations* × *registry size*.

The arithmetic that motivated the policy, using the measured per-pair rate p = 0.0035 at cut-off 0.50:

Existing students N	Expected flags per registration before the fix	Probability of at least one nuisance flag
100	0.35	29.6%
500	1.75	82.7%
1 000	3.5	97.0%
5 000	17.5	~100%

Holding a 1% registration-level nuisance rate at N = 1 000 would need p <= about 0.001%, roughly 350 times lower than the measured 0.35%. Cut-off tuning therefore cannot substitute for better features, blocking or the corroboration policy. The `top-1` change caps the number of cases per registration; it does **not** reduce the probability that a particular legitimate registration receives one.

9. Review policy and what the administrator sees

Field	Computed by	Meaning
`risk_level`	fixed `review`	A case exists and no decision has been made. There is deliberately no `critical` level, because no score makes a decision
`priority`	`reviewPriority({score, highPriorityThreshold, exactIdentifierCount})` in `mlDuplicateReview.js`, shared by the registration response and the queue	`high` when an identifier agrees exactly or the score reaches 0.70; otherwise `standard`
`exact_identifier_matches`	`exactReviewIdentifierMatches`	`['UPI']`, `['birth certificate number']`, `['National ID']` and combinations
`review_reason`	registration controller	An exact-match sentence naming the identifier, or a sentence naming the provisional cut-off
Stored provenance	`duplicate_flags` columns	`model_version`, `review_threshold`, `comparison_features`
Masked identifiers	`duplicateFlags.controller.js:37-42`	Only the last three characters are shown until an audited reveal

The officer's confirmation screen shows plain language and no model internals: a possible-match sentence, the exact-identifier sentence when applicable, and counts ("checked against N existing students; M flagged; K recorded in the audit log without a separate case"). The administrator's queue shows the score, model version, both cut-offs and the aggregation policy, with the explicit statement that the thresholds are provisional.

10. Research artefacts deliberately excluded from the deployed claim

Artefact	Why it cannot be used as evidence for the deployed system
`ml-service/experiments/sbms_dataset/` (ten-feature schema, 16 800/2 400/4 800 split)	Adds four presence and conflict flags and drops `upi_match` and `birth_certificate_match`; the archive has no UPI or birth-certificate values and its school labels are not the five SBMS schools, so the artefact cannot be loaded by `predict.py`
Supplied-archive results (XGBoost precision 0.906, recall 0.846, PR-AUC 0.953 at 0.50)	Properties of a different, incompatible synthetic archive at 25% positive prevalence; the research model was never integrated
`realistic_pairs_xgboost_training_metrics.json` older generations	Superseded; the current holdout figures are the ones in §7, and the integrity checker fails the build if the artifact hash stops matching the metrics
`ML_RELIABILITY_REPORT.md` Random Forest figures	Describe a candidate that the application never loads

Reporting any of these as the production model's performance would be false. They are useful only as comparison evidence and are labelled as research in Chapter Five.

11. What this subsystem cannot support, and why

Claim	Status	Blocked by
"The system detects duplicates accurately"	**Cannot be supported**	No adjudicated labels exist (D5); every label is synthetic
Real-world precision, recall or miss rate	Unmeasured	As above
Applicant-population false-positive rate	Unmeasured	The negative sets are synthetic and either random or deliberately enriched
"The 0.50 cut-off is the right one"	Unsupported	F1 is flat from 0.30 to 0.60; no owner cost policy exists (D2/D3)
Duplicate patterns other than `repeat_application`	Untested	The generator produces one positive pattern
Generalisation beyond the simulator	Unknown	Synthetic name, school and identifier distributions

Claim	Status	Blocked by
Stability across retraining	Unknown	One artifact, one seed; any retrain invalidates the recorded numbers
Real-world latency at registry scale	Unmeasured	Screening is linear in N; the only timing recorded is 22.67 ms for 50 pairs

Two further limitations belong in the report even though they are not model weaknesses. First, registration compares against the entire registry, so the cost per registration grows with the register and the administrator-initiated rescan is quadratic (FR-34). Second, the review queue's ranking is by score and exact-identifier agreement only; it has no notion of reviewer workload, so a surge produces a backlog rather than a prioritised list.

12. Decisions still owned by the programme owner

ID	Decision	Why the model cannot close it
D1	Adjudication protocol and reviewers	Only adjudicated pairs can produce real precision and recall
D2	Acceptable miss rate	A trade-off between missed duplicates and review volume; a value judgement
D3	Review capacity	Determines which cut-off is affordable
D4	Candidate blocking	A behavioural change to the controller; requires approval and new tests
D5	Adjudication ownership and data-protection basis	Required before any real pair is labelled

Providing a recommended default for each of D1-D5 is legitimate; presenting a default as the owner's decision is not. `docs/ml-decision-sheet-2026-10-30.md` keeps that distinction explicit, and this file follows it.

13. Reproduction

```
cd ml-service
.\.venv\Scripts\python.exe -m unittest discover -s tests -v           # 33 tests
.\.venv\Scripts\python.exe .\check_training_integrity.py              # leakage and hash audit, ~30 s
.\.venv\Scripts\python.exe .\evaluate_operating_point.py              # writes the operating-point JSON, ~25
s
.\.venv\Scripts\python.exe .\evaluate_false_positive_rate.py --output .\evaluation\fpr_baseline_2026-10-
30.json
.\.venv\Scripts\python.exe -m unittest discover -s experiments/sbms_dataset -p test_experiment.py   # 7
research tests
```

All five commands are read-only with respect to the artifact: none trains, overwrites or serialises a model. The integrity checker exits non-zero if the committed report stops describing the live artifact, if the holdout and training splits overlap, if serving features drift from the generator's, or if a feature alone predicts the label.

14. Evidence index

Statement in this file	Source
Six feature definitions and missing-value rules	`ml-service/feature_engineering.py:11-116`
Artifact path, version string, protocol	`ml-service/predict.py:14,50-111`
503 on predictor failure	`backend/src/controllers/students.controller.js:74-96`; `docs/FINAL_TEST_REPORT.md`
Cut-off defaults and runtime mutation	`backend/src/config/settings.js`
Holdout metrics at each cut-off	`docs/ml-decision-sheet-2026-10-30.md` §1
False-positive baseline	`docs/ml-evaluation-baseline-2026-10.md` §2
Leakage and learning-curve checks	`docs/ml-training-integrity-2026-10-30.md` §1-§3

Statement in this file	Source
`top-1` aggregation and its limits	`docs/ml-evaluation-baseline-2026-10.md` §4
Review priority and masking	`backend/src/services/mlDuplicateReview.js`; `docs/frontend-verification-2026-10-30.md` §1
Research incompatibility	`ml-service/reports/sbms_dataset_experiment.md` §"Compatibility and limits"

SBMS UI SCREEN AND WORKFLOW INVENTORY

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `frontend/src/App.jsx`, every file in `frontend/src/pages/`, `frontend/src/components/`, `frontend/src/context/AuthContext.jsx`, `frontend/src/hooks/useHashToken.js`, `frontend/src/services/api.js`, and the three browser harnesses in `scripts/`.

---

1. Application shell and routing

The frontend is a single-page React application built with Vite and routed with `react-router-dom`. Every route is declared in `frontend/src/App.jsx`; there is no server-side rendering and no separate admin application.

Path	Component	Guard	Roles admitted
`/login`	`Login`	none	anyone
`/admin/login`	`Login role="admin"`	none	anyone (endpoint decides)
`/field-officer/login`	`Login role="field_officer"`	none	anyone (endpoint decides)
`/verify`	`Verification`	none	public
`/verify-certificate`	`CertificateVerification`	none	public
`/dashboard`	`DashboardHome`	authenticated	admin renders the dashboard; officer is redirected to `/field-officer`
`/field-officer`	`FieldOfficerDashboard`	authenticated	field officer
`/students`	`Students`	`ProtectedPage`	field officer
`/students/register`	`StudentRegistration`	`ProtectedPage`	field officer
`/eligibility`	`Eligibility`	`ProtectedPage`	field officer
`/duplicate-review`	`DuplicateReview`	`ProtectedPage`	admin
`/disbursements`	`Disbursements`	`ProtectedPage`	admin and field officer
`/reports`	`Reports`	`ProtectedPage`	admin
`/settings`	`Settings`	`ProtectedPage`	admin
`/audit`	`AuditLogs`	`ProtectedPage`	admin
`/users`	`UserManagement`	`ProtectedPage`	admin
`/`	redirect	authenticated	to `/dashboard`

`ProtectedRoute` waits for `authLoading`, redirects an unauthenticated visitor to `/login`, and redirects a role mismatch to `/dashboard`. The guard is a **convenience, not a control**: every page's data still arrives from role-guarded endpoints, so a user who forced a route into the DOM would see an empty page and 403 responses rather than another role's data.

2. Shared frontend infrastructure

Element	File	Behaviour
Axios instance	`services/api.js`	`baseURL` is `VITE_API_ORIGIN + /api` when configured, otherwise `/api`; a request interceptor attaches `Authorization: Bearer <token>` from `localStorage`
Auth context	`context/AuthContext.jsx`	Loads the current user from `GET /auth/me` on mount, exposes `isAuthenticated`, `user`, `authLoading`, sign-in and sign-out
Layout	`components/AppLayout.jsx`	Sidebar navigation with role-specific entries, header with the signed-in user and a sign-out action
Public layout	`components/PublicVerificationLayout.jsx`	Minimal centred shell used by both public verification pages, with no navigation and no authenticated data
Hash-token hook	`hooks/useHashToken.js`	Reads `window.location.hash`, captures the token during render (so React StrictMode's double effect cannot strip it before the second pass), removes it with `history.replaceState`, and re-runs on `hashchange`
Certificate panel	`components/AwardCertificatePanel.jsx`	Issue, read, print and revoke controls for an award certificate; used inside the disbursement screen

The hash-token hook exists because of a real defect: verifying once on mount left a stale result when a second QR code was opened in the same tab. That defect, its fix and the StrictMode regression it exposed are recorded in `docs/qr-verification-2026-10-30.md` §4.

3. Public screens

3.1 `/verify` — student QR verification

Property	Detail
Data loaded	None on mount; a token from the fragment is POSTed to `/api/verify`
Success state	A single line, "Record found"
Refusal states	A single line for an unknown, malformed or absent token; "Verification is temporarily unavailable" for a 429 or 503
Data shown	**Nothing else.** No name, school, date of birth, identifier or phone number
URL hygiene	The token is removed from the visible address bar before the request is sent
Evidence	`scripts/qr-smoke.mjs` 9/9 (2026-10-30) asserts each of the above, including that a query-string token is rejected and that a second code in the same tab re-verifies

3.2 `/verify-certificate` — certificate authenticity

Property	Detail
Data loaded	Token from the fragment POSTed to `/api/certificates/verify`
Success state	Authenticity facts: certificate reference, award period, issue date, approval date, issuing authority
Refusal states	`not_recognised`, `revoked`, `not_current`, or a temporary-unavailability line
Data shown	No student name, UPI, school or amount
Evidence	API contracts covered by `certificateVerificationRoute.test.js` (5) and `bursaryCertificates.test.js` (12); **no browser harness exists for this page**

3.3 Sign-in screens

Route	Behaviour
`/login`	Shared sign-in that presents both roles
`/admin/login`	Administrator sign-in; posts to `/api/auth/login/admin`
`/field-officer/login`	Field-officer sign-in; posts to `/api/auth/login/field-officer`

A wrong-role attempt returns 403 from the API with a plain sentence telling the user which door to use, and the page surfaces that sentence rather than a generic failure.

4. Field-officer screens

4.1 `/field-officer` — officer workspace

Property	Detail
Purpose	Entry point for a field officer after sign-in; shows the officer's next actions rather than administrator totals
Data loaded	Uses the authenticated user from context; no administrator endpoint is called
Actions	Navigation to registration, registry and eligibility

4.2 `/students` — registry list

Property	Detail
Data loaded	`GET /students`
Shown	Student rows returned by the endpoint
Actions	Open a student (single read), navigate to registration
Limitation	The endpoint has no pagination or filter parameters, so the screen renders the whole array as returned; a large register would be rendered client-side

4.3 `/students/register` — registration and screening confirmation

Property	Detail
Data loaded	`GET /students/schools` for the school picker
Inputs	Full name (required), date of birth, school (required, five options), national ID, primary phone, guardian ID, **UPI**, **birth certificate number**
Validation	Client-side required fields; the server-side Zod schema is authoritative and its `errors` block is surfaced
Success state	The created student, the QR image, and plain-language screening information: a possible-match sentence, the exact-identifier sentence when applicable, and counts ("checked against N existing students; M flagged; K recorded in the audit log without a separate case")
Refusal state	The API's error sentence in an error panel, with **no confirmation panel**
Deliberate omission	No model version, no cut-off, no score and no `risk_level`: the officer sees wording, not model internals
Evidence	`scripts/browser-smoke.mjs` checks 2-14; `scripts/registration-fields-smoke.mjs` 11/11 against the live deployment

The UPI and birth-certificate inputs were added on 2026-11-01 because the critical exact-identifier rule could otherwise fire only for seeded rows. Both are optional: a guardian may not have the document at the counter, and a blank box is dropped from the payload rather than sent as an empty string.

4.4 `/eligibility` — rule-based scoring

Property	Detail
Data loaded	`GET /students` for the student picker
Inputs	Fee balance, household income, vulnerability status, siblings in school, prior bursary history
Action	`POST /eligibility`
Result shown	Score, category, per-factor breakdown and the reason sentences produced by `services/eligibility.js`
Persistence	Score, category and assessment time are written back to the student row
Limitation	The arithmetic has no automated test (FR-12), so a defect in the banding would not be caught by the suite

4.5 `/disbursements` — awards, payments, reconciliation and certificates

Property	Detail
Data loaded	`GET /awards`, `GET /students`
Award actions	Submit application (`POST /awards`), record verification (`POST /awards/:id/verification`), approve or reject (`PATCH /awards/:id/decision`)
Payment actions	Record payment (`POST /disbursements`), change payment status (`PATCH /disbursements/:id/payment-status`), record acknowledgement (`PATCH /disbursements/:id/acknowledgement`), link a legacy payment (`PATCH /disbursements/:id/award`)
Certificate panel	`AwardCertificatePanel` issues, reads, prints and revokes certificates through `/awards/:id/certificate*`
Role behaviour	The page renders for both roles; controls are permitted by the API, so an officer attempting an administrator-only action receives 403
Known asymmetry	The student picker calls `GET /students`, which is officer-only; for an administrator that call returns 403 and the picker has no data (inspection finding, `SBMS_API_ENDPOINT_CATALOGUE.md` §7.2)
Evidence	Payment and award rules are covered by 20 API tests; **no browser harness drives this screen**

5. Administrator screens

5.1 `/dashboard` — administrative dashboard

Property	Detail
Data loaded	`GET /admin/dashboard`
Shown	Registry, duplicate, award and payment totals; a six-month registration and payment trend; recent audit activity
Limitation	Each counter's arithmetic is not individually tested (FR-25); the endpoint is authorised and the screen is browser-checked as reachable

5.2 `/duplicate-review` — adjudication queue

Property	Detail
Data loaded	`GET /duplicate-flags`
Shown	Per case: priority band, model score, model version, cut-off, reason, masked identifiers. Policy line: both cut-offs, the aggregation policy, and the statement that the thresholds are provisional
Actions	Select a case, reveal identifiers (`GET /duplicate-flags/:id/identifiers`, audited), record a decision (`PATCH /duplicate-flags/:id`) with a reason of at least five characters
Colour rule	Red only for exact identifier agreement; amber for any other review case; green when nothing is flagged
Deliberate design	No control can approve, reject or pay anything; the screen produces a decision record and the award and payment gates read that record
Evidence	`scripts/browser-smoke.mjs` checks 15-23 (queue contents, both cut-offs, model version, "score is not a decision", audit counters)

5.3 `/reports` — PDF export

Property	Detail
Data loaded	`GET /reports/export?start_date=&end_date=`
Actions	Choose a range and download the PDF
Validation	The API refuses a missing range and a reversed range; the screen surfaces the message
Limitation	The PDF's content is verified by inspection only (FR-26)

5.4 `/settings` — duplicate cut-offs

Property	Detail
Data loaded	`GET /settings/threshold`

Property	Detail
Inputs	High-priority cut-off, review cut-off, model review cut-off
Action	`PUT /settings/threshold`
Validation	Each value 0-1; the high-priority cut-off must be greater than or equal to the review cut-off
Persistence	Process memory only; a restart restores environment defaults (`SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §5)

5.5 `/audit` and `/users`

Screen	Data loaded	Actions	Notes
`/audit`	`GET /audit-logs`	Read; screening counters are rendered as readable text rather than raw JSON	Append-only by convention only
`/users`	`GET /users`	Create an account (`POST /users`), change a role (`PATCH /users/:id/role`)	Self-demotion and last-administrator removal are refused by the API

6. Workflow walkthroughs as the interface presents them

Workflow	Screens in order	What the user sees at each step
Register and screen a beneficiary	`/field-officer/login` → `/students/register` → confirmation panel	Sign-in → form → created student, QR image, plain-language screening outcome
Adjudicate a duplicate	`/admin/login` → `/duplicate-review` → case detail → decision form	Queue with priority → masked identifiers, score, cut-offs → reason prompt → confirmation that the decision was recorded
Award and pay	`/students/register` → `/disbursements` (submit, verify, approve) → record payment → clear → acknowledge	Each step either succeeds or returns a named reason (not verified, duplicate open, amount exceeded, evidence missing)
Identify a student	Printed QR → phone camera → `/verify`	One line: record found or not, and nothing else
Verify a certificate	Printed certificate → `/verify-certificate`	Authenticity facts, or revoked / not current / not recognised
Administrative review	`/dashboard`, `/reports`, `/audit`, `/users`, `/settings`	Totals, exported PDF, history, accounts, cut-offs

7. Interface states

State	Presentation
Loading	`ProtectedRoute` renders nothing while `authLoading` is true, so a protected page cannot flash before authentication resolves
Empty	Lists render with no rows rather than a placeholder message
Error	Panels show the server's `message`; a 5xx shows `Internal server error`, never a stack
Refused action	The sentence names the missing precondition, e.g. `Verify the beneficiary before approving this award.`
Role mismatch	Redirect to `/dashboard`, or a 403 sentence naming the correct sign-in page

8. Accessibility and responsiveness: what is and is not established

Aspect	State
Layout	Responsive, single-column on narrow viewports
Keyboard operation	Standard form controls and buttons; no custom widget with bespoke key handling was introduced
Colour	Status colour is always accompanied by text, so meaning does not depend on colour alone
Contrast, font scaling, screen-reader labelling	**Not audited**
Automated accessibility testing	**None**

Aspect	State
Usability testing with officers	**None** (G-06)

NFR-14 must therefore be reported as partially verified on the strength of browser checks that assert rendered text, classes and navigation, not on any accessibility or usability evidence.

9. Screens that do not exist

Absent screen	Consequence
Password reset or change	A user cannot recover an account unaided, and the live administrator password has already drifted from the stored secret
Student edit or delete	The API exposes create, list and read only; a typo cannot be corrected through the interface
QR token re-issue	A student row without a token cannot be given one (FR-35)
Payment execution	Payments are recorded, never transferred (FR-37)
Case assignment	Deliberately absent: any administrator may take any case
Notification or messaging	No email, SMS or in-app notification exists

10. Mapping to the Chapter Four interface figures

Figure in Chapter Four	Screen	Evidence available for the figure
Fig. 4.13 sign-in	`/login`, `/admin/login`, `/field-officer/login`	Screenshot not captured (G-04)
Fig. 4.14 registry list	`/students`	Screenshot not captured
Fig. 4.15 eligibility	`/eligibility`	Screenshot not captured
Fig. 4.16 registration form	`/students/register`	Screenshot not captured; field list verified by the 11-check harness
Fig. 4.17 screening confirmation	`/students/register` success panel	Screenshot not captured; text verified by browser checks 3-7
Fig. 4.18 dashboard	`/dashboard`	Screenshot not captured
Fig. 4.19 duplicate review	`/duplicate-review`	Screenshot not captured; contents verified by browser checks 15-21
Fig. 4.20 awards and certificates	`/disbursements` plus the certificate panel	Screenshot not captured
Fig. 4.21 administrative support screens	`/reports`, `/audit`, `/users`, `/settings`	Screenshot not captured
Fig. 4.22 student verification page	`/verify`	Screenshot not captured; behaviour verified by 9 browser checks
Fig. 4.23 certificate verification page	`/verify-certificate`	Screenshot not captured; behaviour verified at API level only

Every screenshot must be taken against synthetic data and checked for real beneficiary details before insertion (G-05). No screenshot in this folder may show a live deployment record created during testing, including the two `E2E Probe` students named in the deployment log.

SBMS TEST EVIDENCE CATALOGUE

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Run records used:** `docs/FINAL_TEST_REPORT.md` (2026-10-31 run and the 2026-11-01 certificate run), `docs/business-workflow-verification-2026-10-30.md`, `docs/frontend-verification-2026-10-30.md`, `docs/qr-verification-2026-10-30.md`, `docs/bursary-certificates-2026-11-01.md`, `docs/database-recovery-rehearsal-2026-10-31.md`.

What this file is. A catalogue of every test artefact in the repository, what each proves, when it was last executed and by which command. It is not a re-run: the suites listed here were executed on the dates shown, and this catalogue was compiled by reading the suites and the recorded output. Where a suite has never been executed in this environment, the row says so.

---

1. Test levels and tooling

Level	Tool	Location	Count at this revision
Backend unit and integration (HTTP surface, real PostgreSQL)	Jest (`jest --runInBand`)	`backend/tests/` (24 suites)	151 tests
Backend in-process unit	`node --test`	`backend/test/*.node.*`, `.cjs` (7 files)	43 tests
Backend out-of-process smoke	`node --test` against a live API	`backend/test/api.smoke.test.js`	3 tests
ML service	Python `unittest`	`ml-service/tests/` (5 files)	33 tests
ML research helper	Python `unittest`	`ml-service/experiments/sbms_dataset/test_experiment.py`	7 tests
Browser (headless Chrome over DevTools Protocol)	Node script, no new dependency	`scripts/browser-smoke.mjs`	23 checks
Browser, QR page	as above	`scripts/qr-smoke.mjs`	9 checks
Browser, registration fields on the live deployment	as above	`scripts/registration-fields-smoke.mjs`	11 checks
Browser, administrator identifier reveal	as above	`scripts/admin-reveal-smoke.mjs`	Not re-run; blocked by a drifted admin password
Frontend build gate	Vite production build	`frontend/`	exit 0, 2 006 modules, 10.11 s
Database recovery	`pg_dump` / `pg_restore` rehearsal	`docs/database-recovery-rehearsal-2026-10-31.md`	Performed once, 2026-10-31
Load, UAT, phone camera, hosted Azure exercises	—	—	**Not run**

`pytest` is not installed in the ML virtual environment, which is why `unittest` is the runner. That is a recorded substitution, not an oversight.

2. Backend Jest inventory (24 suites, 151 tests)

Suite	Tests	What it proves
`auth.test.js`	4	Sign-in, wrong-endpoint refusal (403), disabled public registration (410), token payload
`roleAuthorization.test.js`	6	Admin/officer boundaries across representative routes; dashboard authorisation; settings denial
`loginRateLimit.test.js`	1	The 11th sign-in attempt in the window is refused with 429
`securityMiddleware.test.js`	9	CORS allowlist behaviour, body-size limit, generic 5xx, header handling
`students.test.js`	9	Registration validation, school constraint, 503 on screening failure, listing, single read
`duplicateCheck.test.js`	15	Predictor invocation, batching, failure handling, response mapping

Suite	Tests	What it proves
`duplicateAggregation.test.js`	13	`top-1` selection, exact-identifier ranking, suppression counts, policy normalisation
`duplicateFlags.test.js`	9	Queue listing, masking, audited reveal, decision recording, terminal-state refusal
`mlDuplicateReview.test.js`	8	Priority banding, exact identifier detection, shared policy between registration and queue
`predictorDateNormalization.test.js`	6	Day-first date parsing, one-day and seven-day tolerances, ambiguous-format refusal
`mlPredictor.integration.test.js`	1	The real Python predictor returns a score through the Node service
`kenyanSyntheticDataset.test.js`	4	Seeded synthetic records behave as the generator intends
`performance.test.js`	1	50-pair screening under the 3 000 ms budget (measured 22.67 ms)
`workflow.test.js`	11	End-to-end business chain: screening, award gate, duplicate gate, clearance, acknowledgement, audit sequence, role boundaries
`bursaryAwards.controller.test.js`	5	Award submission, verification evidence requirement, approval/rejection rules
`disbursements.test.js`	4	Payment creation rules, amount ceiling, award state gate
`disbursementReconciliation.controller.test.js`	4	Clearance evidence, status transitions, exception listing
`bursaryRules.test.js`	4	Cents comparison, receipt status derivation, overdue arithmetic, exception reasons
`bursaryCertificates.test.js`	12	Issue preconditions, QR payload, public answer shape, `not_current`, revocation rules, audit rows
`certificateVerificationRoute.test.js`	5	Public certificate contract, malformed and unknown tokens, its own rate limit, no student data
`verification.test.js`	10	Student QR token format, QR payload, token never returned, `{found}` contract, limiter, forged forwarded header
`verificationRoute.test.js`	4	Public student verification route contract and rate limit
`schemaReadiness.test.js`	5	`/api/ready` fails closed on a missing table, wrong database identity and missing `bursary_certificates`
`health.test.js`	1	`/api/health` liveness

The suite totals are those recorded on **2026-11-01** (`docs/FINAL_TEST_REPORT.md` §10). The 2026-10-31 run recorded 21 suites / 128 tests from the same tree before the certificate feature was added. Both figures are correct for their date; a count must never be cited without its date (G-25).

3. Backend native tests (43 in-process, 3 out-of-process)

File	Environment	What it proves
`databaseBootstrap.node.test.js`	in-process	Database bootstrap path
`databaseStartup.node.test.js`	in-process	Start-up identity checks
`databaseReadiness.node.js`	in-process	Readiness contract and fail-closed behaviour
`errorHandler.unit.test.js`	in-process	No internal detail leaks in a 5xx
`jwtSecret.validation.node.cjs`	in-process	Secret length and placeholder rejection
`migrate.unit.test.js`	in-process	Migration ordering and ledger
`verification.unit.test.js`	in-process	5 QR verification unit tests

File	Environment	What it proves
`api.smoke.test.js`	**out-of-process** against a live API on `http://localhost:5000`	Health, readiness and an authenticated route answer correctly over HTTP

The 2026-10-31 run recorded **43/43** in-process (0.79 s, exit 0) and **3/3** out-of-process. The native layer exists because several behaviours (secret validation, readiness, migration ordering) must be provable without Jest and without a database fixture.

4. Browser evidence

Harness	Checks	Result	Date	What it covers
`scripts/browser-smoke.mjs`	23	**23/23 pass**	2026-10-30	Officer sign-in and registration, screening message, exact identifier sentence, colour change for a suspicious case, aggregation collapsing several candidates to one case, error handling on a blocked request, admin sign-in, queue contents, both cut-offs, model version, "score is not a decision" statement, audit counters, admin redirect
`scripts/qr-smoke.mjs`	9	**9/9 pass**	2026-10-30; **not re-run** 2026-10-31 or 2026-11-01	Scanning opens the page and reports the record, token removed from the URL, no student data, unknown token refused, no-token refused, query-parameter token refused, second scan re-verifies, no page errors
`scripts/registration-fields-smoke.mjs`	11	**11/11 pass** against the live Azure deployment	2026-11-01	Both new identifier inputs present and optional, original six fields still posted, payload carries both markers, blank identifier omitted, blocked request shows an error panel
`scripts/admin-reveal-smoke.mjs`	—	**Not re-run**	—	Blocked because the stored administrator password no longer authenticates

All three executed harnesses drive real Chrome through the DevTools Protocol, sign in through the real forms, fill React-controlled inputs and read the rendered DOM. None of them is a screenshot comparison, and none asserts visual appearance.

5. Python evidence

Suite	Tests	Result	What it proves
`ml-service/tests/` (5 files)	33	**33/33 pass** (1.96 s)	Feature engineering, artifact loading, operating-point report consistency, reliability analysis, training-integrity assertions
`experiments/sbms_dataset/test_experiment.py`	7	**7/7 pass** (0.47 s)	The research experiment's integrity checks (group overlap, label relation, archive hash)

`tests/test_training_integrity.py` is the most consequential of these: it fails the build if a future change retrains on the holdout, regenerates overlapping groups, replaces the artifact without updating its recorded hash, lets the in-sample/holdout gap exceed 0.10, drifts holdout from grouped CV by more than 0.03, introduces a feature with ROC-AUC >= 0.95, or adds training or serialisation calls to the serving path.

6. Database, security and recovery evidence

Check	Result	Date	Source
Schema constraints exercised through the real API	Pass (11 workflow tests plus 12 certificate tests)	2026-10-30, 2026-11-01	`backend/tests/workflow.test.js`, `bursaryCertificates.test.js`
Readiness fails closed on a missing table or wrong identity	Pass (5 tests)	2026-11-01	`schemaReadiness.test.js`
Dump and restore into an independent database	Pass: identity, table count, row digests compared	2026-10-31	`docs/database-recovery-rehearsal-2026-10-31.md`

Check	Result	Date	Source
Scheduled backup job and retention policy	**Absent**	—	NFR-18, G-19
Rate limiters	Pass in-process only	2026-10-30, 2026-11-01	`verification.test.js`, `loginRateLimit.test.js`, `certificateVerificationRoute.test.js`
Concurrency	**No test**	—	NFR-13

7. What has not been run, and what that costs

Not run	Why	Consequence for the report
Phone-camera scan of a printed student QR code	Requires a public HTTPS origin; the deployment has one, but no phone session was recorded	The identification mechanism rests on a headless-browser check (G-11)
Phone-camera scan of a printed certificate	As above	Certificate authenticity is proven at the API, not on paper
Browser check of `/verify-certificate`	No harness exists for it	The certificate page is covered by API tests only
User acceptance study	Not conducted	NFR-14 unverified; no usability claim may be made (G-06)
Load or scale test	Not conducted	No claim about behaviour beyond 50 candidates (NFR-01)
Docker image build and run in this environment	Docker engine not running	Image size and gates are recorded from the CI build instead
Full Azure behaviour of the latest commit	Deployment was exercised on 2026-10-31 and 2026-11-01, not on this revision	Hosted behaviour is reported per date, not as current
Firefox, Safari, mobile browsers	Not exercised	NFR-19 partial
`pytest`	Not installed	`unittest` used; no behavioural difference claimed

8. Run history, including one instability that must be reported

Date	Command	Result	Note
2026-10-30	`npx jest --runInBand tests`	107/107 pass	Before the workflow and payment suites
2026-10-30	Same, after the workflow suite was added	21 suites / 128 tests pass	Recorded in `docs/business-workflow-verification-2026-10-30.md`
2026-10-31	`npm test` (Jest, whole tree)	21 suites / 128 tests pass, 93.6 s	`docs/FINAL_TEST_REPORT.md` §1
2026-10-31	Regression re-run after documentation edits	21 suites / 128 tests pass, 73.5 s	Same tree, shorter run
2026-11-01	`npx jest --runInBand tests`	**24 suites / 151 tests pass**	Certificate feature added; supersedes the totals above
2026-11-01	`npm.cmd --prefix frontend run build`	exit 0, 2 006 modules, 10.11 s	Production build gate

One instability is recorded honestly rather than hidden: during the 2026-10-31 sweeps the same suite was observed to stop at **149 or 150 of 151** when free memory fell to roughly 0.5 GB of 7.4 GB. The mechanism is inferred (memory pressure during a `--runInBand` run against a local database), not measured, and the affected runs were superseded by clean passes. It is carried as G-02 and must be mentioned if the report discusses test stability.

9. Coverage exclusions that matter for the evaluation chapter

Area	What exists	What is missing
Requirement coverage	151 assertions mapped to requirement groups in `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §4	Branch-level coverage of FR-12, FR-19, FR-25, FR-26, FR-28, FR-29

Area	What exists	What is missing
Frontend	Build gate plus 43 browser checks across three harnesses	No unit tests for React components; no accessibility or visual regression
Duplicate model	Synthetic holdout plus integrity gates	No adjudicated data, no real-world metric
Payments	Full state-machine tests at API level	No browser test of the reconciliation screen; no bank integration to test
Certificates	17 API-level tests across two suites	No browser test of `/verify-certificate`; no printed-copy scan
Operations	Readiness and a restore rehearsal	No scheduled backup, no monitoring, no alerting

10. Reproduction commands

```
# Backend Jest suite (needs backend/.env.test with NODE_ENV, DATABASE_URL,
# SBMS_TEST_DATABASE_URL, PUBLIC_APP_URL; values are never printed)
cd backend
npm test

# Native in-process and out-of-process tests
node --test test/*.node.test.js test/*.node.js test/*.unit.test.js
node --test test/api.smoke.test.js            # with API_URL=http://localhost:5000

# Frontend production build
npm.cmd --prefix frontend run build

# ML service and research helper
ml-service\.venv\Scripts\python.exe -m unittest discover -s ml-service/tests -v
ml-service\.venv\Scripts\python.exe -m unittest discover -s ml-service/experiments/sbms_dataset -p
test_experiment.py

# Browser harnesses (each needs the API, the frontend dev server and a seeded account)
node scripts/browser-smoke.mjs
node scripts/qr-smoke.mjs
node scripts/registration-fields-smoke.mjs
```

The Jest guard `backend/tests/testDatabaseGuard.js` fails closed unless `NODE_ENV=test` and both `DATABASE_URL` and `SBMS_TEST_DATABASE_URL` are the same loopback `sbms_test` URL. A run that cannot prove it is pointed at the disposable database does not start, which is why no test in this catalogue can damage the development or deployment database.

11. How to read a test count in the report

1. Always cite the date with the count (`24 suites / 151 tests, 2026-11-01`), because two generations exist in the same file.
2. Never add counts across levels to produce a single "tests passed" figure. Jest, native, browser and Python tests answer different questions.
3. A passing test suite says nothing about the features it does not touch. §7 and §9 are the list of those features, and they are the ones the evaluation chapter must treat as unverified.
4. A green build is not evidence of behaviour; it is evidence that the code compiles and type-checks under the bundler.

SBMS DEPLOYMENT AND OPERATIONS GUIDE

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `docs/azure-deployment-log-2026-10-31.md`, `docs/azure-deployment-plan-2026-10-30.md` §11-§12, `docs/deployment.md`, `docs/database-recovery-rehearsal-2026-10-31.md`, `docs/bursary-certificates-2026-11-01.md` §9, `Dockerfile`, `.github/workflows/publish-api-image.yml`, `render.yaml`.

---

1. Environments

Environment	Purpose	Runtime	Database	Status at this revision
Local development	Authoring and manual checks	Node 22 API on port 5000, Vite frontend on port 4174, Python 3.11 ML subprocess	Local PostgreSQL 17, database `student_beneficiary_system`	Working; the ML virtual environment must exist
Local disposable test	Automated suites	Jest and native runners against the same machine	`sbms_test` on loopback, role `sbms_test_runner`	Working; the guard refuses any other target
Deployment (demonstration)	Live end-to-end checks and the capstone demonstration	Azure Container Apps for the API, Azure App Service F1 for the frontend, Azure Database for PostgreSQL Flexible Server	`sbms` on `pg-sbms.postgres.database.azure.com`	Live and verified 2026-10-31, brought back up 2026-11-01
Render blueprint	Abandoned target	`render.yaml` remains in the repository	—	Superseded by Azure; kept only as history

The deployment is a **demonstration instance**, not a production service. It has one administrator path, demonstration data, no monitoring stack and no backup schedule. That description belongs in Chapter Five wherever the deployment is discussed.

2. Azure resource inventory

Resource	Name	Configuration	Purpose
Resource group	`rg-sbms`	UAE North	Container for everything below
Container Apps environment	`cae-sbms`	UAE North, logs destination none	Hosts the API container
Container app	`ca-sbms-api`	Image `ghcr.io/eggxellent/sbms-api`, `minReplicas 0`, public ingress	API plus ML runtime
App Service plan	`asp-sbms-web`	F1 (free), Linux	Hosts the frontend
Web app	`sbms-web`	F1, static build deployed as a zip	React single-page application
PostgreSQL flexible server	`pg-sbms`	Burstable B1MS, 32 GiB, stopped between demonstrations	`sbms` database
Budget	`budget-sbms`	Cost alert	Spend visibility

Verified facts that constrain any future change (`docs/azure-deployment-log-2026-10-31.md` §2): the subscription is Azure for Students with the spending limit ON, so no out-of-pocket spend is possible; a `sys.regionrestriction` policy denies every region except UAE North, Italy North, Austria East, Switzerland North and Spain Central; Static Web Apps cannot be created on this account because `staticSites` exists only in regions the policy denies; UAE North supports `Microsoft.App`, `Microsoft.DBforPostgreSQL`, `Microsoft.Web` and `Microsoft.ContainerRegistry`.

3. Live endpoints

Service	URL
API	`https://ca-sbms-api.ambitiousstone-f2de8f67.uaenorth.azurecontainerapps.io`
Frontend	`https://sbms-web.azurewebsites.net`
Database host	`pg-sbms.postgres.database.azure.com`, database `sbms`
Health	`GET /api/health` on the API host
Readiness	`GET /api/ready`, which also verifies the expected database name and required tables

4. Runtime architecture as deployed

Property	Value	Consequence
API and ML in one container	`duplicateCheck.js` spawns `predict.py --server` inside the same image	One deployable unit; no separate ML service, no network hop, no second authentication boundary
Image size	1.37 GB after cleanup (2.22 GB before)	Faster cold start; `deploy/` and `.env*` excluded from the image; the ML integrity gate runs inside the build
Image registry	Public `ghcr.io` package built by CI	No registry credential on the container app, and no Azure Container Registry cost
Scale	`minReplicas 0`	Scale to zero between demonstrations, which is why the API can take a cold start
Logs	Destination none	No Log Analytics workspace cost; the only diagnostics are the process's own stderr, which is why the start-up guard prints an authored cause
Frontend hosting	App Service F1, free tier	No cost; no staging slot; deployment is a zip push
Database	Burstable B1MS, stopped on demand	Cost only accrues while running; a stopped server makes the API unready by design
Secrets	`deploy/azure/.secrets.json`, git-ignored	Never printed in any document; rotating them is a manual step

5. Build and release pipeline

Stage	Mechanism	Evidence
API image build	GitHub Actions workflow `publish-api-image.yml`, dispatch only, registered on `dev`	Pushes `ghcr.io/eggxellent/sbms-api`
Build gates inside the image build	Dependency install, ML integrity gate	Recorded in the deployment log
Frontend build	`npm.cmd --prefix frontend run build` with `VITE_API_ORIGIN` set to the API origin	`dist/` written, 2 006 modules
Frontend deploy	`az webapp deploy` with a zip package	Live `index.html` served `assets/index-KTiUJ9pN.js` after the 2026-11-01 change
Migrations	**Manual.** An operator applies them; the pipeline never does	`backend/scripts/migrate.js` plus the backup-then-apply procedure
Local Docker	Docker CLI 29.9.2 present but the engine does not run in this environment	The image was therefore verified from the CI build, not locally

The repository being public is what makes CI minutes and `ghcr.io` packages free; that fact is recorded because it explains why no registry credential exists on the container app.

6. Configuration matrix

Variable	Required	Meaning	Failure if wrong
`DATABASE_URL`	yes	PostgreSQL connection string	API cannot start or reports not ready

Variable	Required	Meaning	Failure if wrong
`JWT_SECRET`	yes	>= 32 random bytes	Start-up aborts with `JWT_SECRET must contain at least 32 bytes and cannot be a placeholder.`
`PUBLIC_APP_URL`	yes	Public HTTPS frontend origin used in QR links	Start-up aborts; a wrong value produces unreachable QR codes
`CORS_ORIGINS`	yes in production	Bare frontend origin allowlist	Browser origins denied
`SBMS_EXPECTED_DATABASE_NAME`	yes in production	Database identity assertion	Readiness fails closed
`PORT`	no (default 5000)	Listen port	—
`NODE_ENV`	yes	`production` changes CORS and `PUBLIC_APP_URL` strictness	Wrong value changes security behaviour
`TRUST_PROXY_HOPS`	recommended behind a proxy	Number of trusted proxies	Unset behind App Service puts every user in one rate-limit bucket
`ACKNOWLEDGEMENT_OVERDUE_DAYS`	optional	1-3650	Unset disables overdue reporting
`ML_REVIEW_SCORE_THRESHOLD`	optional (0.50)	Review cut-off at start-up	A wrong value changes which pairs become cases
`DUPLICATE_HIGH_PRIORITY_THRESHOLD`	optional (0.70)	Presentation band	Changes ordering, not decisions
`DUPLICATE_REVIEW_THRESHOLD`	optional (0.30)	Legacy `standard` band	Cosmetic
`DUPLICATE_AGGREGATION_POLICY`	optional (`top-1`)	`top-1` or `all`	`all` reintroduces one case per pair
`VITE_API_ORIGIN`	frontend build time	API origin baked into the bundle	Frontend calls the wrong host

7. Start-up and readiness sequence

```
1. dotenv loads backend/.env (ignored by git)
2. JWT_SECRET is validated (length, placeholder, character diversity)
3. PUBLIC_APP_URL is resolved and validated (HTTPS in production)
4. Authentication schema is asserted: expected database name, public.users and its login columns
5. Express listens on PORT
6. GET /api/health answers {status: ok} immediately
7. GET /api/ready checks database identity and the required application tables, including
bursary_certificates
```

Any failure in steps 2-4 calls `process.exit(1)` with an authored sentence plus the cause. Steps 6 and 7 are deliberately separate: a live process is not a working system, and readiness fails closed rather than reporting ready because a socket is open. Neither endpoint initialises the database.

8. Database migration procedure

1. Read the SQL of every unapplied file in `backend/src/db/migrations/`.
2. Take a backup and verify it restores into a separate database.
3. Query the identity of the target (`current_user`, `current_database()`, host) and confirm it is the intended one.
4. Apply with `node backend/scripts/migrate.js`; the script records each file in `sbms_schema_migrations`.
5. Call `GET /api/ready` and require `{status: ready}` before announcing the deployment.
6. For the certificate feature specifically, `20261031_bursary_certificates.sql` must be applied before the endpoints are used; until then readiness reports not ready and issuing fails.

The application never applies a migration at start-up. That is a deliberate safety property: a container restart cannot change the schema.

9. Cost model (recorded, not re-derived)

Scenario	Cost
Idle month, database stopped, `minReplicas 0`	**$4.42** (PostgreSQL storage only: 32 GiB at $0.138/GiB-month)
One 8-hour demonstration	**$4.58** (adds about $0.16 of B1MS compute)
A normal month with occasional demonstrations	about **$6.42**
Frontend	$0 on App Service F1
Container registry	$0 (public `ghcr.io` package, no ACR)
Logs	$0 (logging destination none)

These figures come from `docs/azure-deployment-plan-2026-10-30.md` §12 and are owner decisions, not properties of the code. The spending limit on the subscription means the realistic worst case is a suspended service, not a bill.

10. Backup, restore and continuity

Property	State
Rehearsal performed	Yes, 2026-10-31, on the local disposable `sbms_test` database only
Method	`pg_dump -Fc` to a timestamped file outside the repository, then `pg_restore` into a newly created database whose name matched `^sbms_restore_rehearsal_\d{8}_\d{6}$`
Safety rules applied	Identity queried before any write; no overwrite of an existing backup; restore refused if the target name equalled `sbms_test`, `student_beneficiary_system` or `postgres`; no credential printed
Verification after restore	Readiness, table count, migration ledger and per-table row digests compared with the source
Backup file	Still present outside the repository; its hash is recorded in the rehearsal document
Scheduled backup in the deployment	**None**
Retention policy	**None**
Point-in-time recovery	Available as a platform feature of the managed service, never exercised

The rehearsal closes the "a documented, tested restore exists" criterion and nothing more. Continuity for a real service would need a schedule, a retention rule, an off-region copy and a restore drill with a stated recovery objective. None of those exist.

11. Operations runbook

Task	Command or action	Notes
Start the database	`az postgres flexible-server start --name pg-sbms --resource-group rg-sbms`	The API becomes ready once the database answers
Stop the database between demonstrations	`az postgres flexible-server stop --name pg-sbms --resource-group rg-sbms`	Stops accruing compute cost
Check liveness and readiness	`GET /api/health`, `GET /api/ready`	A 503 from readiness is a real fault, not a delay
Rebuild the API image	Dispatch `publish-api-image.yml` on `dev`, then update the container app	Dispatch only; nothing else is merged to `dev`
Deploy the frontend	`npm.cmd --prefix frontend run build` with `VITE_API_ORIGIN`, then `az webapp deploy` the zip	The live bundle name changes, which is the quick visual confirmation
Apply a migration	§8	Manual, after a verified backup
Rotate the JWT secret	Update the secret, restart the container app	Every existing token becomes invalid immediately

Task	Command or action	Notes
Reset an administrator password	Direct `UPDATE` with a freshly generated bcrypt hash, or create a new administrator through `POST /api/users` if any administrator can still sign in	Recorded as a live issue: the stored password drifted from the secret file (`docs/azure-deployment-log-2026-10-31.md` §9)

12. Known operational issues carried forward

Issue	State	What the report must say
Administrator password in `.secrets.json` no longer authenticates (`admin@sbms.com` and `admin@sbms.test` both 401)	Open	Administrator browser smoke checks are blocked until it is reset
Database was still running after the 2026-11-01 session, with a re-stop due by **2026-11-08 11:03Z**	Open at the time of the log	Cost accrues at $0.02/h until stopped
Two `E2E Probe` students and one duplicate case remain in the deployment	Open	The cleanup SQL is recorded in the deployment log
`frontend/src/pages/Verification.jsx` and `backend/src/index.js` were uncommitted in an earlier session	Resolved by later commits, but the log records the state at that time	Cite the commit, not the working tree
`browser-smoke.mjs` expects one shared `SBMS_VERIFY_PASSWORD`, so it cannot drive two accounts with different passwords	Open	Admin-side browser coverage is partial
Docker engine unavailable locally	Environmental	Image builds are evidenced from CI, not from a local build
Two root reports untracked; `.gitignore` lists the model artefact although it is tracked	Open (G-27, G-28)	Affects reproducibility from a clean clone

13. What the deployment does not demonstrate

Not demonstrated	Consequence
Load or concurrency behaviour	No hosted load test; the container scales to zero, so the first request after idle is a cold start
Monitoring, alerting or dashboards	Logs are disabled to save cost; there is no metrics review
Blue/green or canary release	A deploy is a single image swap with no rollback rehearsal
Multi-region or high availability	One region, one database instance, no replica
Backup schedule	§10
Real users	The instance holds demonstration data only
Phone-camera QR scanning against the public origin	Technically possible now that the origin is HTTPS, but no phone session was recorded (G-11)

Chapters Five and Six should describe this as an operated demonstration environment with verified functional behaviour on the dates recorded, and should not describe it as a production deployment.

EVIDENCE AND VERIFICATION REPORT

Purpose. This report identifies the files, code locations, tests, configurations and project documents that support each significant claim in Chapters 4, 5 and 6, and states what was verified by direct inspection during this documentation pass, what was taken from recorded evidence, and what could not be verified at all.

Method. Read-only inspection. No application source file, schema, migration, configuration, environment variable or test was modified, no migration was applied, nothing was deployed, and no test suite was re-executed. The only files written are the six documents in this folder.

---

1. Identity of what was inspected

Item	Value
Repository	`C:\Users\user\Desktop\Projects\student-beneficiary-system`
Branch	`fix/sbms-completion-work`
Commit	`d6b5032` (`chore(repo): drop generated logs, report PDFs and three superseded docs`)
Parent commits of interest	`f3686ec` (certificate feature), `621f23f` (state reviewed by the O10/O11 work)
Working-tree state at inspection	Clean apart from two untracked root reports (`SBMS_O10_O11_IMPLEMENTATION_REPORT.md`, `SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md`)
Other copies of the project on disk	`student-beneficiary-system-dev` (`dev`), `-admin-login-fix`, `sbms-raw-pair-model-training` — **not** used as evidence, because their documents differ
Academic sources consulted	`ICS Project II Proposal Guide 2026_V1.docx` (School template), `Project Proposal- 169613.pdf` (approved SBMS proposal, 34 pages), `CHAPTER_4[1].docx` (Chapter 4 exemplar, used for section numbering)

---

2. Code evidence index

Claim	Evidence
Public registration disabled; two role-specific sign-in routes	`backend/src/routes/auth.routes.js:9-14`
bcrypt cost 12; JWT 8 hours; wrong role refused	`backend/src/controllers/auth.controller.js:26, 37-74`
Role re-read from the database; account must be active	`backend/src/middleware/auth.js:18-25`
Role gate answers 403	`backend/src/middleware/requireRole.js:1-17`
`/api/students` field-officer-only, by design and by test	`backend/src/routes/students.routes.js:8`; `backend/tests/roleAuthorization.test.js`
Registration validates eight fields, resolves policy before the transaction, takes an advisory lock, screens all candidates, returns without the token	`backend/src/controllers/students.controller.js:15-161`
Five approved schools	`backend/src/config/schools.js`; `students_school_supported_check` in `backend/src/db/schema.sql:43-45`
Screening batches at 200, validates count/range/version, raises a typed error on failure	`backend/src/services/mlDuplicateReview.js:4, 84-117`
Predictor is a child process with a 30 s timeout and fail-closed handling	`backend/src/services/duplicateCheck.js:241-301`
Aggregation `top-1` default, corroboration rules, audit counters	`backend/src/services/duplicateAggregation.js:36-144`
Priority shared by registration and queue	`backend/src/services/mlDuplicateReview.js:50-59`
Identifier masking and audited reveal	`backend/src/controllers/duplicateFlags.controller.js:37-42, 434-446`

Claim	Evidence
Terminal review decision with reason, event and audit rows	`backend/src/controllers/duplicateFlags.controller.js:452-503`
Approval gate on verification and duplicates (both sides of a pair)	`backend/src/controllers/bursaryAwards.controller.js:173-189`
Payment gate, balance ceiling, `issued` state	`backend/src/controllers/disbursements.controller.js:65-215`
Payment transition legality and discrepancy arithmetic	`backend/src/services/bursaryRules.js:1-52`
Eligibility scoring rules	`backend/src/services/eligibility.js:1-102`; `docs/eligibility-rules.md`
QR token generation, fragment URL, stripping from responses	`backend/src/controllers/students.controller.js:58-60`; `backend/src/config/publicAppUrl.js:164-177`; `backend/src/models/student.model.js:3-7`
Public verification returns only `{found}`; malformed and unknown indistinguishable	`backend/src/controllers/verification.controller.js:1-24`
Certificate snapshot, four verification states, revocation constraints	`backend/src/controllers/certificates.controller.js:53-89, 95-270`; `backend/src/models/bursaryCertificate.model.js`; migration `20261031_bursary_certificates.sql`
Separate rate-limit budgets and shared counter implementation	`backend/src/middleware/rateLimitWindow.js`; `verificationRateLimit.js`; `certificateRateLimit.js`; `loginRateLimit.js`
Security headers, CORS allowlist, body limit, single registration of `/api/verify`	`backend/src/index.js:33-67`
Sanitised error responses	`backend/src/middleware/errorHandler.js:59-78`
Readiness contract and fail-closed start-up	`backend/src/index.js:73-125`; `backend/src/services/databaseReadiness.js`; `backend/src/services/databaseStartup.js`
Schema: eight baseline tables plus migration tables, constraints and indexes	`backend/src/db/schema.sql`; fourteen files in `backend/src/db/migrations/`
Frontend routes, role-protected layout, public pages	`frontend/src/App.jsx`; `frontend/src/components/AppLayout.jsx`; `frontend/src/pages/Verification.jsx`; `frontend/src/pages/CertificateVerification.jsx`
Officer-facing screening sentence without model internals	`frontend/src/pages/StudentRegistration.jsx:7-22, 65-75`
Six-feature contract and derivation	`ml-service/feature_engineering.py:11-117`
Artefact loading, version hashing, `predict_pairs`	`ml-service/predict.py:14-98`
Container build, inference gate, no CUDA, no secrets	`Dockerfile:1-48`
Image publication gates	`.github/workflows/publish-api-image.yml`
Deployment topology (Render equivalent, documented for reference)	`render.yaml`

3. Test artefact index

Artefact	Location	Evidence it provides	Reliability note
24 Jest suites, 151 tests	`backend/tests/*.test.js` (inventory in Chapter 5 Table 5.9)	Behaviour through the real HTTP surface against PostgreSQL	Counts recorded 2026-11-01; the suites were read during this pass to confirm the described assertions exist
Database guard	`backend/tests/testDatabaseGuard.js`	Fails closed unless the target is the loopback `sbms_test` database	Read directly

Artefact	Location	Evidence it provides	Reliability note
Native test files	`backend/test/databaseReadiness.node.js`, `databaseStartup.node.test.js`, `databaseBootstrap.node.test.js`, `errorHandler.unit.test.js`, `verification.unit.test.js`, `migrate.unit.test.js`, `jwtSecret.validation.node.cjs`	43 in-process tests	Read directly; result recorded 2026-10-31
Out-of-process smoke	`backend/test/api.smoke.test.js`	3 checks against a running API	Read directly
ML service tests	`ml-service/tests/*.py` (6 files)	33 tests: features, operating point, artefact loading, training integrity, reliability	Read directly; result recorded 2026-10-31
Research helper tests	`ml-service/experiments/sbms_dataset/test_experiment.py`	7 tests	Recorded
Browser harnesses	`scripts/browser-smoke.mjs`, `qr-smoke.mjs`, `registration-fields-smoke.mjs`, `admin-reveal-smoke.mjs`	23, 9, 11 and (unrun) reveal checks	Harnesses read; results taken from project records
Training and evaluation metrics	`ml-service/evaluation/realistic_pairs_xgboost_training_metrics.json`	Hyperparameters, holdout metrics, confusion matrix, generated scenarios, limitations	Read directly; every figure in Chapter 5 §5.5.5 comes from this file
Operating-point evidence	`ml-service/evaluation/operating_point_2026-10-30.json`	Threshold table, negative-family rates, masking, blocking projections	Summarised in `docs/ml-decision-sheet-2026-10-30.md`, which was read
False-positive baseline	`ml-service/evaluation/fpr_baseline_2026-10-30.json`; `docs/ml-evaluation-baseline-2026-10.md`	0.35% of 2 000 random unrelated pairs above 0.50	Read through the decision sheet
Training integrity	`ml-service/evaluation/training_integrity_report.json`; `ml-service/check_training_integrity.py`	Group holdout, no memorisation, no single-feature label determination	Read via tests and reports
Research-only artefacts	`ml-service/model/research/rf_attachment_style_candidate.joblib`; `ml-service/experiments/sbms_dataset/artifacts/sbms_dataset_xgb_research_only.joblib`	Kept separate from production; metrics in `ML_RELIABILITY_REPORT.md` and `reports/sbms_dataset_experiment.md`	Read directly; **not** attributed to the deployed system
Production artefact	`ml-service/model/xgb_duplicate_model.joblib`	421 089 bytes, SHA-256 `6ec9f2a3…`	Presence, size and tracked status verified with `git ls-files` and a file check

4. Document evidence index

Document	Used for	Reliability assessment
`docs/FINAL_TEST_REPORT.md`	Test counts, environment, database state, outstanding tests	**Current**, but contains two generations of result; §10 supersedes §1–§3
`docs/business-workflow-verification-2026-10-30.md`	Workflow chain, the payment-clearance defect, behaviour observations	Current and specific
`docs/frontend-verification-2026-10-30.md`	23 browser checks, the priority/`risk_level` defects	Current
`docs/qr-verification-2026-10-30.md`	QR design, 10 API + 9 browser checks, the camera blocker, residual gaps	Current

Document	Used for	Reliability assessment
`docs/bursary-certificates-2026-11-01.md`	Certificate design, endpoint roles, automated coverage, unverified items	Current
`docs/azure-deployment-log-2026-10-31.md`	Deployment identity, resource configuration, live checks, the production QR defect, carry-forward items	Current; §7–§9 record unfinished operator tasks
`docs/ml-decision-sheet-2026-10-30.md`	Operating point, negative families, masking, blocking, D1–D5	Current; explicitly labels all labels synthetic
`docs/ml-evaluation-policy.md`	The five unanswered decisions and where each lands in code	Current
`docs/ml-evaluation-baseline-2026-10.md`	0.35% false-positive baseline and queue arithmetic	Current
`docs/ml-integration-and-school-identifiers.md`	School identifier problem; model interface	Current
`docs/ml_audit.md`	The supplied external dataset and the ten-column research model	Current
`ml-service/ML_RELIABILITY_REPORT.md`	Random Forest candidate metrics; the `raw_pairs.csv` label shortcut	Current, and the basis for excluding that dataset from training
`ml-service/reports/sbms_dataset_experiment.md`	Research-only experiment metrics	Current; describes a model the system does not load
`docs/SBMS_PROJECT_HANDOVER_2026-10.md`	Status matrix, defect history, phase plans	Current as a record; §12+ supersedes its earlier sections
`SBMS_O10_O11_IMPLEMENTATION_REPORT.md` (untracked)	The `/api/students` policy decision and the `/api/verify` double-registration fix	Reliable but **untracked**; gap G-28
`SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` (untracked)	Confirmation that the SBMS-01…18 wording is unrecoverable	Reliable but **untracked**; gap G-28
`docs/verification.md`	**Superseded claims** about the predictor	Do **not** cite; gap G-08/G-23
`docs/uat-test-plan.md`	Five `PASS` results without participants	Do **not** present as acceptance evidence; gap G-24
`docs/database-recovery-rehearsal-2026-10-31.md`	Restore rehearsal procedure and result	Current
`docs/eligibility-rules.md`	Scoring factors and categories	Current; agrees with `services/eligibility.js`

5. Verification status of the chapters' major claims

Claim in the chapters	Status	How it was established
The production classifier is XGBoost over six features, artefact `6ec9f2a3…`	**VERIFIED**	`predict.py`, `feature_engineering.py`, metrics JSON, artefact size and hash
Training used 12 000 pairs with a grouped 80/20 split and an independent 12 000-pair holdout	**VERIFIED**	`realistic_pairs_xgboost_training_metrics.json`
Holdout metrics: accuracy 0.9078, precision 0.9204, recall 0.8927, F1 0.9063, PR-AUC 0.9727, ROC-AUC 0.9685, CM 5 356/463/5 537/644	**VERIFIED**	Same JSON file
The live cut-off is 0.50 and is unvalidated	**VERIFIED**	`config/settings.js:8`; `thresholds_validated: false` in the queue response; `docs/ml-evaluation-policy.md`
Registration compares against the whole registry	**VERIFIED**	`students.controller.js:69-73` (no filter); decision sheet §6
At most one flag per registration by default	**VERIFIED**	`duplicateAggregation.js:36-39, 86-117`

Claim in the chapters	Status	How it was established
A duplicate blocks approval and payment, on either side of the pair	**VERIFIED**	Controller SQL; `workflow.test.js` checks 3–5
Payments are recorded as issued, cleared separately with evidence	**VERIFIED**	`disbursements.controller.js`; `bursaryRules.js`; database `CHECK` constraints
Receipt acknowledgement is officer-assisted	**VERIFIED**	`acknowledgement_method = 'officer_assisted'` required by constraint
Student QR and certificate QR are separate namespaces, endpoints, budgets and pages	**VERIFIED**	`index.js:65-66`; controllers; two page components; two rate limiters
No beneficiary data appears in any public response	**VERIFIED**	`verification.controller.js`; `publicView()`; tests
24 suites / 151 tests passed on 2026-11-01	**RECORDED** (not re-run)	`docs/FINAL_TEST_REPORT.md` §10; `docs/bursary-certificates-2026-11-01.md` §7
43/43 native, 3/3 smoke, 33/33 and 7/7 Python	**RECORDED** (not re-run)	`docs/FINAL_TEST_REPORT.md` §2, §4
23/23, 9/9 and 11/11 browser checks	**RECORDED** (not re-run)	Frontend, QR and deployment records
The system is deployed at the stated Azure endpoints	**RECORDED**	`docs/azure-deployment-log-2026-10-31.md` §1; not independently probed in this pass
Nine defects were found and fixed during testing	**VERIFIED as records**	Each defect is documented with its root cause
The `/api/students` restriction is intentional, not a defect	**VERIFIED**	Code, frontend call sites, regression test, O10/O11 report
The Random Forest is not in production	**VERIFIED**	`predict.py` loads only `xgb_duplicate_model.joblib`
Real-world detection performance is unknown	**VERIFIED as unknown**	No adjudicated labels in the repository or either database
A phone can scan a printed QR code	**NOT VERIFIED**	No camera run recorded; gap G-11
500+ students can be screened within three seconds	**NOT VERIFIED**	No load test; gap G-12
Users find the system usable	**NOT VERIFIED**	No user study; gap G-06

6. What was checked during this pass, and what was not

Checked directly:

1. Every code location cited in the chapters was opened and the cited behaviour confirmed.
2. The suite inventory was regenerated by counting `test(`/`it(` occurrences per file: 24 files, 151 tests, matching the recorded total.
3. Test *names* for the fourteen most relevant suites were extracted and used to write Chapter 5 Table 5.9, so the table describes assertions that exist rather than assertions that were imagined.
4. The training metrics JSON was parsed field by field; every number in Chapter 5 §5.5.5 traces to it.
5. The artefact's size, presence and tracked status were verified (`git ls-files` reports it as tracked although `.gitignore` lists the pattern — gap G-27).
6. `git status` and `git log` fixed the revision under description and recorded the two untracked reports.
7. The proposal PDF was extracted and read for objectives, scope, methodology and the four-feature Random Forest specification, so the deviation in Chapter 4 §4.1 is documented rather than assumed.

Not performed:

1. No test suite was executed, in any language.
2. No database query, migration, backup or restore was run.

3. No deployment, restart or configuration change was made.
4. No screenshot was captured and no figure was drawn.
5. No network call was made to the deployed endpoints; their status is taken from the deployment record.
6. No interview, survey or acceptance session was conducted.

---

7. How to re-verify the main claims

Commands are given as the project records them; they were **not** run in this pass.

```
# 1. Fix the revision under description
git rev-parse HEAD            # expect d6b5032
git status --short            # expect only the two untracked root reports

# 2. Backend suite (needs backend/.env.test with NODE_ENV, DATABASE_URL,
#    SBMS_TEST_DATABASE_URL, and PUBLIC_APP_URL set to an https origin)
cd backend
npx jest --runInBand tests    # expect 24 suites / 151 tests

# 3. Native and smoke tests
node --test backend/test/databaseReadiness.node.js backend/test/databaseStartup.node.test.js
backend/test/databaseBootstrap.node.test.js backend/test/errorHandler.unit.test.js
backend/test/verification.unit.test.js backend/test/migrate.unit.test.js
backend/test/jwtSecret.validation.node.cjs
# expect 43/43

# 4. Frontend build
npm.cmd --prefix frontend run build     # expect exit 0

# 5. ML tests and evaluation
ml-service\.venv\Scripts\python.exe -m unittest discover -s ml-service/tests -v      # expect 33
ml-service\.venv\Scripts\python.exe -m unittest discover -s ml-service/experiments/sbms_dataset -p
test_experiment.py
cd ml-service; .\.venv\Scripts\python.exe .\evaluate_operating_point.py              # regenerates the
operating-point JSON

# 6. Artefact identity
Get-FileHash ml-service\model\xgb_duplicate_model.joblib -Algorithm SHA256
# expect 6EC9F2A3CBE6ED0DCF80C79B15F47364D9F4F7BD3E27DA25C690D0436BB0A6F9
```

Note on disambiguation: `backend/tests/` (plural) holds the Jest suites and `backend/test/` (singular, except for the Jest-named files) holds the native files. Running the native command from the repository root rather than from `backend/` changes the relative paths and will fail for that reason alone.

---

8. Residual risks to the report's accuracy

Risk	Consequence	Mitigation already applied in the chapters
Recorded test counts may not reproduce on a different machine	A reader re-running the suite could see 149–150/151	Chapter 5 §5.4.3 states the instability, its mechanism and the isolation results (gap G-20)
Project records may be superseded after this pass	A citation could point at a stale statement	Every citation names a dated document, and the two known-stale documents are named as stale

Risk	Consequence	Mitigation already applied in the chapters
The deployment identity may change (resources stopped or deleted)	"Deployed" claims could become untrue	The chapters describe the deployment as demonstrated on a stated date and note the stopped-database availability model
Figures may be drawn in a way that contradicts the specifications	Visual content would misstate the system	Each figure carries an explicit content specification, including what must **not** appear
A reader may treat synthetic metrics as accuracy	Over-claiming detection capability	Every metric is labelled synthetic, and Chapter 5 §5.5.5 lists what it does not establish
Chapters 1–3 may describe features that the implementation does not have	Cross-chapter contradiction	The known conflicts are recorded here and in the gap register; Chapters 1–3 were not edited, as that is outside this task

9. Cross-chapter consistency checks performed

Check	Result
Problem in Chapter One matches the solution in Chapters Four and Five	Consistent: within-registry duplicate detection plus an evidence-bearing payment chain; the cross-constituency limitation is stated in Chapter Five §5.6.2
Objectives in Chapter One are addressed in Four, Five and Six	Consistent: each of the four objectives appears in Chapter Four §4.2, is evaluated in Chapter Five §5.5.2 and concluded in Chapter Six §6.2.3–§6.2.6
Chapter Three methodology agrees with the implementation and evaluation	Consistent with DSR plus Agile as described in the proposal §3.2–§3.3; the classifier differs (gap G-21) and the deployment architecture differs (gap G-22), both recorded
Requirements in Chapter Four agree with the implemented system	Consistent: each FR cites a route, service or table; unsatisfied items are marked in Table 4.1
Diagrams agree with the architecture and workflows	The figure specifications are derived from controller and page code; drawing them remains the author's task (gap G-03)
Implementation in Chapter Five agrees with the design in Chapter Four	Consistent: Table 5.3 maps each Chapter Four module to its implementation and test
Testing and evaluation support the conclusions in Chapter Six	Consistent: the conclusion table in `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md` §5 cites the supporting section for each conclusion
Module, role and entity names are used consistently	Consistent: `admin`, `field_officer`, `students`, `duplicate_flags`, `duplicate_review_events`, `bursary_awards`, `bursary_award_verification_events`, `disbursements`, `disbursement_reconciliation_events`, `bursary_certificates`, `audit_logs`, `reports`

10. Summary of the evidence position

The chapters report a system whose code, schema, tests and deployment records are internally consistent and independently checkable for their implementation claims, and whose quantitative claims fall into three groups: (a) implementation facts, which were verified by reading the code; (b) test results, which are recorded with dates and commands but were not re-executed in this pass; and (c) model metrics, which were read directly from the artefact's own report and are explicitly properties of a synthetic generator. The claims that cannot be supported — real-world detection accuracy, physical QR scanning, scale performance, and user acceptance — are named as unsupported rather than inferred, and each is assigned a gap-register item with a route to closure.

OBJECTIVE–REQUIREMENT–TEST TRACEABILITY

Purpose. This file maps each specific objective from Chapter One to the requirements that operationalise it, the code that implements them, the design elements that explain them, the tests that demonstrate them, and the conclusion that reports the result. It is the audit trail supporting the claim in Chapter Six that no conclusion rests on untested functionality.

References: `Project Proposal- 169613.pdf` §1.3.1 (objectives), Chapter 4 of this documentation set (requirements and design), Chapter 5 (implementation and testing), Chapter 6 (conclusions), repository at `fix/sbms-completion-work` @ `d6b5032`.

Status vocabulary: V = verified by an executed test or recorded observation; **P** = partially verified; **N** = implemented but not tested; **X** = not implemented (by scope or as a recorded gap). Where a requirement is implemented and exercised incidentally but the property it names is never asserted, the row is marked **P** and the missing assertion is named; `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` uses the same verdict for the same requirement, so the two files can be read side by side without reconciling numbers.

---

1. Objective → requirement → evidence map

Objective (i): investigate the existing challenges of manual registration and tracking

Element	Content
Requirements serving the objective	The register as a whole encodes the identified problems; the objective itself is documentary rather than functional
Implementation evidence	Chapter Two document analysis (Auditor-General, Controller of Budget, NG-CDF); requirement register derived from it in Chapter 4 §4.2.1
Design elements	Chapter 4 §4.3.1 (context diagram, which fixes the scope of the problem addressed)
Test evidence	Not applicable — investigative, not behavioural
Conclusion	Chapter 6 §6.2.3: **partially achieved** (secondary evidence only; no fieldwork)
Status	**P**

Objective (ii): design architecture, database schema and ML pipeline

Requirement / design element	Implementation	Demonstration	Status
Three-tier architecture (Ch.4 §4.4.1, Fig. 4.12)	`backend/src/index.js`, `backend/src/routes/*`, `frontend/src/**`, `ml-service/predict.py`	Live deployment checks (Ch.5 §5.3.12); `api.smoke.test.js` 3/3	**V**
Relational schema with integrity constraints (Ch.4 §4.6, Fig. 4.10)	`db/schema.sql`, 14 migration files	`schemaReadiness.test.js` (5); `workflow.test.js` (11) exercise the constraints; recorded 12 tables / 56 constraints	**V**
Six-feature ML pipeline shared by training and serving (Ch.4 §4.3.11, §4.4.4)	`ml-service/feature_engineering.py`, `predict.py`	`ml-service/tests` 33/33 including feature-consistency and artefact loading; `mlPredictor.integration.test.js`	**V**
Random Forest classifier as specified in the proposal	Not in production; research candidate only	`ML_RELIABILITY_REPORT.md` records the candidate's metrics	**X (deviation, Ch.6 §6.2.4)**
Deployment architecture (Ch.4 §4.7, Fig. 4.24)	`Dockerfile`, `.github/workflows/publish-api-image.yml`, Azure resources	Image gates; live deployment checks	**V**

Objective (iii): implement registration, unique identifiers, QR identification and RBAC

Requirement	Implementation	Demonstration	Status
FR-05, FR-06 registration and school constraint	`students.controller.js`, `student.model.js`	`students.test.js` (9); `browser-smoke` checks 2–3	**V**
FR-20 unique QR token and image	`students.controller.js:58-60`, `publicAppUrl.js`	`verification.test.js` checks 1–3	**V**
FR-21 public verification, existence only	`verification.controller.js`	`verification.test.js` (10), `verificationRoute.test.js` (4), `qr-smoke` 9/9	**V**
FR-01 to FR-04 authentication and RBAC	`auth.controller.js`, `middleware/auth.js`, `requireRole.js`	`auth.test.js` (4), `roleAuthorization.test.js` (6), `securityMiddleware.test.js` (9)	**V**
FR-22 to FR-24 certificate issue, verification, `not_current`	`certificates.controller.js`, `bursaryCertificate.model.js`	`bursaryCertificates.test.js` (12), `certificateVerificationRoute.test.js` (5)	**V**
Physical QR scanning by a phone	Implemented artefact; no camera run	—	**N (gap G-11)**

Objective (iv): validate, test and evaluate effectiveness

Requirement / claim	Implementation	Demonstration	Status
FR-07 screening on registration	`students.controller.js:69-73`, `mlDuplicateReview.js`	`duplicateFlags.test.js`, `mlPredictor.integration.test.js`, `performance.test.js`	**V** except at registry scale (gap G-12)
FR-08 no automatic rejection	No delete or status-change path exists	`workflow.test.js` check 1; `browser-smoke` check 5	**V**
FR-09 provenance per case	`duplicate_flags` columns; migration `20261022`	`duplicateFlags.test.js`	**V**
FR-10 masked identifiers, audited reveal	`duplicateFlags.controller.js:37-42, 434-446`	`duplicateFlags.test.js`	**V**
FR-11 decision with reason, terminal case	`duplicateFlags.controller.js:452-503`	`duplicateFlags.test.js`; `workflow.test.js` checks 4–6	**V**
FR-15, FR-16 duplicate gates on approval and payment	`bursaryAwards.controller.js:173-189`; `disbursements.controller.js`	`workflow.test.js` checks 2–4; `disbursements.test.js` check 4	**V**
FR-17, FR-18 clearance evidence and discrepancy	`bursaryRules.js`; `disbursements.controller.js`	`disbursementReconciliation.controller.test.js` (4); `workflow.test.js` checks 8–9	**V**
FR-19 exception derivation	`bursaryRules.js:32-52`	`workflow.test.js`; `bursaryRules.test.js`	**P** (not all eleven reason codes asserted)
FR-25 dashboard, FR-26 report	`adminDashboard.controller.js`, `reports.controller.js`	`roleAuthorization.test.js` (authorisation); arithmetic untested	**P**
FR-27 audit trail	`services/auditLog.js`	`workflow.test.js` check 10 (ten actions)	**V**
FR-12 eligibility scoring	`services/eligibility.js`	Route and role only	**P** (gap G-17)
Detection effectiveness on real records	—	Synthetic holdout only	**X** with existing evidence (gap G-18)

---

2. Non-functional requirement traceability

NFR	Implementation location	Demonstration	Status
NFR-01 screening ≤ 3 s	Batching in `mlDuplicateReview.js`	`performance.test.js` (50 candidates)	**P** (small registry only)

NFR	Implementation location	Demonstration	Status
NFR-02 password hashing	`auth.controller.js:26`	`auth.test.js`	**V**
NFR-03 JWT signing and expiry	`auth.controller.js:55-59`; `jwtSecret.js`	`auth.test.js`; `jwtSecret.validation.node.cjs`	**V**
NFR-04 role not trusted from token	`middleware/auth.js:20-24`	`roleAuthorization.test.js`	**V**
NFR-05 identifier masking	`duplicateFlags.controller.js:37-42`	`duplicateFlags.test.js`	**V**
NFR-06 no beneficiary data in public responses	`verification.controller.js`; `certificates.controller.js:53-89`	`verification.test.js`; `bursaryCertificates.test.js`	**V**
NFR-07 public rate limits, separate budgets	`rateLimitWindow.js`, `verificationRateLimit.js`, `certificateRateLimit.js`	`verificationRoute.test.js`; `certificateVerificationRoute.test.js`	**V** (in-process only)
NFR-08 sign-in rate limit	`loginRateLimit.js`	`loginRateLimit.test.js`	**V**
NFR-09 body size limit	`index.js:64`	`securityMiddleware.test.js`	**V**
NFR-10 no internal detail in errors	`errorHandler.js`	`securityMiddleware.test.js`	**V**
NFR-11 CORS allowlist	`config/cors.js`; `index.js:40-47`	`securityMiddleware.test.js`	**V**
NFR-12 database-enforced integrity	`schema.sql`; migrations	`workflow.test.js`; `bursaryCertificates.test.js`	**V**
NFR-13 registration serialisation	advisory lock in `students.controller.js:68`	Every registration test passes through the lock; no test asserts isolation	**P** (lock present; no concurrency assertion; gap G-12)
NFR-14 usability	`AppLayout.jsx`; screen components	`browser-smoke` (behaviour, not usability)	**P** (renders and behaves; no usability study; gap G-06)
NFR-15 maintainability	shared feature module; ordered migrations; three test frameworks	Builds and suites	**V**
NFR-16 cost envelope	`render.yaml`; deployment log	Deployment record	**P** (configuration recorded; no billed month reconciled)
NFR-17 human oversight	role-gated decision endpoints	`workflow.test.js` check 11	**V**
NFR-18 recovery	`docs/database-recovery-rehearsal-2026-10-31.md`	Rehearsal record	**P** (no schedule; gap G-19)
NFR-19 browser compatibility	SPA over HTTPS	Chrome only	**P** (gap G-16)
NFR-20 predictable degradation	503 on predictor failure	`students.test.js`	**V**
NFR-21 audit completeness	`auditLog.js`	`workflow.test.js` check 10	**P** (attributed in tested flows; no immutability control)

3. Figure and table traceability

Design element	Requirement(s) it explains	Where the behaviour is demonstrated
Fig. 4.1 context diagram	Scope of FR-01 to FR-37 (what is inside the boundary)	Ch.5 §5.6.2 (boundaries)
Fig. 4.2 use case diagram	FR-01 to FR-30	Map in Ch.4 §4.3.2
Fig. 4.3 registration activity	FR-05 to FR-11	`students.test.js`; `workflow.test.js` check 1
Fig. 4.4 review/approval/payment activity	FR-11, FR-15 to FR-19	`workflow.test.js` checks 2–9

Design element	Requirement(s) it explains	Where the behaviour is demonstrated
Fig. 4.5 class diagram	Whole domain	Suite inventory in Ch.5 §5.4.3
Fig. 4.6 registration SSD	FR-05, FR-07, FR-20	`verification.test.js`; `students.test.js`
Fig. 4.7 review sequence	FR-10, FR-11	`duplicateFlags.test.js`
Fig. 4.8 approval-to-certificate sequence	FR-13 to FR-18, FR-22 to FR-24	`workflow.test.js`; `bursaryCertificates.test.js`
Fig. 4.9 public verification sequence	FR-21, FR-23, FR-24	`verification.test.js`; `certificateVerificationRoute.test.js`
Fig. 4.10 ERD	FR-09 to FR-27 (storage)	`schemaReadiness.test.js`
Fig. 4.11 ML pipeline	FR-07, FR-09	`ml-service/tests` (33); `duplicateAggregation.test.js`
Fig. 4.12 architecture	All	Deployment checks
Figs. 4.13–4.23 screens	FR-01 to FR-26 (interface realisation)	`browser-smoke` 23/23; `qr-smoke` 9/9; `registration-fields-smoke` 11/11
Fig. 4.24 deployment	NFR-07, NFR-16	Deployment log §1–§2

---

4. Test suite → requirement coverage matrix

Cell values give the number of tests in that suite that bear on the requirement group; a blank means the suite is not relevant. Suite totals sum to 151.

Suite (tests)	Auth / RBAC	Registration	Screening	Review	Awards	Payments	QR	Certificates	Reporting	Ops
`auth` (4)	4									
`students` (9)	2	5	2							
`duplicateFlags` (9)	1		1	6		1				
`duplicateAggregation` (13)			13							
`mlDuplicateReview` (8)			8							
`predictorDateNormalization` (6)			6							
`duplicateCheck` (15)		1	14							
`mlPredictor.integration` (1)			1							
`workflow` (11)	1		2	3	2	3				
`bursaryAwards.controller` (5)	1			1	3					
`disbursements` (4)	1			1		2				
`disbursementReconciliation.controller` (4)						4				
`bursaryRules` (4)						4				
`bursaryCertificates` (12)	1				2			9		
`certificateVerificationRoute` (5)								5		
`verification` (10)							10			
`verificationRoute` (4)							4			
`roleAuthorization` (6)	5								1	
`securityMiddleware` (9)	3									6
`schemaReadiness` (5)										5

Suite (tests)	Auth / RBAC	Registration	Screening	Review	Awards	Payments	QR	Certificates	Reporting	Ops
`loginRateLimit` (1)	1									
`performance` (1)			1							
`kenyanSyntheticDataset` (4)			4							
`health` (1)										1

5. Conclusion → evidence traceability

Conclusion (Chapter 6)	Evidence base	Gaps
§6.2.1 problem addressed	Ch.1 (as approved), Ch.2 literature, Ch.4 §4.2.1 requirement derivation	—
§6.2.2 purpose and DSR approach	Ch.3 methodology (proposal §3.2–§3.3); artefact and evaluation in Ch.4–5	—
§6.2.3 objective (i) partially achieved	Ch.4 §4.2.1; absence of primary fieldwork	G-06 (no user study)
§6.2.4 objective (ii) achieved with deviation	Ch.4 §4.4–§4.7; `feature_engineering.py`; `predict.py`; migration inventory	G-21
§6.2.5 objective (iii) achieved	Ch.5 §5.3.2–§5.3.9, §5.3.12; suites listed in §5.4.3; deployment checks	G-11 (camera scan)
§6.2.6 objective (iv) partially achieved	Ch.5 §5.4 (all suites), §5.5.2, §5.5.5	G-12, G-18
§6.2.7 engineering quality	Ch.5 §5.4.5 (security, database), §5.5.3, §5.5.4	G-17
§6.2.8 contribution	Ch.5 §5.5.5 (aggregation finding); Ch.4 §4.4.4	G-18
§6.2.9 boundaries	Ch.5 §5.6.2–§5.6.6	G-12, G-13, G-19

6. Traceability exceptions to note in the report

1. **Two requirements are demonstrated only by inspection** (FR-26 report content; FR-19 all reason codes). The report says so rather than marking them verified.
2. **Two requirements have no behavioural test** (NFR-13 concurrency; NFR-14 usability). Both are stated as unverified in Chapter Five and carried into the gap register as G-12 and G-06.
3. **One objective's evidence is documentary only** (objective (i)). Chapter Six states this explicitly.
4. **One proposal requirement is not implemented and is reported as a deviation**, not as an omission (Random Forest; G-21).
5. **The production model and the research artefacts are traced separately** so that no metric from `ml-service/experiments/` or `model/research/` is attributed to the deployed system.

SBMS COMPREHENSIVE INFORMATION REPORT FOR CHAPTERS FOUR, FIVE AND SIX

Document type: evidence dossier and drafting resource **System:** Student Beneficiary Management System (SBMS) **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Prepared:** 2026-11-02 **Institution context:** capstone project report; Design Science Research through an Agile cycle

Document control

Item	Value
This report	`SBMS_COMPREHENSIVE_CHAPTERS_4_5_6_INFORMATION_REPORT.md` and `.pdf`
Ten supporting data files	`SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md`, `SBMS_SYSTEM_ARCHITECTURE_AND_MODULE_MAP.md`, `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md`, `SBMS_API_ENDPOINT_CATALOGUE.md`, `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md`, `SBMS_TEST_EVIDENCE_CATALOGUE.md`, `SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md`, `SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md`, `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md`, `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md`
Earlier deliverables re-used, not duplicated	`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md`, `CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md`, `CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md`, `OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md`, `DOCUMENTATION_GAP_REGISTER.md`, `EVIDENCE_AND_VERIFICATION_REPORT.md`, `FIGURE_PLACEHOLDER_CHECKLIST.md`
Evidence position	Implementation facts read from code at the named revision; test counts taken from dated run records and **not re-executed**; ML metrics synthetic only; the unsupported claims are stated in Part III §24 and itemised in `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §4
What this document is not	It is not a replacement for the three chapters. It is the evidence base and drafting aid for them

How to use this report

1. **Part I** supplies everything Chapter Four needs: scope, actors, requirement counts, the analysis and design facts each diagram must encode, the interface inventory, the schema, and the deployment architecture.
2. **Part II** supplies everything Chapter Five needs: the implementation environment, the feature-by-feature implementation map, the test inventory with what each suite proves, the defect log, and the evaluation against objectives and requirements.
3. **Part III** supplies everything Chapter Six needs: an objective-by-objective conclusion with an honest status, the stated contribution, recommendations, and future work with owners.
4. **Part IV** is the assurance layer: a completeness matrix (requested area → status → where it is), the gap summary, reproduction commands and the document map.
5. Every factual statement names its source. Where a statement rests on code reading only, it says so. Where evidence is absent, the text says *not verified* rather than implying success.

Executive summary of the evidence position

Dimension	Position at this revision
Code	Complete three-tier system with a deployed demonstration instance; 45 HTTP endpoints, 13 tables, 14 forward migrations
Automated tests	151 Jest tests in 24 suites (2026-11-01), 43 native in-process tests, 3 live-HTTP smoke tests, 40 Python tests, 43 browser checks across three harnesses
Build	Frontend production build exits 0 (2 006 modules); API image built by CI and running on Azure
Machine learning	One deployed XGBoost artifact over six features; synthetic holdout F1 0.9063 at cut-off 0.50; **no real-world metric exists**
Deployment	Live on Azure (UAE North) as a demonstration instance, verified on 2026-10-31 and 2026-11-01

Dimension	Position at this revision
User evidence	**None.** No user acceptance study, no usability test, no fieldwork
Scale evidence	**None.** No load test; the only timing is 22.67 ms for 50 candidate pairs
Physical QR evidence	**None.** No phone-camera scan of a printed code was recorded

Three sentences summarise the honest position. The system is built, tested at API and browser level, and deployed. Its duplicate-detection model is evaluated only against synthetic data, so no accuracy claim about real beneficiaries is possible. Its usability, scale behaviour and physical QR identification are unverified and must be reported as such.

PART I — EVIDENCE FOR CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN

1. System identity and problem framing

SBMS exists to replace a manual bursary register with a system that (a) registers a beneficiary once, (b) detects a probable repeat application before money is committed, (c) routes every duplicate signal to a human, and (d) leaves an audit trail from registration to receipt. The design's central control is therefore not the classifier: it is the **gate**. A model score may create a case, but only an administrator's decision can release an award for payment, and only a verified beneficiary may be approved at all.

Element	Fact at this revision
Presentation	React 18 SPA, Vite, `react-router-dom`; 14 screens across 16 routes
Application	Node 22 + Express 4; 13 routers, 13 controllers, 10 services, 5 models
Persistence	PostgreSQL 17; 13 tables; 14 forward migrations plus one rollback script
Inference	Python 3.11 + scikit-learn + XGBoost, one artifact, spawned as a stdio subprocess
Delivery	One container image (API plus ML) on Azure Container Apps; static frontend on App Service F1
Identity	JWT, 8-hour expiry, role re-read from the database on every request

The two material departures from the approved proposal must be stated in Chapter Four, not discovered in Chapter Six. First, the deployed classifier is **XGBoost over six features**, not the specified Random Forest over four; the Random Forest survives only as an unevaluated research candidate. Second, the system **exceeds** the proposal on award reconciliation and certificates while **falling short** of the proposal's implied evaluation design, because registration still compares a new record against the entire registry with no candidate blocking.

2. Requirements: what the chapter can state as fact

Group	Count	Verified	Partial	Not implemented
Functional requirements delivered (FR-01 to FR-30)	30	24	6	0
Proposal requirements carried as deviations (FR-31 to FR-33)	3	0	0	3
Functional gaps stated as gaps (FR-34 to FR-37)	4	0	0	4
Non-functional requirements (NFR-01 to NFR-21)	21	14	7	0

The register, with the statement, actor, priority, implementation location, test and status of each requirement, is `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §6-§7. Chapter Four should reproduce the register as its Table 4.1 and Table 4.2 rather than restating requirements in prose, because prose drifts and the register does not.

The six gaps that must appear as gaps: no candidate blocking (FR-34), no QR re-issue (FR-35), no token expiry (FR-36), no payment execution (FR-37), no NEMIS integration (FR-32), no offline or native mobile operation (FR-33).

3. Actors and their interests

Actor	Interest	Interface surface	Evidence of role enforcement
Field Officer	Register accurately, see whether a case was raised, record evidence, submit awards	`/students`, `/students/register`, `/eligibility`, `/disbursements`, `/field-officer`	403 on adjudication and approval (`workflow.test.js` #11)
Administrator	Decide cases, approve awards, control payments, issue certificates, read the audit trail	`/dashboard`, `/duplicate-review`, `/disbursements`, `/reports`, `/settings`, `/audit`, `/users`	403 on registration and award submission (`workflow.test.js` #11)
Public verifier	Confirm a printed code is genuine without learning anything about the holder	`/verify`, `/verify-certificate`	Boolean and authenticity facts only (15 tests)

Actor	Interest	Interface surface	Evidence of role enforcement
Operator	Apply migrations, manage secrets, start and stop the deployment, restore a backup	CLI, `deploy/azure/**`	Start-up guards; readiness contract
Programme owner	Set the cut-off policy, adjudication protocol and acknowledgement deadline	Not in the interface	Open decisions D1-D5

The role split is by *duty*, not seniority: the officer who registers a beneficiary cannot approve money for that beneficiary, and the administrator who approves cannot register. That separation is the design's answer to the collusion and error risk described in Chapter Two.

4. Analysis artefacts: what each diagram must encode

Chapter Four specifies eleven analysis artefacts. Each must be drawn from the implemented system, so each row gives the facts the drawing has to carry and the code that establishes them.

Figure	What it must show	Facts that must appear	Source
Context diagram (4.3.1)	SBMS and its external parties	Field officer, administrator, public verifier, PostgreSQL, the ML predictor as an internal component, no NEMIS link	`backend/src/index.js` mount table
Use case diagram (4.3.2)	Actors to use cases	The 14 screens' capabilities; public verification as unauthenticated	`frontend/src/App.jsx`; API catalogue §3
Activity: registration and screening (4.3.3)	Decision points	Validation, advisory lock, whole-registry scoring, qualification, `top-1` aggregation, 503 on predictor failure, student still created	`students.controller.js:31-161`
Activity: review to payment gate (4.3.4)	Two gates	Approval requires verified and no open or confirmed case; payment requires approved, verified, no open case and amount ceiling	`bursaryAwards.controller.js:151-209`; `disbursements.controller.js:65-136`
Class diagram (4.3.5)	Entities and services	Student, DuplicateCase, ReviewEvent, Award, VerificationEvent, Disbursement, ReconciliationEvent, Certificate, User	Data dictionary §4
Sequence: registration (4.3.6)	Message order across tiers	SPA → API → lock → SELECT FOR UPDATE → predictor → aggregation → insert → audit → response	Architecture map §5
Sequence: review decision (4.3.7)	Decision to terminal state	Masked queue, audited reveal, decision plus event plus audit in one transaction	`duplicateFlags.controller.js`
Sequence: award to certificate (4.3.8)	Four gates and a snapshot	Verification evidence, approval, payment ceiling, certificate snapshot with unique award	API catalogue §3.5-§3.6
Sequence: public verification (4.3.9)	Token handling	Fragment token, `replaceState`, single-key POST, existence query, boolean answer	`docs/qr-verification-2026-10-30.md` §1
ER diagram (4.3.10)	13 tables and relationships	Cascades for review data; RESTRICT for money, certification and verification records	Data dictionary §5
ML pipeline diagram (4.3.11)	Six stages	Generation, feature construction, training, validation, artifact, serving	ML documentation §2-§3

`FIGURE_PLACEHOLDER_CHECKLIST.md` already separates diagrams that can be drawn from specifications from those that need screenshots. No screenshot exists yet (G-04).

5. Design: architecture and module responsibilities

Layer	Responsibility	Notable design choice	Where the evidence sits
Presentation	Rendering, role routing, plain-language messaging	Officers never see model internals; administrators see score, version and cut-offs	UI inventory §3-§5

Layer	Responsibility	Notable design choice	Where the evidence sits
Application	Auth, validation, workflow, audit	The role is re-read from the database, so a stale token cannot carry an old privilege	Security catalogue §4
Domain services	Screening, aggregation, rules, documents, readiness	One shared module per rule set, so policy cannot drift between screens	Architecture map §3.2
Persistence	Constraints and referential actions	The database refuses an unverified approval and a half-recorded clearance	Data dictionary §7
Inference	Feature construction and scoring	Training and serving share one feature module	ML documentation §3

Three design decisions deserve explicit treatment because they are what the chapter's "design justification" section should argue:

1. **A human gate enforced by the schema.** `bursary_awards_approval_verification_check` makes it impossible to store an approved award for an unverified beneficiary, regardless of which code path attempted it. The control is not in the controller alone.
2. **Snapshot rather than reference for certificates.** A certificate copies the facts it prints, so a later award edit cannot silently alter what a printed document says; verification compares snapshot with live award and answers `not_current` on a mismatch.
3. **Aggregation of cases, not of scores.** `top-1` caps cases per registration at one while still counting suppressed candidates in the audit trail. It manages workload without pretending to improve the model.

6. Design: workflow and state

Entity	States	Transition enforcer	Refusal message
Duplicate case	`pending` → `confirmed_duplicate` / `not_duplicate` / `needs_verification` (terminal)	Controller plus event history	`This duplicate flag has already been decided`
Award	`submitted` → `approved` / `rejected`	Schema CHECK plus controller	`Verify the beneficiary before approving this award.` / `Resolve duplicate review cases before approving this award.`
Verification	`not_checked` → `verified` / `not_found` / `needs_investigation`	Schema CHECK on evidence	`Provide an outcome, notes, and evidence for verified cases.`
Payment	`issued` → `cleared` / `failed` / `cancelled`	`canTransitionPayment` plus CHECK	`This payment status cannot be changed.`
Receipt	`null` → `awaiting_acknowledgement` → `acknowledged` / `discrepancy`	`receiptStatus` in cents	`Enter the received amount and evidence reference.`
Certificate	`active` → `revoked`	CHECK plus UNIQUE award	`This certificate has already been revoked.`

A confirmed duplicate is deliberately **terminal**: it leaves the beneficiary unpayable and cannot be reversed through the API. That is correct policy, and it also means a mistaken confirmation can be corrected only in the database. The chapter should say so rather than describe the design as forgiving.

7. Design: access control, security and validation

Control	Implementation	What must be said about it
Authentication	bcrypt cost 12; JWT 8 h; secret >= 32 bytes validated at start-up	Implemented and tested; no multi-factor and no token revocation list
Authorisation	`requireRole` per route or router; role re-read from the row	Tested; matrix in the security catalogue §3
Account state	`is_active` checked on every request	Deactivation is the only revocation mechanism

Control	Implementation	What must be said about it
Origin control	Allowlist checked before routing plus `cors`	A browser policy, not authentication
Headers	`nosniff`, `DENY`, `no-referrer`, restricted permissions	No CSP and no HSTS from the API
Body bound	100 kB JSON limit	Tested
Error hygiene	5xx always `Internal server error`; allowlisted diagnostics only	Tested
Rate limits	10 sign-ins, 10 student verifications, 20 certificate verifications per IP per 15 min	In-process only; not shared across replicas
Validation	Zod per endpoint with cross-field rules	Tested through the API
Identifier masking	Last three characters until an audited reveal	Tested
Privacy of public endpoints	Boolean and authenticity facts only	15 tests

The residual risks in `SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md` §11 belong in the chapter's security-evaluation subsection. They are: a JWT in `localStorage`, no revocation list, no per-account lockout, no MFA, an in-process limiter, no CSP or HSTS, no row-level security, a conventionally-immutable audit trail, a temporary admin-login diagnostic, no penetration test and no dependency-audit gate.

8. Design: interface inventory

Screen group	Routes	Purpose	Evidence
Public verification	`/verify`, `/verify-certificate`	Existence and authenticity, one line or a few facts	9 browser checks; 15 API tests
Sign-in	`/login`, `/admin/login`, `/field-officer/login`	Role-specific entry	4 auth tests; browser check 1
Officer workspace	`/field-officer`, `/students`, `/students/register`, `/eligibility`	Register, review the register, score eligibility	23 browser checks; 9 student tests
Award and payment	`/disbursements` (+ certificate panel)	Submit, verify, approve, pay, clear, acknowledge, certify	20 API tests; no browser harness
Administration	`/dashboard`, `/duplicate-review`, `/reports`, `/settings`, `/audit`, `/users`	Adjudicate, monitor, tune, administer	33 API tests; 9 browser checks

Full detail, including the deliberate asymmetry between what an officer and an administrator may see, is in `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md`. Two interface facts matter for the chapter's honesty: the registry screen renders whatever the endpoint returns because the endpoint has no pagination, and the administrator's award picker has no data because `GET /api/students` is officer-only.

9. Design: database

Aspect	Fact
Tables	13: users, students, duplicate_flags, duplicate_review_events, bursary_awards, bursary_award_verification_events, disbursements, disbursement_reconciliation_events, bursary_certificates, audit_logs, reports, beneficiaries, sbms_schema_migrations
Migrations	14 forward files, dated 20261019 to 20261031, plus one rollback script
Constraints that matter most	approval requires verified; clearance requires evidence; acknowledgement requires officer-assisted method and evidence; one certificate per award; one case per unordered pair; token format `^[a-f0-9]{64}$`
Referential asymmetry	Review data cascades; money, certification and verification data restrict deletion
Indexes	Nine purpose-built indexes; none on student name, school or date of birth, consistent with whole-registry scanning
Legacy	`beneficiaries` table and `scholarship_amount` with unresolved meaning, deliberately not mapped to awards

The full column-level dictionary is `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` §4. That file is the source for the chapter's Table 4.4 (column design) and must be cited rather than paraphrased.

10. Design: physical deployment

Element	Fact
API	Azure Container Apps, `minReplicas 0`, image `ghcr.io/eggxellent/sbms-api` (1.37 GB)
Frontend	Azure App Service F1 (free), zip deployment of the Vite build
Database	Azure Database for PostgreSQL Flexible Server, B1MS, 32 GiB, stopped between demonstrations
Region	UAE North, forced by a `sys.regionrestriction` policy; Static Web Apps is not creatable on this subscription
Cost	$4.42 per idle month; $4.58 per 8-hour demonstration; about $6.42 in a normal month
Logs	Destination none; diagnostics are the process's own output plus the start-up guard message

Chapter Four's deployment figure must show one container (API plus ML), one web app, one managed database and the public HTTPS ingress, and must state that this is a demonstration topology rather than a production one.

11. What Chapter Four must not claim

Tempting claim	Why it fails
"The architecture scales horizontally"	Rate limits, thresholds and the ML subprocess are per-process; nothing was tested under load
"The design eliminates duplicate payments"	It blocks payment only when a case exists and is open or confirmed; an undetected duplicate still passes
"The ML model was specified as XGBoost"	The proposal specified Random Forest; this is a documented deviation (G-21)
"The system integrates with national registries"	It does not (FR-32)
"Certificates prove payment"	They prove an approval; the PDF says so explicitly

PART II — EVIDENCE FOR CHAPTER FIVE: IMPLEMENTATION, TESTING AND EVALUATION

12. Implementation environment

Item	Value	Status of the evidence
Operating system	Windows 11 (win32)	Recorded
Node.js	v22.11.0	Recorded
npm	10.9.0	Recorded
Python	3.11.9 with a service virtual environment	Recorded
PostgreSQL	17.11 client and server	Recorded
Docker	CLI 29.9.2, engine **not** running	Recorded; image built by CI instead
Browser automation	Headless Chrome over the DevTools Protocol from Node 22's built-in `WebSocket`	Recorded
Hardware specification	**Not recorded** (G-01); logs show 7.4 GB RAM with about 0.5 GB free	Blocking gap
Test database	Disposable `sbms_test` on loopback, role `sbms_test_runner`, guarded by `testDatabaseGuard.js`	Recorded

The hardware gap is the chapter's weakest environmental statement. It is honest to write that the workstation specification was not captured, that available memory was observed at 7.4 GB with approximately 0.5 GB free during the sweeps, and that one sweep produced 149 or 150 of 151 tests under memory pressure (G-02), rather than to claim a clean environment.

13. Implementation by feature, and where the code is

Feature	Implementation	Tests
Authentication and RBAC	Two sign-in endpoints, JWT, role re-read, user administration	4 + 6 + 1
Registration and validation	Zod schema, five-school constraint, phone and identifier normalisation	9
Duplicate screening	Advisory lock, whole-registry comparison, Python subprocess, `top-1` aggregation, provenance	15 + 13 + 8 + 6 + 1 + 4
Duplicate review	Masked queue, audited reveal, decision with reason, terminal states	9
Eligibility scoring	Five weighted factors, four categories, persisted result	Route and validation only
Award lifecycle	Submission, verification with evidence, approval and rejection with reasons	5
Payments and reconciliation	Issue, clearance with evidence, officer-assisted acknowledgement, discrepancy detection, exceptions	4 + 4 + 4
Student QR identification	32 random bytes, fragment token, existence-only endpoint, dedicated limiter	10 + 4
Award certificates	Snapshot row, reference, token, A4 landscape PDF, revocation, `not_current`	12 + 5
Dashboard, reports, audit, settings	Totals and trend, date-range PDF, audit trail, three cut-offs	6 (authorisation); content by inspection
Operations	Health, readiness, start-up guards, migration ledger	1 + 5 + migration unit tests

14. Test evidence, by level

Level	Command	Result	Date
Backend Jest, whole tree	`npx jest --runInBand tests`	**24 suites / 151 tests pass**	2026-11-01
Backend Jest, earlier generation	`npm test`	21 suites / 128 tests pass	2026-10-31

Level	Command	Result	Date
Backend native, in-process	`node --test` over seven files	**43/43 pass**	2026-10-31
Backend native, out-of-process	`node --test test/api.smoke.test.js`	**3/3 pass**	2026-10-31
Frontend production build	`npm.cmd --prefix frontend run build`	**exit 0**, 2 006 modules, 10.11 s	2026-11-01
ML service	`unittest discover -s ml-service/tests`	**33/33 pass**	2026-10-31
ML research helper	`unittest discover -s experiments/sbms_dataset`	**7/7 pass**	2026-10-31
Browser, registration and review	`node scripts/browser-smoke.mjs`	**23/23 pass**	2026-10-30
Browser, QR page	`node scripts/qr-smoke.mjs`	**9/9 pass**	2026-10-30
Browser, registration fields on the live deployment	`node scripts/registration-fields-smoke.mjs`	**11/11 pass**	2026-11-01
Database restore	`pg_dump` then `pg_restore` into an independent database	Pass, digests compared	2026-10-31
Phone-camera QR scan	manual	**Not run**	—
Browser check of `/verify-certificate`	no harness	**Not run**	—
User acceptance	—	**Not conducted**	—
Load or scale test	—	**Not run**	—
Docker build in this environment	—	**Not run** (engine unavailable)	—

The count that may be quoted as the current automated position is **24 suites / 151 tests, 2026-11-01**, plus the native, browser and Python layers separately. Counts must never be summed across levels, and never quoted without a date.

15. Defects found during testing, and what closed them

Six defects are tabled here because each has a code-level root cause, a fix and a test or browser check at this revision. They are a selection, not a register: the session documents record others, including a date-of-birth feature skew that made two identical dates score `dob_match: 0`, a registration 500 caused by an origin variable read at request time, a deployed SPA that returned 404 for every `/api/verify` call, a review queue that closed a case as soon as it was opened, and a readiness check that passed against a schema missing a table. Those are recorded where they happened, in `docs/SBMS_PROJECT_HANDOVER_2026-10.md` §13.2-§13.4 and `docs/azure-deployment-log-2026-10-31.md` §7-§8, and this report does not restate them.

Defect	Effect before the fix	Root cause	Closed by
Payment clearance returned 500 for every status change	No payment could ever be marked cleared; all reconciliation reporting was unreachable	One SQL parameter reused across `VARCHAR(20)`, `VARCHAR(50)` and a text comparison (PostgreSQL 42P08)	Explicit casts, with a comment recording why they must stay
Stale QR result on a second scan in the same tab	A valid code followed by an invalid one still read "Record found"	Verification ran once on mount; a hash-only change does not remount	`hashchange` listener
StrictMode regression introduced by the first fix	Development runs reported "invalid" for a valid token	The effect re-read an already-stripped URL	Token captured during render, effect keyed on it
`risk_level: critical` unreachable in the UI	A suspicious case could never be styled as such	The API only emitted `review`	API now publishes `priority` and `exact_identifier_matches`
High-priority cut-off was dead configuration	Every model case was `high`; screen showed "high 0.00"	Queue prioritised on the review cut-off and published a null threshold	Priority computed on the high-priority cut-off and published

Defect	Effect before the fix	Root cause	Closed by
Registration form omitted UPI and birth certificate	The critical exact-identifier rule could only fire for seeded rows	Form posted five fields; API accepted eight	Two optional inputs added and verified live with 11 checks

Two of these are the strongest evidence that testing found real faults rather than confirming what was written: payment clearance was completely broken and invisible to the suite until a workflow test exercised it, and the QR defect lived in the interaction between hash routing and React's mount behaviour, where no unit test would have reached it.

16. Machine-learning evaluation

Aspect	Fact	What it supports
Artifact	`xgboost:6ec9f2a3…`, six features, one file	Provenance per case
Holdout	12 000 synthetic pairs; 0 shared groups, 0 shared identities, 0 identical rows	Leakage check
In-sample versus holdout F1	0.9196 versus 0.9063	No memorisation
Grouped cross-validation	0.9067 mean F1, within 0.0004 of the holdout	Stability inside the training file
Learning curve	0.8886 at 10% of groups rising to 0.9063 at 100%	Near-flat, not memorisation
Live cut-off	0.50: precision 0.9204, recall 0.8927, F1 0.9063, miss 10.73%, FPR on hard negatives 7.72%	Synthetic operating point only
Independent FPR baseline	0.35% of 2 000 synthetic unrelated pairs at 0.50	A different question from 7.72%; report both or neither
Per-feature AUC	0.34 to 0.71	No single-feature shortcut
Adjudicated data	**None**	No real-world metric is possible

The reported numbers describe a generator, not Kenyan bursary records. Three limitations carry into Chapter Six: positives are one duplicate pattern (`repeat_application`), the holdout was used for model selection (so the figures are selection-optimistic), and one artifact with one seed means a retrain invalidates them.

17. Evaluation against the objectives

Objective	Status	Evidence	What is missing
(i) Investigate the existing challenges of manual registration and tracking	**Partially achieved**	Chapter Two document analysis; requirement register derived from it	No primary fieldwork or interviews (G-06)
(ii) Design architecture, database schema and ML pipeline	**Achieved with one deviation**	Three-tier architecture, 13 tables, six-feature pipeline, deployed	Random Forest replaced by XGBoost (G-21)
(iii) Implement registration, unique identifiers, QR identification and RBAC	**Achieved**	45 endpoints; QR flow with 10 + 4 tests and 9 browser checks; role matrix with 6 + 11 tests	Phone-camera scan not performed (G-11)
(iv) Validate, test and evaluate effectiveness	**Partially achieved**	151 Jest tests, 43 native, 40 Python, 43 browser checks; synthetic ML evaluation; live deployment checks	No real-user evaluation, no load test, no real-world ML metric

18. Evaluation against non-functional requirements

NFR	Verdict	Basis
NFR-01 performance	Partial	22.67 ms for 50 pairs; no registry-scale measurement
NFR-02 to NFR-06 security and privacy	Verified	Tests plus implementation
NFR-07 to NFR-11 rate limits, bodies, errors, origins	Verified	Tests; in-process limitation recorded

NFR	Verdict	Basis
NFR-12 database integrity	Verified	Constraint tests across workflow and certificate suites
NFR-13 concurrency	Partial	Advisory lock present; no concurrency test
NFR-14 usability	Partial	Browser-checked; no usability study
NFR-15 maintainability	Verified	Shared feature module, ordered migrations, three test runners
NFR-16 zero-cost operation	Partial	Zero-cost configuration is recorded (scale-to-zero, free tiers); no billed month reconciled
NFR-17 human decisions	Verified	Score creates cases only; gates are role-gated
NFR-18 recovery	Partial	Rehearsal done; no schedule or retention
NFR-19 browser compatibility	Partial	Chrome only
NFR-20 degradation	Verified	503 on predictor failure
NFR-21 audit completeness	Partial	Attributed in every tested flow; append-only is convention, not a constraint

In requirement terms this is 14 verified and 7 partial, the same split as §2; the grouped rows above cover NFR-02 to NFR-11 individually.

19. Limits of this evaluation

1. No user acceptance study, so nothing about usability, comprehension or workflow fit can be concluded.
2. No load test, so nothing about concurrency, throughput or latency at scale can be concluded.
3. No real-world duplicate labels, so nothing about model accuracy can be concluded.
4. No phone-camera session, so physical QR identification is unverified even though the code path is tested headlessly.
5. No penetration test or dependency audit, so the security section is a control inventory, not an assurance statement.
6. The deployment is a demonstration instance: one region, no monitoring, no backup schedule, demonstration data.
7. Test counts come from dated runs; none was re-executed for this report.

PART III — EVIDENCE FOR CHAPTER SIX: CONCLUSIONS, RECOMMENDATIONS AND FUTURE WORK

20. Conclusions, objective by objective

Objective	Conclusion the evidence supports	Wording to use
(i) Investigate the challenges	Partially achieved: the problem is characterised from secondary sources, not fieldwork	The investigation establishes the problem from documentary evidence and does not claim primary data
(ii) Design the architecture, schema and pipeline	Achieved, with a stated deviation from the proposal	The artefact is a three-tier system with a 13-table schema and a six-feature pipeline; the classifier is XGBoost rather than the proposed Random Forest, and that change is recorded rather than reconciled
(iii) Implement registration, identifiers, QR identification and RBAC	Achieved	Every capability named in the objective is implemented and covered by executed tests, with the physical scan and the scale behaviour reported as unverified
(iv) Validate, test and evaluate	Partially achieved	The artefact is tested at API, browser and database level and evaluated against synthetic data only; the absence of adjudicated data and of user involvement bounds the conclusion

21. The stated contribution

Contribution	Statement the evidence supports	Evidence
Engineering	A working, deployed bursary system in which an algorithmic signal can never authorise payment, because the gates are enforced in the database as well as the API	`workflow.test.js` #2-#6; schema CHECK constraints
Methodological	The registration-level flag arithmetic: because every registration is compared with the whole registry, a per-pair false-positive rate p produces a nuisance flag with probability 1 − (1 − p)^N, so queue volume grows with registry size independently of cut-off. The `top-1` aggregation policy is the response	`docs/ml-evaluation-baseline-2026-10.md` §3-§4
Evaluation	A reproducible synthetic pipeline with leakage gates, an operating-point sweep, and integrity tests that fail the build if the artifact and its recorded metrics diverge	`docs/ml-training-integrity-2026-10-30.md`; `ml-service/tests/`
Transparency	A documented set of claim limits and stale-document exclusions that a reader can audit	`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md`

The aggregation finding is the most defensible contribution because it is arithmetic rather than assertion: it follows from the measured per-pair rate and does not depend on any accuracy claim.

22. Recommendations

#	Recommendation	For whom	Basis
1	Obtain adjudicated duplicate pairs before making any accuracy statement	Programme owner	D1, D5
2	Choose a cut-off only after stating an acceptable miss rate and the weekly review capacity	Programme owner	D2, D3
3	Introduce candidate blocking before the register exceeds a few thousand rows	Project owner	FR-34; the N x p arithmetic
4	Replace the in-process rate limiter with a shared store before running more than one instance	Operator	NFR-07
5	Schedule and retain database backups, and rehearse a restore against the deployment, not only locally	Operator	NFR-18, G-19
6	Perform at least one phone-camera scan of a printed student code and a printed certificate	Author	G-11

#	Recommendation	For whom	Basis
7	Conduct a small structured usability session with officers and record consent	Author	G-06, NFR-14
8	Mark `docs/verification.md` and `docs/uat-test-plan.md` superseded or withdraw them	Author	G-07, G-08
9	Commit the two untracked root reports and resolve the model-artefact ignore rule	Author	G-27, G-28
10	Capture the workstation and device specification, or state explicitly that it was not recorded	Author	G-01

23. Future work

Area	Work	Why it matters	Depends on
Candidate generation	Name-token blocking with a capped candidate list; keep `school_match` as a feature; treat a missing school as a wildcard	Removes the linear and quadratic cost and makes the queue affordable	D4 approval and new tests
Model governance	A registry entry per artifact, with the metrics file bound to the hash	Makes a retrain auditable	The current integrity gate as a starting point
Calibration	Calibrate scores before calling them probabilities, or stop calling them probabilities	Prevents a 0.6 being read as "60% likely a duplicate"	Adjudicated data
Adjudication	In-app review of sampled pairs, stratified by score band, two independent reviewers	Produces the only evidence that can support an accuracy claim	D5
Operations	Scheduled backups, retention, monitoring, alerting and a log workspace	Turns a demonstration into an operated service	Budget decision
Security	Short-lived tokens with refresh, server-side revocation, per-account lockout, MFA for administrators, CSP and HSTS, dependency auditing, a penetration test	Closes the residual risks in the security catalogue §11	Time and review
Data model	Remove the legacy `beneficiaries` module; add the constraint tying a payment's student to its award's student	Removes two parallel registries and one unchecked pairing	Migration and data cleanup
Accessibility	Keyboard and screen-reader audit plus a contrast check	NFR-14 cannot be claimed without it	Design time
Physical identification	Token rotation and re-issue, and a printed-card process	FR-35 and FR-36 are currently gaps	Policy decision on re-issue

24. Boundaries of the conclusions

Boundary	What it excludes
Data boundary	Every metric is synthetic; no claim about Kenyan bursary records is supported
User boundary	No officer, student, guardian or school participated; usability is unmeasured
Scale boundary	Nothing above 50 candidate pairs was timed; no hosted load test exists
Device boundary	Chrome only; no phone camera, no printed output on a physical printer
Operational boundary	Demonstration instance; no monitoring, no backup schedule, no high availability
Security boundary	Control inventory, not an assurance statement; no review or penetration test
Governance boundary	The five owner decisions are unanswered, and no supervisor finding can be closed without its wording

PART IV — ASSURANCE

25. Completeness matrix (requested area to status to location)

Requested area	Status	Where it is covered
Scope, actors and requirement register	Complete	Part I §1-§3; `SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md` §4-§7
Functional requirements with verification status	Complete	Requirements catalogue §6
Non-functional requirements with status	Complete	Requirements catalogue §7
Analysis artefacts and what each must encode	Complete	Part I §4
Architecture, modules and cross-cutting concerns	Complete	Part I §5; `SBMS_SYSTEM_ARCHITECTURE_AND_MODULE_MAP.md` §1-§10
Access control matrix and security controls	Complete	Part I §7; `SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md` §1-§12
Database schema and full data dictionary	Complete	Part I §9; `SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md` §2-§11
API endpoint catalogue with contracts	Complete	`SBMS_API_ENDPOINT_CATALOGUE.md` §2-§7 (45 endpoints)
ML subsystem: features, artifact, metrics, limits	Complete	Part II §16; `SBMS_ML_SUBSYSTEM_DOCUMENTATION.md` §1-§14
UI screens, workflows and figure mapping	Complete	Part I §8; `SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md` §1-§10
Implementation environment	Complete for software; **gap** for hardware	Part II §12 (G-01)
Test inventory with what each suite proves	Complete	Part II §13-§14; `SBMS_TEST_EVIDENCE_CATALOGUE.md` §2-§11
Defect log and resolution	Complete	Part II §15
Evaluation against objectives and NFRs	Complete	Part II §17-§18
Conclusions, contribution, recommendations, future work	Complete	Part III §20-§24
Claim limits and stale-document exclusions	Complete	Part III §24; `SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md` §3-§5
Figures and screenshots	**Not produced**	`FIGURE_PLACEHOLDER_CHECKLIST.md`; G-03, G-04, G-05
User acceptance evidence	**Not available**	G-06
Physical QR scan evidence	**Not available**	G-11
Load and scale evidence	**Not available**	NFR-01
Production metrics, uptime, monitoring	**Not available**	Part II §19
Supervisor findings SBMS-01…18	**Not recoverable**	`SBMS_PHASE_2_SUPERVISOR_FINDINGS_REVIEW.md` (repository root) §11

26. Gap summary, ordered by submission risk

Priority	Gap	Effect if unresolved	Owner
Blocking	G-01 workstation and device specification	Chapter Five §5.2.1 cannot be completed	Author

Priority	Gap	Effect if unresolved	Owner
Blocking	G-03/G-04/G-05 figures and screenshots, plus the synthetic-data check	Chapters Four and Five cannot be illustrated	Author
Blocking	G-06 a user-acceptance position, or an explicit statement that none was conducted	NFR-14 stays unverified and Chapter Five §5.5.6 is empty	Author
Important	G-11 one phone-camera scan of a printed code	The identification mechanism rests on a headless-browser check	Author
Important	G-21 the Random Forest to XGBoost deviation	Must be stated in Chapters Four and Six	Author
Important	G-07/G-08 stale `uat-test-plan.md` and `verification.md`	Two documents contradict the implementation	Author
Important	G-02 the 149/150-of-151 instability	Test stability claim needs a caveat	Author
Important	G-25 two test-count generations in one file	A count quoted without a date is misleading	Author
Optional	G-27/G-28 untracked reports and the artefact ignore rule	Reproducibility from a clean clone	Author
Optional	G-19 retention policy, G-17 monitoring	Operational maturity	Operator

27. Reproduction: how a reader can re-establish every claim

```
# 1. Fix the revision under description
git checkout fix/sbms-completion-work
git rev-parse HEAD          # expect d6b5032

# 2. Backend suite (needs backend/.env.test; values are never printed)
cd backend; npm test        # expect 24 suites / 151 tests on 2026-11-01

# 3. Native tests
node --test test/*.node.test.js test/*.node.js test/*.unit.test.js     # expect 43/43
node --test test/api.smoke.test.js                                     # expect 3/3 against a live API

# 4. Frontend build
npm.cmd --prefix frontend run build                                    # expect exit 0

# 5. ML suite and research helper, then the integrity audit and the operating point
cd ..\ml-service
.\.venv\Scripts\python.exe -m unittest discover -s tests -v           # expect 33/33
.\.venv\Scripts\python.exe -m unittest discover -s experiments\sbms_dataset -p test_experiment.py   # expect
7/7
.\.venv\Scripts\python.exe .\check_training_integrity.py
.\.venv\Scripts\python.exe .\evaluate_operating_point.py

# 6. Browser checks (each needs the API, the frontend dev server and a seeded account)
cd ..
node scripts/browser-smoke.mjs                                         # 23 checks
node scripts/qr-smoke.mjs                                              # 9 checks
node scripts/registration-fields-smoke.mjs                             # 11 checks (live deployment)
# node scripts/admin-reveal-smoke.mjs                                   # not re-run: admin password drifted
```

Together steps 5 and 6 reproduce the recorded 40 Python tests and 43 browser checks. The fourth browser harness is listed but commented out deliberately: it cannot pass until the administrator password is reset, and a script that silently skips would misrepresent the evidence.

The Jest guard `backend/tests/testDatabaseGuard.js` refuses to run unless the target database is the disposable loopback `sbms_test`, so step 2 cannot damage the development or deployment database.

28. Document map

File	What it is for	Read it when
This report	The consolidated evidence base and drafting aid	Starting any of Chapters Four to Six
`SBMS_REQUIREMENTS_AND_SCOPE_CATALOGUE.md`	The numbered requirement register with status	Writing requirement tables or conclusion claims
`SBMS_SYSTEM_ARCHITECTURE_AND_MODULE_MAP.md`	Modules, flows, decisions, weaknesses	Writing the architecture and implementation sections
`SBMS_DATABASE_SCHEMA_AND_DATA_DICTIONARY.md`	Every table and column, plus state machines	Writing the database section or an ER diagram
`SBMS_API_ENDPOINT_CATALOGUE.md`	All 45 endpoints with guards and contracts	Writing the interface or API appendix
`SBMS_ML_SUBSYSTEM_DOCUMENTATION.md`	Features, artifact, metrics, limits, decisions	Writing the ML and evaluation sections
`SBMS_TEST_EVIDENCE_CATALOGUE.md`	Every test artefact, date and what it proves	Writing the testing section
`SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md`	Controls, matrix, residual risks	Writing the security section
`SBMS_DEPLOYMENT_AND_OPERATIONS_GUIDE.md`	Resources, pipeline, config, runbook, costs	Writing the deployment section
`SBMS_UI_SCREEN_AND_WORKFLOW_INVENTORY.md`	Screens, workflows, figure mapping	Writing the interface design section or capturing screenshots
`SBMS_EVIDENCE_PROVENANCE_AND_CLAIM_LIMITS.md`	Evidence tiers, forbidden claims, open decisions	Before writing any claim
`CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN.md`	The drafted chapter	Drafting or checking Chapter Four
`CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION.md`	The drafted chapter	Drafting or checking Chapter Five
`CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORKS.md`	The drafted chapter	Drafting or checking Chapter Six
`OBJECTIVE_REQUIREMENT_TEST_TRACEABILITY.md`	Objective to requirement to test to conclusion	Defending a conclusion
`DOCUMENTATION_GAP_REGISTER.md`	Every open gap with priority	Before submission
`EVIDENCE_AND_VERIFICATION_REPORT.md`	Code, test and document evidence index	Auditing a specific claim
`FIGURE_PLACEHOLDER_CHECKLIST.md`	What each figure must show	Producing the figures

Entry set added later, same folder. Nine files now form the front door to this evidence base, and each is summarised in `CLAUDE_HANDOFF_GUIDE.md` §2: `ACADEMIC_GUIDE_REQUIREMENTS.md`, `CHAPTER_4_SYSTEM_ANALYSIS_AND_DESIGN_EVIDENCE.md`, `CHAPTER_5_IMPLEMENTATION_TESTING_AND_EVALUATION_EVIDENCE.md`, `CHAPTER_6_CONCLUSIONS_RECOMMENDATIONS_AND_FUTURE_WORK_EVIDENCE.md`, `CROSS_CHAPTER_CONSISTENCY_AND_TRACEABILITY.md`, `DIAGRAM_SPECIFICATIONS.md`, `MASTER_EVIDENCE_REGISTER.md` (51 evidence IDs), `ACADEMIC_COMPLETENESS_AND_GAP_AUDIT.md` and `CLAUDE_HANDOFF_GUIDE.md`. Read those first; the table above is the deeper tier they point into.

29. Final statement

The evidence supports a clear, bounded conclusion: SBMS is a working, tested, deployed three-tier system whose duplicate-detection signal is deliberately subordinate to human decision, whose payment path is gated by verification and duplicate state in the database as well as the API, and whose machine-learning component is evaluated only against synthetic data. The engineering is real and inspectable. The evaluation is partial, and this documentation set states exactly where it stops: no user acceptance, no load test, no physical QR scan, no real-

world accuracy figure, no production assurance. A report that keeps those boundaries visible is stronger than one that quietly crosses them.